

Punto 1 — Introduzione al progetto MicroBot

Negli ultimi decenni la robotica ha attraversato una trasformazione profonda, spostandosi progressivamente dal paradigma del robot singolo, centralizzato e programmato in modo deterministico, verso sistemi distribuiti composti da molte unità autonome capaci di cooperare, adattarsi e produrre comportamenti collettivi complessi a partire da regole locali semplici. Questo cambio di paradigma, noto come *swarm robotics*, si ispira direttamente ai sistemi biologici — colonie di insetti, stormi di uccelli, banchi di pesci — nei quali l'intelligenza non risiede nel singolo individuo ma emerge dall'interazione tra individui, dall'ambiente e dalle regole di prossimità che governano i contatti locali. Il progetto MicroBot nasce precisamente in questo contesto, con l'obiettivo di esplorare, progettare e realizzare un sistema di *swarm robotics* modulare in cui l'aggregazione fisica, la comunicazione distribuita e il controllo immersivo da parte dell'utente umano costituiscono i tre assi portanti dell'intera architettura.

L'idea fondante di MicroBot è che un insieme di moduli robotici compatti — denominati MicroBot — possa dare origine, attraverso l'aggregazione magnetica controllata e la coordinazione wireless, a strutture fisiche dinamiche capaci di cambiare forma, rispondere a comandi esterni e manifestare comportamenti emergenti che trascendono le capacità del singolo modulo. Ogni MicroBot è progettato come un'unità autonoma completa: possiede un proprio microcontrollore, una batteria ricaricabile, un sistema di segnalazione visiva tramite LED RGB e un apparato magnetico composto sia da magneti permanenti sia da elettromagneti attivi, che consente l'aggancio e il disaggancio controllato con gli altri moduli. La geometria del MicroBot — un doppio cono troncato che racchiude una sfera centrale — non è una scelta arbitraria ma riflette precisi vincoli funzionali: massimizzare il volume interno per l'elettronica, garantire superfici di contatto geometricamente compatibili per l'aggancio magnetico e mantenere il peso complessivo entro un intervallo che rende efficace l'interazione tra le forze magnetiche disponibili e la resistenza all'attrito con la superficie di appoggio.

Il sistema non si limita ai soli MicroBot. Attorno allo swarm fisico è costruita un'architettura a cinque livelli che comprende gli strumenti di input dell'utente, le interfacce software di simulazione e visualizzazione, il motore di calcolo dei pattern collettivi, il controller centrale di coordinamento e infine i MicroBot stessi come livello di attuazione fisica. Questa separazione in livelli non è puramente concettuale ma rispecchia una precisa scelta ingegneristica: mantenere disaccoppiata la logica di alto livello — gestita in ambienti come Unity e JavaScript — dalla logica di basso livello che governa il firmware degli ESP32 e il controllo real-time dell'hardware. Il risultato è un sistema in cui la complessità è distribuita e gestibile, in cui ciascuno strato può essere sviluppato, testato e sostituito in modo indipendente senza compromettere il funzionamento dell'intero ecosistema.

Un elemento distintivo del progetto è l'attenzione all'interfaccia uomo-macchina come componente progettuale di primo piano e non come elemento accessorio. L'utente interagisce con lo swarm attraverso gesture della mano rilevate da MediaPipe, espressioni facciali, movimenti del corpo catturati da un wristband dotato di IMU a nove assi e, in prospettiva, attraverso un visore VR prototipale che offre una rappresentazione tridimensionale immersiva dell'intero sistema. A questo si aggiunge il framework 4D/5D, che

introduce il concetto di stato interno invisibile del sistema — una quarta dimensione che modula le trasformazioni geometriche dello swarm in risposta a parametri acustici, rendendo il suono un meccanismo attivo di controllo e non semplicemente un effetto di feedback. Questa scelta teorica distingue MicroBot dai sistemi di swarm robotics tradizionali, nei quali l'interfaccia utente è quasi sempre ridotta a una console testuale o a una GUI bidimensionale.

Lo sviluppo del progetto segue una roadmap rigorosamente incrementale, strutturata in versioni successive che corrispondono a livelli crescenti di complessità e integrazione. La versione 0.1 ha definito l'architettura concettuale e le scelte progettuali fondamentali. La versione 0.2 ha tradotto queste scelte in una prima simulazione funzionale basata su microcontrollore Arduino, introducendo la logica a stati, la gestione del tempo non bloccante e i primi comportamenti collettivi dello swarm rappresentati tramite LED. La versione 0.3, obiettivo della fase corrente, prevede la realizzazione del primo prototipo fisico con comunicazione wireless reale, gestione della batteria, controllo magnetico attivo e chiusura del ciclo completo tra input umano e risposta fisica dello swarm. Le versioni successive — 0.4 e 0.5 — introdurranno autonomia locale, algoritmi di intelligenza distribuita e l'integrazione completa con il visore VR.

MicroBot si configura quindi non come un prodotto finito ma come una piattaforma aperta di ricerca e sviluppo, nella quale ogni componente — dalla geometria del guscio stampato in 3D alla scelta del protocollo di comunicazione wireless, dalla struttura del firmware alla logica dei pattern collettivi — è documentato, misurabile e sostituibile. La presente documentazione tecnica raccoglie in forma sistematica tutte le scelte progettuali effettuate, i parametri dimensionali definitivi, le specifiche dei componenti hardware, i modelli matematici adottati e la struttura del software, costituendo il riferimento ufficiale per tutte le fasi di sviluppo successive.

Punto 2 — Contesto scientifico e tecnologico

La swarm robotics è una disciplina che affonda le proprie radici nella biologia dei sistemi complessi e nell'intelligenza artificiale distribuita. Il termine stesso richiama esplicitamente il comportamento degli sciami biologici — le api, le formiche, le termiti, i murmuri degli storni — nei quali migliaia di individui dotati di capacità cognitive e sensoriali estremamente limitate sono in grado, attraverso interazioni locali ripetute e regole di prossimità semplici, di costruire strutture architettonicamente sofisticate, risolvere problemi di ottimizzazione combinatoria e rispondere in modo adattivo a perturbazioni ambientali di grande entità. La proprietà fondamentale che rende questi sistemi così affascinanti dal punto di vista scientifico e così promettenti dal punto di vista ingegneristico è l'emergenza: la capacità del sistema collettivo di manifestare comportamenti globali che non sono presenti, né prevedibili, analizzando il singolo individuo in isolamento.

Nell'ambito della robotica, questo paradigma è stato formalizzato a partire dagli anni Novanta del Novecento, con i lavori pionieristici di Gerardo Beni e Jing Wang, che nel 1989 introdussero il concetto di swarm intelligence come approccio computazionale ispirato ai sistemi biologici. Da allora, la ricerca in swarm robotics ha percorso una traiettoria di crescente complessità, passando dai primi esperimenti con robot Khepera su superfici piane ai sistemi contemporanei di droni autonomi in formazione tridimensionale, dai modelli

puramente simulativi alle implementazioni hardware su piattaforme miniaturizzate come i Kilobot del Harvard Wyss Institute, capaci di coordinarsi in sciame di centinaia di unità attraverso comunicazione a infrarosso e movimenti micrometrici indotti da vibrazioni.

Il contributo teorico più rilevante per la comprensione del comportamento collettivo nei sistemi multi-agente è rappresentato dal modello Boids, proposto da Craig Reynolds nel 1987. Reynolds dimostrò che tre sole regole locali — coesione verso il centro di massa del gruppo, allineamento con la velocità media dei vicini e separazione per evitare le collisioni — erano sufficienti a riprodurre con estremo realismo il comportamento dei banchi di pesci e degli stormi di uccelli in ambiente simulato. Questo risultato ha avuto implicazioni profonde non solo per la computer grafica, dove il modello fu originariamente sviluppato, ma per tutta la teoria dei sistemi complessi adattativi, dimostrando che la complessità del comportamento collettivo non richiede necessariamente una pianificazione centralizzata né una comunicazione globale tra gli agenti. Il progetto MicroBot adotta il modello Boids come nucleo del proprio motore di simulazione, estendendolo con regole aggiuntive legate all'aggancio magnetico, alla sincronizzazione temporale e alla gestione dei pattern geometrici definiti dal controller centrale.

Dal punto di vista della fisica applicata, il progetto MicroBot si inserisce nel filone della materia programmabile, un concetto introdotto da Neil Gershenfeld al MIT Media Lab per descrivere materiali e sistemi in grado di modificare le proprie proprietà fisiche — forma, rigidità, conduttività, comportamento meccanico — in risposta a segnali di controllo esterni. La materia programmabile rappresenta una delle frontiere più ambiziose della robotica moderna, con applicazioni che spaziano dalla chirurgia minimamente invasiva alla costruzione adattiva in ambienti estremi, dall'assemblaggio autonomo di strutture architettoniche alla realizzazione di interfacce tangibili che rispondono al tocco umano modificando la propria geometria. MicroBot si posiziona deliberatamente in questo spazio concettuale, progettando i propri moduli non come robot nel senso tradizionale del termine ma come unità di una materia attiva capace di auto-organizzarsi in strutture fisicamente reali e riconfigurabile in tempo reale.

Il contesto tecnologico in cui MicroBot si sviluppa è caratterizzato dalla straordinaria maturità raggiunta da alcune tecnologie abilitanti che fino a pochi anni fa sarebbero state inaccessibili a un progetto di ricerca indipendente. I microcontrollori della famiglia ESP32, prodotti da Espressif Systems, integrano in un singolo chip un processore dual-core a 240 MHz, moduli Wi-Fi e Bluetooth, convertitori analogico-digitali, interfacce I2C e SPI e decine di pin di input/output configurabili, il tutto in un package che misura meno di 2 centimetri per lato e consuma pochi decine di milliampere in condizioni operative normali. La stampa 3D FDM ha reso economicamente accessibile la produzione di gusci meccanici personalizzati con tolleranze dell'ordine del decimo di millimetro, eliminando la necessità di costose lavorazioni CNC per la realizzazione di prototipi. Le librerie di computer vision come MediaPipe di Google hanno portato il riconoscimento delle gesture e il tracciamento dei landmark facciali direttamente nel browser e su dispositivi mobili, senza richiedere hardware dedicato. Unity ha trasformato lo sviluppo di interfacce VR da attività riservata a grandi studi industriali a pratica accessibile a singoli sviluppatori con hardware consumer.

È in questo ecosistema tecnologico favorevole che MicroBot diventa non solo concettualmente interessante ma praticamente realizzabile. La combinazione di ESP32,

stampa 3D, magneti al neodimio, librerie open source per la visione artificiale e ambienti di sviluppo VR accessibili crea le condizioni per costruire, con risorse limitate, un sistema che fino a pochi anni fa avrebbe richiesto laboratori specializzati e budget di ricerca significativi. Questa accessibilità non è un dettaglio marginale ma una scelta progettuale esplicita: MicroBot è progettato per essere replicabile, documentato e aperto, nella convinzione che la democratizzazione degli strumenti di ricerca in robotica rappresenti un valore scientifico e culturale autonomo, indipendente dai risultati specifici del progetto.

Punto 3 — Obiettivi del progetto e contributi originali

Il progetto MicroBot persegue obiettivi che si articolano su tre livelli distinti ma interdipendenti: un livello scientifico, legato alla comprensione e alla modellazione del comportamento collettivo emergente in sistemi multi-agente fisici; un livello ingegneristico, relativo alla progettazione, realizzazione e integrazione di componenti hardware e software in un sistema funzionante; e un livello metodologico, concernente lo sviluppo di un approccio di progettazione incrementale e documentata che possa servire da riferimento per progetti analoghi.

Sul piano scientifico, l'obiettivo principale è dimostrare che un insieme di moduli robotici semplici, dotati di capacità computazionali e sensoriali limitate, è in grado di produrre comportamenti collettivi complessi e fisicamente significativi attraverso la sola interazione locale. In particolare, il progetto intende verificare se le soglie di aggancio e disaggancio magnetico, combinate con la sincronizzazione temporale distribuita e le regole di tipo Boids, siano sufficienti a garantire la formazione stabile di pattern geometrici predefiniti — linee, quadrati, cerchi, spirali, onde — in condizioni operative reali, ovvero in presenza di attrito, disallineamenti meccanici, variazioni nella carica delle batterie e ritardi nella comunicazione wireless. Questo obiettivo si distingue dagli analoghi studi in letteratura perché integra l'aggregazione fisica tramite forze magnetiche controllate — un meccanismo relativamente poco esplorato nella swarm robotics a scala centimetrica — con un sistema di controllo immersivo che include gesture, espressioni facciali e parametri acustici come canali di input.

Sul piano ingegneristico, il progetto si propone di realizzare un prototipo fisico completo e funzionante entro la versione 0.3, integrando in un unico ecosistema operativo componenti eterogenei che spaziano dalla meccanica di precisione stampata in 3D all'elettronica embedded, dalla comunicazione wireless real-time al rendering tridimensionale in Unity. Questo obiettivo implica la risoluzione di un insieme di sfide progettuali concrete: il dimensionamento di un guscio meccanico che ospiti elettronica, batteria e sistema magnetico in un volume esterno di soli 70 millimetri di altezza e 20 millimetri di diametro alle estremità; la progettazione di un circuito di pilotaggio delle coil elettromagnetiche che rispetti i vincoli termici ed energetici imposti da una batteria da 40–60 mAh; la definizione di un protocollo di comunicazione UDP su rete SoftAP capace di coordinare decine di nodi con latenze inferiori ai 50 millisecondi; e l'implementazione di un safety layer embedded che protegga i componenti da surriscaldamento, sovracorrente e instabilità del firmware senza introdurre latenze incompatibili con il controllo in tempo reale.

Sul piano metodologico, MicroBot si propone come dimostrazione concreta di un approccio alla progettazione robotica che privilegia la separazione netta tra livelli di astrazione, la simulazione sistematica prima della costruzione fisica e la documentazione continua come

strumento attivo di sviluppo e non come attività accessoria. Ogni scelta progettuale è giustificata quantitativamente — attraverso formule, stime d'ordine di grandezza e simulazioni — prima di essere tradotta in hardware o firmware. Questo approccio, ispirato alle pratiche dell'ingegneria industriale e della ricerca accademica, distingue MicroBot dai progetti hobbistici analoghi e ne costituisce uno dei contributi originali più significativi.

I contributi originali del progetto possono essere identificati con precisione in cinque aree. Il primo è l'integrazione del framework 4D/5D come meccanismo di modulazione dello stato interno dello swarm attraverso parametri acustici, un approccio che non trova precedenti diretti nella letteratura sulla swarm robotics e che apre la possibilità di utilizzare il suono come canale di controllo continuo e naturale per sistemi di materia programmabile. Il secondo è la geometria a doppio cono troncato con sfera centrale, progettata per ottimizzare simultaneamente la distribuzione interna dei componenti, le proprietà di aggancio magnetico e l'estetica visiva del modulo, con un peso target di 8–12 grammi che lo colloca in una fascia dimensionale particolarmente favorevole per l'interazione magnetica a contatto. Il terzo è il sistema di autenticazione biometrica facciale come cancello di sicurezza all'accesso al controller dello swarm, un elemento inusuale nei sistemi di swarm robotics che riflette una sensibilità verso i temi della sicurezza e del controllo accessi nei sistemi robotici distribuiti. Il quarto è l'architettura software a separazione netta tra strato Unity/C# per l'alto livello e strato ESP32/C++ per il basso livello, con un ponte di comunicazione UDP ben definito che permette lo sviluppo indipendente dei due ambienti. Il quinto è la prospettiva evolutiva verso la MicroHand, un'evoluzione futura del sistema in cui moduli specializzati con dita articolate trasformerebbero lo swarm da sistema di aggregazione passiva a sistema di manipolazione attiva degli oggetti fisici nell'ambiente.

Punto 4 — Visione e filosofia progettuale

Ogni progetto di ingegneria complesso è sorretto, al di sotto delle specifiche tecniche e delle scelte implementative, da una visione che ne orienta le decisioni nelle situazioni di ambiguità e ne definisce i criteri di priorità quando obiettivi diversi entrano in conflitto. Nel caso di MicroBot, questa visione può essere sintetizzata in un'unica proposizione: costruire un sistema in cui la frontiera tra il digitale e il fisico diventi permeabile, in cui l'intenzione umana si traduca in materia che si organizza, e in cui la complessità emerga dalla semplicità attraverso l'interazione. Questa visione non è puramente estetica né puramente funzionale — è al tempo stesso una posizione teorica sulla natura dell'intelligenza distribuita, una scelta ingegneristica sulla struttura dei sistemi complessi e una proposta culturale sul rapporto tra l'essere umano e le macchine che lo circondano.

La prima componente della filosofia progettuale di MicroBot è il principio di emergenza come obiettivo primario. In molti sistemi robotici il comportamento complessivo è il risultato diretto della programmazione esplicita di ogni azione: il robot esegue ciò che gli è stato ordinato, nella sequenza in cui gli è stato ordinato, con i parametri che sono stati specificati. MicroBot rifiuta questo paradigma come unico modello possibile e lo affianca — senza sostituirlo — con un modello in cui il comportamento globale dello swarm non è specificato direttamente ma emerge dall'interazione tra regole locali semplici, forze fisiche reali e vincoli ambientali. Questo non significa che il sistema sia imprevedibile o incontrollabile: significa che il controller definisce le condizioni dell'emergenza — i pattern target, le soglie magnetiche, le regole di coordinamento — e poi osserva, misura e adatta. Il controllo è indiretto,

parametrico, orientato alle condizioni al contorno piuttosto che alle traiettorie puntuali. Questa distinzione è profonda e ha implicazioni pratiche dirette sulla struttura del firmware, sulla scelta dei protocolli di comunicazione e sulla progettazione dell'interfaccia utente.

La seconda componente è il principio di incrementalità come metodo di sviluppo e come garanzia di qualità. MicroBot non è stato concepito come un sistema completo da realizzare in una singola fase, ma come una sequenza di versioni successive in cui ogni iterazione è completa e funzionante di per sé, aggiunge un livello di complessità ben definito rispetto alla versione precedente e non rompe la compatibilità con le scelte progettuali già consolidate. Questo principio, mutuato dalle metodologie di sviluppo software agile e dalle pratiche della ricerca sperimentale, ha una conseguenza diretta sulla qualità del progetto: ogni componente viene validato prima di essere integrato nel sistema successivo, ogni assunzione teorica viene verificata sperimentalmente prima di essere assunta come dato di progetto, ogni scelta di implementazione viene documentata insieme alla motivazione che la sorregge. Il risultato è un sistema in cui la comprensione cresce insieme alla complessità, e in cui il rischio di fallimento per accumulo di errori non rilevati è strutturalmente minimizzato.

La terza componente è il principio di separazione delle responsabilità come struttura organizzativa fondamentale. MicroBot distingue nettamente tra livello fisico e livello logico, tra hardware e firmware, tra simulazione e attuazione, tra interfaccia utente e controllo distribuito. Questa separazione non è solo una buona pratica ingegneristica — è una posizione filosofica sulla natura dei sistemi complessi: la complessità governabile è quella che può essere decomposta in sottosistemi con interfacce ben definite e responsabilità chiaramente delimitate. Un sistema in cui tutto dipende da tutto è fragile, difficile da testare, impossibile da modificare senza effetti collaterali incontrollabili. Un sistema in cui ogni livello comunica con quello adiacente attraverso protocolli stabili e ben documentati è robusto, estensibile e comprensibile anche a distanza di mesi dalla sua progettazione iniziale. MicroBot applica questo principio a ogni scala: tra Unity e ESP32, tra controller e MicroBot, tra firmware di basso livello e algoritmi di alto livello, tra la logica di aggancio magnetico e la logica di formazione dei pattern.

La quarta componente è il principio di fisicità come vincolo e come risorsa. A differenza dei sistemi puramente simulativi, MicroBot si confronta con la realtà fisica in tutta la sua complessità: attrito, tolleranze costruttive, dissipazione termica, caduta di tensione nelle batterie, ritardi nella comunicazione wireless, disallineamenti meccanici negli agganci magnetici. Questi fenomeni non sono trattati come fastidi da eliminare o da nascondere sotto strati di compensazione software, ma come elementi costitutivi del sistema da comprendere, modellare e integrare nel progetto. La geometria conica del MicroBot sfrutta la fisica del contatto per guidare passivamente l'allineamento durante l'aggancio. L'isteresi nelle soglie magnetiche sfrutta la fisica dell'interazione tra dipoli per evitare oscillazioni indesiderate. Il peso contenuto di 8–12 grammi sfrutta la scala dimensionale per rendere le forze magnetiche disponibili sufficienti a vincere l'attrito statico. In MicroBot la fisica non è un ostacolo da superare — è una risorsa progettuale da valorizzare.

La quinta componente, infine, è il principio di apertura come valore intrinseco. MicroBot è progettato per essere documentato, replicabile e comprensibile. Le specifiche sono pubbliche, le motivazioni delle scelte progettuali sono esplicitate, il codice è organizzato in moduli con responsabilità chiare e nomi autoesplicativi, i parametri dimensionali sono

raccolti in tabelle di riferimento utilizzabili direttamente in CAD e simulatori. Questa apertura non è solo un gesto di generosità verso la comunità — è una garanzia di qualità per il progetto stesso, perché un sistema documentato al punto da essere replicabile da terzi è necessariamente un sistema compreso fino in fondo dal suo autore. La documentazione, in questa prospettiva, non è il prodotto finale del processo di progettazione: è parte integrante del processo stesso, uno strumento attivo di chiarificazione concettuale che procede in parallelo con lo sviluppo tecnico e lo alimenta continuamente.

Punto 5 — Metodologia di sviluppo e approccio alla simulazione

Uno degli elementi che distinguono MicroBot dai progetti di robotica amatoriale o puramente dimostrativi è l'adozione sistematica di una metodologia di sviluppo strutturata, nella quale la simulazione precede sempre la costruzione fisica, la validazione precede sempre l'integrazione e la documentazione accompagna ogni fase del processo senza essere mai posticipata a una fase conclusiva che, nell'esperienza pratica della ricerca e dello sviluppo tecnologico, tende a non arrivare mai. Questa metodologia non è stata scelta per ragioni estetiche o per aderenza formale a standard accademici, ma per una ragione pratica molto concreta: in un sistema complesso come MicroBot, in cui componenti meccanici, elettronici, firmware e software di alto livello interagiscono attraverso interfacce fisiche e protocolli digitali, un errore non rilevato in una fase precoce si propaga inevitabilmente attraverso tutti i livelli successivi, moltiplicando i costi di correzione e rendendo sempre più difficile isolare la causa dei malfunzionamenti osservati. Simulare prima di costruire non è un lusso — è la forma più efficiente di gestione del rischio disponibile a chi progetta sistemi complessi con risorse limitate.

La metodologia adottata in MicroBot si articola in tre piani di simulazione distinti e complementari, ciascuno dei quali risponde a una domanda specifica e produce risultati che alimentano i piani successivi. Il primo piano è quello della simulazione elettrica, condotta con strumenti come LTspice o il simulatore Falstad, e ha come oggetto il comportamento dei circuiti che governano il pilotaggio delle coil elettromagnetiche. In questa fase vengono costruiti modelli circuitali che includono la sorgente di alimentazione — la batteria Li-Po con la sua resistenza interna e la sua curva di scarica — il MOSFET di potenza con le sue caratteristiche di soglia e di conduzione, l'induttanza della coil con la resistenza serie del filo di avvolgimento e il diodo di ricircolo per la protezione dagli spike di tensione indotti dalla commutazione. Su questi modelli vengono eseguite simulazioni di tipo transitorio, nelle quali si osserva come evolve la corrente nella bobina in risposta a un segnale PWM generato dall'ESP32, quali picchi di tensione compaiono ai capi del MOSFET durante la fase di spegnimento, quanta potenza viene dissipata sui vari elementi del circuito e quale sia la costante di tempo dell'avvolgimento, che determina la velocità con cui il campo magnetico si instaura e decade. I risultati di questa fase non restano confinati all'analisi circuitale: diventano i parametri di ingresso per il dimensionamento del safety layer nel firmware — i limiti di duty cycle, le soglie di temperatura, i tempi minimi di raffreddamento tra attivazioni successive.

Il secondo piano è quello della simulazione logica a livello di firmware, condotta con strumenti come Wokwi o Tinkercad Circuits, e ha come oggetto il comportamento del codice Arduino o ESP32 in risposta agli input digitali e ai segnali di controllo. In questo ambiente virtuale è possibile posizionare una scheda ESP32, collegare pin digitali a LED, MOSFET

simbolici o altri attuatori, e verificare in tempo reale che la logica di controllo si comporti come previsto: che i tempi di attivazione e disattivazione siano corretti, che la gestione del tempo non bloccante basata su `millis()` funzioni correttamente in presenza di più task concorrenti, che i messaggi UDP vengano inviati e ricevuti con la struttura attesa, che il watchdog si attivi correttamente in caso di freeze del firmware. Questa fase è particolarmente preziosa per identificare race condition, problemi di priorità tra task e comportamenti inattesi dovuti all'interazione tra la logica di controllo e il sistema operativo real-time sottostante, senza dover costruire l'hardware fisico né rischiare di danneggiare componenti durante le fasi di debug.

Il terzo piano è quello della simulazione logica astratta, condotta in JavaScript su canvas HTML o in C/C++ su PC, e ha come oggetto il comportamento collettivo dello swarm come sistema multi-agente. In questo ambiente ogni MicroBot è rappresentato come una struttura dati con un identificativo univoco, una posizione nello spazio bidimensionale, una velocità, uno stato dei magneti e un livello di carica della batteria. Il controller è implementato come un insieme di funzioni che aggiornano questi stati ad ogni tick temporale, applicando le regole di Boids, le soglie di aggancio e disaggancio magnetico, gli algoritmi di formazione dei pattern e i meccanismi di sincronizzazione temporale. Ad ogni iterazione il sistema visualizza le posizioni dei bot sullo schermo, rendendo immediatamente evidente se il pattern converge verso la configurazione target, se si formano oscillazioni indesiderate, se alcuni bot rimangono isolati o se la velocità di convergenza è compatibile con i vincoli temporali del sistema reale. Questa simulazione è il laboratorio virtuale in cui vengono sviluppati e raffinati tutti gli algoritmi che poi verranno implementati nel firmware del controller centrale.

I tre piani di simulazione non sono sequenziali in senso stretto ma si sviluppano in parallelo e si alimentano reciprocamente. La simulazione elettrica fornisce i parametri di pilotaggio delle coil che vengono usati nella simulazione firmware per calibrare i profili di corrente. La simulazione firmware valida la struttura del codice che poi viene riutilizzata nella simulazione astratta per implementare la logica di comunicazione. La simulazione astratta genera i pattern e gli algoritmi collettivi che vengono poi trasformati in comandi concreti da implementare nel firmware del controller. Questa circolarità non è un difetto metodologico ma una caratteristica deliberata: rispecchia la natura iterativa del processo di progettazione, in cui la comprensione di un livello modifica e affina la comprensione degli altri livelli in modo continuo.

Un aspetto metodologico di particolare rilevanza è la gestione sistematica del gap tra simulazione e realtà fisica. Ogni simulazione è basata su ipotesi semplificative — resistenza interna della batteria costante, coefficiente di attrito uniforme sulla superficie di appoggio, assenza di interferenze elettromagnetiche tra le coil di bot adiacenti, latenza di rete costante e prevedibile — che nella realtà fisica non sono mai perfettamente verificate. MicroBot affronta questo gap non ignorandolo né cercando di eliminarlo attraverso modelli sempre più dettagliati e computazionalmente costosi, ma introducendo esplicitamente nei modelli simulativi dei margini di sicurezza e dei parametri di tolleranza che tengono conto dell'incertezza della realtà. Le soglie magnetiche sono dimensionate con un margine del 20–30% rispetto ai valori minimi teorici. I limiti di corrente nel firmware sono impostati al 70–80% dei valori massimi ammissibili dai componenti. Le latenze di rete sono stimate nel caso peggiore e non nel caso medio. Questa filosofia del margine di sicurezza, ispirata alle

pratiche dell'ingegneria strutturale e dell'aeronautica, garantisce che il sistema fisico reale si comporti in modo robusto anche quando la realtà si discosta dalle ipotesi del modello, che è la condizione normale e non l'eccezione nel mondo dell'hardware embedded.

La metodologia di sviluppo di MicroBot si completa con una pratica di documentazione continua che accompagna ogni fase del processo. Ogni parametro dimensionale è raccolto in tabelle di riferimento con la propria unità di misura, il proprio valore nominale e i propri limiti di tolleranza. Ogni scelta progettuale è accompagnata da una motivazione esplicita che ne illustra le alternative considerate e i criteri di selezione adottati. Ogni algoritmo implementato nel firmware è documentato con la descrizione del comportamento atteso, le assunzioni su cui si basa e i casi limite che gestisce. Questo livello di documentazione non è una formalità accademica: è lo strumento che rende possibile riprendere il progetto dopo settimane di interruzione senza perdere il filo delle decisioni prese, integrare nel sistema componenti sviluppati in momenti diversi senza introdurre inconsistenze, e trasferire la conoscenza accumulata a collaboratori o alla comunità scientifica in modo efficace e rigoroso.

Punto 6 — Roadmap del progetto e visione evolutiva

La struttura evolutiva di MicroBot non è stata definita retrospettivamente per dare ordine a uno sviluppo caotico, ma è stata progettata a priori come parte integrante dell'architettura del sistema, con la stessa attenzione dedicata alla geometria del guscio o alla scelta del protocollo di comunicazione. Definire una roadmap rigorosa prima di scrivere la prima riga di codice o stampare il primo componente non è un esercizio di pianificazione burocratica: è il modo in cui un sistema complesso mantiene la coerenza attraverso le iterazioni, evitando la deriva progressiva verso soluzioni localmente ottimali ma globalmente inconsistenti che affligge molti progetti di ricerca e sviluppo nelle fasi di crescita accelerata. Ogni versione di MicroBot è quindi un traguardo autonomo e verificabile, non un punto intermedio su un percorso indefinito verso una destinazione vaga.

La versione 0.1 costituisce la fase fondativa del progetto, dedicata interamente alla definizione dell'architettura concettuale e alla formalizzazione delle scelte progettuali fondamentali. In questa fase non viene prodotto alcun artefatto fisico funzionante, ma viene costruita la base cognitiva su cui tutte le versioni successive si appoggiano: la geometria del MicroBot con le sue implicazioni meccaniche e magnetiche, la separazione in livelli dell'architettura software, la scelta dei componenti hardware principali, la definizione dei protocolli di comunicazione, la strutturazione della roadmap stessa. Il prodotto principale della versione 0.1 è la documentazione tecnica iniziale, che raccoglie in forma sistematica tutte queste scelte con le relative motivazioni. Questa documentazione non è un allegato del progetto — è il progetto stesso nella sua prima forma concreta, il punto di riferimento rispetto al quale tutte le decisioni future vengono valutate e giustificate.

La versione 0.2 rappresenta il primo passaggio dalla visione alla realizzazione operativa, attraverso la costruzione di una simulazione funzionale del comportamento collettivo dello swarm su piattaforma Arduino. In questa versione ogni MicroBot è rappresentato da un LED collegato a un pin digitale dedicato, e il controller centrale è implementato come un insieme di funzioni che aggiornano lo stato dei LED in risposta a comandi inviati via interfaccia seriale. Nonostante la semplicità apparente di questa implementazione, la versione 0.2

introduce concetti architetturali fondamentali che rimarranno invariati in tutte le versioni successive: la logica a stati per ogni singolo bot, la gestione del tempo non bloccante basata su `millis()`, la separazione tra strategia globale del controller e esecuzione locale di ciascun nodo, e i primi comportamenti collettivi dello swarm come la propagazione lineare, la rotazione, l'onda e il lampeggio sincronizzato. La versione 0.2 dimostra che il concetto di swarm cooperante è tecnicamente valido e pone le basi per l'integrazione hardware delle versioni successive.

La versione 0.3 è la versione corrente del progetto e rappresenta il primo vero prototipo fisico integrato. Gli obiettivi di questa versione sono ambiziosi e coprono l'intero stack del sistema: la stampa 3D del guscio con la geometria a doppio cono troncato e sfera centrale, la realizzazione del PCB circolare da 14–16 millimetri con l'ESP32 e i componenti di controllo, l'avvolgimento a mano delle coil elettromagnetiche e il test del sistema di aggancio magnetico, l'implementazione del firmware di comunicazione UDP su rete SoftAP, la costruzione del controller centrale con ESP32-S3 e IMU a nove assi, la realizzazione del wristband con ESP32-C3 e sensore BNO055 e la chiusura del ciclo completo tra input gestuale e risposta fisica dello swarm. Al termine della versione 0.3 sarà possibile costruire un MicroBot fisico funzionante, farlo comunicare con il controller, attivare i magneti in risposta a comandi remoti, rilevare gesture dal wristband e visualizzare le coordinate dei bot su un'interfaccia grafica di base. Questa versione costituisce il punto di non ritorno del progetto: il momento in cui la teoria incontra la realtà fisica e vengono raccolti i primi dati sperimentali sul comportamento reale del sistema.

La versione 0.4 introduce autonomia locale e intelligenza distribuita a livello di singolo MicroBot. In questa versione ogni bot non si limita a eseguire passivamente i comandi del controller ma acquisisce una capacità di elaborazione locale limitata: può rilevare la propria posizione relativa rispetto ai bot vicini tramite sensori di prossimità, può prendere micro-decisioni locali compatibili con il pattern globale assegnato dal controller, può gestire autonomamente situazioni di emergenza come la caduta critica della batteria o il surriscaldamento delle coil senza attendere una risposta dal controller centrale. Questa versione introduce anche i primi algoritmi di mesh networking tra bot, che consentono la comunicazione diretta tra moduli adiacenti attraverso ESP-NOW senza passare obbligatoriamente per il controller, aumentando la robustezza del sistema in presenza di perdite di connettività. Sul piano dell'interfaccia utente, la versione 0.4 completa l'integrazione del framework 4D/5D, rendendo operativa la modulazione dello stato interno dello swarm attraverso parametri acustici.

La versione 0.5 rappresenta la versione matura del sistema e introduce l'integrazione completa con il visore VR prototipale e le interfacce immersive. In questa versione l'utente interagisce con lo swarm esclusivamente attraverso l'interfaccia tridimensionale del visore, che offre una rappresentazione in tempo reale della posizione e dello stato di ogni bot nello spazio, la possibilità di selezionare, spostare e riconfigurare i pattern con gesture naturali tracciate dalla camera integrata nel visore, e un feedback visivo e sonoro che chiude il ciclo percettivo tra l'intenzione dell'utente e la risposta fisica dello swarm. La versione 0.5 introduce anche i primi algoritmi di intelligenza artificiale locale distribuita, nei quali i bot apprendono progressivamente le preferenze dell'utente e anticipano le transizioni di pattern più frequenti, riducendo la latenza percepita tra il comando e l'esecuzione.

Oltre la versione 0.5 si apre lo spazio della visione a lungo termine, che include due direzioni di sviluppo principali. La prima è la scalabilità dello swarm verso centinaia di unità, che richiede una revisione dei protocolli di comunicazione per gestire reti di questa dimensione senza degradazione delle prestazioni, e l'introduzione di algoritmi di coordinamento gerarchico in cui gruppi di bot eleggono dinamicamente nodi leader responsabili della coordinazione locale. La seconda direzione è l'evoluzione verso la MicroHand, il sistema di manipolazione attiva degli oggetti fisici nell'ambiente attraverso moduli specializzati con dita articolate. La MicroHand non è una versione numerata della roadmap principale ma una biforcazione evolutiva del progetto: un sistema che condivide con MicroBot l'architettura di base, i protocolli di comunicazione e la filosofia progettuale, ma che aggiunge la capacità di esercitare forze controllate sull'ambiente attraverso strutture articolate ispirate alla biomeccanica della mano umana, trasformando lo swarm da sistema di aggregazione passiva a sistema di manipolazione attiva e aprendo applicazioni che spaziano dalla logistica automatizzata all'assistenza in ambienti non strutturati.

Punto 7 — Architettura generale del sistema

L'architettura di MicroBot è il risultato di un processo di progettazione che ha privilegiato la chiarezza strutturale rispetto alla compattezza implementativa, la separazione delle responsabilità rispetto all'ottimizzazione locale e la robustezza sistemica rispetto alle prestazioni di picco del singolo componente. Descrivere l'architettura di MicroBot significa quindi descrivere non solo come i componenti sono collegati tra loro, ma perché sono stati organizzati in quel modo specifico, quali alternative sono state considerate e scartate, e quali proprietà emergono dall'organizzazione scelta che non sarebbero presenti in architetture alternative. Questa prospettiva è essenziale per comprendere il sistema non come una collezione di componenti giustapposti ma come un organismo coerente in cui ogni scelta strutturale ne implica e ne giustifica altre.

Il sistema MicroBot è organizzato in cinque livelli gerarchici distinti, disposti secondo una struttura che va dall'input umano all'attuazione fisica passando attraverso strati progressivi di elaborazione, traduzione e coordinamento. Questa organizzazione a livelli non è semplicemente una metafora descrittiva ma una struttura implementativa concreta: ogni livello comunica esclusivamente con i livelli adiacenti attraverso interfacce ben definite, non ha accesso diretto allo stato interno dei livelli non adiacenti e può essere modificato o sostituito senza impattare i livelli con cui non è in contatto diretto. Questa proprietà, nota in ingegneria del software come *information hiding*, è ciò che rende il sistema manutenibile nel tempo e integrabile con componenti nuovi senza richiedere una riscrittura globale.

Il primo livello è quello degli input utente, e comprende tutti i meccanismi attraverso i quali l'intenzione umana viene catturata e trasformata in segnali digitali elaborabili dai livelli successivi. Questo livello include il riconoscimento delle gesture della mano tramite MediaPipe Hands, che identifica in tempo reale la posizione di ventuno landmark per mano e li interpreta come comandi — pinch, grip, tap, swipe, rotazione — con una frequenza di aggiornamento compatibile con i tempi di risposta percepiti come naturali dall'utente umano. Include il riconoscimento delle espressioni facciali tramite MediaPipe Face Landmarker, che traccia 468 punti del volto e mappa configurazioni muscolari specifiche su stati del sistema o comandi discreti. Include il wristband con IMU a nove assi, che fornisce informazioni sull'orientamento della mano nello spazio tridimensionale — pitch, yaw e roll — e riconosce

gesture dinamiche basate sull'accelerazione come spinte, scosse e movimenti rapidi. Include infine il framework 4D/5D, che cattura parametri acustici dall'ambiente e li mappa su modificazioni dello stato interno $W(t)$ dello swarm. La molteplicità dei canali di input non è ridondanza ma ricchezza espressiva: canali diversi sono adatti a tipi diversi di comandi, e la loro combinazione permette all'utente di comunicare con il sistema in modo naturale e contestualmente appropriato.

Il secondo livello è quello dell'interfaccia software, e comprende gli ambienti in cui i segnali di input vengono visualizzati, elaborati e trasformati in comandi logici destinati al motore di simulazione. Questo livello include la simulazione JavaScript su canvas HTML, che fornisce una rappresentazione visiva in tempo reale del comportamento dello swarm e permette il test degli algoritmi di coordinamento senza richiedere hardware fisico. Include il visore VR sviluppato in Unity con C#, che offre una rappresentazione tridimensionale immersiva dello swarm e gestisce il tracking della testa a sei gradi di libertà, il riconoscimento delle gesture in ambiente VR e il rendering in tempo reale dei MicroBot come oggetti dinamici nello spazio virtuale. Include il sistema di autenticazione biometrica facciale, che costituisce il cancello di sicurezza attraverso il quale l'utente deve passare prima di ottenere accesso al controller dello swarm. La separazione di questo livello dal livello di input garantisce che la stessa logica di controllo dello swarm possa essere guidata da interfacce diverse — gesture, VR, comandi testuali, interfacce web — senza richiedere modifiche al motore di simulazione o al firmware del controller.

Il terzo livello è quello del motore di simulazione, e costituisce il nucleo computazionale del sistema. È qui che le regole di Boids vengono applicate per calcolare le forze di coesione, allineamento e separazione tra i bot, che i pattern geometrici vengono definiti come insiemi ordinati di posizioni target nello spazio bidimensionale, che il framework 4D/5D trasforma i parametri acustici in modificazioni della dinamica collettiva e che gli algoritmi di assegnazione dei bot alle posizioni target vengono eseguiti per minimizzare la distanza totale percorsa durante le transizioni di pattern. Questo livello opera principalmente in ambiente software — JavaScript, Python o C++ su PC — e produce come output un insieme di comandi logici che descrivono lo stato desiderato del sistema fisico: quali bot devono attivare i magneti, quali LED devono accendersi di quale colore, quali posizioni target devono essere raggiunte entro quale finestra temporale.

Il quarto livello è quello del controller centrale, implementato su ESP32-S3 in un involucro fisico di 70×70×30 millimetri. Il controller riceve i comandi logici dal livello superiore tramite connessione Wi-Fi, li interpreta e li trasforma in pacchetti UDP destinati ai singoli MicroBot. Svolge la funzione di clock master del sistema, inviando con ogni comando un timestamp globale che permette a tutti i bot di sincronizzare le proprie azioni senza accumulare deriva temporale. Gestisce il safety layer globale, monitorando lo stato di batteria, temperatura e connettività di ogni bot e intervenendo con comandi di emergenza quando vengono rilevate condizioni anomale. Mantiene una tabella di stato aggiornata dell'intero swarm, che include la posizione stimata, lo stato dei magneti, il livello di carica della batteria e la temperatura interna di ogni bot connesso alla rete. Il controller è il solo componente del sistema che ha una visione globale dello stato dello swarm — tutti gli altri livelli lavorano con informazioni parziali o aggregate.

Il quinto e ultimo livello è quello dei MicroBot fisici, le unità di attuazione del sistema. Ogni MicroBot riceve i pacchetti UDP dal controller, verifica che il proprio identificativo corrisponda al destinatario del comando o che il comando sia destinato a tutti i bot, e lo esegue attivando o disattivando i propri elettromagneti, modificando il colore del proprio LED RGB o aggiornando i parametri del proprio algoritmo locale. A intervalli regolari, ogni bot invia al controller un pacchetto di stato che riporta il livello di carica della batteria, la temperatura interna, lo stato corrente dei magneti e un contatore di messaggi ricevuti che permette al controller di stimare la qualità della connessione wireless. I MicroBot sono progettati per funzionare in modalità degradata anche in caso di perdita temporanea della connessione con il controller, mantenendo l'ultimo stato ricevuto fino al ripristino della comunicazione o all'intervento del watchdog.

La comunicazione tra i livelli segue due topologie distinte a seconda della direzione del flusso di informazione. Il flusso discendente — dall'input utente ai MicroBot fisici — segue una topologia a cascata in cui ogni livello riceve informazioni elaborate dal livello superiore, le trasforma aggiungendo il proprio contributo computazionale e le passa al livello inferiore. Il flusso ascendente — dai MicroBot fisici all'interfaccia utente — segue una topologia a consolidamento in cui le informazioni di stato dei singoli bot vengono aggregate dal controller in una rappresentazione globale dello swarm, che viene poi visualizzata nell'interfaccia software e resa accessibile all'utente attraverso feedback visivi, sonori e aptici. Questa asimmetria tra flusso discendente e flusso ascendente riflette la diversa natura delle informazioni che scorrono nelle due direzioni: i comandi sono strutturati, sintetici e prioritari; i dati di stato sono distribuiti, ridondanti e tolleranti alla perdita.

L'architettura a cinque livelli di MicroBot presenta proprietà sistemiche che non appartengono a nessuno dei livelli considerati singolarmente. La prima è la scalabilità: aggiungere nuovi MicroBot al sistema non richiede modifiche ai livelli superiori, perché il controller gestisce dinamicamente la tabella dei bot connessi e il motore di simulazione opera su un numero arbitrario di agenti. La seconda è la sostituibilità: l'interfaccia VR può essere sostituita con un'interfaccia web, il controller ESP32-S3 può essere sostituito con un Raspberry Pi Zero 2W, il protocollo UDP può essere sostituito con ESP-NOW, senza che queste modifiche si propaghino agli altri livelli. La terza è la testabilità: ogni livello può essere testato in isolamento attraverso la simulazione del livello adiacente, riducendo drasticamente la complessità del debug nei sistemi integrati. La quarta è la comprensibilità: un nuovo collaboratore può acquisire una comprensione approfondita di un singolo livello senza dover padroneggiare l'intero sistema, abbassando significativamente la barriera di ingresso al progetto.

Punto 8 — Protocollo di comunicazione e sincronizzazione temporale

La comunicazione in un sistema swarm distribuito è una sfida ingegneristica che va ben oltre la semplice trasmissione di dati tra nodi. In MicroBot la comunicazione non è un canale neutro attraverso cui scorrono informazioni — è una componente attiva del sistema che determina la qualità della sincronizzazione, la robustezza del comportamento collettivo, il consumo energetico di ogni bot e i limiti pratici sulla dimensione massima dello swarm gestibile in tempo reale. Progettare il protocollo di comunicazione significa quindi prendere decisioni che hanno conseguenze dirette su tutti gli altri aspetti del sistema, e che non

possono essere delegate a una fase successiva di ottimizzazione senza rischiare di costruire un'architettura incompatibile con i requisiti operativi reali.

La scelta fondamentale alla base del protocollo di comunicazione di MicroBot è l'adozione di una topologia a stella con il controller centrale come nodo hub, nella quale tutti i MicroBot si connettono al controller come stazioni client e non comunicano direttamente tra loro se non attraverso meccanismi di tipo ESP-NOW nelle versioni più avanzate del sistema. Questa scelta è stata preferita a topologie mesh più distribuite per ragioni concrete legate alla fase di sviluppo corrente: una topologia a stella è più semplice da implementare, più facile da debuggare, più prevedibile nelle prestazioni e più controllabile dal punto di vista della sicurezza. La centralizzazione della comunicazione nel controller non contraddice la filosofia distribuita del sistema — l'intelligenza collettiva emerge comunque dalle interazioni locali tra i bot — ma delimita chiaramente il perimetro di complessità della versione 0.3, lasciando la migrazione verso topologie mesh a versioni successive quando la base tecnologica sarà sufficientemente consolidata.

Il meccanismo di rete adottato è quello del SoftAP, ovvero la modalità Access Point software dell'ESP32-S3 del controller. In questa configurazione il controller crea una rete Wi-Fi dedicata con un SSID specifico del progetto, e tutti i MicroBot si connettono a questa rete come stazioni nella fase di inizializzazione. Questa scelta elimina la dipendenza da infrastrutture di rete esterne — router domestici, reti aziendali, accessi Internet — rendendo il sistema completamente autonomo e operabile in qualsiasi ambiente, inclusi spazi espositivi, laboratori senza infrastruttura di rete o ambienti outdoor. Il controller assegna a ogni bot un indirizzo IP fisso basato sul suo identificativo univoco, garantendo che la mappatura tra identificativo logico del bot e indirizzo di rete sia stabile e prevedibile durante tutta la sessione operativa.

Il protocollo di trasporto scelto è UDP, il User Datagram Protocol, preferito al TCP per ragioni legate alla natura specifica del traffico dati in un sistema di swarm robotics in tempo reale. TCP offre garanzie di consegna ordinata e affidabile dei pacchetti attraverso meccanismi di acknowledgment e ritrasmissione, ma questi meccanismi introducono latenze variabili e imprevedibili che sono incompatibili con i requisiti di sincronizzazione temporale di MicroBot. In un sistema swarm, un comando che arriva con 50 millisecondi di ritardo a causa di una ritrasmissione TCP è spesso meno utile di nessun comando, perché il bot ha già preso una micro-decisione locale basata sull'ultimo stato noto. UDP è connectionless, non offre garanzie di consegna ma garantisce latenze minime e prevedibili, e si adatta perfettamente a un sistema in cui i comandi sono frequenti, di piccole dimensioni e tolleranti alla perdita occasionale di singoli pacchetti.

La struttura di ogni pacchetto UDP inviato dal controller ai bot è definita in modo rigoroso e comprende un insieme minimo di campi che bilanciano la completezza informativa con la necessità di mantenere i pacchetti di dimensioni ridotte per minimizzare la latenza di trasmissione. Il campo di sequenza è un intero a 16 bit che identifica univocamente ogni pacchetto e permette ai bot di rilevare pacchetti persi o arrivati fuori ordine. Il campo timestamp è un intero a 32 bit che riporta il tempo in millisecondi dal momento di accensione del controller e costituisce il fondamento del meccanismo di sincronizzazione temporale. Il campo identificativo destinatario è un intero a 8 bit che specifica il bot a cui il comando è destinato, con il valore 255 riservato ai comandi broadcast indirizzati a tutti i bot.

simultaneamente. Il campo tipo comando è un intero a 8 bit che identifica la categoria dell'azione richiesta — attivazione magneti, modifica LED, aggiornamento posizione target, sincronizzazione clock, comando di emergenza. Il campo parametri è un blocco di dimensione variabile che contiene i dati specifici associati al tipo di comando — intensità del campo magnetico espressa come duty cycle PWM, colore RGB del LED, coordinate della posizione target, identificativo del pattern da eseguire. Il campo checksum è un intero a 16 bit calcolato sul contenuto del pacchetto che permette al bot ricevente di verificare l'integrità dei dati e scartare i pacchetti corrotti prima di eseguire i comandi in essi contenuti.

La sincronizzazione temporale è uno degli aspetti più critici dell'intero sistema e merita una trattazione separata e approfondita. In MicroBot la sincronizzazione non serve semplicemente a far sapere ai bot che ore sono — serve a garantire che tutti i bot che devono attivare i propri magneti in risposta a un comando di formazione lo facciano entro una finestra temporale sufficientemente stretta da produrre un comportamento collettivo coerente. Se alcuni bot attivano i magneti con 100 millisecondi di anticipo rispetto ad altri, il pattern si forma in modo disordinato, con bot che si agganciano prematuramente ad altri ancora in movimento, generando forze non previste che possono destabilizzare l'intera configurazione. La finestra di sincronizzazione target per MicroBot è dell'ordine di 10–20 millisecondi, che corrisponde alla differenza di latenza tipica tra i bot più vicini e quelli più lontani dal controller in una rete SoftAP con 10–20 nodi.

Il meccanismo di sincronizzazione adottato è ispirato al Precision Time Protocol e funziona nel modo seguente. Il controller, che funge da clock master del sistema, include in ogni pacchetto UDP il proprio timestamp locale al momento dell'invio. Ogni bot, al momento della ricezione del pacchetto, misura il proprio timestamp locale e calcola la differenza rispetto al timestamp del controller, ottenendo una stima dell'offset del proprio clock rispetto al clock master. Questa stima viene filtrata con un filtro a media mobile per eliminare le fluttuazioni dovute alla variabilità della latenza di rete, e utilizzata per correggere progressivamente il clock locale del bot. Con questo meccanismo, dopo pochi cicli di sincronizzazione tutti i bot del sistema condividono una base temporale comune con una precisione dell'ordine di pochi millisecondi, sufficiente a garantire la finestra di sincronizzazione target. Il controller invia periodicamente pacchetti di sincronizzazione puri — pacchetti senza payload di comando il cui unico scopo è aggiornare il clock dei bot — a una frequenza di circa dieci hertz, garantendo che la deriva temporale accumulata tra due sincronizzazioni successive resti al di sotto della soglia di 2–3 millisecondi.

Il flusso di comunicazione ascendente — dai bot al controller — è strutturato come un meccanismo di heartbeat periodico nel quale ogni bot invia al controller, a intervalli regolari di circa 500 millisecondi, un pacchetto di stato che riporta il livello di tensione della batteria misurato dall'ADC interno dell'ESP32, la temperatura stimata delle coil calcolata integrando il profilo di corrente nel tempo, lo stato corrente dei magneti espresso come duty cycle PWM attivo, il numero di pacchetti di comando ricevuti dall'ultimo heartbeat come indicatore della qualità della connessione wireless, e un flag di stato che segnala eventuali condizioni anomale rilevate dal safety layer locale. Il controller aggrega questi dati nella propria tabella di stato globale e li rende disponibili al livello di interfaccia software per la visualizzazione in tempo reale. Se un bot non invia un heartbeat entro un timeout configurabile — tipicamente 1500 millisecondi, corrispondenti a tre intervalli mancati — il controller lo marca come

disconnesso, rimuove la sua posizione target dal pattern corrente e notifica l'interfaccia utente attraverso un messaggio di avviso.

La gestione delle collisioni di pacchetti e della congestione della rete è un aspetto che diventa rilevante quando il numero di bot supera i 10–15 nodi, soglia oltre la quale il traffico di heartbeat aggregato inizia a competere con il traffico di comando per la banda disponibile della rete SoftAP. MicroBot affronta questo problema attraverso due strategie complementari. La prima è il temporal spreading degli heartbeat: invece di ricevere tutti gli heartbeat contemporaneamente in risposta a un segnale di sincronizzazione globale, il controller assegna a ogni bot uno slot temporale dedicato all'interno di una finestra di 500 millisecondi, distribuendo il traffico ascendente in modo uniforme nel tempo. La seconda è la compressione dei pacchetti di stato attraverso la codifica delta: invece di inviare i valori assoluti di tutti i parametri ad ogni heartbeat, il bot invia solo i parametri che sono cambiati rispetto all'heartbeat precedente, riducendo significativamente la dimensione media dei pacchetti in condizioni operative stabili.

La robustezza del sistema di comunicazione in presenza di perturbazioni — interferenze elettromagnetiche, attenuazione del segnale dovuta alla posizione relativa dei bot, picchi di traffico generati da eventi di ricalibrazione simultanea — è garantita da un insieme di meccanismi difensivi implementati sia nel firmware dei bot sia nel software del controller. I bot implementano un meccanismo di jitter randomizzato sulle ritrasmissioni, che evita la sincronizzazione involontaria dei tentativi di accesso al canale in seguito a una perdita di pacchetto generalizzata. Il controller implementa un meccanismo di rate limiting sui comandi broadcast, che impone un intervallo minimo di 20 millisecondi tra comandi successivi destinati a tutti i bot, evitando la saturazione del canale durante le transizioni di pattern. Entrambi i livelli implementano validazione dell'integrità dei pacchetti ricevuti tramite checksum, con scarto silenzioso dei pacchetti corrotti senza generazione di messaggi di errore che potrebbero a loro volta contribuire alla congestione della rete.

Punto 9 — Modello matematico del sistema swarm

La descrizione matematica di un sistema swarm non è un esercizio accademico fine a sé stesso, ma lo strumento attraverso cui le intuizioni qualitative sul comportamento collettivo vengono tradotte in algoritmi implementabili, i parametri di controllo vengono dimensionati quantitativamente e le proprietà del sistema — stabilità, convergenza, robustezza — vengono verificate prima ancora di costruire il primo prototipo fisico. In MicroBot il modello matematico svolge un ruolo operativo diretto: le equazioni che descrivono la dinamica dei bot, le forze di interazione magnetica e i criteri di formazione dei pattern sono le stesse equazioni che vengono implementate nel motore di simulazione JavaScript e nel firmware del controller centrale, con le semplificazioni e le approssimazioni necessarie a renderle computabili in tempo reale su un microcontrollore con risorse limitate. Comprendere il modello matematico significa quindi comprendere il comportamento del sistema a un livello che nessuna descrizione qualitativa può raggiungere.

Il punto di partenza del modello è la rappresentazione dello stato di ogni MicroBot come un vettore che cattura le informazioni necessarie a descriverne completamente la condizione in un dato istante. Nella versione bidimensionale del modello, adottata per la simulazione nella versione 0.2 e 0.3, lo stato del bot i -esimo è descritto dalla coppia di coordinate di posizione

nel piano, indicata con il vettore $r_i(t) = (x_i(t), y_i(t))$, e dalla coppia di componenti della velocità, indicata con il vettore $v_i(t) = (v_{x_i}(t), v_{y_i}(t))$. La relazione cinematica fondamentale che collega posizione e velocità è la derivata temporale della posizione rispetto al tempo, espressa dalla relazione $v_i(t) = dr_i(t)/dt$, che nella discretizzazione numerica adottata nel firmware diventa l'equazione di aggiornamento $r_i(t + \Delta t) = r_i(t) + v_i(t) \cdot \Delta t$, dove Δt è il passo temporale della simulazione, tipicamente compreso tra 20 e 100 millisecondi in funzione della frequenza di aggiornamento del controller.

La dinamica di ogni bot è governata dalla seconda legge di Newton applicata al moto traslazionale nel piano, che nella forma vettoriale si scrive come $m_i \cdot dv_i/dt = F_{i,tot}$, dove m_i è la massa del bot i-esimo e $F_{i,tot}$ è la somma vettoriale di tutte le forze che agiscono su di esso. Nella discretizzazione numerica questa relazione diventa l'equazione di aggiornamento della velocità $v_i(t + \Delta t) = v_i(t) + (F_{i,tot} / m_i) \cdot \Delta t$, che insieme all'equazione di aggiornamento della posizione costituisce il nucleo dell'integratore numerico del sistema. La forza totale $F_{i,tot}$ che agisce sul bot i-esimo è la somma di quattro contributi distinti, ciascuno dei quali cattura un aspetto specifico del comportamento collettivo dello swarm e corrisponde a un termine separato nel codice del motore di simulazione.

Il primo contributo è la forza di inseguimento del target, che rappresenta l'attrazione del bot verso la posizione target assegnatagli dal controller nel pattern corrente. Questa forza è modellata come una forza elastica proporzionale alla distanza tra la posizione corrente del bot e la posizione target, espressa dalla relazione $F_{i,target} = k_{\square} \cdot (r_{i,target} - r_i(t))$, dove $k_{\square}$ è la costante di rigidità del campo di attrazione verso il target e $r_{i,target}$ è il vettore posizione del target assegnato al bot i-esimo. Questa modellazione produce un comportamento di avvicinamento progressivo al target con velocità proporzionale alla distanza, che garantisce movimenti fluidi e naturali piuttosto che salti discontinui verso la posizione finale. Il parametro $k_{\square}$ controlla la velocità di convergenza del sistema verso il pattern target: valori elevati producono convergenza rapida ma possono generare oscillazioni intorno al target se non opportunamente smorzate, mentre valori bassi producono convergenza lenta ma stabile.

Il secondo contributo è la forza di attrazione magnetica tra bot vicini, che modella l'interazione tra i magneti permanenti di bot che si trovano entro la soglia di aggancio. Questa forza è attiva solo quando la distanza tra due bot scende al di sotto della soglia di aggancio d_{agg} e il sistema ha deciso di attivare l'interazione magnetica tra quella coppia di bot. Nella sua forma semplificata, valida per la simulazione bidimensionale, la forza di attrazione magnetica del bot j sul bot i è espressa dalla relazione $F_{i\square,attr} = k_{\square} \cdot (r_{\square}(t) - r_i(t)) / |r_{\square}(t) - r_i(t)|^2$, dove $k_{\square}$ è una costante che dipende dai parametri fisici dei magneti — momento di dipolo, permeabilità del mezzo, distanza effettiva tra i poli. Il denominatore quadratico nella distanza riflette la dipendenza dalla distanza al cubo della forza tra dipoli magnetici nella direzione radiale, approssimata come distanza al quadrato nel modello bidimensionale per semplificare il calcolo real-time.

Il terzo contributo è la forza di repulsione, che impedisce la sovrapposizione fisica tra bot e produce il comportamento di separazione tipico dei sistemi Boids. Questa forza è repulsiva a corto raggio e decade rapidamente con la distanza, ed è espressa nella forma $F_{i\square,rep} = -k_r \cdot (r_{\square}(t) - r_i(t)) / |r_{\square}(t) - r_i(t)|^3$, dove k_r è la costante di repulsione. La somma delle forze di repulsione di tutti i bot vicini produce il comportamento di separazione collettiva che evita le

collisioni fisiche e mantiene una distanza minima tra i moduli durante le transizioni di pattern. Il raggio entro cui la forza di repulsione è considerata attiva — il raggio di separazione — è un parametro critico del sistema: valori troppo piccoli permettono collisioni fisiche, valori troppo grandi impediscono l'aggregazione magnetica producendo uno swarm troppo disperso.

Il quarto contributo è la forza di allineamento, che tende a uniformare le direzioni di moto dei bot vicini producendo il comportamento di volo coordinato tipico degli stormi biologici. Questa forza non è una forza nel senso meccanico del termine — non deriva da un'interazione fisica tra bot — ma è un termine di controllo che il controller introduce nella dinamica di ogni bot per produrre un comportamento collettivo desiderato. Nella formulazione adottata in MicroBot, la forza di allineamento del vicinato N_i sul bot i è espressa come $F_{i,align} = k_a \cdot (\bar{v}_i - v_i(t))$, dove $\bar{v}_i$ è la velocità media di tutti i bot nel vicinato di i e k_a è la costante di allineamento. Questo termine produce una convergenza progressiva della velocità di ogni bot verso la velocità media del proprio vicinato, creando il comportamento di moto coordinato caratteristico degli swarm biologici.

La gestione dell'aggancio e del disaggancio magnetico introduce nel modello una non-linearità importante che non può essere catturata dai soli termini di forza continui descritti in precedenza. L'aggancio tra due bot non è un fenomeno graduale ma una transizione di stato discreta che avviene quando la distanza tra i due bot scende al di sotto della soglia di aggancio d_{agg} e che, una volta avvenuta, modifica qualitativamente la dinamica del sistema perché i due bot diventano un sistema rigido con nuova massa totale, nuovo centro di massa e nuovo momento di inerzia. Il disaggancio è analogamente una transizione di stato discreta che avviene quando la forza di separazione esercitata sulla coppia agganciata supera la forza di ritenuta magnetica $F_{\square_{ax}}$. Per evitare oscillazioni rapide tra stato agganciato e stato sganciato nelle vicinanze della soglia — il fenomeno del chattering ben noto nei sistemi di controllo a commutazione — MicroBot implementa un meccanismo di isteresi a due soglie: l'aggancio si attiva quando la distanza scende al di sotto di d_{agg} , ma il disaggancio si attiva solo quando la distanza supera una soglia di distacco $d_{dist} > d_{agg}$. La zona compresa tra d_{agg} e d_{dist} è la zona di isteresi, nella quale lo stato del sistema — agganciato o sganciato — dipende dalla storia passata e non solo dalla posizione corrente.

I pattern geometrici implementati in MicroBot sono definiti matematicamente come insiemi ordinati di posizioni target nel piano, parametrizzati da un numero ridotto di variabili di controllo. Il pattern lineare di N bot con passo d è definito dall'insieme di posizioni target $r_{i,target} = r_0 + i \cdot (d, 0)$ per $i = 0, 1, \dots, N-1$, dove r_0 è la posizione del primo bot. Il pattern circolare di N bot su un cerchio di raggio R centrato in r_0 è definito dalle posizioni $r_{i,target} = r_0 + R \cdot (\cos(2\pi i/N), \sin(2\pi i/N))$. Il pattern a onda sinusoidale è un pattern lineare modulato in ampiezza, con posizioni $r_{i,target} = (i \cdot d, A \cdot \sin(2\pi i/\lambda))$, dove A è l'ampiezza e λ è la lunghezza d'onda del pattern. Il pattern a spirale di Archimede è definito dalle posizioni $r_{i,target} = (a \cdot \theta_i \cdot \cos(\theta_i), a \cdot \theta_i \cdot \sin(\theta_i))$ con $\theta_i = 2\pi i/N \cdot n$, dove a è il parametro di passo della spirale e n è il numero di giri. Il pattern a vortice combina il pattern circolare con un termine di velocità angolare imposto che produce una rotazione coordinata dell'intero swarm attorno al centro del cerchio.

La stabilità del sistema swarm — intesa come la proprietà di convergere verso il pattern target e di mantenersi in quella configurazione in presenza di piccole perturbazioni — può essere analizzata formalmente attraverso la teoria di Lyapunov applicata al sistema dinamico complessivo. La funzione di Lyapunov naturale per questo sistema è la somma delle distanze al quadrato tra le posizioni correnti dei bot e le rispettive posizioni target, espressa come $V(t) = \sum_i |r_i(t) - r_{i,target}|^2$. Questa funzione è definita positiva — è zero solo quando tutti i bot si trovano esattamente nelle posizioni target — e la sua derivata temporale può essere espressa in funzione delle forze che agiscono sui bot. Affinché il sistema sia asintoticamente stabile verso il pattern target, è sufficiente che la derivata temporale di V sia negativa definita in un intorno della configurazione target, condizione che viene verificata analiticamente per il sistema di forze descritto in precedenza quando i parametri k_θ , k_ϕ , k_r e k_a sono scelti in modo appropriato e le forze di attrito e le perturbazioni esterne restano al di sotto di soglie determinate dall'analisi di robustezza del sistema.

Punto 10 — Integrazione dei componenti e flusso operativo del sistema

Descrivere i singoli componenti di MicroBot in isolamento — il protocollo di comunicazione, il modello matematico dello swarm, l'architettura a livelli — è necessario ma non sufficiente per comprendere come il sistema funziona nella pratica. La comprensione completa richiede di seguire il flusso operativo end-to-end: dal momento in cui l'utente esprime un'intenzione attraverso un gesto della mano o un movimento del polso fino al momento in cui i MicroBot fisici modificano la propria configurazione nello spazio reale in risposta a quell'intenzione. Questo percorso attraversa tutti i cinque livelli dell'architettura, coinvolge componenti hardware e software eterogenei, e si svolge in un intervallo di tempo dell'ordine di poche decine di millisecondi — abbastanza rapido da essere percepito come immediato dall'utente, abbastanza lungo da richiedere una progettazione attenta di ogni fase della catena per evitare l'accumulo di latenze che degraderebbero l'esperienza di controllo.

Il flusso operativo inizia nel momento in cui il sistema viene acceso e tutti i componenti completano la fase di inizializzazione. Il controller ESP32-S3 crea la rete SoftAP dedicata, assegna gli indirizzi IP ai bot che si connettono progressivamente, inizializza la tabella di stato globale dello swarm e comincia a inviare i pacchetti di sincronizzazione temporale a frequenza di dieci hertz. Ogni MicroBot, dopo aver completato il proprio boot e aver stabilito la connessione Wi-Fi con il controller, invia il proprio primo pacchetto di heartbeat con i parametri di stato iniziali — batteria carica, magneti inattivi, temperatura ambiente — e comincia ad attendere i comandi del controller in ascolto sulla propria porta UDP. Il wristband completa la calibrazione dell'IMU attraverso una sequenza di movimenti guidati, impara la posizione neutra della mano dell'utente come riferimento per il calcolo degli angoli di pitch, yaw e roll, e si mette in trasmissione continua dei dati di orientamento verso il controller. Il sistema di autenticazione biometrica rimane attivo in primo piano fino a quando non riceve una conferma di identità valida, impedendo l'accesso al controller e bloccando l'invio di comandi allo swarm fino al completamento dell'autenticazione.

Una volta completata l'inizializzazione e superata l'autenticazione, il sistema entra nella fase operativa normale. L'utente può ora interagire con lo swarm attraverso uno qualsiasi dei canali di input disponibili. Consideriamo il caso più rappresentativo: l'utente vuole formare il pattern a cerchio con tutti i bot disponibili. Esegue il gesto di pinch con la mano destra tenendo le dita disposte in arco — un gesto che MediaPipe Hands riconosce come comando

di selezione del pattern circolare — e poi ruota il polso verso l'alto per aumentare il raggio del cerchio. MediaPipe estrae i ventuno landmark della mano dall'immagine della telecamera, calcola gli angoli delle articolazioni, classifica la configurazione come gesto di selezione con parametro di intensità proporzionale al grado di apertura delle dita, e genera un evento di comando che viene passato al motore di simulazione nel livello software. Il wristband rileva simultaneamente la rotazione del polso come variazione dell'angolo di pitch, e invia al controller un pacchetto BLE con il valore aggiornato dell'orientamento.

Il motore di simulazione riceve l'evento di comando dal livello di input, lo interpreta come richiesta di transizione al pattern circolare con raggio proporzionale al parametro estratto dal gesto, e calcola le posizioni target per tutti i bot attualmente connessi al sistema. Per un insieme di N bot su un cerchio di raggio R centrato nell'origine del piano di simulazione, le posizioni target sono distribuite uniformemente sulla circonferenza secondo la formula parametrica già descritta nel modello matematico. Il motore esegue quindi l'algoritmo di assegnazione, che mappa ogni bot reale alla posizione target che minimizza la distanza complessiva da percorrere — un problema di assegnazione ottimale risolvibile in tempo polinomiale con l'algoritmo ungherese per swarm di dimensioni moderate, o con euristiche greedy per swarm di grandi dimensioni. Il risultato dell'assegnazione è una tabella che mappa ogni identificativo di bot alla propria posizione target nel nuovo pattern, insieme a un piano di transizione che specifica l'ordine temporale in cui i bot devono iniziare a muoversi per evitare collisioni durante la riconfigurazione.

Il piano di transizione viene inviato al controller centrale attraverso la connessione Wi-Fi tra il livello di interfaccia software e il livello hardware. Il controller riceve il piano, lo valida verificando la coerenza con lo stato corrente dello swarm — controllando che tutti i bot assegnati siano effettivamente connessi e operativi — e comincia a inviare i pacchetti di comando ai singoli bot seguendo il piano temporale specificato. Per ogni bot, il comando di transizione contiene le coordinate della posizione target, il timestamp globale al quale il bot deve iniziare il movimento — tipicamente 50–100 millisecondi nel futuro rispetto al momento di invio, per dare a tutti i bot il tempo di ricevere il proprio comando prima che il primo cominci a muoversi — e i parametri di attivazione magnetica che specificano quando e con quale intensità il bot deve attivare le proprie coil durante l'avvicinamento alla posizione target.

Ogni MicroBot riceve il proprio pacchetto di comando, verifica il checksum per assicurarsi dell'integrità dei dati, confronta il proprio timestamp locale — opportunamente sincronizzato con il clock del controller — con il timestamp di inizio specificato nel comando, e quando i due valori coincidono inizia l'esecuzione. Nella versione 0.3, in cui i bot non hanno ancora propulsori fisici indipendenti, l'esecuzione del comando di posizione corrisponde all'attivazione selettiva delle coil magnetiche con i parametri specificati, che produce una forza di attrazione verso i bot vicini già posizionati nella configurazione target, guidando il bot verso la propria posizione per effetto dell'interazione magnetica collettiva. I LED RGB si aggiornano per riflettere lo stato operativo corrente — blu durante la transizione, verde quando la posizione target è raggiunta entro la tolleranza di posizionamento, giallo in condizione di attesa, rosso in caso di anomalia rilevata dal safety layer locale.

Durante tutta la fase di transizione il controller monitora continuamente lo stato dello swarm attraverso i pacchetti di heartbeat ricevuti dai bot, confronta le posizioni stimate con le

posizioni target previste dal piano di transizione e interviene con comandi correttivi se rileva deviazioni significative rispetto alla traiettoria pianificata. Questo meccanismo di controllo in retroazione a livello di sistema è distinto dal controllo locale di ciascun bot — è una forma di supervisione globale che complementa l'esecuzione locale, garantendo che il pattern emergente a livello di sistema corrisponda all'intenzione originale dell'utente anche in presenza di perturbazioni locali. Quando tutti i bot hanno raggiunto le proprie posizioni target entro la tolleranza di posizionamento, il controller invia un comando di consolidamento che attiva i magneti permanenti in modalità di mantenimento a basso consumo — duty cycle ridotto, appena sufficiente a mantenere l'aggancio contro le perturbazioni ambientali — e aggiorna lo stato del sistema da transizione in corso a pattern stabile. L'interfaccia utente riflette questo cambiamento con un feedback visivo e sonoro che conferma il completamento della formazione.

Il flusso operativo descritto presuppone condizioni nominali, ma il sistema è progettato per gestire con grazia le deviazioni da queste condizioni. Se un bot perde la connessione Wi-Fi durante una transizione, il controller lo rimuove dal piano di transizione, redistribuisce la sua posizione target ai bot rimanenti se il pattern lo permette e notifica l'utente attraverso l'interfaccia. Se la batteria di un bot scende al di sotto della soglia critica durante l'operazione, il bot invia un flag di emergenza nel proprio heartbeat, il controller lo esclude dal pattern corrente e lo porta in modalità di risparmio energetico con magneti disattivati e LED che lampeggia in rosso lentamente per indicare la necessità di ricarica. Se la temperatura interna di un bot supera la soglia di sicurezza termica a causa di un'attivazione prolungata delle coil, il safety layer locale del bot interviene autonomamente riducendo il duty cycle PWM al di sotto del valore di regime termico sicuro, anche in assenza di un comando esplicito dal controller, e segnala la condizione al controller nel successivo pacchetto di heartbeat.

L'integrazione tra i componenti di MicroBot produce proprietà di sistema che non emergono dall'analisi dei singoli componenti in isolamento e che costituiscono il valore aggiunto dell'approccio architetturale adottato. La prima è la graceful degradation: il sistema continua a funzionare in modo ridotto anche quando alcuni componenti non sono disponibili — senza visore VR il controllo avviene tramite wristband e gesture, senza wristband tramite interfaccia web, senza alcuni bot il pattern si adatta alle unità disponibili. La seconda è la trasparenza operativa: l'utente riceve in ogni momento un feedback completo sullo stato del sistema attraverso i LED dei bot, l'interfaccia grafica e i segnali acustici, senza dover interpretare messaggi di errore tecnici o consultare log di sistema. La terza è la coerenza temporale: grazie alla sincronizzazione clock distribuita, tutti i componenti del sistema condividono una visione temporale comune che garantisce la coerenza causale degli eventi — un comando generato prima di un altro viene sempre eseguito prima, indipendentemente dalle variazioni di latenza della rete. La quarta è la scalabilità incrementale: aggiungere un nuovo bot al sistema richiede solo che il bot si connetta alla rete SoftAP e invii il proprio primo heartbeat — il controller lo rileva automaticamente, gli assegna una posizione nella tabella di stato globale e lo include nei pattern successivi senza alcuna configurazione manuale.

Punto 11 — Struttura fisica del MicroBot

Il MicroBot è l'elemento terminale e più visibile dell'intera architettura di MicroBot — l'oggetto fisico che materializza nello spazio reale i comportamenti collettivi calcolati dai livelli superiori del sistema. La sua progettazione fisica è il risultato di un processo iterativo che ha dovuto conciliare vincoli simultaneamente attivi e spesso in conflitto: miniaturizzare il volume per favorire l'interazione magnetica tra moduli, massimizzare lo spazio interno per ospitare elettronica e batteria, garantire resistenza meccanica sufficiente a sopravvivere agli urti durante l'aggregazione, mantenere il peso complessivo entro un intervallo che renda le forze magnetiche disponibili competitive con l'attrito, e produrre una geometria stampabile con tecnologia FDM senza richiedere supporti complessi o tolleranze impossibili da ottenere con stampanti desktop. Il risultato di questo processo è una forma che non ha precedenti diretti nel panorama della swarm robotics a scala centimetrica: il doppio cono troncato con sfera centrale, una geometria che risolve simultaneamente tutti i vincoli elencati attraverso una distribuzione spaziale dei componenti funzionalmente ottimizzata.

La geometria del MicroBot può essere descritta come la giustapposizione assiale di tre elementi strutturali distinti. Il cono troncato superiore si sviluppa dalla sfera centrale verso l'alto, terminando con una base circolare di diametro 20 millimetri all'estremità superiore del modulo. Il cono troncato inferiore è geometricamente simmetrico rispetto al cono superiore, si sviluppa dalla sfera centrale verso il basso e termina anch'esso con una base circolare di diametro 20 millimetri all'estremità inferiore. La sfera centrale, con diametro di 18 millimetri, costituisce la zona di massimo ingombro trasversale del modulo e funge da elemento di raccordo tra i due coni, ospitando nel proprio interno il nucleo elettronico principale. L'altezza complessiva del modulo, misurata dalla base inferiore alla base superiore, è di 70 millimetri. Questa proporzione — due coni relativamente snelli che si allargano verso una sfera centrale — non è arbitraria ma riflette la necessità di concentrare la massa elettronica nella zona centrale per abbassare il baricentro e migliorare la stabilità dinamica durante le interazioni magnetiche, mentre i coni ospitano la batteria e il sistema magnetico nelle zone dove la necessità di accesso dall'esterno è maggiore.

Il guscio esterno del MicroBot è progettato per la produzione tramite stampa 3D FDM in materiale PLA, con uno spessore di parete compreso tra 1.2 e 1.5 millimetri. Questo valore è il risultato di un compromesso tra resistenza meccanica e peso: uno spessore inferiore a 1.2 millimetri produce pareti fragili soggette a rottura per impatto durante l'aggregazione magnetica, mentre uno spessore superiore a 1.5 millimetri aumenta il peso del guscio senza produrre benefici meccanici significativi data la geometria curvilinea che distribuisce naturalmente le sollecitazioni. Nella pratica della stampa FDM, uno spessore di 1.2–1.5 millimetri corrisponde a tre o quattro linee di estrusione con ugello da 0.4 millimetri, il che garantisce una buona coesione tra i perimetri e una resistenza agli strati sufficiente per le sollecitazioni operative previste. Il guscio è progettato in due metà separabili lungo un piano di separazione orizzontale passante per il centro della sfera, con un sistema di chiusura tramite viti metriche M2 o incastro a scatto che permette l'accesso all'elettronica interna per la manutenzione e la sostituzione dei componenti senza distruggere il guscio.

La sfera centrale è l'elemento strutturalmente più critico del modulo perché deve ospitare il PCB principale in uno spazio interno di diametro netto inferiore a 16 millimetri, tenendo conto dello spessore del guscio. Il PCB è circolare con diametro compreso tra 14 e 16 millimetri e spessore di circa 1 millimetro, e deve ospitare il microcontrollore, i componenti passivi associati, il driver per le coil, il circuito di gestione della batteria e i connettori per i

LED RGB. L'altezza massima dei componenti montabili sul PCB è vincolata dalla geometria interna della sfera a non superare 4–5 millimetri per lato, richiedendo l'uso di componenti in package SMD di dimensioni ridotte e la disposizione attenta di tutti gli elementi per evitare interferenze fisiche con la superficie interna del guscio. I LED RGB SMD 3535 sono posizionati sul perimetro del PCB in corrispondenza di piccole finestre trasparenti nel guscio della sfera, che permettono la visibilità della segnalazione luminosa dall'esterno senza compromettere la resistenza meccanica della struttura.

I due coni troncati ospitano gli elementi del sistema di alimentazione e del sistema magnetico. Il cono inferiore contiene la batteria Li-Po da 3.7 volt con capacità compresa tra 40 e 60 milliampereora, caratterizzata da dimensioni fisiche di 20×12×4 millimetri che si adattano bene al volume disponibile nella zona conica inferiore. La scelta di una batteria di questa capacità è il risultato di un equilibrio tra autonomia operativa e peso: batterie di capacità maggiore garantirebbero autonomie più lunghe ma aumenterebbero il peso del modulo oltre il limite di 12 grammi necessario per mantenere efficace l'interazione magnetica tra moduli. Con un consumo medio stimato del sistema di 100–150 milliampere in condizioni operative normali, una batteria da 50 milliampereora garantisce un'autonomia di circa 20–30 minuti di operazione continua, valore che può essere esteso significativamente attraverso strategie di gestione energetica che riducono il duty cycle delle coil durante le fasi di mantenimento del pattern e disattivano i sottosistemi non necessari durante le fasi di attesa.

Il sistema magnetico è distribuito tra entrambi i coni e comprende due categorie di elementi con funzioni complementari. I magneti permanenti cilindrici, in numero variabile da quattro a otto per modulo, hanno diametro di 5 millimetri e spessore di 2 millimetri, e sono realizzati in neodimio N52 per massimizzare la forza di attrazione a parità di volume. Questi magneti sono disposti simmetricamente nelle zone apicali dei due coni, in posizioni che coincidono con le zone di contatto tra moduli adiacenti durante l'aggregazione, e svolgono la funzione di pre-allineamento passivo: quando due bot si avvicinano entro il raggio di influenza dei magneti permanenti, l'interazione tra i dipoli tende spontaneamente ad allineare i moduli nella configurazione di aggancio ottimale, riducendo i gradi di libertà del sistema prima che gli elettromagneti vengano attivati. Ogni MicroBot ospita quattro coil elettromagnetiche, due nel cono superiore e due in quello inferiore, ciascuna con una sezione di avvolgimento di 10×10 millimetri e uno spessore di 3–4 millimetri. Le coil sono avvolte a mano con filo di rame smaltato AWG 30–34, con un numero di spire sufficiente a produrre il campo magnetico necessario entro i limiti di corrente imposti dalla batteria. La distanza tra le coil dello stesso cono è di circa 8 millimetri, scelta per creare due zone di campo indipendenti ma sovrapponibili, che il firmware può attivare separatamente o in combinazione per modulare l'intensità e la direzione della forza magnetica prodotta.

Le tolleranze costruttive del guscio stampato in 3D sono un aspetto progettuale che influenza direttamente la funzionalità del sistema di aggancio magnetico. La tolleranza dimensionale della stampa FDM con ugello da 0.4 millimetri su una stampante desktop calibrata è dell'ordine di ± 0.2 – 0.4 millimetri per dimensioni fino a 50 millimetri, e di ± 1 – 2 millimetri per dimensioni maggiori. Nel caso del MicroBot, le tolleranze più critiche riguardano la posizione delle coil e dei magneti permanenti rispetto alle superfici di contatto del guscio: un errore di 1–2 millimetri nella posizione di un magnete permanente rispetto alla posizione nominale riduce significativamente la forza di pre-allineamento e può rendere l'aggancio instabile o dipendente dall'orientamento relativo dei moduli. Per questa ragione il

guscio include alloggiamenti dedicati per ogni magnete e ogni coil, con pareti di guida che vincolano la posizione dei componenti entro le tolleranze richieste, e il processo di assemblaggio prevede una fase di verifica dell'allineamento con calibro digitale prima della chiusura definitiva del guscio.

Il peso complessivo del MicroBot, nell'insieme di guscio stampato, PCB con componenti, batteria, coil e magneti permanenti, è stimato tra 8 e 12 grammi. Questo intervallo è il risultato della combinazione delle masse dei singoli componenti: il guscio in PLA pesa circa 3–4 grammi tenendo conto del volume di materiale con densità 1.24 grammi per centimetro cubo e percentuale di riempimento del 20%; il PCB con componenti pesa circa 1–2 grammi; la batteria da 50 milliamperora pesa circa 1.5–2 grammi; le quattro coil in rame pesano complessivamente circa 1–2 grammi; i magneti permanenti, in numero di sei con dimensioni 5×2 millimetri e densità del neodimio di circa 7.5 grammi per centimetro cubo, contribuiscono con circa 0.7 grammi. La somma di questi contributi colloca il peso totale nel range target, garantendo che le forze magnetiche disponibili — dell'ordine di 0.5–1.5 newton per coppia di magneti in contatto — siano sufficienti a vincere l'attrito statico sulla superficie di appoggio, stimato tra 10 e 30 millinewton per un modulo di questa massa, con un margine di sicurezza adeguato anche in condizioni di superficie non ideale.

Punto 12 — Elettronica del MicroBot: microcontrollore, PCB e gestione dell'alimentazione

L'elettronica del MicroBot è il sistema nervoso del modulo fisico — l'insieme di componenti che trasforma i pacchetti UDP ricevuti via wireless in correnti elettriche nelle coil, in colori nei LED, in decisioni locali sulla sicurezza termica ed energetica del sistema. Progettare l'elettronica di un MicroBot significa affrontare simultaneamente vincoli che appartengono a domini ingegneristici distinti: il vincolo dimensionale impone che tutti i componenti si inseriscano in un PCB circolare di diametro massimo 16 millimetri; il vincolo energetico impone che il consumo totale del sistema sia compatibile con una batteria da 40–60 milliamperora; il vincolo termico impone che la dissipazione di potenza nelle coil e nei componenti di potenza non produca surriscaldamenti pericolosi in un volume così ridotto; il vincolo funzionale impone che il sistema sia in grado di ricevere comandi wireless, elaborarli in tempo reale, pilotare le coil con profili PWM controllati e segnalare il proprio stato tramite LED con una latenza complessiva inferiore a 10 millisecondi. La soluzione a questi vincoli simultanei richiede scelte di componentistica estremamente conservative — ogni componente deve essere il più piccolo, il più efficiente e il più semplice disponibile che soddisfi i requisiti funzionali, senza margini di over-engineering che aumenterebbero inutilmente l'ingombro e il consumo.

Il microcontrollore è il componente centrale attorno a cui ruota l'intera progettazione del PCB. La famiglia ESP32 di Espressif Systems è la scelta naturale per MicroBot per ragioni che vanno oltre la semplice disponibilità commerciale: l'ESP32 integra nel medesimo package un processore dual-core Xtensa LX6 a 240 megahertz, un modulo Wi-Fi 802.11 b/g/n completo di stack TCP/IP, un modulo Bluetooth 4.2 con supporto BLE, 520 kilobyte di RAM interna, un convertitore analogico-digitale a 12 bit su più canali, interfacce SPI, I2C e UART, e un sistema di gestione dell'alimentazione con modalità di deep sleep che riduce il consumo a pochi microampere nelle fasi di inattività. Questa integrazione è cruciale per MicroBot perché elimina la necessità di chip separati per la comunicazione wireless, il

controllo dei sensori e la gestione dell'alimentazione, riducendo significativamente il numero di componenti sul PCB e quindi l'area occupata e il peso complessivo. Per la versione 0.3 del MicroBot fisico, il modulo raccomandato è l'ESP32-PICO-D4 o l'ESP32-C3 SuperMini, entrambi caratterizzati da dimensioni estremamente compatte — inferiori a 20×20 millimetri nella versione PICO-D4 e a 23×18 millimetri nella versione SuperMini — compatibili con il diametro del PCB circolare del modulo.

La tensione logica operativa dell'ESP32 è di 3.3 volt, mentre la batteria Li-Po eroga una tensione nominale di 3.7 volt con una variazione tipica compresa tra 3.0 volt a scarica quasi completa e 4.2 volt a carica completa. Questa differenza richiede la presenza di un regolatore di tensione che stabilizzi l'alimentazione del microcontrollore indipendentemente dallo stato di carica della batteria. La soluzione adottata è un regolatore LDO a basso dropout — tipicamente un ME6206 o equivalente in package SOT-23 — che converte la tensione della batteria in 3.3 volt stabili con una caduta di tensione minima e una corrente quiescente dell'ordine di pochi microampere, garantendo un'efficienza energetica elevata anche nelle fasi di consumo ridotto. Il circuito di ricarica della batteria è implementato tramite un modulo TP4056 in package SOP-8, che gestisce la ricarica tramite la porta USB-C del controller centrale nella versione 0.3, con una corrente di ricarica configurabile tramite una resistenza esterna tipicamente impostata a 100–200 milliampere per batterie di piccola capacità.

Il driver per le coil elettromagnetiche è il componente di potenza più critico dell'intera elettronica del MicroBot. Le coil richiedono correnti dell'ordine di alcune centinaia di milliampere per produrre il campo magnetico necessario all'aggancio, correnti che l'ESP32 non è in grado di erogare direttamente dai propri pin di output — la corrente massima per pin dell'ESP32 è di 40 milliampere. Il driver è implementato tramite MOSFET a canale N in package SOT-23, come il 2N7002 o il BSS138, che vengono pilotati dai pin PWM dell'ESP32 e commutano la corrente della batteria verso le coil con una frequenza tipica di 20–50 kilohertz. Ogni coil è associata a un MOSFET dedicato, permettendo l'attivazione indipendente di ciascuna delle quattro coil del modulo con duty cycle PWM configurabile via software tra 0 e 100 percento. In parallelo a ciascun MOSFET è presente un diodo di ricircolo a recupero rapido — tipicamente un 1N4148 in package SOD-323 — che protegge il MOSFET dagli spike di tensione indotti dalla commutazione dell'induttanza della coil, uno dei fenomeni più comuni di guasto nei circuiti di pilotaggio di carichi induttivi non adeguatamente protetti.

La gestione termica del circuito di pilotaggio è un aspetto che non può essere trascurato data la densità di potenza elevata nel volume ridotto del MicroBot. La potenza dissipata in ciascun MOSFET durante il pilotaggio di una coil è la somma di una componente di conduzione, proporzionale alla resistenza di canale del MOSFET in stato on e al quadrato della corrente, e di una componente di commutazione, proporzionale alla frequenza di commutazione e alle energie di carica e scarica delle capacità di gate e drain. Per i MOSFET in package SOT-23 tipicamente utilizzati, la resistenza termica giunzione-ambiente è dell'ordine di 300–500 gradi per watt, il che significa che con una potenza dissipata di 50 milliwatt la giunzione opera a circa 15–25 gradi al di sopra della temperatura ambiente — un valore accettabile. Tuttavia, con quattro MOSFET e quattro coil attivi simultaneamente a duty cycle elevato, la potenza totale dissipata può avvicinarsi a 200–300 milliwatt, producendo un incremento di temperatura nell'ordine di 60–90 gradi rispetto all'ambiente in

assenza di dissipazione aggiuntiva. Per questa ragione il firmware implementa un limite automatico sul numero di coil attivabili simultaneamente e sul duty cycle massimo in funzione della temperatura misurata, garantendo che la temperatura interna del modulo non superi la soglia di sicurezza di 60–70 gradi centigradi durante l'operazione normale.

La segnalazione visiva dello stato operativo è affidata a LED RGB di tipo SMD 3535, scelti per le loro dimensioni compatte — 3.5×3.5 millimetri di footprint — e per l'elevata luminosità che permette la visibilità attraverso il guscio semitrasparente del MicroBot anche in condizioni di illuminazione ambiente moderata. I LED sono alimentati direttamente dai pin GPIO dell'ESP32 tramite resistenze di limitazione della corrente dimensionate per una corrente di lavoro di 10–15 milliampere per canale, compatibile con la corrente massima dei pin dell'ESP32. In alternativa, per applicazioni che richiedono effetti di colore più sofisticati o il controllo di più LED da un singolo pin, è possibile utilizzare LED RGB addressable di tipo WS2812B in package 5050, che integrano il proprio driver e possono essere controllati in cascata tramite un singolo pin digitale dell'ESP32 con il protocollo a un filo della libreria FastLED. I codici colore standard adottati in MicroBot sono il blu per lo stato di idle in attesa di comandi, il verde per la condizione di pattern stabile raggiunto, il ciano per la fase di transizione in corso, il giallo per la condizione di batteria in esaurimento e il rosso per qualsiasi condizione di errore o di emergenza rilevata dal safety layer.

Il PCB circolare che ospita tutti questi componenti è progettato con un diametro nominale di 15 millimetri, valore intermedio nell'intervallo ammissibile di 14–16 millimetri, che offre un compromesso accettabile tra spazio disponibile per il routing delle piste e facilità di inserimento nel guscio sferico. Il substrato è un laminato FR4 a doppia faccia con spessore di 1 millimetro, che garantisce rigidità meccanica sufficiente a resistere alle vibrazioni e agli urti dell'aggregazione magnetica senza delaminarsi o flettersi in modo inaccettabile. La distribuzione dei componenti sulle due facce del PCB segue una logica di separazione funzionale: la faccia superiore ospita il microcontrollore, il regolatore LDO, i condensatori di bypass e i componenti di segnale a bassa potenza; la faccia inferiore ospita i MOSFET di potenza, i diodi di ricircolo, i connettori per le coil e il connettore per la batteria. Questa separazione riduce il coupling elettromagnetico tra la parte di potenza e la parte di segnale, migliorando la stabilità del microcontrollore e l'immunità ai disturbi generati dalla commutazione dei MOSFET. Le piste di potenza che portano la corrente verso le coil sono dimensionate per una densità di corrente massima di 20 ampere per millimetro quadrato di sezione del rame, con uno spessore di rame di 35 micrometri — lo standard per PCB a doppia faccia — il che richiede larghezze di pista di almeno 0.5–0.8 millimetri per correnti dell'ordine di 300–500 milliampere.

Il connettore tra il PCB e la batteria è un elemento progettuale che richiede attenzione particolare in un sistema così miniaturizzato. Il connettore deve essere meccanicamente robusto per resistere alle vibrazioni durante l'operazione, elettricamente affidabile per garantire la continuità del circuito di alimentazione, e facilmente removibile per permettere la sostituzione della batteria senza dissaldare componenti. La soluzione adottata è un connettore JST-PH a 2 pin con passo 2 millimetri, il connettore standard de facto per le batterie Li-Po di piccola capacità nel settore dei droni e dei dispositivi portatili, che offre il giusto equilibrio tra compattezza, robustezza e disponibilità commerciale. Il circuito di protezione della batteria — che include la protezione da sovraccarica, sovrascarica e cortocircuito — è integrato nella cella Li-Po stessa attraverso un modulo di protezione PCM

incorporato nel pack, eliminando la necessità di circuiteria dedicata sul PCB del MicroBot e semplificando il design.

L'insieme di queste scelte progettuali produce un sistema elettronico che, nonostante la sua complessità funzionale, si adatta al volume interno del MicroBot con margini accettabili e rispetta i vincoli energetici imposti dalla batteria di piccola capacità. La sfida progettuale maggiore non è la complessità dei singoli componenti — tutti ampiamente documentati e disponibili commercialmente — ma la loro integrazione in uno spazio così ridotto con routing delle piste ottimizzato, gestione termica adeguata e affidabilità meccanica sufficiente per un sistema che deve sopravvivere a centinaia di cicli di aggregazione e disaggancio magnetico nel corso della propria vita operativa.

Punto 13 — Sistema magnetico: coil elettromagnetiche, magneti permanenti e logica di aggancio

Il sistema magnetico è il cuore fisico di MicroBot — il meccanismo attraverso cui moduli separati si trasformano in strutture aggregate, attraverso cui forze invisibili organizzano nello spazio oggetti materiali seguendo intenzioni digitali. Comprendere il sistema magnetico di MicroBot significa comprendere non solo la fisica delle coil e dei magneti permanenti, ma la logica complessiva con cui forze passive e forze attive cooperano per produrre un aggancio che sia al tempo stesso robusto, reversibile, selettivo e energeticamente sostenibile con una batteria da 40–60 milliampereora. Questi quattro requisiti — robustezza, reversibilità, selettività, efficienza energetica — sono in tensione strutturale tra loro: un aggancio più robusto richiede forze maggiori e quindi correnti maggiori, ma correnti maggiori consumano più energia e producono più calore; un aggancio più selettivo richiede un controllo più preciso del campo magnetico, ma un controllo più preciso richiede circuiti più complessi che occupano spazio prezioso sul PCB. La soluzione adottata in MicroBot non elimina questa tensione ma la gestisce attraverso una separazione funzionale tra il compito dei magneti permanenti e il compito delle coil elettromagnetiche, assegnando a ciascuno il ruolo per cui è fisicamente più adatto.

I magneti permanenti svolgono il ruolo di sistema di pre-allineamento passivo e di forza di ritenuta a riposo. Sono realizzati in lega di neodimio ferro boro di grado N52, il grado commercialmente disponibile con la densità energetica magnetica più elevata, caratterizzato da un'induzione residua B_r di circa 1.44–1.52 tesla e una coercitività intrinseca H_{ci} superiore a 875 kiloampere per metro. Le dimensioni adottate — diametro 5 millimetri e spessore 2 millimetri — producono una forza di attrazione tipica tra due magneti identici in contatto diretto dell'ordine di 0.8–1.2 newton, valore che scende rapidamente con la distanza: a 2 millimetri di gap la forza si riduce a circa il 25–35% del valore di contatto, e a 5 millimetri di gap a meno del 5%. Questa dipendenza forte dalla distanza è al tempo stesso un vantaggio e una limitazione: è un vantaggio perché significa che la forza di attrazione diventa significativa solo quando i bot si trovano già in prossimità della configurazione di aggancio, evitando interazioni indesiderate a distanze operative normali; è una limitazione perché significa che il sistema di pre-allineamento ha un raggio d'azione molto ridotto e che la fase di avvicinamento iniziale deve essere gestita da altri meccanismi.

La disposizione geometrica dei magneti permanenti nel guscio del MicroBot è progettata per massimizzare la simmetria dell'interazione magnetica tra moduli adiacenti. In ogni modulo

sono presenti da quattro a otto magneti, distribuiti simmetricamente nelle zone apicali dei due coni troncati. Nella configurazione a quattro magneti per modulo, due magneti sono posizionati nel cono superiore e due nel cono inferiore, disposti a 180 gradi l'uno dall'altro rispetto all'asse di simmetria verticale del modulo. Questa disposizione garantisce che qualsiasi orientamento relativo tra due moduli che porti le loro zone apicali in contatto produca un'interazione magnetica positiva — un'attrazione netta verso la configurazione di aggancio — piuttosto che una repulsione che allontani i moduli. Nella configurazione a otto magneti, quattro per cono disposti a 90 gradi, la simmetria di aggancio è ancora più elevata e la forza di pre-allineamento è distribuita su un numero maggiore di punti di contatto, producendo un sistema di aggancio più stabile e meno sensibile ai disallineamenti angolari tra moduli.

La polarità dei magneti permanenti è un aspetto progettuale che richiede attenzione durante l'assemblaggio. I magneti devono essere orientati in modo che i poli opposti si affaccino verso l'esterno nelle posizioni di aggancio corrispondenti tra moduli adiacenti — polo nord verso l'esterno in un modulo e polo sud verso l'esterno nel modulo adiacente nella zona di contatto — per produrre attrazione invece di repulsione. Dato che la geometria del MicroBot prevede agganci sia tra coni superiori di moduli diversi sia tra cono superiore e cono inferiore di moduli adiacenti a seconda della configurazione dello swarm, la polarizzazione deve essere scelta in modo che entrambi i tipi di aggancio producano attrazione. La soluzione adottata è la polarizzazione alternata all'interno dello stesso modulo — i magneti nel cono superiore e nel cono inferiore hanno polarità opposte — in modo che qualsiasi zona apicale di un modulo sia attratta magneticamente da qualsiasi zona apicale di un altro modulo indipendentemente dalla loro orientazione relativa.

Le coil elettromagnetiche hanno un ruolo radicalmente diverso rispetto ai magneti permanenti. Non sono progettate per produrre la forza di aggancio principale — per questo sono fisicamente troppo piccole e energeticamente troppo costose in un sistema a batteria miniaturizzata — ma per svolgere tre funzioni specifiche che i magneti permanenti non sono in grado di realizzare. La prima funzione è il rinforzo selettivo dell'aggancio: quando il controller decide che due bot specifici devono aggregarsi, le coil di entrambi i moduli vengono attivate con correnti tali da produrre un campo magnetico aggiuntivo che si somma al campo dei magneti permanenti, aumentando la forza totale di aggancio a un valore superiore alla soglia necessaria per vincere l'attrito statico e completare il contatto fisico tra i moduli. La seconda funzione è il disaggancio controllato: quando il controller decide che due bot devono separarsi, le coil vengono attivate con una corrente inversa che produce un campo magnetico opposto a quello dei magneti permanenti, riducendo o invertendo la forza netta di interazione tra i moduli fino a permetterne la separazione fisica. La terza funzione è la modulazione continua della rigidità dell'aggregato: variando il duty cycle PWM delle coil in modo continuo tra zero e il valore massimo, il sistema può realizzare una transizione graduale tra uno stato fluido — in cui i moduli sono debolmente attratti e possono scorrere l'uno rispetto all'altro — e uno stato solido — in cui i moduli sono fortemente agganciati e resistono alle perturbazioni esterne — aprendo la possibilità di controllare le proprietà meccaniche macroscopiche dell'aggregato in tempo reale.

Ogni MicroBot ospita quattro coil, distribuite due nel cono superiore e due in quello inferiore. Ogni coil ha una sezione di avvolgimento di 10×10 millimetri e uno spessore di 3–4 millimetri, ed è avvolta con filo di rame smaltato di diametro AWG 30–34. La scelta del

diametro del filo è il risultato di un compromesso tra resistenza elettrica della coil e numero di spire realizzabili nel volume disponibile: un filo più sottile — AWG 34, diametro 0.16 millimetri — permette di avvolgere più spire nello stesso volume e quindi di produrre un campo magnetico più intenso a parità di corrente, ma ha una resistenza per unità di lunghezza circa quattro volte superiore rispetto a un filo AWG 30, dissipando più potenza e limitando la corrente massima gestibile dalla batteria. Il numero ottimale di spire per ogni coil, nella geometria adottata, è dell'ordine di 50–100 spire con filo AWG 32, che produce un'induttanza di circa 50–150 microhenry e una resistenza di avvolgimento di 5–15 ohm. Con una tensione di alimentazione di 3.7 volt e una resistenza di avvolgimento di 10 ohm, la corrente massima nella coil è dell'ordine di 370 milliampere, che produce un campo magnetico al centro dell'avvolgimento dell'ordine di 0.5–2 millitesla — sufficiente per le funzioni di rinforzo e disaggancio nelle condizioni operative di MicroBot.

La distanza tra le due coil dello stesso cono è di circa 8 millimetri, scelta per creare due zone di campo magnetico indipendenti ma spazialmente vicine. Questa separazione permette di attivare le coil di uno stesso cono con correnti indipendenti per creare un gradiente di campo nella zona apicale del modulo, che può essere sfruttato per guidare l'allineamento del modulo adiacente durante la fase di aggancio. In pratica, attivando una coil con intensità maggiore dell'altra, il sistema produce una forza asimmetrica che tende a ruotare il modulo adiacente verso la configurazione di allineamento ottimale, funzionando come un sistema di sterzo magnetico passivo che non richiede motori fisici per correggere i disallineamenti angolari durante l'aggregazione.

La logica di aggancio implementata nel firmware combina le soglie di distanza del modello matematico con i profili di corrente ottimizzati per minimizzare il consumo energetico durante le diverse fasi dell'aggancio. La fase di approccio inizia quando la distanza tra due bot scende al di sotto della soglia di pre-attivazione — tipicamente 15–20 millimetri — e il controller invia a entrambi i moduli il comando di attivare le coil con un duty cycle basso, dell'ordine del 20–30 percento, sufficiente a produrre un leggero incremento dell'attrazione magnetica senza dissipare potenza eccessiva. La fase di contatto inizia quando la distanza scende al di sotto della soglia di aggancio — tipicamente 5–8 millimetri — e le coil vengono portate al duty cycle massimo per un breve impulso di durata 50–100 millisecondi, producendo la forza massima disponibile che completa il contatto fisico tra i moduli e vince l'attrito statico dell'ultimo millimetro di avvicinamento. La fase di mantenimento inizia quando il contatto fisico è confermato — rilevato attraverso la variazione dell'induttanza delle coil o tramite un sensore di prossimità a infrarossi — e le coil vengono ridotte a un duty cycle di mantenimento basso, dell'ordine del 10–15 percento, sufficiente a rinforzare i magneti permanenti contro le perturbazioni ambientali senza dissipare energia inutilmente. La fase di disaggancio viene attivata su comando del controller e prevede l'inversione della corrente nelle coil per un impulso di durata 100–200 millisecondi con duty cycle massimo, sufficiente a vincere la forza di ritenuta dei magneti permanenti e a separare i moduli, seguita dalla disattivazione immediata delle coil una volta che i moduli si sono allontanati oltre la soglia di distacco.

L'isteresi nella logica di aggancio e disaggancio è implementata a livello firmware attraverso due soglie di distanza distinte: la soglia di aggancio d_{agg} al di sotto della quale l'aggancio viene attivato, e la soglia di distacco d_{dist} superiore a d_{agg} al di sopra della quale il disaggancio viene confermato. Nel firmware del MicroBot questi valori sono configurabili

come parametri, con valori di default di $d_{agg} = 5$ millimetri e $d_{dist} = 8$ millimetri, che definiscono una zona di isteresi di 3 millimetri nella quale lo stato del sistema dipende dalla storia passata. Questa isteresi è fondamentale per la stabilità del sistema: senza di essa, piccole oscillazioni nella distanza tra due bot intorno alla soglia di aggancio produrrebbero cicli rapidi di attivazione e disattivazione delle coil — il fenomeno del chattering — che non solo renderebbero il comportamento del sistema imprevedibile ma dissiperebbero rapidamente l'energia della batteria attraverso commutazioni inutili dei MOSFET.

Punto 14 — Firmware del MicroBot: architettura software embedded e gestione real-time

Il firmware del MicroBot è il livello più basso della gerarchia software del sistema — il codice che gira direttamente sull'ESP32 di ogni modulo fisico, senza sistema operativo di alto livello né interprete, con accesso diretto all'hardware e responsabilità piena sulla correttezza temporale di ogni operazione. Progettare il firmware di un sistema embedded real-time come MicroBot è un'attività fondamentalmente diversa dalla programmazione di applicazioni software convenzionali: non esistono eccezioni non gestite che terminano il programma con un messaggio di errore, non esiste un garbage collector che libera la memoria inutilizzata, non esiste un framework che astrae la gestione del tempo e degli interrupt. Ogni millisecondo di latenza introdotto da una scelta implementativa errata si traduce in un comportamento fisicamente osservabile del robot — un aggancio mancato, un LED che lampeggia nel momento sbagliato, una coil che rimane attiva un istante di troppo e surriscalda il modulo. Questa responsabilità diretta del codice sul comportamento fisico del sistema è la ragione per cui il firmware di MicroBot è progettato con la stessa attenzione metodologica dedicata all'hardware: struttura modulare con responsabilità chiaramente delimitate, gestione del tempo rigorosamente non bloccante, documentazione interna sistematica e un safety layer che opera come guardia indipendente su tutti gli altri componenti software.

L'architettura del firmware è organizzata in moduli software distinti, ciascuno responsabile di un aspetto specifico del comportamento del MicroBot e comunicante con gli altri attraverso interfacce ben definite. Questa organizzazione modulare non è una questione estetica ma una necessità pratica: un firmware monolitico in cui tutte le funzionalità sono intrecciate in un unico file di codice diventa rapidamente impossibile da testare, da modificare e da debuggare, soprattutto quando il comportamento del sistema dipende da interazioni temporali precise tra componenti che operano a frequenze diverse. La separazione in moduli permette di sviluppare e testare ogni componente in isolamento — su simulatore Wokwi o su hardware fisico dedicato — prima di integrarlo nel firmware completo, riducendo drasticamente il rischio di regressioni introdotte dalle modifiche.

Il modulo principale, implementato nel file `main.cpp`, svolge il ruolo di orchestratore dell'intera esecuzione. Contiene la funzione `setup()` che viene eseguita una sola volta all'avvio del sistema e inizializza tutti i componenti hardware e software — configura i pin GPIO, inizializza il driver delle coil, stabilisce la connessione Wi-Fi con il controller, sincronizza il clock locale con il clock master e porta il sistema nello stato operativo iniziale con LED blu acceso e magneti disattivati. Contiene la funzione `loop()` che viene eseguita continuamente durante l'intera vita operativa del sistema e coordina l'esecuzione di tutti gli altri moduli attraverso il meccanismo di scheduling cooperativo basato su `millis()`. La

funzione `loop()` non chiama direttamente le funzioni di controllo delle coil, di gestione della comunicazione o di lettura dei sensori, ma delega queste responsabilità ai moduli specializzati attraverso chiamate a funzioni di aggiornamento che ogni modulo espone come interfaccia pubblica. Questo pattern di progettazione, noto come cooperative multitasking, permette a più componenti di condividere il processore senza richiedere un sistema operativo real-time, mantenendo la semplicità implementativa del modello Arduino mentre si avvicina alle proprietà di concorrenza necessarie per un sistema multi-funzione.

Il modulo di comunicazione, implementato nel file `communication.cpp`, gestisce tutto ciò che riguarda la connettività Wi-Fi e il protocollo UDP. Mantiene aperto un socket UDP in ascolto sulla porta dedicata del MicroBot, riceve i pacchetti inviati dal controller, verifica il checksum di integrità, decodifica il tipo di comando e i parametri associati, e deposita il comando decodificato in una struttura dati condivisa che gli altri moduli possono leggere nella loro fase di aggiornamento. La ricezione dei pacchetti UDP è implementata in modalità non bloccante: la funzione di aggiornamento del modulo di comunicazione controlla se ci sono dati disponibili sul socket e li legge se presenti, ma non attende il loro arrivo bloccando l'esecuzione del loop principale. Questo è il principio fondamentale della gestione non bloccante del tempo in MicroBot: nessuna funzione può chiamare `delay()` o attendere attivamente un evento per un periodo superiore a pochi decine di microsecondi, perché qualsiasi attesa bloccante impedirebbe l'aggiornamento degli altri moduli e potrebbe causare il mancato rispetto delle scadenze temporali del safety layer. Il modulo di comunicazione gestisce anche l'invio periodico dei pacchetti di heartbeat al controller, schedulato attraverso un timer software basato su `millis()` con intervallo di 500 millisecondi, e implementa il meccanismo di sincronizzazione del clock locale con il timestamp ricevuto nei pacchetti del controller.

Il modulo di controllo delle coil, implementato nel file `magnet_driver.cpp`, è il componente più critico dal punto di vista della sicurezza e della correttezza temporale. Mantiene lo stato corrente di ciascuna delle quattro coil — attiva o inattiva, duty cycle corrente, energia totale dissipata dall'ultima riattivazione, tempo dall'ultima attivazione — e implementa la logica di transizione tra le fasi di approccio, contatto, mantenimento e disaggancio descritta nel capitolo precedente. Il controllo del duty cycle PWM è implementato attraverso il canale LEDC dell'ESP32, che genera segnali PWM hardware indipendenti dall'esecuzione del firmware e garantisce la correttezza temporale dei profili di corrente anche quando il processore è occupato nell'elaborazione di altri task. La frequenza PWM è impostata a 25 kilohertz, valore che cade al di sopra della soglia udibile dall'orecchio umano — eliminando il fastidioso ronzio che si produrrebbe a frequenze inferiori a 20 kilohertz — e al di sotto della frequenza di risonanza del circuito LC formato dalla coil e dalla sua capacità parassita, garantendo che il MOSFET commuti in condizioni di corrente ben controllate. Il modulo implementa anche la logica di isteresi nelle soglie di aggancio e disaggancio, mantenendo una variabile di stato per ogni coppia di bot potenzialmente agganciabili che ricorda se la coppia è attualmente nello stato agganciato o sganciato e applica la soglia appropriata in funzione di questo stato.

Il modulo di gestione della batteria e della temperatura, implementato nel file `power_manager.cpp`, monitora continuamente i parametri energetici e termici del sistema e interviene con misure correttive quando vengono rilevate condizioni anomale. La tensione della batteria è misurata tramite il convertitore ADC interno dell'ESP32, collegato al

terminale positivo della batteria attraverso un partitore resistivo che scala la tensione nel range di ingresso dell'ADC — 0–3.3 volt. La misurazione viene campionata a 10 hertz con una media mobile su 16 campioni per filtrare il rumore di quantizzazione e le fluttuazioni dovute ai picchi di corrente durante l'attivazione delle coil. Quando la tensione misurata scende al di sotto della soglia di batteria scarica — tipicamente 3.3 volt — il modulo invia un flag di avviso al controller nel successivo pacchetto di heartbeat, riduce il duty cycle massimo consentito alle coil al 50% per estendere l'autonomia residua e imposta il LED RGB su giallo lampeggiante per segnalare visivamente la condizione all'utente. Quando la tensione scende ulteriormente al di sotto della soglia critica — tipicamente 3.0 volt — il modulo forza l'inattivazione immediata di tutte le coil per evitare la scarica profonda della batteria Li-Po, che causerebbe un danno irreversibile alla cella, e imposta il LED su rosso fisso. La temperatura interna è stimata indirettamente integrando il profilo di corrente nelle coil nel tempo — una tecnica nota come modello termico equivalente — che calcola l'energia dissipata nei MOSFET e nelle resistenze di avvolgimento e la converte in un incremento di temperatura rispetto alla temperatura ambiente tramite la resistenza termica equivalente del circuito. Questa tecnica è preferibile alla misurazione diretta con un sensore NTC o DS18B20 perché elimina la necessità di un componente aggiuntivo sul PCB già densamente popolato, al costo di una stima della temperatura meno precisa ma sufficiente per le funzioni di protezione richieste.

Il modulo di gestione dello stato e dei LED, implementato nel file `state_manager.cpp`, centralizza la logica di transizione tra gli stati operativi del MicroBot e aggiorna il colore del LED RGB in funzione dello stato corrente. Gli stati operativi implementati sono: IDLE — il bot è connesso al controller e in attesa di comandi, LED blu fisso; MOVING — il bot sta eseguendo una transizione verso una nuova posizione target, LED ciano lampeggiante lento; DOCKING — il bot sta eseguendo la fase di aggancio magnetico, LED ciano lampeggiante veloce; LOCKED — il bot è agganciato ad almeno un altro modulo e mantiene il pattern stabile, LED verde fisso; LOW_BATTERY — la batteria è al di sotto della soglia di avviso, LED giallo lampeggiante; EMERGENCY — il safety layer ha rilevato una condizione di errore critica, LED rosso fisso; SAFE_MODE — il bot ha attivato la modalità di sicurezza autonoma, LED rosso lampeggiante lento. Le transizioni tra stati sono governate da una macchina a stati finiti esplicita, con condizioni di transizione ben definite per ogni arco del grafo degli stati, garantendo che il sistema non possa trovarsi in stati inconsistenti o transizioni circolari non previste.

Il safety layer è implementato come un modulo separato, `safety_layer.cpp`, che viene aggiornato con priorità più alta rispetto a tutti gli altri moduli nel loop principale — viene chiamato per primo in ogni iterazione del loop, prima ancora del modulo di comunicazione. Questo non è un dettaglio implementativo ma una scelta architetturale fondamentale: il safety layer deve avere accesso allo stato più aggiornato del sistema in ogni istante e deve poter intervenire prima che qualsiasi altro modulo prenda decisioni basate su uno stato potenzialmente non sicuro. Il safety layer monitora quattro condizioni di sicurezza in parallelo: il limite di corrente nelle coil, verificato confrontando il duty cycle richiesto con il duty cycle massimo consentito in funzione della temperatura stimata; il limite di temperatura, verificato confrontando la temperatura stimata con la soglia di sicurezza di 65 gradi centigradi; il livello di tensione della batteria, verificato confrontando la tensione misurata con le soglie di avviso e di emergenza; e il watchdog software, un timer che viene resettato ad ogni iterazione del loop principale e che, se non resettato entro un timeout di 5 secondi —

indicazione di un freeze del firmware — forza il riavvio completo del sistema attraverso il watchdog hardware dell'ESP32. In caso di rilevamento di qualsiasi condizione di sicurezza violata, il safety layer esegue immediatamente l'azione correttiva appropriata — riduzione del duty cycle, disattivazione delle coil, transizione allo stato di emergenza — e registra l'evento in una variabile di stato che viene inclusa nel successivo pacchetto di heartbeat inviato al controller, permettendo il monitoraggio remoto della salute del sistema dall'interfaccia utente.

Punto 15 — Controller centrale: hardware, firmware e gestione dello swarm

Il controller centrale è il componente che trasforma MicroBot da un insieme di moduli indipendenti in un sistema coordinato. Se i MicroBot sono le mani del sistema — gli elementi che agiscono fisicamente nel mondo reale — il controller è il cervello che pianifica, coordina e supervisiona, mantenendo in ogni istante una visione globale dello stato dello swarm che nessun singolo bot possiede. Questa posizione privilegiata nella gerarchia del sistema comporta responsabilità precise e vincoli precisi: il controller deve essere abbastanza potente da gestire il traffico di comunicazione con decine di bot simultaneamente, abbastanza reattivo da rispettare le scadenze temporali della sincronizzazione clock, abbastanza robusto da non diventare il punto singolo di fallimento dell'intero sistema, e abbastanza compatto da essere portatile e alimentato a batteria in un involucro di 70×70×30 millimetri. La soluzione hardware e firmware adottata per il controller nella versione 0.3 del progetto rappresenta il punto di equilibrio tra questi requisiti, con un margine di crescita esplicito verso le versioni successive in cui la complessità della gestione dello swarm aumenterà significativamente.

L'hardware del controller è basato sull'ESP32-S3, la versione della famiglia ESP32 con le prestazioni computazionali più elevate disponibile in package compatto. L'ESP32-S3 integra un processore dual-core Xtensa LX7 a 240 megahertz — leggermente più potente del LX6 dei modelli precedenti — con 512 kilobyte di SRAM interna e supporto per memoria flash esterna fino a 16 megabyte, modulo Wi-Fi 802.11 b/g/n con supporto per modalità Access Point, Station e AP+Station simultanee, modulo Bluetooth 5.0 con supporto BLE e Bluetooth Classic, e un insieme di periferiche hardware che include interfacce SPI, I2C, UART, canali PWM e convertitori ADC. La scelta dell'ESP32-S3 rispetto all'ESP32 standard dei MicroBot non è motivata esclusivamente dalla maggiore potenza computazionale, ma dalla necessità di gestire simultaneamente la rete SoftAP con connessioni multiple, l'interfaccia con l'IMU a nove assi tramite I2C, la comunicazione con il livello di interfaccia software tramite Wi-Fi in modalità Station e il calcolo in tempo reale degli algoritmi di coordinamento dello swarm — un carico computazionale che può saturare i core di un ESP32 standard quando il numero di bot connessi supera la decina.

Il PCB del controller ha dimensioni di circa 60×60 millimetri e spessore di 1.5 millimetri, con substrato FR4 a doppia faccia. Ospita l'ESP32-S3 nella versione con antenna PCB integrata, l'unità IMU a nove assi — preferibilmente una Bosch BNO055 per la sua capacità di fornire quaternioni di orientamento pre-elaborati direttamente in output, riducendo il carico computazionale sull'ESP32 rispetto a soluzioni basate su IMU raw come la MPU6050 — il circuito di gestione dell'alimentazione con regolatore LDO, il modulo di ricarica TP4056 connesso alla porta USB-C, e un'interfaccia per un'eventuale antenna Wi-Fi esterna da montare sull'involucro del controller per migliorare la copertura wireless in ambienti con

attenuazione del segnale. La batteria del controller è una Li-Po da 1200–2000 milliampereora con dimensioni fisiche di circa 50×30×7 millimetri, che garantisce un'autonomia del controller di diverse ore in condizioni operative normali — significativamente superiore all'autonomia dei MicroBot, in modo che la batteria del controller non diventi il fattore limitante delle sessioni operative.

L'involucro del controller è stampato in PLA o ABS con pareti di spessore 1.8–2 millimetri, dimensioni esterne di 70×70×30 millimetri e chiusura tramite viti M2 o sistema di incastro. Il peso complessivo stimato del controller completo di PCB, batteria e involucro è di 50–70 grammi, compatibile con l'utilizzo come dispositivo portatile appoggiato su una superficie di lavoro o montato su un supporto regolabile nelle installazioni più elaborate. Il pannello superiore dell'involucro espone la porta USB-C per la ricarica, un LED di stato che segnala lo stato della connessione Wi-Fi e il livello di carica della batteria, e un pulsante di reset per il riavvio del firmware senza dover aprire l'involucro.

Il firmware del controller è strutturato secondo gli stessi principi architetturali del firmware dei MicroBot — modularità, gestione non bloccante del tempo, safety layer indipendente — ma con una complessità significativamente maggiore dovuta alla necessità di gestire lo stato globale dello swarm e di implementare gli algoritmi di coordinamento collettivo. Il modulo principale orchestra l'esecuzione di tutti i sottomoduli attraverso il meccanismo di cooperative multitasking, con la differenza rispetto al firmware dei bot che il loop principale del controller deve gestire comunicazioni con un numero variabile di client simultanei — i bot connessi — oltre che con il livello di interfaccia software che invia i comandi di pattern dall'alto.

Il modulo di gestione della rete SoftAP inizializza l'access point Wi-Fi del controller con un SSID dedicato del formato MicroBot_Network_XXXX, dove XXXX è un identificativo univoco derivato dall'indirizzo MAC dell'ESP32-S3. Assegna indirizzi IP statici ai bot in fase di connessione basandosi sull'identificativo univoco di ciascun bot, registrato nel database di configurazione del controller, e mantiene una tabella delle connessioni attive aggiornata in tempo reale. Il modulo implementa anche la logica di autenticazione delle connessioni in ingresso: ogni bot che si connette alla rete SoftAP deve presentare un token di autenticazione derivato dal proprio identificativo e da una chiave condivisa nota al controller, impedendo la connessione di moduli non autorizzati alla rete dello swarm.

Il modulo di gestione dello stato globale dello swarm è il componente più complesso del firmware del controller. Mantiene una struttura dati per ogni bot connesso che include l'identificativo univoco, l'indirizzo IP nella rete SoftAP, la posizione stimata nel piano di simulazione — aggiornata a partire dai dati di telemetria ricevuti nei pacchetti di heartbeat — il livello di carica della batteria, la temperatura stimata delle coil, lo stato corrente dei magneti, il timestamp dell'ultimo heartbeat ricevuto e un indicatore della qualità della connessione wireless calcolato come percentuale di pacchetti di comando ricevuti correttamente nell'ultimo secondo. Questa struttura dati è il fondamento su cui tutti gli altri moduli del controller basano le proprie decisioni: il modulo di pianificazione dei pattern la consulta per decidere l'assegnazione dei bot alle posizioni target, il modulo di sicurezza globale la monitora per rilevare condizioni anomale, il modulo di interfaccia verso il livello software la serializza e la trasmette all'interfaccia utente per la visualizzazione.

Il modulo di pianificazione e assegnazione dei pattern riceve dal livello di interfaccia software la specifica del pattern desiderato — tipo di pattern, parametri geometrici, numero di bot da coinvolgere — e calcola l'assegnazione ottimale dei bot disponibili alle posizioni target. Per swarm di piccole dimensioni — fino a circa 15–20 bot — viene utilizzato l'algoritmo ungherese per la risoluzione del problema di assegnazione ottimale, che garantisce la minimizzazione della somma delle distanze tra le posizioni correnti dei bot e le posizioni target assegnate in tempo $O(n^3)$, dove n è il numero di bot. Per swarm di dimensioni maggiori viene utilizzata un'euristica greedy che assegna iterativamente ogni posizione target al bot più vicino non ancora assegnato, con complessità $O(n^2 \log n)$ che permette l'esecuzione in tempo reale anche con 50–100 bot. Una volta calcolata l'assegnazione, il modulo genera il piano di transizione temporale che specifica l'ordine di attivazione dei bot per evitare collisioni durante la riconfigurazione, e lo invia al modulo di comunicazione per la trasmissione ai singoli bot.

Il modulo di sincronizzazione temporale implementa il meccanismo di clock master descritto nel capitolo sulla comunicazione. Mantiene un contatore di tempo globale aggiornato con precisione di un millisecondo attraverso il timer hardware dell'ESP32-S3, e include il valore corrente di questo contatore in ogni pacchetto UDP inviato ai bot — sia nei pacchetti di comando sia nei pacchetti di sincronizzazione puri inviati a 10 hertz. Il modulo monitora la deriva temporale stimata di ciascun bot confrontando i timestamp locali riportati negli heartbeat con il proprio clock master, e interviene con pacchetti di correzione se la deriva di un bot supera la soglia di 5 millisecondi — indicazione di un problema nel meccanismo di sincronizzazione locale del bot che potrebbe compromettere la coerenza temporale del sistema.

Il safety layer del controller opera a un livello di astrazione superiore rispetto al safety layer dei singoli bot: invece di monitorare parametri fisici locali come la temperatura delle coil o la tensione della batteria, monitora proprietà globali dello swarm come la coerenza del pattern — verificando che le posizioni stimate dei bot siano compatibili con il pattern assegnato entro le tolleranze previste — la salute della comunicazione — verificando che tutti i bot connessi stiano inviando heartbeat con la regolarità attesa — e la coerenza energetica — verificando che la somma dei consumi stimati di tutti i bot sia compatibile con le autonomie residue dichiarate. In caso di rilevamento di anomalie globali, il safety layer del controller può emettere comandi di emergenza broadcast che portano tutti i bot contemporaneamente in modalità sicura, indipendentemente dal loro stato locale corrente.

Il firmware del controller implementa infine un sistema di logging persistente che registra su memoria flash i principali eventi operativi — connessioni e disconnessioni dei bot, transizioni di pattern, attivazioni del safety layer, anomalie di comunicazione — con timestamp precisi al millisecondo. Questo log è accessibile tramite interfaccia seriale USB o tramite un'apposita API REST esposta dal controller sul proprio indirizzo IP nella rete SoftAP, e costituisce uno strumento fondamentale per il debug del comportamento del sistema nelle fasi di sviluppo e per l'analisi post-hoc delle sessioni operative nelle fasi di ricerca sperimentale. La capacità di correlare temporalmente eventi avvenuti su componenti diversi del sistema — un comando inviato dal controller, la sua ricezione da parte di un bot, la risposta fisica dello swarm — è indispensabile per comprendere le cause dei comportamenti inattesi e per raffinare progressivamente i parametri degli algoritmi di coordinamento.

Punto 16 — Il Wristband: hardware, firmware e riconoscimento delle gesture

Il wristband è il componente di MicroBot che stabilisce il contatto più diretto e fisico tra l'utente e il sistema — il dispositivo che trasforma il movimento naturale del corpo umano in comandi digitali comprensibili al controller dello swarm. A differenza del visore VR, che media l'interazione attraverso uno schermo e una rappresentazione virtuale, il wristband è indossato sul polso e risponde ai movimenti della mano con la stessa immediatezza con cui un guanto risponde alla stretta della mano che lo indossa. Questa immediatezza non è solo una caratteristica dell'esperienza utente ma un requisito tecnico preciso: il ritardo percepito tra il gesto dell'utente e la risposta dello swarm deve essere sufficientemente ridotto da mantenere la sensazione di controllo diretto, e il wristband è il primo anello della catena di latenza che contribuisce a questo ritardo. Progettare il wristband significa quindi progettare simultaneamente l'hardware di acquisizione del movimento, il firmware di elaborazione locale dei dati inerziali, il protocollo di trasmissione wireless dei comandi e la logica di riconoscimento delle gesture che trasforma sequenze di dati grezzi dell'IMU in comandi discreti comprensibili al controller.

L'hardware del wristband è costruito attorno a due componenti principali: il microcontrollore ESP32-C3 mini e l'unità di misura inerziale BNO055 di Bosch Sensortec. La scelta dell'ESP32-C3 mini è motivata dalle sue dimensioni estremamente compatte — circa 23×18 millimetri nella versione SuperMini — che permettono l'integrazione in un involucro indossabile senza compromettere il comfort dell'utente durante l'uso prolungato.

L'ESP32-C3 è un microcontrollore single-core RISC-V a 160 megahertz con Wi-Fi e Bluetooth integrati, 400 kilobyte di SRAM e 4 megabyte di flash, sufficiente per le funzioni di elaborazione locale richieste dal wristband. La versione C3 è preferita alla versione standard dell'ESP32 per il consumo energetico ridotto — importante in un dispositivo alimentato da una batteria Li-Po da 500 milliampereora che deve garantire diverse ore di autonomia — e per il form factor compatto che si adatta meglio all'involucro del wristband.

La scelta dell'IMU BNO055 merita una giustificazione approfondita perché determina in modo decisivo le capacità di riconoscimento delle gesture del wristband. La BNO055 è un sensore di fusione inerziale che integra in un singolo chip un accelerometro triassiale, un giroscopio triassiale e un magnetometro triassiale, insieme a un processore ARM Cortex-M0 dedicato che esegue l'algoritmo di fusione sensoriale internamente, producendo direttamente in output quaternioni di orientamento assoluto, angoli di Eulero e vettori di accelerazione lineare compensati dalla gravità. Questa capacità di fusione sensoriale interna è il vantaggio competitivo della BNO055 rispetto a soluzioni più economiche come la MPU6050: con la MPU6050 il firmware del wristband deve implementare internamente l'algoritmo di fusione — tipicamente un filtro di Madgwick o di Mahony — che richiede risorse computazionali significative e produce stime di orientamento meno accurate in condizioni di movimento rapido. Con la BNO055 il firmware riceve direttamente quaternioni di orientamento di alta qualità tramite interfaccia I2C, con una frequenza di aggiornamento fino a 100 hertz e un'accuratezza angolare statica dell'ordine di un grado.

L'orientamento del polso nello spazio tridimensionale è descritto dai tre angoli di Eulero estratti dal quaternioni fornito dalla BNO055: il pitch, che rappresenta la rotazione attorno all'asse trasversale del polso e corrisponde al gesto di inclinare la mano verso l'alto o verso il basso; il roll, che rappresenta la rotazione attorno all'asse longitudinale dell'avambraccio e

corrisponde al gesto di ruotare la mano come per girare una manopola; e il yaw, che rappresenta la rotazione attorno all'asse verticale e corrisponde al gesto di ruotare il polso verso sinistra o verso destra. Questi tre angoli sono i parametri di controllo continui fondamentali del wristband: il controller può mapparli direttamente su parametri geometrici dei pattern dello swarm — il pitch controlla il raggio del pattern circolare, il roll controlla l'angolo di rotazione del pattern, il yaw controlla la direzione della propagazione nell'onda — producendo un sistema di controllo intuitivo in cui i movimenti naturali del polso corrispondono a modificazioni naturali della forma dello swarm.

Accanto ai parametri di orientamento continui, il firmware del wristband implementa il riconoscimento di un insieme di gesture discrete che corrispondono a comandi qualitativi — cambia pattern, attiva emergenza, sincronizza clock, seleziona gruppo di bot. Il riconoscimento delle gesture discrete è implementato attraverso un algoritmo di rilevamento di eventi basato su soglie e finestre temporali. Il gesto di scossa rapida — un'accelerazione lineare superiore a una soglia configurabile in una direzione qualsiasi, seguita da una decelerazione di pari entità entro una finestra temporale di 200–300 millisecondi — è riconosciuto come comando di cambio pattern e produce una transizione al pattern successivo nella lista configurata nel controller. Il gesto di rotazione rapida del polso di 180 gradi attorno all'asse longitudinale dell'avambraccio — rilevato come una variazione del roll superiore a 150 gradi entro 500 millisecondi — è riconosciuto come comando di inversione del pattern corrente, che specchia geometricamente la configurazione attuale dello swarm. Il gesto di doppia scossa — due accelerazioni rapide successive entro una finestra temporale di 600 millisecondi — è riconosciuto come comando di attivazione della modalità di emergenza globale, che porta tutti i bot in stato di sicurezza indipendentemente dalla loro attività corrente. La scelta di gesture con caratteristiche cinematiche distinte e difficilmente confondibili — in termini di intensità dell'accelerazione, durata del movimento e numero di eventi — è fondamentale per minimizzare i falsi positivi nel riconoscimento, che produrrebbero comandi indesiderati durante i normali movimenti del polso dell'utente.

Il firmware del wristband è organizzato in modo più semplice rispetto al firmware dei MicroBot, riflettendo la minore complessità funzionale del dispositivo. Il modulo principale coordina l'esecuzione di quattro sottomoduli: il modulo di lettura dell'IMU, che interroga la BNO055 tramite I2C a 50 hertz e aggiorna le variabili di orientamento corrente e di accelerazione lineare; il modulo di riconoscimento gesture, che analizza la sequenza temporale dei dati IMU alla ricerca di pattern cinematici corrispondenti alle gesture discrete registrate; il modulo di comunicazione, che trasmette i dati di orientamento e i comandi gesture al controller tramite Bluetooth Low Energy in modalità centrale-periferica, con il wristband nel ruolo di periferica e il controller nel ruolo di centrale; e il modulo di gestione dell'alimentazione, che monitora la tensione della batteria e implementa le strategie di risparmio energetico per estendere l'autonomia operativa.

La trasmissione dei dati dal wristband al controller avviene tramite Bluetooth Low Energy piuttosto che Wi-Fi per ragioni legate al consumo energetico: il BLE consuma tipicamente dieci volte meno energia del Wi-Fi in modalità di trasmissione continua, e con una batteria da 500 milliampereora questa differenza si traduce in diverse ore di autonomia aggiuntiva. Il protocollo di comunicazione BLE adottato è basato sul profilo GATT standard, con una caratteristica personalizzata che trasporta i dati di orientamento — tre angoli di Eulero a 16 bit ciascuno — e i flag di gesture discrete — un byte di flag con un bit per ogni gesture

riconosciuta — in un singolo pacchetto da 8 byte trasmesso a 50 hertz. Questa frequenza di aggiornamento è sufficiente per i parametri di controllo continui del wristband, dato che i movimenti del polso umano hanno tipicamente componenti frequenziali significative fino a circa 10 hertz, e un campionamento a 50 hertz garantisce un margine di sicurezza di cinque volte rispetto al criterio di Nyquist. La latenza end-to-end del canale BLE — dal momento in cui il gesto viene eseguito al momento in cui il controller riceve il comando — è dell'ordine di 10–30 millisecondi in condizioni operative normali, valore che si somma alle latenze degli strati successivi della catena di controllo.

Il calibrazione del wristband è una fase operativa che deve essere eseguita all'inizio di ogni sessione per compensare le variazioni nell'orientamento di montaggio del sensore sul polso e stabilire la posizione neutra di riferimento per il calcolo degli angoli. La procedura di calibrazione è guidata dal firmware attraverso una sequenza di feedback visivi tramite un piccolo LED integrato nell'involucro del wristband: l'utente mantiene il polso in posizione neutra per tre secondi con LED verde fisso, esegue una rotazione completa del polso attorno all'asse longitudinale con LED lampeggiante lento, e poi inclina la mano verso l'alto e verso il basso per la calibrazione dell'asse di pitch con LED lampeggiante veloce. Al termine della sequenza il firmware salva i parametri di calibrazione in memoria flash non volatile, in modo che siano disponibili anche dopo un riavvio del dispositivo senza richiedere una nuova calibrazione.

L'involucro del wristband è progettato per essere indossato comodamente sul polso durante sessioni operative di durata tipica di 30–60 minuti. È realizzato in PETG stampato in 3D con pareti di spessore 2 millimetri, ha forma di parallelepipedo arrotondato con dimensioni di circa 55×35×15 millimetri che ospita il PCB con ESP32-C3 e BNO055, la batteria Li-Po da 500 milliampereora e i connettori per la ricarica USB-C. Il fissaggio al polso è realizzato tramite fasce elastiche con chiusura a strappo, regolabili per adattarsi a polsi di diverse dimensioni, che mantengono il dispositivo in posizione stabile durante i movimenti del braccio senza stringere eccessivamente né scivolare durante i gesti rapidi. La posizione ottimale di montaggio è sul dorso del polso con l'asse longitudinale del sensore allineato all'asse dell'avambraccio, orientamento che massimizza la sensibilità ai movimenti di roll e pitch che costituiscono i gesti di controllo più utilizzati nel sistema MicroBot.

Punto 17 — Il Visore VR: hardware, software e interfaccia immersiva

Il visore VR è il componente di MicroBot che rappresenta la visione più ambiziosa del progetto — l'interfaccia attraverso cui l'utente non si limita a controllare lo swarm ma lo abita, lo percepisce nello spazio tridimensionale e interagisce con esso con la stessa naturalezza con cui si interagisce con oggetti fisici reali. Se il wristband traduce il movimento del corpo in comandi digitali, il visore inverte questa direzione trasformando lo stato digitale dello swarm in percezione sensoriale immersiva, chiudendo il ciclo tra intenzione umana e risposta fisica del sistema in un modo che nessuna interfaccia bidimensionale — uno schermo, una finestra di browser, una mappa di LED — è in grado di replicare. La progettazione del visore VR prototipale di MicroBot affronta questa sfida con l'approccio caratteristico dell'intero progetto: massimizzare le capacità funzionali entro i vincoli stringenti della produzione artigianale, della stampa 3D e dei componenti commerciali accessibili, senza rinunciare alla solidità ingegneristica che distingue un prototipo di ricerca da un oggetto dimostrativo.

Il telaio del visore è progettato interamente per la produzione tramite stampa 3D in PETG, un materiale preferito al PLA per questa applicazione per le sue proprietà meccaniche superiori a temperature moderate — il PETG mantiene la propria rigidità fino a circa 70–80 gradi centigradi contro i 50–60 del PLA — e per la sua migliore resistenza agli urti e alla flessione ciclica che si producono nelle zone di fissaggio delle fasce elastiche durante l'indossaggio e la rimozione ripetuta del dispositivo. Le dimensioni esterne del telaio sono di 165 millimetri in larghezza, 85 millimetri in altezza e 95 millimetri in profondità, valori che derivano dal compromesso tra la necessità di ospitare le lenti ottiche con la corretta distanza focale e la necessità di contenere il peso e l'ingombro del dispositivo entro limiti compatibili con un utilizzo prolungato senza affaticamento cervicale. Le pareti del telaio hanno uno spessore di 2 millimetri, sufficiente a garantire la rigidità strutturale necessaria a mantenere l'allineamento delle lenti rispetto agli occhi dell'utente durante l'uso, con rinforzi aggiuntivi nelle zone di montaggio delle lenti e nella zona di collegamento con le fasce di fissaggio laterali.

Il sistema ottico del visore è basato su lenti biconvesse da 40 millimetri di diametro, una per occhio, che fungono da oculari di ingrandimento per la visualizzazione degli schermi o dei display posizionati nella parte anteriore del telaio. La distanza focale delle lenti — tipicamente 40–50 millimetri per lenti biconvesse di questo diametro — determina la distanza ottimale tra le lenti e gli schermi, che nel visore prototipale è compresa tra 35 e 45 millimetri, regolabile attraverso una guida a scorrimento che permette la messa a fuoco personalizzata per utenti con diversi gradi di miopia o ipermetropia leggera. La distanza interpupillare — la distanza tra i centri delle due lenti — è regolabile meccanicamente tra 58 e 68 millimetri, coprendo la variazione interpupillare della maggior parte degli utenti adulti che è tipicamente compresa tra 60 e 70 millimetri. La distanza tra le lenti e gli occhi dell'utente è di circa 12 millimetri, valore che bilancia la necessità di un campo visivo ampio — che aumenta avvicinando gli occhi alle lenti — con la necessità di spazio per gli utenti che indossano occhiali correttivi. Il campo visivo orizzontale risultante con questa configurazione ottica è di circa 90–100 gradi, valore inferiore ai visori VR commerciali di alta gamma ma sufficiente per creare un senso di immersione nella visualizzazione tridimensionale dello swarm.

La versione prototipale del visore prevede due approcci alternativi per la generazione delle immagini, selezionabili in funzione delle risorse disponibili e del livello di integrazione desiderato. Il primo approccio, più semplice e accessibile, utilizza uno smartphone come display: il telaio del visore include un alloggiamento standard per smartphone da 5–6 pollici nella parte anteriore, che posiziona lo schermo del telefono alla distanza ottimale dalle lenti. L'applicazione Unity gira direttamente sullo smartphone, che gestisce sia il rendering della scena 3D sia il tracking della testa tramite i propri sensori inerziali integrati, comunicando con il controller dello swarm tramite Wi-Fi. Questo approccio è compatibile con qualsiasi smartphone moderno dotato di accelerometro e giroscopio, e permette di prototipare rapidamente l'interfaccia VR senza sviluppare hardware dedicato. Il secondo approccio, più integrato e tecnicamente più interessante, utilizza display OLED da 2.9 pollici con risoluzione 1440×1440 pixel per occhio — componenti disponibili come spare parts per visori VR commerciali — collegati a un ESP32-S3 che gestisce il rendering della scena e la comunicazione con il controller. Questo approccio richiede uno sviluppo hardware e firmware più complesso ma produce un dispositivo completamente standalone, senza dipendenza da uno smartphone esterno.

Il tracking della testa è implementato tramite una IMU dedicata, distinta da quella del wristband, posizionata in un alloggiamento di 15×15×3 millimetri integrato nel telaio del visore. La scelta del sensore ricade sulla stessa BNO055 utilizzata nel wristband per le ragioni già discusse — accuratezza degli angoli di Eulero, fusione sensoriale interna, consumo energetico contenuto. Il tracking della testa a sei gradi di libertà — tre gradi di rotazione e tre gradi di traslazione — è fondamentale per l'esperienza VR: senza tracking di rotazione la testa virtuale non segue i movimenti della testa reale e l'esperienza immersiva collassa immediatamente. Il tracking di traslazione — la rilevazione dei movimenti lineari della testa nello spazio — è più complesso da implementare con una sola IMU, perché richiede l'integrazione doppia dell'accelerazione lineare che accumula errori rapidamente, ed è per questa ragione che nella versione prototipale 0.1 il tracking di traslazione è approssimato o assente, con un impatto limitato sull'esperienza complessiva dato che i movimenti di traslazione della testa durante l'uso normale del visore sono tipicamente piccoli rispetto ai movimenti di rotazione.

La mini camera integrata nel visore ha un campo visivo di 120 gradi e dimensioni fisiche di 25×25×25 millimetri, ed è posizionata nella parte anteriore del telaio in modo da riprendere l'ambiente esterno nella direzione in cui l'utente sta guardando. Questa camera svolge due funzioni distinte nel sistema MicroBot. La prima funzione è il tracking esterno delle mani: il flusso video della camera viene elaborato da MediaPipe Hands in esecuzione sullo smartphone o sull'ESP32-S3, rilevando la posizione delle mani dell'utente nello spazio davanti al visore e permettendo il riconoscimento delle gesture in realtà aumentata — l'utente vede le proprie mani sovrapposte alla visualizzazione dello swarm e può interagire con i bot virtuali attraverso gesti naturali. La seconda funzione è la visualizzazione in realtà aumentata dello swarm fisico reale: il flusso video della camera viene combinato con la rappresentazione digitale dei bot in tempo reale, sovrapposto ai MicroBot fisici nella scena ripresa dalla camera, permettendo all'utente di vedere simultaneamente la realtà fisica e l'overlay digitale con le informazioni di stato di ogni bot — livello di carica, stato dei magneti, pattern assegnato.

Il software del visore è sviluppato in Unity 3D con C# e sfrutta l'ecosistema XR Toolkit di Unity per la gestione del tracking della testa, il rendering stereoscopico e il riconoscimento delle interazioni. La scena Unity che rappresenta lo swarm è strutturata come un ambiente tridimensionale in cui ogni MicroBot è rappresentato da un oggetto 3D — un modello semplificato del modulo fisico con la sua geometria a doppio cono — il cui colore, luminosità e animazione riflettono in tempo reale lo stato del bot corrispondente nel sistema fisico. Lo stato dei bot viene ricevuto dal controller tramite socket UDP, deserializzato e applicato agli oggetti Unity attraverso un sistema di aggiornamento a 30 hertz che garantisce la fluidità della visualizzazione senza saturare la banda della connessione Wi-Fi. L'interfaccia utente del visore include un menu di selezione del pattern visualizzato come pannello tridimensionale fluttuante nel campo visivo dell'utente, selezionabile tramite il gesto di pinch rilevato dalla camera o tramite i pulsanti fisici del wristband, e un dashboard di stato dello swarm che mostra in forma aggregata il numero di bot connessi, il pattern corrente, la media del livello di carica delle batterie e eventuali avvisi del safety layer.

La comunicazione tra il visore e il controller avviene tramite socket UDP su rete Wi-Fi, con il visore connesso alla rete SoftAP creata dal controller come stazione client. Il visore invia al controller i comandi di pattern generati dall'interfaccia utente — selezione del pattern,

modifica dei parametri geometrici, attivazione della modalità di emergenza — e riceve dal controller il flusso di aggiornamenti dello stato globale dello swarm. La latenza di questa connessione Wi-Fi si aggiunge alla latenza della catena di controllo complessiva, con un contributo tipico dell'ordine di 5–15 millisecondi in una rete locale senza congestione, compatibile con i requisiti di responsività dell'interfaccia VR. La sincronizzazione temporale tra il visore e il controller è gestita con lo stesso meccanismo di timestamp già descritto per i MicroBot, garantendo che la rappresentazione virtuale dello swarm nel visore sia temporalmente coerente con lo stato fisico reale dei bot.

L'alimentazione del visore è garantita da una cella 18650 con alloggiamento di 18×18×68 millimetri integrato nel telaio, oppure da una Li-Po da 1200 milliampereora nella variante con form factor più compatto. La cella 18650 è preferita per le applicazioni che richiedono autonomia prolungata — fornisce tipicamente 2600–3500 milliampereora a 3.7 volt nominali, sufficienti per due o tre ore di utilizzo continuo del visore — mentre la Li-Po da 1200 milliampereora è preferita quando la compattezza e il peso ridotto sono prioritari rispetto all'autonomia. Il circuito di gestione dell'alimentazione del visore include un modulo di ricarica compatibile con la tipologia di batteria scelta, un regolatore di tensione che alimenta l'ESP32-S3 e i display a 3.3 volt e un convertitore boost che alimenta i display OLED alla loro tensione operativa di 5 volt. Il consumo totale del visore in condizioni operative normali — ESP32-S3 in trasmissione Wi-Fi attiva, due display OLED a luminosità media, camera attiva, IMU in modalità di aggiornamento a 100 hertz — è stimato nell'ordine di 400–600 milliampere a 3.7 volt, producendo un'autonomia di circa 2–3 ore con la cella 18650 standard.

Punto 18 — Sistema di autenticazione biometrica facciale

Il sistema di autenticazione biometrica facciale è il componente di MicroBot che presidia l'accesso al controller dello swarm, garantendo che solo utenti autorizzati possano inviare comandi ai MicroBot fisici. La sua presenza nell'architettura del sistema non è un elemento decorativo né una dimostrazione di capacità tecniche accessorie al progetto principale — è una scelta progettuale che riflette una consapevolezza precisa: un sistema di swarm robotics fisico che può aggregare, disaggregare e riconfigurare oggetti nello spazio reale è intrinsecamente un sistema che può causare danni se controllato da utenti non autorizzati o se attivato accidentalmente. Il cancello di autenticazione biometrica introduce una barriera di sicurezza che è al tempo stesso robusta — difficilmente aggirabile senza la presenza fisica dell'utente autorizzato — e trasparente per l'utente legittimo — il riconoscimento avviene in pochi secondi senza richiedere password, PIN o dispositivi fisici aggiuntivi. Questa combinazione di sicurezza e usabilità è il motivo per cui la biometria facciale è stata preferita ad altri meccanismi di autenticazione come la password testuale o il token hardware.

Il sistema è implementato come modulo software indipendente, sviluppato in Java con le librerie OpenCV per la visione artificiale, e opera in esecuzione separata rispetto al resto dell'interfaccia utente. Questa separazione implementativa non è accidentale: il modulo di autenticazione deve poter operare come un guardiano autonomo che non può essere bypassato o disabilitato da altri componenti del sistema, e la sua implementazione come processo indipendente con comunicazione esplicita verso il controller garantisce questa proprietà. Il modulo si avvia automaticamente all'accensione del sistema, prima che qualsiasi altro componente dell'interfaccia utente venga inizializzato, e il controller non

accetta comandi di pattern dallo strato software superiore fino a quando il modulo di autenticazione non ha emesso un token di autorizzazione valido. Il token ha una durata configurabile — tipicamente 30–60 minuti — trascorsa la quale il sistema richiede una nuova autenticazione, garantendo che una sessione lasciata incustodita non rimanga indefinitamente accessibile.

La pipeline di autenticazione si articola in sei fasi sequenziali che trasformano il flusso video grezzo della telecamera in una decisione binaria di accesso concesso o negato. La prima fase è l'acquisizione del flusso video dalla telecamera collegata al sistema, tipicamente una webcam USB con risoluzione minima di 640×480 pixel e frequenza di aggiornamento di 30 fotogrammi al secondo. Il flusso viene acquisito tramite la classe VideoCapture di OpenCV e pre-elaborato per normalizzare la luminosità e il contrasto attraverso l'equalizzazione dell'istogramma sull'immagine in scala di grigi, una tecnica che migliora la robustezza del rilevamento facciale in condizioni di illuminazione variabile riducendo la sensibilità alle variazioni dell'esposizione della telecamera.

La seconda fase è il rilevamento del volto nell'immagine pre-elaborata, implementato tramite il classificatore a cascata di Viola-Jones basato sulle feature di Haar, disponibile nelle librerie OpenCV come file di configurazione XML pre-addestrato. L'algoritmo di Viola-Jones scansiona l'immagine a diverse scale utilizzando una finestra scorrevole e valuta in ogni posizione un insieme di feature rettangolari — differenze di intensità tra regioni adiacenti dell'immagine — attraverso una cascata di classificatori deboli che rigetta rapidamente le regioni non contenenti volti e concentra il calcolo sulle regioni candidate. La complessità computazionale ridotta di questo algoritmo — dell'ordine di pochi millisecondi per frame su hardware desktop standard — lo rende adatto all'elaborazione in tempo reale del flusso video senza richiedere accelerazione hardware dedicata. Il risultato della fase di rilevamento è un insieme di bounding box rettangolari che identificano le regioni dell'immagine contenenti volti, caratterizzate dalle coordinate del vertice superiore sinistro, dalla larghezza e dall'altezza del rettangolo.

La terza fase è il rilevamento dei landmark facciali nel volto rilevato, implementato attraverso un modello di regressione addestrato che predice la posizione di 68 punti caratteristici del volto — angoli degli occhi, sopracciglia, punta e narici del naso, angoli e contorno delle labbra, profilo del mento e delle orecchie — a partire dalla regione dell'immagine contenente il volto. Per applicazioni che richiedono maggiore precisione e sono disposte a sopportare un costo computazionale superiore, è possibile utilizzare il modello MediaPipe Face Landmarker che predice 468 landmark con accuratezza superiore e robustezza maggiore alle variazioni di posa. I landmark rilevati costituiscono la mappa geometrica del volto che viene utilizzata nella fase successiva per l'estrazione del vettore di caratteristiche biometriche.

La quarta fase è l'estrazione del vettore di caratteristiche biometriche dal volto rilevato e normalizzato. Le distanze euclidee tra coppie significative di landmark — distanza tra gli angoli degli occhi, rapporto tra larghezza e altezza del volto, distanza tra la punta del naso e il mento, proporzioni delle labbra — vengono calcolate e normalizzate rispetto alla dimensione complessiva del volto per produrre un vettore di caratteristiche invariante rispetto alla distanza dell'utente dalla telecamera. Questo vettore di caratteristiche, tipicamente di dimensione 128–512 elementi a seconda del modello di estrazione utilizzato,

rappresenta la firma biometrica del volto in uno spazio vettoriale ad alta dimensione in cui volti simili producono vettori simili e volti diversi producono vettori distanti.

La quinta fase è il confronto del vettore estratto con i template biometrici degli utenti autorizzati registrati nel database del sistema. Il confronto è implementato attraverso due metriche complementari: la distanza euclidea tra il vettore estratto e ogni template nel database, espressa come $D = \sqrt{\sum (f_i - g_i)^2}$, e la similarità coseno tra gli stessi vettori, espressa come $\cos(\theta) = (F \cdot G) / (|F| |G|)$. La decisione di autenticazione è basata sulla combinazione pesata di queste due metriche: l'utente viene autenticato se la distanza euclidea con il template più simile nel database è inferiore alla soglia di accettazione τ_D e la similarità coseno è superiore alla soglia $\tau_{\cos}$. La scelta della soglia di accettazione determina il bilanciamento tra il False Acceptance Rate — la probabilità che un impostore venga erroneamente autenticato — e il False Rejection Rate — la probabilità che un utente legittimo venga erroneamente rifiutato. Il punto di Equal Error Rate, in cui i due tassi di errore sono uguali, rappresenta il punto di bilanciamento ottimale per applicazioni in cui i due tipi di errore hanno costi comparabili.

La sesta fase è la verifica anti-spoofing, che distingue un volto reale presente fisicamente di fronte alla telecamera da una fotografia o da un video riprodotto su schermo per ingannare il sistema. Il liveness detection è implementato attraverso tre meccanismi complementari. Il primo è l'analisi del movimento dei landmark facciali nel tempo: un volto reale produce movimenti naturali — microsaccadi degli occhi, piccoli movimenti della testa, variazioni dell'espressione — che non sono presenti in una fotografia statica e sono difficilmente replicabili in un video pre-registrato in modo convincente. Il secondo meccanismo è l'analisi della texture dell'immagine attraverso i Local Binary Pattern: la texture di un volto reale illuminato dalla luce ambiente differisce dalla texture di una fotografia stampata o riprodotta su schermo in modo rilevabile attraverso l'analisi statistica dei pattern locali di intensità. Il terzo meccanismo, disponibile quando la telecamera fornisce informazioni di profondità o quando sono presenti due telecamere in configurazione stereo, è la verifica della tridimensionalità del volto attraverso l'analisi della disparità tra le immagini delle due telecamere.

Il database dei template biometrici è gestito come file cifrato sul filesystem del sistema, con cifratura AES-256 della chiave simmetrica a sua volta protetta tramite RSA con chiave pubblica dell'utente amministratore. La registrazione di un nuovo utente autorizzato richiede l'acquisizione di un insieme di immagini del volto dell'utente in diverse condizioni di illuminazione e con diverse espressioni facciali — tipicamente 20–50 immagini — dal quale vengono estratti i vettori di caratteristiche biometriche che vengono mediati e normalizzati per produrre il template di riferimento. Il template viene aggiornato incrementalmente ad ogni autenticazione riuscita attraverso la formula $T_{\text{updated}} = \alpha \cdot T_{\text{old}} + (1 - \alpha) \cdot T_{\text{new}}$, dove α è un parametro di aggiornamento tipicamente impostato a 0.9 che garantisce una lenta adattamento del template alle variazioni graduali dell'aspetto dell'utente — invecchiamento, cambiamenti di acconciatura, variazioni del peso — senza renderlo vulnerabile ad attacchi di adattamento graduale.

Il modulo di autenticazione implementa anche un meccanismo di logging degli accessi che registra ogni tentativo di autenticazione — riuscito o fallito — con timestamp, immagine del volto rilevato e decisione del sistema. Questo log è accessibile solo all'utente amministratore

tramite interfaccia dedicata protetta da password, e serve sia come strumento di audit per verificare chi ha avuto accesso al sistema in una determinata sessione sia come strumento di debug per analizzare i casi di falso rifiuto e regolare le soglie di accettazione in modo empirico basandosi sulla distribuzione reale delle distanze biometriche osservate nel contesto operativo specifico del sistema.

Punto 19 — Il framework 4D/5D: stato interno, suono e modulazione dello swarm

Il framework 4D/5D è l'elemento concettualmente più originale di MicroBot — quello che più di ogni altro distingue il progetto dai sistemi di swarm robotics convenzionali e lo posiziona in un territorio di ricerca che non ha precedenti diretti nella letteratura del settore.

Comprendere il framework 4D/5D richiede di abbandonare temporaneamente il linguaggio dell'ingegneria embedded e della robotica distribuita per adottare una prospettiva più teorica, che riguarda la natura dello spazio in cui i MicroBot operano e il tipo di informazione che può essere usata per modularli. L'idea fondante è semplice nella sua formulazione ma ricca di implicazioni pratiche: il comportamento dello swarm non è completamente descritto dalle posizioni e dalle velocità dei bot nello spazio tridimensionale fisico, ma dipende da uno stato interno aggiuntivo — una quarta dimensione W — che non è direttamente osservabile dall'esterno ma che governa le trasformazioni geometriche della struttura collettiva e la sua risposta agli input dell'utente. La quinta dimensione è la variazione temporale di questo stato interno — $W(t)$ — che conferisce al sistema una memoria dinamica capace di produrre comportamenti diversi in risposta agli stessi input fisici a seconda della storia passata del sistema.

La motivazione per introdurre questo livello di astrazione aggiuntivo non è puramente teorica ma nasce da un'osservazione pratica: i sistemi di controllo degli swarm convenzionali basati esclusivamente su coordinate spaziali e velocità producono comportamenti che, per quanto complessi, sono fondamentalmente reattivi e privi di contesto. Un bot che riceve un comando di movimento verso una posizione target si muove verso quella posizione indipendentemente da tutto ciò che è accaduto in precedenza — dalla forma che lo swarm aveva un secondo prima, dall'intensità sonora dell'ambiente, dall'umore dell'utente inferibile dalla sua espressione facciale. Il framework 4D/5D introduce un meccanismo attraverso cui queste informazioni contestuali possono influenzare il comportamento dello swarm in modo continuo e naturale, producendo un sistema che risponde non solo a dove l'utente vuole che i bot vadano ma a come vuole che si muovano, con quale fluidità, con quale intensità, con quale ritmo.

La quarta dimensione W è formalmente definita come uno scalare o un vettore di bassa dimensionalità — tipicamente due o quattro componenti — che parametrizza la famiglia di trasformazioni geometriche applicabili al pattern corrente dello swarm. Nella formulazione più semplice, W è uno scalare compreso tra zero e uno che interpola continuamente tra due comportamenti estremi del sistema: per W uguale a zero lo swarm è completamente fluido — i bot si muovono con transizioni morbide e gradualità, le forze di aggancio magnetico sono al minimo, la rigidità dell'aggregato è bassa — mentre per W uguale a uno lo swarm è completamente solido — le transizioni sono rapide e precise, le forze di aggancio sono al massimo, la rigidità dell'aggregato è alta. I valori intermedi di W producono comportamenti intermedi, con una transizione continua tra i due estremi che può essere controllata in tempo reale. In formulazioni più ricche, W diventa un vettore bidimensionale le cui due componenti

controllano indipendentemente la fluidità delle transizioni di posizione e la rigidità degli agganci magnetici, permettendo combinazioni come uno swarm fluido nel movimento ma rigido negli agganci, o uno swarm rigido nel movimento ma capace di disagganciarsi rapidamente su comando.

La quinta dimensione è la dimensione temporale dello stato interno — $W(t)$ — che trasforma W da un parametro statico configurabile dall'utente in una traiettoria dinamica nello spazio degli stati interni del sistema. Questa traiettoria non è necessariamente deterministica: può essere guidata da input esterni come i parametri acustici dell'ambiente, può evolvere seguendo dinamiche proprie del sistema come oscillatori accoppiati o attrattori caotici, oppure può essere il risultato di una combinazione di questi meccanismi. La traiettoria $W(t)$ conferisce al sistema una memoria temporale: il comportamento dello swarm in un dato istante dipende non solo dallo stato corrente di W ma dalla direzione e dalla velocità con cui W sta cambiando, producendo effetti di inerzia e anticipazione che rendono il comportamento del sistema più naturale e prevedibile per l'utente.

Il meccanismo attraverso cui i parametri acustici dell'ambiente vengono mappati sullo stato interno $W(t)$ è il cuore operativo del framework. Il segnale audio acquisito dal microfono del sistema — che può essere la voce dell'utente, la musica dell'ambiente, suoni generati specificamente per controllare il sistema o qualsiasi altra sorgente sonora — viene analizzato in tempo reale attraverso una pipeline di elaborazione del segnale che estrae un insieme di parametri acustici significativi. Il primo parametro è l'intensità istantanea del segnale, calcolata come radice quadrata della media dei quadrati dei campioni in una finestra temporale scorrevole di 50–100 millisecondi, che rappresenta l'energia sonora dell'ambiente in quel momento e viene mappata tipicamente sull'ampiezza delle oscillazioni di $W(t)$. Il secondo parametro è la frequenza fondamentale del segnale, estratta attraverso l'algoritmo di autocorrelazione o attraverso la trasformata di Fourier rapida applicata alla finestra temporale corrente, che rappresenta il tono dominante del suono e viene mappata tipicamente sulla frequenza delle oscillazioni di $W(t)$. Il terzo parametro è il centroide spettrale — la media ponderata delle frequenze presenti nel segnale pesate per la loro ampiezza — che rappresenta la brillantezza o la durezza del timbro sonoro e viene mappata tipicamente sulla componente di rigidità del vettore W . Il quarto parametro è il ritmo, estratto attraverso un algoritmo di beat detection che identifica i picchi periodici di energia nel segnale, che viene mappato sulla frequenza dei cambi di pattern del sistema producendo uno swarm che si riconfigura spontaneamente in sincronia con il ritmo della musica ambiente.

La mappatura tra parametri acustici e componenti del vettore W non è fissa ma configurabile dall'utente attraverso una matrice di mappatura che specifica il peso con cui ogni parametro acustico contribuisce a ogni componente di W . Questa configurabilità permette di adattare il comportamento del sistema a contesti operativi diversi — una performance artistica interattiva in cui il suono guida completamente il comportamento dello swarm, un laboratorio di ricerca in cui il suono è uno tra molti input di controllo, un'installazione pubblica in cui il suono dell'ambiente trasforma spontaneamente la struttura dello swarm senza intervento esplicito dell'utente — senza richiedere modifiche al firmware o al codice del sistema. La matrice di mappatura è memorizzata come file di configurazione JSON sul controller e può essere modificata in tempo reale tramite l'interfaccia utente del visore o tramite comandi inviati dal livello di interfaccia software.

L'implementazione del framework 4D/5D nel motore di simulazione e nel firmware del controller richiede l'introduzione di termini aggiuntivi nelle equazioni del modello matematico dello swarm. Il vettore $W(t)$ entra direttamente nei parametri delle forze che governano la dinamica dei bot: la costante di rigidità del campo di attrazione verso il target k_a diventa una funzione di W espressa come $k_a(W) = k_{a,min} + W \cdot (k_{a,max} - k_{a,min})$, dove $k_{a,min}$ e $k_{a,max}$ sono i valori minimo e massimo della costante di rigidità corrispondenti rispettivamente agli stati completamente fluido e completamente solido. Analogamente, la costante magnetica k_m e il duty cycle massimo delle coil diventano funzioni di W che aumentano con l'aumentare del valore del parametro. La frequenza di aggiornamento del piano di transizione dei pattern diventa una funzione della componente di ritmo di W , aumentando o diminuendo in sincronia con il beat detection del segnale audio. Il risultato di queste modificazioni è un sistema in cui lo stato interno $W(t)$ — guidato dal suono — modula in modo continuo e coerente tutti i parametri comportamentali dello swarm simultaneamente, producendo un effetto di risonanza tra il suono e il comportamento fisico dei robot che è percepibile e sorprendente anche per osservatori che non conoscono il meccanismo sottostante.

Il framework 4D/5D ha implicazioni che si estendono ben oltre la semplice modulazione dei parametri di controllo. Introduce nel sistema MicroBot la possibilità di comportamenti emergenti di secondo ordine — comportamenti del comportamento, per così dire — in cui la traiettoria temporale di $W(t)$ produce sequenze di configurazioni dello swarm che non sono mai state esplicitamente programmate ma emergono dall'interazione tra la dinamica di W e la fisica del sistema. Un esempio concreto: se $W(t)$ oscilla sinusoidalmente con una frequenza di 0.5 hertz guidata dal ritmo di un brano musicale, e la costante di rigidità k_a è modulata da W , lo swarm alterna periodicamente tra una configurazione fluida in cui i bot si disperdono seguendo le forze di Boids e una configurazione rigida in cui si compattano verso il pattern target. Il risultato visivo è uno swarm che respira in sincronia con la musica — si espande e si contrae ritmicamente come un organismo vivente — un comportamento emergente che nessuna delle regole programmate esplicitamente produce da sola ma che emerge dalla loro interazione con la modulazione temporale di W .

La validazione sperimentale del framework 4D/5D richiede la definizione di metriche quantitative che permettano di misurare l'effetto dello stato interno W sui parametri osservabili del comportamento dello swarm. La metrica principale adottata in MicroBot è la varianza temporale della configurazione dello swarm — calcolata come la somma delle varianze delle distanze tra i bot nelle diverse componenti della posizione — che aumenta quando W è basso e lo swarm è fluido e diminuisce quando W è alto e lo swarm è rigido. Una seconda metrica è il tempo di convergenza verso il pattern target a partire da una configurazione casuale, che diminuisce al crescere di W riflettendo la maggiore rigidità e reattività del sistema. Una terza metrica è la correlazione tra il ritmo del segnale audio e la frequenza dei cambi di configurazione del sistema, che dovrebbe essere elevata quando la componente di beat detection di W è attiva e vicina a zero quando è disattivata. Queste metriche possono essere misurate automaticamente dalla simulazione JavaScript e dal sistema di logging del controller, producendo dati quantitativi che permettono di calibrare la matrice di mappatura e di verificare che il framework si comporti come atteso in condizioni operative reali.

Punto 20 — Simulazione JavaScript e motore di visualizzazione dello swarm

La simulazione JavaScript è il laboratorio virtuale di MicroBot — il luogo in cui gli algoritmi di coordinamento collettivo vengono sviluppati, testati e raffinati prima di essere tradotti in firmware embedded, in cui i pattern geometrici vengono visualizzati e validati senza richiedere hardware fisico, e in cui l'intera logica del sistema può essere esplorata interattivamente con la velocità di iterazione che solo un ambiente software puro può offrire. La sua importanza nel progetto non è subordinata alla fase di sviluppo hardware — non è semplicemente uno strumento provvisorio destinato a essere abbandonato quando i MicroBot fisici saranno disponibili — ma è un componente permanente dell'architettura del sistema che continuerà a svolgere funzioni specifiche anche nelle versioni più mature del progetto: il test di nuovi algoritmi prima del deployment sul firmware, la visualizzazione dello stato dello swarm su interfacce web accessibili senza visore VR, la simulazione di scenari con numeri di bot superiori a quelli fisicamente disponibili, e la generazione di dati di training per algoritmi di apprendimento automatico che verranno introdotti nelle versioni successive.

La simulazione è implementata in JavaScript puro con rendering su elemento Canvas HTML5, senza dipendenze da framework grafici esterni o motori di gioco. Questa scelta è deliberata e riflette la filosofia di controllo completo sulla catena di esecuzione che caratterizza l'intero progetto: una dipendenza da Three.js o da altri framework grafici introdurrebbe strati di astrazione che nascondono il comportamento del rendering e complicano il debug delle interazioni tra la logica degli algoritmi e la visualizzazione. Il Canvas HTML5 espone un'API di drawing bidimensionale sufficientemente ricca per produrre una visualizzazione chiara e informativa dello swarm, e la sua integrazione nativa nel browser elimina qualsiasi problema di compatibilità con ambienti diversi. La simulazione gira direttamente nel browser senza installazione, è accessibile da qualsiasi dispositivo connesso alla rete locale del sistema e non richiede server backend dedicati per le sessioni di test degli algoritmi.

Il motore di simulazione è strutturato come un ciclo di aggiornamento principale — il game loop — eseguito a frequenza fissa tramite la funzione `requestAnimationFrame` del browser, che sincronizza l'esecuzione con la frequenza di aggiornamento del display — tipicamente 60 hertz — producendo un rendering fluido senza sprechi computazionali. Ad ogni iterazione del game loop il motore esegue tre fasi sequenziali: la fase di aggiornamento della fisica, in cui le forze che agiscono su ogni bot vengono calcolate e integrate per produrre le nuove posizioni e velocità; la fase di aggiornamento dello stato, in cui le condizioni di aggancio e disaggancio magnetico vengono valutate, i cambi di stato dei bot vengono applicati e il safety layer virtuale viene verificato; e la fase di rendering, in cui la configurazione corrente dello swarm viene disegnata sul canvas con tutte le informazioni di stato visibili all'utente.

La fase di aggiornamento della fisica implementa il modello matematico descritto nel punto 9 con un integratore numerico di Euler del primo ordine, scelto per la sua semplicità implementativa e la sua trasparenza computazionale a scapito della precisione numerica assoluta. Per un sistema come MicroBot, in cui le forze sono continue e ben comportate e il passo temporale di simulazione è dell'ordine di 16 millisecondi — corrispondente a 60 hertz — l'integratore di Euler produce risultati sufficientemente accurati per la validazione degli algoritmi, con errori di troncamento che non si accumulano in modo significativo nell'arco temporale tipico di una sessione di simulazione. Per le versioni future che richiedono simulazioni più lunghe o con dinamiche più rapide, è previsto il passaggio a un integratore di Runge-Kutta del quarto ordine che offre una precisione significativamente superiore a parità

di passo temporale. Il calcolo delle forze è ottimizzato attraverso una struttura dati di partizionamento spaziale — una griglia uniforme in cui ogni cella contiene i riferimenti ai bot che si trovano nella propria area — che riduce la complessità del calcolo delle interazioni tra bot da $O(n^2)$ a $O(n \cdot k)$, dove k è il numero medio di bot in ciascuna cella, permettendo la simulazione efficiente di swarm con centinaia di bot in tempo reale nel browser.

La fase di rendering produce una visualizzazione ricca di informazioni che va ben oltre la semplice rappresentazione dei bot come punti sullo schermo. Ogni bot è disegnato come un cerchio con raggio proporzionale alla propria massa simulata, colorato secondo il proprio stato operativo corrente con lo stesso schema cromatico del LED RGB fisico — blu per idle, ciano per movimento, verde per pattern stabile, giallo per batteria in esaurimento, rosso per emergenza. Attorno a ogni bot vengono disegnati cerchi concentrici semitrasparenti che indicano visivamente le soglie di aggancio e disaggancio — il cerchio interno di raggio d_{agg} e il cerchio esterno di raggio d_{dist} — permettendo di osservare direttamente quando due bot entrano nella zona di isteresi e come si risolve la loro interazione. Le connessioni magnetiche attive tra bot agganciati vengono visualizzate come linee la cui larghezza è proporzionale alla forza dell'aggancio e il cui colore varia dal blu tenue per agganci deboli al rosso intenso per agganci a piena potenza. Le forze che agiscono su ogni bot vengono visualizzate come frecce vettoriali — colorate per tipo di forza, con lunghezza proporzionale alla magnitudine — che rendono immediatamente visibile la composizione delle interazioni in ogni punto dello swarm. Le posizioni target del pattern corrente vengono visualizzate come croci o asterischi nella posizione assegnata a ciascun bot, con una linea tratteggiata che collega ogni bot alla propria posizione target, rendendo visibile il processo di convergenza verso il pattern in tempo reale.

Il pannello di controllo della simulazione è implementato come un overlay HTML sovrapposto al canvas, che espone all'utente tutti i parametri configurabili del sistema attraverso controlli interattivi. Il selettore di pattern permette di scegliere tra tutti i pattern implementati — LINE, WAVE, BOIDS, VORTEX, CIRCLE, FIGURE-8, SPIRAL, SQUARE, FLOW FIELD — e di modificare i parametri geometrici di ciascuno tramite slider numerici: raggio del cerchio, ampiezza e lunghezza d'onda del pattern a onda, parametro di passo della spirale, dimensione del lato del quadrato. Il pannello dei parametri fisici espone i parametri del modello matematico — costanti di forza $k_{\square}$, $k_{\square}$, k_r , k_a , soglie di aggancio e distacco, parametro di smorzamento della velocità — modificabili in tempo reale durante la simulazione per osservare immediatamente l'effetto delle variazioni sul comportamento del sistema. Il pannello del framework 4D/5D espone il valore corrente del vettore W e i parametri della matrice di mappatura dal segnale audio a W , con la possibilità di attivare o disattivare la modulazione acustica e di impostare manualmente il valore di W tramite slider per testare l'effetto di diversi stati interni sul comportamento dello swarm indipendentemente dall'input sonoro.

Il generatore di scenari di test è un componente della simulazione che permette di creare condizioni iniziali controllate per la validazione sistematica degli algoritmi. Permette di posizionare un numero arbitrario di bot in configurazioni iniziali predefinite — griglia uniforme, distribuzione casuale, cluster, configurazione specifica definita dall'utente — e di registrare l'evoluzione del sistema nel tempo producendo metriche quantitative come il tempo di convergenza verso il pattern target, la varianza della configurazione finale rispetto alla configurazione target ideale, il numero di agganci effettuati e disagganci indesiderati

durante la transizione, e l'energia totale dissipata dalle coil simulate. Queste metriche vengono registrate in un log testuale scaricabile che permette l'analisi comparativa di diverse configurazioni di parametri attraverso strumenti esterni come Python con NumPy e Matplotlib, chiudendo il ciclo tra la simulazione nel browser e l'analisi quantitativa offline.

La comunicazione tra la simulazione JavaScript e il controller fisico reale è implementata tramite WebSocket, un protocollo di comunicazione bidirezionale full-duplex che opera su connessione TCP persistente e permette l'invio di messaggi in entrambe le direzioni senza il polling continuo che caratterizza le connessioni HTTP tradizionali. Il controller espone un server WebSocket sulla propria rete SoftAP alla quale la simulazione si connette quando è in modalità di controllo hardware reale. In questa modalità la simulazione non si limita a calcolare posizioni virtuali ma invia i comandi di pattern calcolati al controller fisico attraverso il WebSocket, riceve dal controller gli aggiornamenti dello stato reale dello swarm — posizioni stimate, stati delle batterie, temperature delle coil — e li fonde con lo stato simulato per produrre una visualizzazione ibrida che mostra simultaneamente la configurazione target simulata e la configurazione reale misurata, rendendo immediatamente visibile il gap tra il comportamento ideale del modello e il comportamento reale del sistema fisico. Questo confronto diretto tra simulazione e realtà è uno degli strumenti più potenti per la calibrazione dei parametri del modello fisico: la differenza sistematica tra posizione simulata e posizione reale rivela quali aspetti del modello sono imprecisi — attrito sottostimato, forze magnetiche sovrastimate, latenze di comunicazione non modellate — e guida il processo di raffinamento iterativo del modello verso una rappresentazione sempre più fedele della realtà fisica.

La simulazione implementa anche una modalità di replay che permette di riprodurre sessioni operative precedenti — caricate da file di log generati dal controller — con velocità variabile da 0.1x a 10x rispetto al tempo reale, permettendo l'analisi dettagliata di eventi interessanti o anomali che si sono verificati durante sessioni fisiche reali. In modalità replay tutti i controlli di visualizzazione sono attivi — è possibile abilitare o disabilitare la visualizzazione delle forze, delle soglie magnetiche, dei vettori velocità — e l'utente può mettere in pausa la riproduzione in qualsiasi istante per esaminare in dettaglio la configurazione del sistema in quel preciso momento, misurare distanze tra bot, verificare lo stato degli agganci e confrontare la configurazione osservata con il pattern target teorico. Questa capacità di analisi post-hoc delle sessioni operative è indispensabile per comprendere il comportamento del sistema in condizioni al limite — agganci multipli simultanei, transizioni di pattern durante disagganci parziali, comportamenti anomali durante la scarica della batteria — che sono difficili da osservare in tempo reale e impossibili da riprodurre in modo controllato senza un sistema di logging accurato.

Punto 21 — Algoritmi di coordinamento collettivo e formazione dei pattern

Gli algoritmi di coordinamento collettivo sono il cuore computazionale di MicroBot — il luogo in cui la matematica dello swarm si trasforma in comportamento fisicamente osservabile, in cui le equazioni del modello dinamico diventano traiettorie reali di robot reali che si muovono su una superficie reale. Progettare questi algoritmi significa affrontare un insieme di problemi che appartengono simultaneamente alla teoria del controllo, all'ottimizzazione combinatoria, alla geometria computazionale e all'informatica distribuita: come assegnare ogni bot alla posizione target che minimizza la distanza totale percorsa durante la transizione, come orchestrare il movimento di tutti i bot in modo che non si verifichino collisioni durante la

riconfigurazione, come gestire le transizioni tra pattern in presenza di bot che si aggregano e disaggregano magneticamente, come garantire che il comportamento emergente dello swarm corrisponda all'intenzione dell'utente anche quando i bot fisici si discostano dalle posizioni simulate a causa di attrito, disallineamenti e latenze di comunicazione. Ciascuno di questi problemi ha una letteratura scientifica dedicata e soluzioni note in condizioni ideali — il problema di assegnazione ottimale, la pianificazione del moto senza collisioni, la teoria del controllo dei sistemi multi-agente — ma la loro combinazione in un sistema fisico real-time con vincoli energetici stringenti e comunicazione wireless inaffidabile richiede soluzioni pragmatiche che privilegiano la robustezza e la semplicità implementativa rispetto all'ottimalità teorica.

Il problema di assegnazione bot-posizioni è il primo problema computazionale che il controller deve risolvere ogni volta che viene richiesta una transizione verso un nuovo pattern. Dato un insieme di N bot con posizioni correnti note e un insieme di N posizioni target definite dal pattern desiderato, il problema consiste nel trovare la corrispondenza biunivoca tra bot e posizioni target che minimizza la somma totale delle distanze che ogni bot deve percorrere per raggiungere la propria posizione assegnata. Questo problema è formalmente equivalente al problema di assegnazione lineare, risolvibile in tempo polinomiale attraverso l'algoritmo ungherese con complessità $O(n^3)$. Per swarm di piccole dimensioni — fino a circa 20 bot — l'algoritmo ungherese viene eseguito direttamente nel firmware del controller sull'ESP32-S3, con un tempo di esecuzione tipico dell'ordine di pochi millisecondi che non introduce latenze percepibili nella risposta del sistema ai comandi dell'utente. Per swarm di dimensioni maggiori, nei quali il costo computazionale dell'algoritmo esatto diventerebbe incompatibile con i requisiti di risposta in tempo reale, viene utilizzata un'euristica greedy che assegna iterativamente ogni posizione target al bot non ancora assegnato che si trova a distanza minima da essa. L'euristica greedy non garantisce l'ottimalità della soluzione ma produce assegnazioni con una qualità tipicamente entro il 10–20% dall'ottimo, sufficiente per le applicazioni pratiche di MicroBot.

Un aspetto dell'algoritmo di assegnazione che merita attenzione specifica è la gestione dei bot già agganciati magneticamente ad altri bot al momento della richiesta di transizione. Quando due bot sono agganciati, si muovono come un corpo rigido unico e non possono essere assegnati indipendentemente a posizioni target distanti tra loro senza prima disagganciare la coppia. Il controller gestisce questa situazione attraverso una fase di pre-elaborazione che identifica i cluster di bot agganciati prima dell'esecuzione dell'algoritmo di assegnazione, tratta ogni cluster come un'unità logica con posizione pari al centroide del cluster, assegna il cluster a una sottoregione del pattern target compatibile con la sua configurazione geometrica, e genera un piano di disaggancio sequenziale che separa i bot del cluster nell'ordine ottimale per minimizzare le forze di disaggancio necessarie e il tempo di separazione. Solo dopo che il piano di disaggancio è stato eseguito con successo i singoli bot vengono assegnati individualmente alle proprie posizioni target finali nel pattern.

La pianificazione del moto senza collisioni è il secondo problema computazionale fondamentale del sistema di coordinamento. Anche con un'assegnazione ottimale delle posizioni target, i percorsi diretti dal bot alla propria posizione target possono intersecarsi, producendo collisioni fisiche durante la transizione che destabilizzerebbero l'intera configurazione. Il controller affronta questo problema attraverso un approccio in due stadi. Il primo stadio è la rilevazione preventiva delle collisioni potenziali: per ogni coppia di bot con

percorsi diretti che si intersecano, il controller calcola il tempo e il luogo dell'intersezione e identifica la coppia come potenzialmente pericolosa. Il secondo stadio è la risoluzione delle collisioni rilevate attraverso un meccanismo di priorità temporale: per ogni coppia in conflitto, il controller assegna priorità al bot che deve percorrere la distanza minore verso il proprio target — quello che arriverà prima nella zona di conflitto — e ritarda l'avvio del movimento del bot a priorità inferiore di un intervallo di tempo sufficiente a permettere al bot prioritario di attraversare la zona di conflitto prima che il secondo bot vi entri. Questo meccanismo produce sequenze di movimento in cui i bot si muovono in modo coordinato evitando le collisioni, al costo di un tempo di transizione complessivo leggermente superiore rispetto al caso ideale in cui tutti i bot si muovono simultaneamente senza mai interferire.

Il pattern LINE è il più semplice tra i pattern implementati ed è composto da N bot distribuiti uniformemente lungo un segmento di lunghezza determinata dalla distanza standard d_std tra bot adiacenti. Le posizioni target sono calcolate come $r_{i,target} = r_0 + i \cdot (d_std, 0)$ per $i = 0, 1, \dots, N-1$, dove r_0 è la posizione del primo bot nella linea e la direzione della linea è specificata dall'utente tramite l'angolo di orientamento del pattern. Il pattern LINE è il pattern di riferimento per la validazione del sistema di coordinamento perché la semplicità della sua geometria rende immediatamente visibile qualsiasi deviazione del comportamento reale dal comportamento atteso — bot che non raggiungono la propria posizione target, spaziatura non uniforme, disallineamenti angolari — e permette di isolare la causa dei problemi senza la confusione di geometrie più complesse.

Il pattern WAVE è un'estensione del pattern LINE che introduce una modulazione sinusoidale nella coordinata trasversale delle posizioni target, producendo una configurazione a onda. Le posizioni target sono calcolate come $r_{i,target} = (i \cdot d_std, A \cdot \sin(2\pi \cdot i \cdot d_std / \lambda))$, dove A è l'ampiezza dell'onda e λ è la lunghezza d'onda. Il pattern WAVE è particolarmente interessante per l'integrazione con il framework 4D/5D: quando lo stato interno $W(t)$ è modulato da un segnale audio periodico, l'ampiezza A può essere fatta variare in sincronia con il segnale, producendo un'onda che pulsa in risposta alla musica. Nella sua versione dinamica, il pattern WAVE implementa anche una propagazione temporale della fase: le posizioni target vengono aggiornate ad ogni tick temporale come $r_{i,target}(t) = (i \cdot d_std, A \cdot \sin(2\pi \cdot i \cdot d_std / \lambda - \omega \cdot t))$, dove ω è la velocità angolare di propagazione dell'onda, producendo una struttura che si propaga longitudinalmente nel tempo come un'onda meccanica visibile.

Il pattern VORTEX è uno dei pattern visivamente più spettacolari di MicroBot e implementa una rotazione coordinata dell'intero swarm attorno a un centro di rotazione configurabile. Il pattern è definito come un insieme di bot disposti su uno o più anelli concentrici attorno al centro, con posizioni target che ruotano nel tempo secondo la relazione $r_{i,target}(t) = r_centro + R_i \cdot (\cos(\theta_i + \omega \cdot t), \sin(\theta_i + \omega \cdot t))$, dove R_i è il raggio dell'anello su cui si trova il bot i , θ_i è la fase angolare iniziale del bot e ω è la velocità angolare di rotazione. L'implementazione del pattern VORTEX richiede che il controller aggiorni le posizioni target di tutti i bot ad ogni tick temporale e invii i comandi di aggiornamento con frequenza sufficiente a mantenere la fluidità della rotazione — tipicamente 10–20 comandi al secondo — producendo un carico di comunicazione più elevato rispetto ai pattern statici ma ancora gestibile con il protocollo UDP su rete SoftAP.

Il pattern FLOW FIELD è il pattern concettualmente più complesso di MicroBot e implementa un comportamento di tipo fluido in cui i bot si muovono seguendo le linee di flusso di un campo vettoriale definito nello spazio. Il campo vettoriale è specificato come una griglia di vettori velocità campionati su una griglia regolare del piano di simulazione, con valori intermedi ottenuti per interpolazione bilineare. Il controller calcola ad ogni tick temporale la velocità target di ogni bot come il vettore del campo nel punto corrispondente alla posizione corrente del bot, e invia comandi di aggiornamento della velocità piuttosto che comandi di posizione. Questo produce un comportamento in cui i bot scorrono fluentemente attraverso lo spazio seguendo le linee di flusso del campo — che possono formare vortici, divergenze, convergenze e strutture topologiche più complesse — senza una configurazione target fissa ma con un comportamento collettivo emergente determinato dalla geometria del campo. Il campo vettoriale può essere configurato staticamente dall'utente attraverso l'interfaccia del visore, oppure generato algoritmicamente in funzione del vettore W del framework 4D/5D, oppure derivato dall'analisi del flusso ottico del video della telecamera del visore — in questo caso i bot seguono letteralmente il movimento dell'ambiente ripreso dalla camera, producendo un sistema di realtà aumentata fisicamente materializzata in cui i MicroBot diventano indicatori fisici del flusso dell'ambiente visivo.

Il meccanismo di transizione tra pattern è un aspetto critico del sistema di coordinamento che determina la qualità dell'esperienza utente durante il controllo dello swarm. Una transizione brusca che teletrasporta istantaneamente tutti i bot dalle posizioni target del pattern corrente alle posizioni target del nuovo pattern produrrebbe movimenti caotici, collisioni multiple e disagganci indesiderati che degraderebbero sia l'esperienza visiva sia la stabilità fisica dello swarm. MicroBot implementa invece un meccanismo di interpolazione morfologica tra pattern che genera una sequenza di configurazioni intermedie che transitano gradualmente dalla geometria del pattern di partenza alla geometria del pattern di arrivo. L'interpolazione è implementata attraverso una media ponderata delle posizioni target dei due pattern, con un parametro α che varia da zero a uno nel tempo secondo una funzione di easing — tipicamente una funzione sigmoideale o una funzione smoothstep — che produce una transizione con accelerazione graduale all'inizio e decelerazione graduale alla fine, eliminando i movimenti bruschi che si produrrebbero con un'interpolazione lineare pura. La durata della transizione è configurabile dall'utente tramite l'interfaccia del visore, con un valore di default di 2–3 secondi che produce transizioni visivamente fluide senza essere percepite come eccessivamente lente.

Punto 22 — Safety layer globale: protezione, emergenza e affidabilità del sistema

Il safety layer è la componente di MicroBot che trasforma il sistema da un insieme di componenti funzionanti in un sistema affidabile — la differenza tra un prototipo di laboratorio che funziona in condizioni controllate e un sistema ingegneristicamente serio che mantiene il proprio comportamento corretto anche nelle situazioni limite, nei casi d'uso non previsti e nelle condizioni operative degradate che la realtà fisica inevitabilmente produce. La progettazione di un safety layer efficace richiede un cambio di prospettiva rispetto alla progettazione funzionale: invece di chiedersi come il sistema deve comportarsi quando tutto funziona correttamente, bisogna chiedersi sistematicamente cosa può andare storto, con quale probabilità, con quali conseguenze e con quale meccanismo di rilevamento e risposta. Questo processo di analisi dei modi di guasto — formalmente noto come Failure Mode and Effects Analysis nei contesti ingegneristici — è il fondamento metodologico su cui il safety

layer di MicroBot è costruito, e la sua esecuzione sistematica prima della realizzazione del primo prototipo fisico è una delle manifestazioni più concrete della filosofia progettuale del sistema.

I rischi fisici principali identificati nell'analisi dei modi di guasto di MicroBot appartengono a tre categorie distinte che richiedono meccanismi di protezione di natura diversa. La prima categoria è quella dei rischi energetici, legati alla gestione della batteria Li-Po in un sistema miniaturizzato con vincoli termici stringenti. La seconda categoria è quella dei rischi termici, legati alla dissipazione di potenza nelle coil elettromagnetiche e nei MOSFET di pilotaggio in un volume interno molto ridotto. La terza categoria è quella dei rischi di controllo, legati alla possibilità di perdita della connessione wireless tra il controller e i bot, di freeze del firmware, di corruzione dei dati di configurazione o di ricezione di comandi fisicamente impossibili o pericolosi.

La protezione dai rischi energetici è implementata attraverso un sistema a soglie multiple che interviene con intensità crescente in funzione della gravità della condizione rilevata. La soglia di avviso è impostata a 3.5 volt — circa il 30% della capacità residua della batteria Li-Po — e la sua attivazione produce due azioni simultanee: la segnalazione visiva tramite LED giallo lampeggiante e l'invio di un flag di batteria scarsa nel pacchetto di heartbeat verso il controller. Questo avviso è puramente informativo e non limita l'operatività del bot, ma permette all'utente di pianificare la ricarica prima che il sistema entri in condizioni critiche. La soglia di riduzione delle prestazioni è impostata a 3.3 volt e produce una limitazione automatica del duty cycle massimo delle coil al 50%, riducendo la potenza disponibile per l'aggancio magnetico ma estendendo significativamente l'autonomia residua. La soglia di sicurezza critica è impostata a 3.0 volt e produce l'inattivazione immediata e completa di tutte le coil, l'impostazione del LED su rosso fisso e la trasmissione di un pacchetto di emergenza al controller che segnala la condizione di batteria esaurita. Questa soglia protegge la cella Li-Po dalla scarica profonda che causerebbe un degradamento irreversibile della capacità e un potenziale rischio di gonfiamento della cella con rilascio di gas. Una quarta soglia, impostata a 2.7 volt, attiva il deep sleep forzato dell'ESP32 che riduce il consumo a pochi microampere, proteggendo la cella anche da una perdita di corrente residua che potrebbe portarla a zero volt in caso di mancata ricarica prolungata.

La protezione dai rischi termici è implementata attraverso il modello termico equivalente già descritto nel capitolo sull'elettronica, integrato da un sistema di limitazione dinamica del duty cycle che opera come un regolatore proporzionale sulla temperatura stimata. Quando la temperatura stimata delle coil supera la soglia di attenzione — impostata a 50 gradi centigradi — il safety layer riduce il duty cycle massimo consentito di un quantità proporzionale alla deviazione dalla soglia, secondo la relazione $\text{duty_max}(T) = \text{duty_nominale} \cdot (1 - k_T \cdot \max(0, T - T_{\text{soglia}}))$, dove k_T è la costante di proporzionalità termica impostata in modo che il duty cycle si azzeri quando la temperatura raggiunge 70 gradi centigradi. Questo meccanismo di de-rating termico continuo è preferibile a una soglia di shutdown binaria perché degrada le prestazioni del sistema gradualmente invece di produrre un'interruzione improvvisa che potrebbe destabilizzare una struttura aggregata già formata. Quando la temperatura stimata supera la soglia di emergenza termica a 70 gradi centigradi, il safety layer attiva lo shutdown completo delle coil e imposta un timer di cooldown di 30 secondi durante il quale le coil non possono essere riattivate.

indipendentemente dai comandi ricevuti dal controller, garantendo un raffreddamento sufficiente prima della ripresa dell'operazione normale.

La protezione dai rischi di controllo è implementata attraverso tre meccanismi complementari che operano a livelli diversi della gerarchia del sistema. Il primo meccanismo è il watchdog hardware dell'ESP32, un timer hardware indipendente dal firmware che viene resettato periodicamente dal loop principale del programma e che, se non resettato entro il timeout configurato — tipicamente 5 secondi — forza il riavvio completo del microcontrollore. Il watchdog hardware è l'ultima linea di difesa contro i guasti del firmware: qualsiasi condizione che blocchi l'esecuzione del loop principale — un loop infinito, un deadlock, una corruzione dello stack — viene automaticamente risolta dal riavvio forzato senza richiedere intervento umano. Il secondo meccanismo è il timeout di comunicazione: se un bot non riceve alcun pacchetto dal controller per un intervallo superiore al timeout configurato — tipicamente 3 secondi — entra autonomamente in modalità sicura locale, disattivando tutte le coil e impostando il LED su arancione lampeggiante per indicare la perdita di connessione. Questa modalità garantisce che i bot non rimangano con coil attive in caso di guasto del controller o di perdita della connessione wireless, evitando surriscaldamenti prolungati in assenza di supervisione. Il terzo meccanismo è la validazione dei comandi ricevuti: prima di eseguire qualsiasi comando ricevuto via UDP il bot verifica che i parametri contenuti nel pacchetto siano fisicamente plausibili — duty cycle nell'intervallo 0–100%, identificativo del bot valido, tipo di comando riconosciuto — e scarta silenziosamente i comandi che non superano la validazione, proteggendosi da comandi corrotti, replay attack e errori di programmazione nel firmware del controller.

Il safety layer del controller centrale opera a un livello di astrazione superiore rispetto ai safety layer dei singoli bot, monitorando proprietà globali dello swarm che non sono osservabili da nessun bot individuale. Il monitoraggio della coerenza globale del pattern verifica periodicamente che la configurazione stimata dello swarm — ricostruita a partire dalle posizioni riportate negli heartbeat dei bot — sia compatibile con il pattern assegnato entro una tolleranza configurabile, e genera un avviso se la deviazione supera la soglia per un numero di bot superiore a una frazione configurabile dello swarm. Il monitoraggio della salute della rete verifica che tutti i bot registrati stiano inviando heartbeat con la regolarità attesa e genera eventi di disconnessione per i bot che superano il timeout di heartbeat, aggiornando la tabella di stato globale e adattando il piano di pattern ai bot effettivamente disponibili. Il monitoraggio dell'energia globale dello swarm calcola la somma delle capacità residue stimate di tutti i bot e genera un avviso quando la capacità residua media scende al di sotto del 30%, permettendo all'utente di pianificare una sessione di ricarica coordinata prima che i bot inizino a disattivarsi autonomamente per batteria scarica.

Il comando di emergenza globale è il meccanismo di intervento più drastico del safety layer del controller e può essere attivato sia manualmente dall'utente tramite il gesto di doppia scossa del wristband o tramite il pulsante di emergenza nell'interfaccia del visore, sia automaticamente dal safety layer del controller in risposta a condizioni critiche rilevate nel monitoraggio globale. Il comando di emergenza globale è un pacchetto UDP broadcast — indirizzato a tutti i bot simultaneamente — con priorità massima che istruisce ogni bot a disattivare immediatamente tutte le coil, impostare il LED su rosso fisso e entrare in modalità di attesa sicura in cui nessun altro comando viene accettato fino alla ricezione esplicita di un segnale di reset. La trasmissione del pacchetto di emergenza viene ripetuta tre volte a

intervalli di 50 millisecondi per garantire la ricezione anche in presenza di perdita occasionale di pacchetti UDP, e il controller monitora i pacchetti di heartbeat successivi per verificare che tutti i bot abbiano ricevuto e applicato correttamente il comando. Il reset dalla modalità di emergenza richiede un'autenticazione esplicita dell'utente attraverso il sistema biometrico facciale — una misura che garantisce che il sistema non possa essere riattivato accidentalmente dopo un'emergenza e che un utente autorizzato sia sempre consapevole del motivo che ha causato l'emergenza prima di riprendere il controllo dello swarm.

Il sistema di logging del safety layer registra ogni evento di protezione rilevato — attivazione di una soglia di batteria, intervento del de-rating termico, timeout di comunicazione, esecuzione del comando di emergenza — con timestamp preciso al millisecondo, identificativo del bot coinvolto, valore del parametro che ha causato l'attivazione e azione correttiva intrapresa. Questo log è uno strumento fondamentale per l'analisi post-hoc delle sessioni operative, permettendo di identificare pattern ricorrenti di attivazione del safety layer che indicano problemi sistemici — un bot con una batteria che si scarica più rapidamente degli altri, una zona dello spazio operativo in cui la connessione wireless è degradata, un pattern di aggregazione che produce sistematicamente surriscaldamento delle coil — e di intervenire con modifiche al design fisico, ai parametri del firmware o alle procedure operative per eliminare la causa radice dei problemi piuttosto che limitarsi a gestirne i sintomi.

Punto 23 — Gestione energetica del sistema: autonomia, ricarica e ottimizzazione dei consumi

La gestione energetica è uno dei vincoli progettuali più stringenti di MicroBot e uno di quelli che più direttamente condizionano le scelte architettoniche a tutti i livelli del sistema. In un sistema di swarm robotics miniaturizzato alimentato a batteria, l'energia non è una risorsa abbondante da allocare liberamente ma una risorsa scarsa da gestire con attenzione millimetrica, bilanciando continuamente le esigenze funzionali del sistema — la forza delle coil magnetiche, la potenza del segnale wireless, la luminosità dei LED — con i limiti fisici imposti dalla capacità della batteria e dalla densità energetica dei materiali disponibili. Comprendere la gestione energetica di MicroBot significa comprendere non solo quanta energia è disponibile e quanto ne consuma ogni componente, ma come il sistema distribuisce dinamicamente questa energia in funzione dello stato operativo corrente, delle priorità definite dal safety layer e delle previsioni sull'evoluzione futura della sessione operativa.

Il bilancio energetico del MicroBot fisico è il punto di partenza obbligatorio di qualsiasi analisi della gestione energetica. La fonte di energia è la batteria Li-Po da 3.7 volt nominali con capacità compresa tra 40 e 60 milliampereora, che contiene un'energia totale disponibile espressa dalla relazione $E = V \cdot Q$ pari a circa 148–222 millijoule — un valore che può sembrare piccolo ma che è sufficiente per decine di minuti di operazione se la gestione energetica è accurata. I consumatori di energia nel MicroBot si dividono in quattro categorie con caratteristiche molto diverse. Il microcontrollore ESP32 in modalità operativa attiva — Wi-Fi acceso, processore in esecuzione, periferiche abilitate — consuma tipicamente 80–120 milliampere a 3.3 volt, corrispondenti a 264–396 milliwatt di potenza, e rappresenta il consumo di base del sistema che non può essere ridotto senza compromettere le funzionalità fondamentali di comunicazione e controllo. Le coil elettromagnetiche, quando

attive al duty cycle nominale del 50%, assorbono una corrente dell'ordine di 150–200 milliampere per coil attiva, con una potenza di circa 555–740 milliwatt per coil — il componente di gran lunga più energivoro del sistema, che può aumentare il consumo totale di un fattore tre o quattro rispetto al consumo di base quando tutte le quattro coil sono attive simultaneamente. I LED RGB consumano tipicamente 10–15 milliampere per canale colore attivo, corrispondenti a una potenza di 33–50 milliwatt, un contributo modesto ma non trascurabile quando si considera che i LED rimangono accesi continuamente durante l'intera sessione operativa. I sensori e i circuiti ausiliari — il sensore di prossimità a infrarossi, il circuito di protezione della batteria, il regolatore LDO — consumano complessivamente meno di 5 milliampere in condizioni normali, un contributo trascurabile rispetto agli altri componenti.

Il consumo totale del MicroBot in condizioni operative tipiche — microcontrollore attivo, una coil attiva al 30% di duty cycle di mantenimento, LED a luminosità ridotta — è stimato nell'ordine di 150–200 milliampere, che con una batteria da 50 milliampereora produce un'autonomia teorica di circa 15–20 minuti. Questo valore è significativamente inferiore alle autonomie tipiche dei dispositivi IoT commerciali, ma è accettabile per le sessioni operative di MicroBot che raramente superano i 30 minuti di utilizzo continuo nella fase prototipale. L'autonomia può essere estesa a 30–40 minuti adottando strategie di gestione energetica adattiva che riducono il consumo nei periodi di bassa attività — duty cycle delle coil ridotto durante la fase di mantenimento del pattern stabile, LED a luminosità ridotta quando non è necessario feedback visivo all'utente, frequenza di aggiornamento Wi-Fi ridotta durante le fasi in cui non sono attesi comandi imminenti — senza compromettere le prestazioni del sistema durante le fasi di transizione e di aggancio attivo.

La strategia di gestione energetica adattiva implementata nel firmware del MicroBot è basata su un modello a tre modalità operative con consumi e prestazioni diverse. La modalità Full Active è la modalità di massima funzionalità, adottata durante le fasi di transizione tra pattern e di aggancio magnetico attivo: tutte le coil possono essere attivate fino al duty cycle massimo consentito dalla temperatura, la frequenza di aggiornamento Wi-Fi è massima per minimizzare la latenza di ricezione dei comandi, i LED sono alla luminosità nominale. Il consumo in questa modalità può raggiungere 400–600 milliampere durante i picchi di attivazione multipla delle coil, ma la durata di queste fasi è tipicamente breve — pochi secondi per ogni transizione — e il loro contributo all'energia totale consumata nella sessione è limitato. La modalità Pattern Hold è la modalità di mantenimento del pattern stabile, adottata quando il bot ha raggiunto la propria posizione target e le coil sono in fase di mantenimento a basso duty cycle: il duty cycle delle coil è ridotto al 10–15% sufficiente a mantenere l'aggancio, la frequenza di aggiornamento Wi-Fi è ridotta da 100 a 20 hertz per i pacchetti di heartbeat, i LED sono ridotti al 30% della luminosità nominale. Il consumo in questa modalità è dell'ordine di 100–130 milliampere, significativamente inferiore alla modalità Full Active, e questa è la modalità più frequente durante una sessione operativa tipica. La modalità Standby è la modalità di attesa inattiva, adottata quando il bot non ha ricevuto comandi per un periodo superiore a 30 secondi: le coil sono completamente disattivate, la frequenza Wi-Fi è ridotta al minimo compatibile con il mantenimento della connessione — circa 1 heartbeat ogni 2 secondi — il LED lampeggia una volta ogni tre secondi per indicare la presenza del bot, e il processore entra in modalità di light sleep tra un heartbeat e l'altro. Il consumo in modalità Standby è dell'ordine di 30–40 milliampere,

sufficientemente basso da permettere tempi di attesa di diverse ore senza scaricare completamente la batteria.

La ricarica delle batterie dei MicroBot è un aspetto operativo che richiede attenzione nella gestione delle sessioni di utilizzo prolungato con un numero elevato di bot. La ricarica di ogni bot richiede la connessione fisica di un cavo USB-C al connettore del modulo, il che implica che il bot deve essere rimosso dalla superficie operativa e connesso a una sorgente di alimentazione esterna durante la ricarica. Con una corrente di ricarica di 100–200 milliampere — il valore tipico per il chip TP4056 con resistenza di programmazione appropriata per batterie da 40–60 milliampereora — il tempo di ricarica completa da zero è dell'ordine di 20–40 minuti. Questa durata è compatibile con un piano operativo che alterna sessioni di utilizzo di 20–30 minuti a sessioni di ricarica di 30–40 minuti, mantenendo i bot sempre disponibili attraverso la rotazione di un insieme di unità in carica con un altro insieme in operazione. Per sessioni che richiedono operatività continua con un numero elevato di bot, il sistema può essere configurato per scalare la dimensione dello swarm operativo in funzione del numero di bot con carica sufficiente, rimuovendo automaticamente dal piano di pattern i bot la cui carica scende al di sotto della soglia critica e sostituendoli con bot precedentemente in ricarica quando disponibili.

Il controller centrale adotta una strategia di gestione energetica diversa rispetto ai MicroBot, riflettendo le sue diverse priorità operative. La batteria Li-Po da 1200–2000 milliampereora garantisce un'autonomia del controller di diverse ore in condizioni operative normali, sufficiente per gestire multiple sessioni operative dei MicroBot senza necessità di ricarica intermedia. Il consumo del controller è dominato dal modulo Wi-Fi in modalità Access Point — che richiede tipicamente 150–200 milliampere in trasmissione continua — e dal processore ESP32-S3 in esecuzione attiva. Le strategie di riduzione del consumo del controller si concentrano quindi sulla gestione intelligente del traffico Wi-Fi: durante le fasi di swarm stabile il controller riduce la frequenza di broadcast dei pacchetti di sincronizzazione da 10 a 2 hertz, riduce il potere trasmissivo del modulo Wi-Fi al valore minimo sufficiente a mantenere la connessione con tutti i bot, e aumenta l'intervallo tra i pacchetti di heartbeat richiesti ai bot da 500 millisecondi a 2 secondi — riducendo il traffico di rete aggregato e il conseguente consumo energetico senza compromettere la qualità della supervisione dello stato dello swarm.

Il wristband e il visore VR hanno profili energetici e strategie di gestione distinti che riflettono le loro specifiche funzioni nel sistema. Il wristband con batteria da 500 milliampereora e consumo operativo di circa 50–80 milliampere in modalità BLE attiva garantisce un'autonomia di circa 6–10 ore, sufficiente per coprire una giornata intera di utilizzo senza necessità di ricarica. Il visore VR con batteria da 1200 milliampereora e consumo operativo di 400–600 milliampere in modalità di rendering attivo garantisce un'autonomia di circa 2–3 ore, il fattore limitante per le sessioni di utilizzo prolungato. La strategia di risparmio energetico del visore include la riduzione automatica della luminosità dei display dopo 60 secondi di inattività dell'utente, la sospensione del rendering della scena 3D quando il sensore di prossimità rileva che il visore non è indossato, e la riduzione della frequenza di aggiornamento della scena da 60 a 30 hertz durante le fasi in cui lo swarm è stabile e non ci sono transizioni di pattern in corso.

L'ottimizzazione energetica del sistema nel suo complesso è un problema di coordinamento che va oltre la gestione individuale di ogni componente e richiede una visione globale del bilancio energetico dell'intera sessione operativa. Il controller implementa un algoritmo di energy-aware scheduling che, nel pianificare le transizioni di pattern, tiene conto non solo della qualità estetica della transizione ma anche del costo energetico complessivo: tra due sequenze di transizione che producono risultati visivi equivalenti, l'algoritmo preferisce quella che minimizza la somma delle energie dissipate nelle coil di tutti i bot, ottenuta scegliendo l'assegnazione bot-posizioni che minimizza le distanze di spostamento e quindi la durata delle attivazioni magnetiche necessarie. Questo approccio energy-aware non elimina il problema dell'autonomia limitata dei MicroBot — la fisica dei magneti e le dimensioni della batteria impongono limiti che nessun algoritmo può superare — ma estende l'autonomia operativa del sistema del 10–20% rispetto a un algoritmo che ignori completamente le considerazioni energetiche, un margine che in una sessione di 20 minuti può fare la differenza tra completare o non completare una dimostrazione pianificata.

Punto 24 — Prototipazione fisica: materiali, strumenti e processo di assemblaggio

La prototipazione fisica è il momento in cui il progetto MicroBot attraversa la soglia che separa il mondo della simulazione e della documentazione dal mondo degli oggetti reali — il momento in cui le equazioni del modello matematico, le specifiche dimensionali delle tabelle di parametri e le scelte architetture dei diagrammi software si confrontano per la prima volta con la realtà concreta del materiale plastico, del filo di rame, della saldatura e del campo magnetico fisico. Questo confronto è quasi sempre sorprendente, e non sempre in modo piacevole: tolleranze che sembravano abbondanti sulla carta si rivelano insufficienti quando il coperchio del guscio stampato non si chiude per un decimo di millimetro di deformazione termica durante il raffreddamento, forze magnetiche che il modello prevedeva sufficienti si rivelano appena al limite quando l'attrito della superficie reale è leggermente superiore al valore assunto, connessioni WiFi che nella simulazione sono istantanee introducono latenze variabili che destabilizzano gli algoritmi di sincronizzazione. Documentare il processo di prototipazione significa quindi documentare non solo la sequenza di operazioni necessarie per assemblare un MicroBot funzionante ma anche i problemi che si incontrano inevitabilmente lungo questo percorso e le soluzioni adottate per risolverli — una forma di conoscenza pratica che non può essere derivata dalla teoria ma solo acquisita attraverso l'esperienza diretta con il materiale.

La scelta del materiale di stampa 3D è il primo punto di decisione del processo di prototipazione e condiziona tutte le scelte successive. Il PLA è il materiale primario per la versione 0.3 del MicroBot per ragioni che combinano accessibilità, lavorabilità e proprietà meccaniche sufficienti per le sollecitazioni operative previste. Il PLA è il materiale più diffuso tra le stampanti FDM desktop, con temperature di estrusione di 200 gradi centigradi e temperatura del piano di stampa di 60 gradi centigradi che non richiedono attrezzature speciali né precauzioni particolari di sicurezza. La sua densità di 1.24 grammi per centimetro cubo è la più bassa tra i materiali di stampa FDM comuni, un vantaggio diretto per il peso complessivo del MicroBot che deve rimanere entro il limite di 12 grammi. La rigidità del PLA — con modulo di Young tipicamente dell'ordine di 3.5 gigapascal — è sufficiente per mantenere la geometria del guscio nelle condizioni operative normali, con la caveat che il PLA ammorbidisce significativamente al di sopra di 50–60 gradi centigradi, temperatura che potrebbe essere raggiunta nelle immediate vicinanze delle coil durante attivazioni prolungate.

ad alto duty cycle. Per questa ragione il guscio del MicroBot include un gap d'aria di almeno 2 millimetri tra le coil e la parete interna dello shell, riducendo la conduzione termica verso il guscio e mantenendo la temperatura della parete al di sotto della soglia di rammollimento del PLA anche durante le attivazioni più intense. Il PETG è il materiale alternativo considerato per le iterazioni successive, con una temperatura di transizione vetrosa di circa 80 gradi centigradi che elimina il rischio di deformazione termica ma introduce una maggiore flessibilità che potrebbe compromettere la rigidità degli alloggiamenti delle coil e dei magneti permanenti in strutture così piccole. Il PETG richiede anche temperature di estrusione più elevate — 230–245 gradi centigradi — e una maggiore attenzione alle condizioni di stampa per evitare stringhie e distacchi tra gli strati che degraderebbero la qualità superficiale e la resistenza meccanica del pezzo.

La stampante 3D utilizzata per il prototipo deve soddisfare requisiti specifici di precisione che non tutte le macchine desktop garantiscono nella pratica. La risoluzione dimensionale necessaria per stampare correttamente gli alloggiamenti delle coil da 10×10 millimetri, i fori di passaggio per i magneti da 5 millimetri e i canali di instradamento dei cavi interni con sezione minima di 1.5×1.5 millimetri richiede una stampante con ugello da 0.4 millimetri o inferiore, calibrata per produrre un flusso di materiale accurato e uniforme senza sotto-estrazione né sovra-estrazione. L'altezza di strato ottimale per combinare qualità superficiale e velocità di stampa nel MicroBot è di 0.2 millimetri — metà del diametro dell'ugello, valore che garantisce una buona adesione tra gli strati e una superficie esterna sufficientemente liscia per ridurre l'attrito nelle zone di contatto durante l'aggregazione. La percentuale di riempimento interna è impostata al 20% con pattern a griglia, che produce la rigidità strutturale necessaria con un consumo di materiale contenuto. Le pareti sono impostate a un numero di perimetri di quattro — corrispondenti a 1.6 millimetri di spessore con ugello da 0.4 millimetri — per le zone strutturali principali e a tre perimetri nelle zone meno sollecitate, in modo da bilanciare resistenza e peso. Il tempo di stampa del guscio completo del MicroBot — le due metà separabili del doppio cono con sfera centrale — è stimato in 2–4 ore su una stampante desktop calibrata, valore compatibile con un ciclo di prototipazione rapida che permette di testare modifiche al design e stampare nuove versioni in tempi ragionevoli.

Il processo di assemblaggio del MicroBot è strutturato in una sequenza di operazioni che rispetta un ordine preciso dettato dai vincoli di accessibilità ai componenti interni nelle diverse fasi dell'assemblaggio. La prima operazione è la verifica dimensionale del guscio stampato tramite calibro digitale, che controlla che le dimensioni critiche — diametro interno della sfera, dimensioni degli alloggiamenti delle coil, posizione dei fori per i magneti — rientrino nelle tolleranze previste di ± 0.3 millimetri. Eventuali deviazioni eccessive richiedono la modifica dei parametri di stampa o del modello CAD prima di procedere all'assemblaggio, evitando lo spreco di componenti elettronici in un guscio dimensionalmente non conforme. La seconda operazione è l'inserimento e il fissaggio dei magneti permanenti negli alloggiamenti dedicati nel cono inferiore, con attenzione alla polarità corretta di ciascun magnete verificata tramite una bussola o un secondo magnete di riferimento. I magneti sono fissati con una goccia di colla cianoacrilica che polimerizza in pochi secondi e garantisce la ritenzione meccanica del magnete anche durante le sollecitazioni di attrazione e repulsione magnetica con altri moduli. La polarità di ogni magnete deve essere verificata dopo il fissaggio perché la forte forza magnetica può causare la rotazione spontanea del magnete

prima della completa polimerizzazione della colla se non viene mantenuto in posizione con una pinzetta o uno strumento non magnetico durante l'attesa.

L'avvolgimento delle coil è l'operazione più delicata e laboriosa del processo di assemblaggio, che richiede manualità, pazienza e un'attrezzatura di avvolgimento anche se rudimentale. Il nucleo di avvolgimento — un blocchetto di materiale non magnetico con le dimensioni della sezione interna della coil, 10×10 millimetri — viene inserito nel mandrino di un semplice avvolgitore manuale. Il filo di rame smaltato AWG 32 viene avvolto attorno al nucleo in strati successivi contando le spire, fino al raggiungimento del numero di spire target — tipicamente 70–90 spire per uno spessore di avvolgimento di 3–4 millimetri. Le estremità del filo vengono saldate a due pin di connessione che permettono il collegamento della coil al PCB. La resistenza della coil viene misurata con un multimetro dopo l'avvolgimento per verificare la corrispondenza con il valore calcolato — tipicamente 8–12 ohm per la configurazione di avvolgimento descritta — e l'induttanza viene misurata se è disponibile un misuratore LCR. Coil con resistenza fuori specifica vengono riavvolte prima di procedere con l'assemblaggio per evitare asimmetrie nel pilotaggio che potrebbero produrre forze magnetiche non uniformi tra le quattro coil del modulo.

La saldatura del PCB è l'operazione che richiede la strumentazione più specializzata — un saldatore a punta fine con temperatura controllata, preferibilmente con punta a matita da 0.8 millimetri, flusso di saldatura e lente di ingrandimento o microscopio da laboratorio — e la maggiore esperienza nella lavorazione di componenti SMD di piccole dimensioni. I componenti SMD come i MOSFET in package SOT-23, le resistenze e i condensatori in package 0402 e il regolatore LDO in package SOT-23 hanno pad di saldatura con dimensioni dell'ordine di 0.5–1 millimetro che richiedono precisione e controllo della temperatura — tipicamente 320–350 gradi centigradi per leghe di stagno senza piombo — per evitare sia saldature fredde che danneggiamento dei componenti per surriscaldamento. Il microcontrollore ESP32 in versione modulo — come il ESP32-C3 SuperMini — è significativamente più semplice da saldare rispetto alla versione bare chip perché i pad di connessione sono accessibili su bordo del modulo con passo di 2.54 millimetri, compatibile con la saldatura manuale senza attrezzature SMD specializzate. Dopo la saldatura di tutti i componenti il PCB viene ispezionato visivamente sotto lente di ingrandimento per verificare l'assenza di cortocircuiti tra pad adiacenti e di saldature fredde, e poi testato con un multimetro in modalità diodo per verificare l'assenza di cortocircuiti tra le linee di alimentazione prima di connettere la batteria.

Il test funzionale del MicroBot assemblato segue una sequenza di verifiche progressive che isola i potenziali problemi prima di procedere al test integrato. Il primo test è la verifica dell'alimentazione: con la batteria connessa e il firmware caricato, si verifica che il LED si illumini correttamente e che la tensione di alimentazione dell'ESP32 sia di 3.3 volt stabili misurata ai terminali del regolatore LDO. Il secondo test è la verifica della comunicazione wireless: si verifica che il MicroBot compaia nella lista delle stazioni connesse al SoftAP del controller e che i pacchetti di heartbeat vengano ricevuti correttamente dal controller con frequenza regolare. Il terzo test è la verifica del pilotaggio delle coil: tramite un comando inviato dal controller si attiva sequenzialmente ciascuna delle quattro coil al 20% di duty cycle e si verifica con un misuratore di campo magnetico o con un piccolo magnete di test che ciascuna coil produca il campo magnetico atteso nella direzione corretta. Il quarto test è la verifica dell'aggancio magnetico: si avvicinano due MicroBot assemblati e si verifica che il

sistema di pre-allineamento dei magneti permanenti guidi spontaneamente i moduli nella configurazione di aggancio e che l'attivazione delle coil via firmware produca una forza di attrazione percepibilmente superiore alla sola attrazione dei magneti permanenti. Solo dopo aver superato positivamente tutti e quattro i livelli di test il MicroBot viene considerato pronto per l'integrazione nel sistema swarm completo.

Punto 25 — Validazione sperimentale e metriche di performance del sistema

La validazione sperimentale è la fase in cui il progetto MicroBot smette di essere una proposta teorica e diventa un sistema verificato — il momento in cui le affermazioni sulle prestazioni del sistema cessano di essere stime basate su modelli e diventano misurazioni basate su dati reali raccolti in condizioni operative controllate. La distinzione tra stima e misurazione non è semantica ma epistemologica: una stima è vera nel modello che la produce, una misurazione è vera nel mondo fisico reale, e il gap tra le due contiene informazioni preziose sulle approssimazioni del modello, sugli effetti fisici non modellati e sui limiti pratici del sistema che nessuna analisi teorica può rivelare con altrettanta precisione. Progettare la validazione sperimentale di MicroBot significa quindi progettare un insieme di esperimenti che mettano sistematicamente alla prova le assunzioni del modello, quantifichino le prestazioni reali del sistema nelle dimensioni più rilevanti per le applicazioni previste e producano dati sufficientemente dettagliati da guidare il processo di raffinamento iterativo del design nelle versioni successive.

Le metriche di performance di MicroBot sono organizzate in cinque famiglie che coprono le dimensioni fondamentali del comportamento del sistema: le prestazioni di comunicazione, le prestazioni magnetiche, le prestazioni di coordinamento collettivo, le prestazioni energetiche e le prestazioni dell'interfaccia utente. Ciascuna famiglia comprende un insieme di metriche specifiche misurabili con strumentazione standard, con protocolli di misura definiti che garantiscono la riproducibilità dei risultati e la comparabilità tra versioni successive del sistema.

Le prestazioni di comunicazione sono caratterizzate da tre metriche principali. La latenza di comando è il tempo che intercorre tra il momento in cui il controller invia un pacchetto UDP a un bot e il momento in cui il bot inizia l'esecuzione del comando contenuto nel pacchetto, misurata tramite la differenza tra il timestamp di invio incluso nel pacchetto e il timestamp locale del bot al momento dell'inizio dell'esecuzione. La misurazione richiede che i clock del controller e del bot siano sincronizzati tramite il meccanismo di Precision Time Protocol descritto nel capitolo sulla comunicazione, e che la deriva residua del clock del bot — tipicamente inferiore a 2 millisecondi dopo la sincronizzazione — sia sottratta dal valore misurato come correzione. Il valore target per la latenza di comando è inferiore a 10 millisecondi per il 95° percentile della distribuzione, con un valore massimo assoluto di 30 millisecondi per garantire la coerenza della sincronizzazione temporale durante le transizioni di pattern. Il packet loss rate è la frazione di pacchetti UDP inviati dal controller che non vengono ricevuti correttamente dai bot, misurata come rapporto tra il numero di comandi eseguiti rilevato dai bot nel loro log interno e il numero di comandi inviati registrato nel log del controller in un intervallo temporale definito. Il valore target è inferiore al 2% in condizioni operative normali — distanza tra controller e bot inferiore a 3 metri, assenza di ostacoli fisici nel percorso del segnale, assenza di interferenze Wi-Fi significative nel canale utilizzato. La frequenza di aggiornamento dello stato è il numero di pacchetti di heartbeat ricevuti dal

controller per ogni bot nell'unità di tempo, che deve essere vicina al valore nominale di 2 heartbeat al secondo in modalità Pattern Hold e di 0.5 heartbeat al secondo in modalità Standby, con deviazioni massime del 20% rispetto al valore nominale come indicatore della qualità della connessione wireless.

Le prestazioni magnetiche sono caratterizzate da quattro metriche specifiche che quantificano la qualità del sistema di aggancio e disaggancio. La forza di aggancio nominale è la forza di trazione necessaria a separare due bot agganciati in condizioni di allineamento ottimale, misurata tramite un dinamometro digitale con risoluzione di 0.01 newton applicato perpendicolarmente alla superficie di contatto tra i moduli. Questa misurazione viene eseguita in tre condizioni distinte: con sole magneti permanenti attivi e coil spente, con magneti permanenti e coil al duty cycle di mantenimento del 15%, e con magneti permanenti e coil al duty cycle massimo. I valori target sono rispettivamente 0.5–0.8 newton, 0.8–1.2 newton e 1.5–2.0 newton, con variazione inferiore al 15% tra misurazioni ripetute nelle stesse condizioni come indicatore della riproducibilità del sistema. Il tempo di aggancio è il tempo che intercorre tra il momento in cui due bot vengono posizionati alla distanza di pre-attivazione e il momento in cui il contatto fisico è confermato dal sistema, misurato tramite il log del firmware che registra il timestamp di ogni transizione di stato del sistema di aggancio. Il valore target è inferiore a 500 millisecondi per il 90° percentile della distribuzione, con il 10% restante corrispondente a situazioni di disallineamento angolare iniziale che richiedono una fase di pre-allineamento più lunga prima del contatto. Il tasso di successo dell'aggancio è la frazione di tentativi di aggancio — definiti come avvicinamenti di due bot a distanza inferiore alla soglia di pre-attivazione con comando di aggancio attivo — che producono un contatto fisico stabile entro il timeout configurato di 2 secondi. Il valore target è superiore al 90% su superfici con coefficiente di attrito nell'intervallo 0.2–0.5, che copre la maggior parte delle superfici di lavoro comuni — tavoli in legno, superfici in plastica, superfici in vetro. La stabilità dell'aggancio è la durata media dell'aggancio tra due bot in condizioni di mantenimento a duty cycle ridotto in presenza di perturbazioni esterne standardizzate — vibrazioni della superficie di appoggio prodotte da un vibratore calibrato a 10 hertz e 0.5 millimetri di ampiezza — prima che si verifichi un disaggancio non voluto. Il valore target è superiore a 60 secondi in queste condizioni di perturbazione, garantendo una stabilità sufficiente per le sessioni operative tipiche.

Le prestazioni di coordinamento collettivo sono le metriche più complesse da misurare perché riguardano proprietà emergenti del sistema che dipendono dall'interazione di tutti i componenti simultaneamente. Il tempo di convergenza al pattern è il tempo che intercorre tra l'emissione del comando di transizione verso un nuovo pattern e il momento in cui tutti i bot si trovano entro la tolleranza di posizionamento di ± 5 millimetri rispetto alle proprie posizioni target, misurato tramite il log del controller che riceve gli aggiornamenti di posizione dagli heartbeat dei bot. Questa metrica viene misurata per ogni tipo di pattern implementato e per diverse dimensioni dello swarm — 4, 8 e 16 bot — producendo una matrice di tempi di convergenza che caratterizza le prestazioni del sistema di coordinamento in funzione della complessità del pattern e della dimensione dello swarm. Il valore target per un pattern circolare con 8 bot è inferiore a 10 secondi dal comando alla convergenza completa, con variabilità inferiore al 30% tra ripetizioni nelle stesse condizioni. La precisione di posizionamento è la deviazione media tra le posizioni target assegnate e le posizioni reali raggiunte dai bot al termine della convergenza, stimata a partire dalle posizioni riportate negli heartbeat e corretta per l'incertezza nella stima della posizione. Questa metrica

quantifica quanto fedelmente il sistema fisico replica la configurazione geometrica calcolata dal modello, e le sue deviazioni sistematiche rivelano effetti fisici non modellati come disallineamenti meccanici, forze magnetiche parassite e attrito non uniforme. Il valore target è una deviazione media inferiore a 8 millimetri per pattern con passo nominale di 30 millimetri tra bot adiacenti, corrispondente a un errore relativo inferiore al 27%. Il tasso di successo della transizione è la frazione di transizioni di pattern che completano la convergenza entro il timeout di 15 secondi senza attivare il safety layer per collisioni, surriscaldamento o perdita di connessione. Il valore target è superiore all'85% per transizioni tra pattern della stessa famiglia geometrica e superiore al 70% per transizioni tra pattern di famiglie diverse — come da lineare a circolare — che richiedono disagganci e reagganci magnetici più complessi.

Le prestazioni energetiche sono misurate attraverso un insieme di test standardizzati che caratterizzano il consumo del sistema nelle diverse modalità operative. Il test di autonomia in Pattern Hold misura il tempo di operazione continua del MicroBot a partire da una batteria completamente carica fino all'attivazione della soglia di emergenza a 3.0 volt, con il bot in modalità Pattern Hold con una coil attiva al 15% di duty cycle — la condizione operativa più comune durante una sessione tipica. Il valore misurato viene confrontato con il valore teorico calcolato dal modello energetico per quantificare l'accuratezza del modello e identificare eventuali consumi parassiti non modellati. Il test di ciclo di vita della batteria misura la capacità della batteria Li-Po dopo un numero crescente di cicli di carica e scarica — tipicamente 50, 100, 200 cicli — verificando che la degradazione della capacità resti al di sotto del 20% dopo 200 cicli, valore che corrisponde a circa un anno di utilizzo normale del sistema con una sessione al giorno. Questa misurazione richiede tempi lunghi ma è fondamentale per caratterizzare la durata operativa del sistema e pianificare la frequenza di sostituzione delle batterie nei prototipi.

Le prestazioni dell'interfaccia utente sono le più difficili da quantificare con metriche oggettive perché dipendono dalla percezione soggettiva dell'utente e dal contesto di utilizzo. La latenza percepita del controllo gestuale è misurata come il tempo che intercorre tra l'esecuzione di un gesto riconoscibile dall'utente e la risposta visiva dello swarm — un cambio di LED, l'inizio di una transizione di pattern — che fornisce feedback del recepimento del comando. Questa metrica include tutti i contributi di latenza nella catena di controllo: rilevamento del gesto da MediaPipe, elaborazione nel motore di simulazione, trasmissione al controller, trasmissione ai bot, risposta fisica. Il valore target è inferiore a 150 millisecondi per il 95° percentile, soglia al di sotto della quale la maggior parte degli utenti percepisce la risposta come immediata e al di sopra della quale inizia a percepirsi un ritardo che degrada il senso di controllo diretto. Il tasso di riconoscimento corretto delle gesture è la frazione di gesture eseguite dall'utente che vengono classificate correttamente dal sistema — senza falsi positivi né falsi negativi — misurata in sessioni di test con utenti diversi e in condizioni di utilizzo normale. Il valore target è superiore al 90% per le gesture discrete fondamentali del wristband — scossa singola, scossa doppia, rotazione rapida — misurato su un campione di almeno 100 esecuzioni per tipo di gesture con 5 utenti diversi.

Il protocollo di validazione sperimentale prevede che ogni metrica venga misurata in tre condizioni distinte che coprono l'intervallo di variabilità operativa atteso. La condizione nominale corrisponde all'utilizzo in condizioni ottimali — superficie orizzontale uniforme, temperatura ambiente di 22 gradi centigradi, assenza di interferenze wireless significative,

bot completamente carichi, distanza di 1 metro dal controller. La condizione degradata corrisponde all'utilizzo in condizioni sfavorevoli ma realistiche — superficie con piccole irregolarità, temperatura ambiente di 35 gradi centigradi, presenza di altre reti WiFi nel canale, bot a 50% di carica, distanza di 3 metri dal controller. La condizione limite corrisponde all'utilizzo alla soglia delle specifiche operative — superficie con attrito elevato, temperatura ambiente di 40 gradi centigradi, interferenze WiFi significative, bot a 30% di carica, distanza di 5 metri dal controller. Il confronto delle metriche nelle tre condizioni quantifica la robustezza del sistema alle variazioni delle condizioni operative e identifica i parametri ambientali che hanno l'impatto maggiore sulle prestazioni, guidando le priorità di ottimizzazione nelle versioni successive del progetto.

Punto 26 — Simulazioni elettriche: SPICE, modellazione delle coil e verifica dei circuiti

La simulazione elettrica è il primo livello della catena di validazione di MicroBot e il più lontano dall'esperienza diretta dell'utente finale — non produce pattern visibili sullo schermo né comportamenti collettivi emergenti, ma genera i dati quantitativi fondamentali senza i quali tutte le scelte successive sul dimensionamento dei componenti, sulla protezione del firmware e sulla gestione energetica sarebbero basate su assunzioni non verificate. La simulazione elettrica risponde a domande concrete e critiche: la corrente che scorre nella coil quando il MOSFET è saturo è compatibile con i limiti della batteria? Il picco di tensione ai capi del MOSFET durante la commutazione è inferiore alla tensione di breakdown del componente? La potenza dissipata nella resistenza di avvolgimento della coil produce un surriscaldamento accettabile nelle condizioni operative peggiori? Senza rispondere quantitativamente a queste domande prima di costruire il circuito fisico, il rischio di danneggiare componenti durante i test — MOSFET bruciati dalla tensione inversa delle coil, batterie in cortocircuito per assorbimenti eccessivi, coil surriscaldate per duty cycle troppo elevati — è concreto e costoso. La simulazione SPICE elimina o riduce drasticamente questo rischio permettendo di esplorare virtualmente l'intero spazio dei parametri operativi prima di applicare tensione al circuito reale.

L'ambiente di simulazione SPICE adottato per MicroBot è LTspice XVII di Analog Devices, disponibile gratuitamente per Windows e macOS e dotato di una libreria di modelli di componenti sufficientemente ricca da coprire i dispositivi utilizzati nel progetto. In alternativa, il simulatore Falstad è accessibile direttamente via browser senza installazione e offre un'interfaccia grafica più immediata e una visualizzazione animata delle correnti e delle tensioni nei nodi del circuito, particolarmente utile per una comprensione intuitiva del comportamento dei circuiti di pilotaggio delle coil. I due strumenti sono complementari: LTspice offre maggiore precisione numerica, modelli di componenti più accurati e la possibilità di eseguire analisi parametriche e sweep in frequenza; Falstad offre maggiore immediatezza visiva e una curva di apprendimento più bassa che permette di costruire e simulare circuiti semplici in pochi minuti.

Il circuito base di pilotaggio di una singola coil del MicroBot può essere modellato in SPICE come un insieme di quattro elementi fondamentali. Il primo elemento è la sorgente di tensione che rappresenta la batteria Li-Po, modellata come una tensione ideale di 3.7 volt in serie con una resistenza interna di 0.5–1 ohm che rappresenta la resistenza interna della cella e la resistenza dei connettori e delle piste. La resistenza interna è un parametro critico da includere nel modello perché determina la caduta di tensione durante i picchi di corrente

— quando quattro coil sono attive simultaneamente con correnti dell'ordine di 300 milliampere ciascuna, la corrente totale assorbita dalla batteria può raggiungere 1.2 ampere, producendo una caduta di tensione di 0.6–1.2 volt sulla resistenza interna che riduce significativamente la tensione disponibile per i componenti di carico. Il secondo elemento è il MOSFET a canale N, modellato tramite il modello SPICE standard che include la resistenza di canale in stato on — $R_{DS(on)}$ — tipicamente dell'ordine di 0.3–1 ohm per MOSFET logic-level in package SOT-23 come il BSS138, le capacità di gate-source e gate-drain che determinano la velocità di commutazione, e la tensione di soglia che determina la tensione minima di gate necessaria all'accensione. Il terzo elemento è la coil, modellata come un'induttanza L in serie con una resistenza R che rappresenta la resistenza ohmica del filo di avvolgimento — tipicamente 8–12 ohm per la configurazione di avvolgimento di MicroBot — e in parallelo con una capacità parassita C che rappresenta la capacità distribuita tra le spire adiacenti. Il quarto elemento è il diodo di ricircolo, modellato tramite il modello SPICE del diodo con la sua tensione di soglia — 0.3–0.4 volt per diodi Schottky, 0.6–0.7 volt per diodi di silicio convenzionali come il 1N4148 — e la sua corrente di saturazione inversa. Il diodo di ricircolo è connesso in antiparallelo rispetto alla serie MOSFET-coil — anodo verso il drain del MOSFET, catodo verso l'alimentazione positiva — in modo da fornire un percorso di scarica dell'energia accumulata nella coil quando il MOSFET si spegne, evitando il picco di tensione distruttivo che si produrrebbe senza questo percorso di scarica.

La simulazione transitoria del circuito base è la prima analisi da eseguire e ha l'obiettivo di osservare come evolvono nel tempo la corrente nella coil e la tensione ai capi del MOSFET in risposta a un segnale PWM applicato al gate. Il segnale di gate è modellato come un generatore di tensione che produce un'onda quadra tra 0 e 3.3 volt — la tensione logica dell'ESP32 — con frequenza di 25 kilohertz e duty cycle configurabile. L'analisi transitoria viene eseguita per un intervallo di tempo di almeno 10 periodi del segnale PWM — 400 microsecondi — con un passo temporale massimo dell'ordine di 10 nanosecondi per risolvere accuratamente i transitori di commutazione che avvengono in tempi dell'ordine di 100–500 nanosecondi. I risultati chiave da osservare in questa simulazione sono quattro. Il primo è la forma d'onda della corrente nella coil, che deve mostrare la caratteristica forma a dente di sega — crescita lineare durante il periodo di accensione del MOSFET e decrescita lineare durante il periodo di spegnimento con ricircolo attraverso il diodo — con un valore medio proporzionale al duty cycle e un ripple di corrente inversamente proporzionale all'induttanza della coil. Il secondo risultato chiave è il picco di corrente durante la fase di accensione, che non deve superare la corrente massima ammissibile dalla batteria — tipicamente 1 ampere per batterie da 50 milliampereora — né la corrente di saturazione del MOSFET. Il terzo risultato è il picco di tensione ai capi del MOSFET durante la fase di spegnimento, che deve essere inferiore alla tensione di breakdown del componente — tipicamente 20–30 volt per MOSFET in package SOT-23 — con un margine di sicurezza di almeno il 50%. Il quarto risultato è la potenza media dissipata nel MOSFET e nella resistenza di avvolgimento, calcolata come prodotto della tensione e della corrente istantanee mediate su un periodo, che determina il surriscaldamento dei componenti nelle condizioni operative di regime.

L'analisi parametrica è il tipo di simulazione più utile per il dimensionamento dei componenti del circuito di pilotaggio. In LTspice l'analisi parametrica viene eseguita tramite la direttiva .STEP che itera la simulazione per valori diversi di un parametro specificato — il duty cycle del PWM, la frequenza di commutazione, il numero di spire della coil, la resistenza interna

della batteria — producendo famiglie di curve che mostrano come le grandezze di interesse variano al variare del parametro. Questo tipo di analisi è particolarmente utile per rispondere a domande come: a quale valore di duty cycle la potenza dissipata nella coil supera la soglia termica critica? A quale frequenza di commutazione le perdite di commutazione nel MOSFET diventano comparabili alle perdite di conduzione? Qual è il numero ottimale di spire che massimizza la forza magnetica prodotta dalla coil rispettando simultaneamente i vincoli di corrente della batteria e di temperatura? Le risposte a queste domande, ottenute dalla simulazione parametrica, definiscono i limiti operativi del sistema che vengono poi codificati come parametri del safety layer nel firmware.

La simulazione del circuito completo con quattro coil attive simultaneamente è un'estensione naturale della simulazione del circuito base che rivela effetti di interazione tra le coil che non sono visibili nel circuito con una sola coil. Quando quattro MOSFET commutano alla stessa frequenza con fase diversa — come avviene quando il firmware attiva le quattro coil in sequenza sfasata per distribuire il carico sulla batteria — le correnti di commutazione si sommano in modo che dipende dalla relazione di fase tra i segnali di gate. Se i quattro segnali sono in fase — tutti i MOSFET si accendono e si spengono simultaneamente — i picchi di corrente si sommano e la corrente massima assorbita dalla batteria è quattro volte quella di una singola coil, con una caduta di tensione sulla resistenza interna corrispondentemente elevata che può ridurre la tensione di alimentazione a valori incompatibili con il corretto funzionamento dell'ESP32. Se i quattro segnali sono sfasati di un quarto di periodo l'uno rispetto all'altro — offset di 10 microsecondi a 25 kilohertz — i picchi di corrente si distribuiscono nel tempo e la corrente massima assorbita dalla batteria è significativamente inferiore, con una riduzione del ripple di corrente sull'alimentazione che migliora la stabilità della tensione e riduce le interferenze elettromagnetiche generate dal circuito. La simulazione del circuito completo con quattro coil in sfasamento configurabile permette di quantificare questo effetto e di scegliere la configurazione di sfasamento ottimale che minimizza i picchi di corrente rispettando i vincoli di sincronizzazione temporale richiesti dalla logica di aggancio magnetico.

La verifica sperimentale dei risultati delle simulazioni SPICE è il passo conclusivo del processo di modellazione elettrica e permette di quantificare l'accuratezza dei modelli adottati. La verifica viene eseguita costruendo il circuito di pilotaggio su una breadboard o su un PCB di prototipazione, applicando il segnale PWM tramite un generatore di funzioni o direttamente dall'ESP32, e misurando le grandezze di interesse con un oscilloscopio USB — sufficiente per le frequenze e le tensioni in gioco nel circuito di MicroBot — e un multimetro digitale. Le misurazioni di tensione e corrente nel tempo vengono confrontate con le curve prodotte dalla simulazione SPICE, e le discrepanze osservate vengono analizzate per identificare le assunzioni del modello che si discostano maggiormente dalla realtà — tipicamente la resistenza interna della batteria sotto carico impulsivo, la tensione di soglia reale del MOSFET che dipende dalla temperatura e dalla corrente di gate, e l'induttanza effettiva della coil che può differire significativamente dal valore calcolato analiticamente a causa degli effetti geometrici dell'avvolgimento a mano. I parametri del modello SPICE vengono aggiornati in base alle misurazioni per produrre un modello calibrato che può essere utilizzato con maggiore confidenza per il dimensionamento definitivo dei componenti e per la previsione del comportamento del sistema nelle condizioni operative che non è possibile testare direttamente — temperature elevate, correnti di picco, condizioni di guasto controllate.

Punto 27 — Sviluppi futuri: scalabilità, intelligenza distribuita e materia programmabile

La roadmap numerata di MicroBot — dalla versione 0.1 alla versione 0.5 — definisce un percorso di sviluppo concreto e raggiungibile con le risorse e le tecnologie attualmente disponibili. Ma oltre questo orizzonte a medio termine esiste uno spazio di sviluppo più ampio e speculativo, in cui le scelte architetture del progetto aprono direzioni di ricerca che potrebbero trasformare MicroBot da sistema prototipale a piattaforma di ricerca matura, capace di contribuire in modo originale al panorama scientifico della swarm robotics, della materia programmabile e dell'interazione uomo-macchina. Esplorare questi sviluppi futuri non è un esercizio di fantascienza ingegneristica ma una pratica progettuale necessaria: le scelte fatte oggi sull'architettura del sistema, sui protocolli di comunicazione, sulla geometria dei moduli e sulla struttura del firmware determinano quanto facilmente il sistema potrà essere esteso domani verso applicazioni più ambiziose. Un'architettura che non è stata progettata con la scalabilità in mente diventa rapidamente un ostacolo allo sviluppo, mentre un'architettura progettata con lungimiranza si rivela uno strumento che cresce con il progetto e lo abilita a raggiungere obiettivi che inizialmente sembravano irraggiungibili.

La scalabilità dello swarm verso centinaia di unità è il primo e più immediato sviluppo futuro di MicroBot, e il suo raggiungimento richiede di affrontare tre categorie di problemi che non si manifestano con swarm di 10–20 bot ma emergono inevitabilmente quando la dimensione dello swarm aumenta di un ordine di grandezza. Il primo problema è la scalabilità del protocollo di comunicazione: la rete SoftAP con protocollo UDP funziona efficacemente fino a circa 20–30 client connessi simultaneamente, limite oltre il quale la congestione del canale wireless produce perdite di pacchetti e latenze variabili incompatibili con i requisiti di sincronizzazione temporale del sistema. La soluzione più promettente per superare questo limite è l'adozione di una topologia di comunicazione gerarchica a due livelli, in cui il controller comunica con un insieme di nodi aggregatori — bot designati come subcontroller — ciascuno responsabile di un cluster di 10–15 bot ordinari con cui comunica tramite ESP-NOW. Questa topologia riduce il numero di comunicazioni dirette con il controller da N a $N/15$, eliminando la congestione del canale principale senza richiedere modifiche fondamentali all'architettura del sistema. Il secondo problema è la scalabilità degli algoritmi di coordinamento: il problema di assegnazione ottimale risolto con l'algoritmo ungherese ha complessità $O(n^3)$ che diventa computazionalmente proibitiva per swarm di 100 bot su un ESP32-S3. La soluzione prevista è la migrazione verso algoritmi di assegnazione decentralizzati — in cui ogni bot calcola autonomamente la propria posizione target attraverso un processo di negoziazione locale con i bot vicini — che scalano linearmente con la dimensione dello swarm e producono assegnazioni di qualità comparabile all'ottimo globale in tempi significativamente inferiori. Il terzo problema è la scalabilità della gestione fisica dello spazio: con 100 bot su una superficie di lavoro tipica di 1×1 metro, la densità media è di un bot ogni 100 centimetri quadrati, sufficientemente elevata da rendere le collisioni durante le transizioni di pattern non più eventi rari ma fenomeni sistemici che il coordinamento deve gestire come norma piuttosto che come eccezione. La soluzione prevista combina algoritmi di pianificazione del moto multi-robot basati su grafi di visibilità con regole di priorità locali che permettono ai bot di negoziare autonomamente il diritto di precedenza nelle zone di conflitto senza richiedere l'intervento centralizzato del controller per ogni coppia di bot in potenziale collisione.

L'intelligenza distribuita è la direzione di sviluppo concettualmente più ambiziosa di MicroBot e la più lontana dagli approcci convenzionali della swarm robotics. L'idea fondante è che ogni bot, anziché essere un esecutore passivo dei comandi del controller, acquisisca progressivamente una capacità di apprendimento locale che gli permette di migliorare il proprio comportamento nel tempo a partire dall'esperienza accumulata durante le sessioni operative. Questa capacità di apprendimento locale non richiede algoritmi di deep learning — incompatibili con le risorse computazionali di un ESP32 — ma può essere implementata attraverso tecniche di reinforcement learning leggero, come il Q-learning tabulare, in cui ogni bot mantiene una tabella di valori che associa ad ogni coppia stato-azione una stima della ricompensa futura attesa. Lo stato del bot è descritto da un insieme discretizzato di variabili locali — distanza dai bot vicini, stato del magnete, livello di carica della batteria, tipo di pattern corrente — e le azioni disponibili includono le decisioni di basso livello che il bot prende autonomamente come il duty cycle delle coil, la soglia di aggancio e il ritardo nell'inizio del movimento. La ricompensa è definita come una combinazione della qualità dell'aggancio prodotto — forza di tenuta, tempo impiegato, consumo energetico — e della coerenza del comportamento locale con il pattern globale assegnato dal controller. Nel tempo, i bot che apprendono attraverso questo meccanismo sviluppano strategie di comportamento locale progressivamente più efficaci — imparando ad anticipare i comandi del controller in base al pattern in corso, ad adottare profili di corrente nelle coil adattati alla temperatura corrente, ad accelerare l'aggancio in condizioni di buon allineamento e a rallentarlo in condizioni di allineamento incerto — senza che queste strategie siano state esplicitamente programmate ma emergano dall'esperienza accumulata nelle sessioni operative.

La condivisione dell'esperienza tra bot è un'estensione naturale dell'apprendimento locale che può produrre effetti di intelligenza collettiva particolarmente interessanti. Se ogni bot, oltre ad apprendere dalla propria esperienza, riceve periodicamente aggiornamenti della tabella di Q-learning degli altri bot tramite il controller — una forma di federated learning adattata alle risorse limitate dell'ESP32 — l'intera flotta beneficia dell'esperienza accumulata da ogni singolo modulo, convergendo verso strategie di comportamento ottimali molto più rapidamente di quanto potrebbe fare ciascun bot apprendendo in isolamento. Questo meccanismo apre la possibilità di sessioni di training dedicate in cui i bot vengono esposti sistematicamente a situazioni operative diverse — diverse superfici di attrito, diverse temperature, diversi pattern di aggregazione — producendo una base di esperienza distribuita che migliora le prestazioni del sistema nelle condizioni operative reali senza richiedere la riscrittura del firmware.

Il collegamento con la visione della materia programmabile è il filo che unisce tutti gli sviluppi futuri di MicroBot in una prospettiva coerente. La materia programmabile — nella definizione più ambiziosa del concetto — è un materiale che può modificare arbitrariamente la propria forma, le proprie proprietà meccaniche e le proprie funzionalità in risposta a segnali di controllo, realizzando in modo fisico qualsiasi struttura che possa essere descritta digitalmente. MicroBot nella sua versione attuale è una realizzazione parziale e approssimata di questa visione: può modificare la propria forma aggregandosi in pattern geometrici diversi, può modificare le proprie proprietà meccaniche variando la rigidità degli agganci magnetici tramite il framework 4D/5D, ma non può ancora modificare arbitrariamente la propria funzionalità perché tutti i bot sono identici e svolgono le stesse funzioni. Il passo verso la materia programmabile completa richiede l'introduzione di moduli

specializzati con funzionalità diverse — moduli di sensorizzazione, moduli di attuazione meccanica, moduli di elaborazione intensiva, moduli di comunicazione a lungo raggio — che possono essere aggregati in strutture funzionalmente eterogenee capaci di adattare non solo la propria forma ma anche le proprie capacità computazionali e sensoriali alla specifica applicazione richiesta dall'utente.

La MicroHand è il caso d'uso più concreto e tecnicamente più sviluppato di questa visione di moduli specializzati. Come anticipato nella descrizione della roadmap, la MicroHand non è semplicemente un robot con dita articolate ma è una struttura che emerge dall'aggregazione di moduli MicroBot standard con moduli specializzati di attuazione — moduli che invece delle coil elettromagnetiche integrano piccoli servomotori o attuatori a memoria di forma — producendo una mano robotica riconfigurabile la cui morfologia può essere adattata in tempo reale alla forma dell'oggetto da manipolare. Questa adattabilità morfologica è il vantaggio principale della MicroHand rispetto alle mani robotiche convenzionali a struttura fissa: invece di progettare una geometria di pinza ottimale per una singola categoria di oggetti, la MicroHand può assumere configurazioni diverse ottimizzate per oggetti di forme diverse — una presa a tre dita per oggetti cilindrici, una presa palmare piatta per oggetti lastriformi, una presa a pinza di precisione per oggetti piccoli — semplicemente riassegnando i ruoli dei moduli specializzati nella struttura aggregata.

Lo sviluppo dell'interfaccia di controllo verso paradigmi sempre più naturali e immersivi è la direzione che riguarda più direttamente l'esperienza dell'utente e che determina in modo cruciale l'accessibilità del sistema a utenti non tecnici. L'interfaccia attuale di MicroBot — gesture del polso tramite wristband, selezione del pattern tramite pinch nel visore, modulazione tramite suono attraverso il framework 4D/5D — è già significativamente più naturale delle interfacce testuali o a joystick tipiche dei sistemi di swarm robotics di ricerca, ma rimane lontana dall'ideale di un'interfaccia completamente trasparente in cui l'utente interagisce con lo swarm con la stessa naturalezza con cui interagisce con oggetti fisici nell'ambiente quotidiano. Le direzioni di sviluppo dell'interfaccia che MicroBot intende esplorare nelle versioni future includono il controllo aptico — l'utente sente nella propria mano la resistenza e la consistenza della struttura aggregata attraverso un guanto con attuatori aptici — il controllo vocale — l'utente descrive verbalmente la configurazione desiderata e il sistema interpreta la descrizione naturale attraverso un modello linguistico leggero eseguito sul controller — e il controllo basato sull'intenzione — sensori EMG sul polso rilevano i segnali muscolari che precedono il movimento fisico della mano e li traducono in comandi prima ancora che il movimento sia completato, riducendo la latenza percepita del sistema a valori inferiori al tempo di reazione umano e producendo un'esperienza di controllo che trascende il confine tra il corpo dell'utente e il sistema robotico.

L'integrazione di MicroBot con sistemi di realtà aumentata spaziale — proiettori che sovrappongono informazioni visive direttamente sui bot fisici senza richiedere il visore — apre la possibilità di installazioni pubbliche e performative in cui l'interfaccia digitale è condivisa da molteplici osservatori simultaneamente senza che ciascuno debba indossare un dispositivo dedicato. In questa configurazione i bot diventano pixel fisici di un display tridimensionale che popola lo spazio reale, la cui configurazione è controllata dall'interazione collettiva di più utenti simultaneamente attraverso gesture rilevate da telecamere ambientali, producendo un'esperienza di interazione collettiva con la materia programmabile che non ha

precedenti nel panorama delle installazioni artistiche interattive e che apre applicazioni in ambiti che vanno dall'educazione scientifica alla performance artistica, dalla progettazione collaborativa alla terapia motoria e cognitiva.

Punto 28 — Architettura del software Unity: rendering, sincronizzazione e pipeline XR

Il software Unity che anima il visore VR di MicroBot è il componente che trasforma i dati grezzi del sistema — coordinate dei bot, stati delle batterie, configurazioni degli agganci magnetici, valore del vettore W del framework 4D/5D — in un'esperienza percettiva coerente e immersiva che l'utente può abitare e attraverso cui può esercitare il controllo sullo swarm fisico. Progettare questo software non significa semplicemente creare una visualizzazione tridimensionale dei bot in tempo reale — un problema tecnicamente accessibile con gli strumenti Unity di base — ma significa costruire un sistema che mantiene la coerenza tra la rappresentazione virtuale e la realtà fisica in presenza di latenze di rete variabili, che gestisce il rendering stereoscopico a 60 fotogrammi al secondo senza introdurre latenze percettibili nel tracking della testa, che integra la visione artificiale di MediaPipe con il rendering 3D senza sincronizzazione esplicita dei thread, e che espone all'utente un'interfaccia intuitiva abbastanza semplice da usare durante il controllo dello swarm senza distrarre l'attenzione dalla scena fisica. La complessità di questo sistema non è concentrata in nessun singolo componente ma è distribuita nelle interazioni tra componenti che operano a velocità diverse — il rendering a 60 hertz, il tracking della testa a 100 hertz, la ricezione degli aggiornamenti di stato dello swarm a 20 hertz, il riconoscimento delle gesture a 30 hertz — e che devono cooperare producendo un'esperienza percettiva unificata e temporalmente coerente.

L'architettura del progetto Unity è organizzata attorno a tre livelli di astrazione che rispecchiano la separazione di responsabilità dell'intero sistema MicroBot. Il livello di presentazione contiene tutti i componenti responsabili del rendering visivo della scena — i GameObject che rappresentano i bot, le luci, la griglia di riferimento, il pannello di interfaccia utente — e si aggiorna a ogni frame del ciclo di rendering di Unity, tipicamente a 60 hertz, applicando alle trasformazioni visive degli oggetti i valori più recenti disponibili nelle strutture dati condivise. Il livello di stato contiene le strutture dati che rappresentano lo stato corrente dello swarm — un array di oggetti SwarmBotState che contiene la posizione stimata, lo stato operativo, il livello di carica e la configurazione degli agganci di ogni bot — e viene aggiornato dal livello di comunicazione a ogni ricezione di un pacchetto di aggiornamento dal controller. Il livello di comunicazione gestisce tutta la comunicazione di rete con il controller e con i moduli di visione artificiale, riceve i pacchetti UDP con gli aggiornamenti di stato dello swarm, li deserializza, e aggiorna le strutture dati del livello di stato in modo thread-safe attraverso un meccanismo di double buffering che evita le race condition tra il thread di rete e il thread di rendering principale di Unity.

Il double buffering è un pattern architetturale fondamentale per garantire la coerenza dei dati in un sistema che combina aggiornamenti asincroni di rete con un ciclo di rendering sincrono. L'idea è semplice: invece di un singolo array di SwarmBotState che viene letto dal renderer e scritto dal thread di rete simultaneamente — con il rischio di leggere uno stato parzialmente aggiornato che mescola dati del frame precedente con dati del frame corrente — il sistema mantiene due array: un buffer di lettura che il renderer utilizza durante il frame corrente e un buffer di scrittura che il thread di rete aggiorna con i nuovi dati ricevuti. Al

termine di ogni frame il renderer e il thread di rete si sincronizzano attraverso un lock di breve durata — dell'ordine di pochi microsecondi — durante il quale i puntatori ai due buffer vengono scambiati: il buffer di scrittura diventa il nuovo buffer di lettura per il frame successivo e il vecchio buffer di lettura diventa il nuovo buffer di scrittura disponibile per il thread di rete. Questo meccanismo garantisce che il renderer lavori sempre su uno snapshot coerente dello stato dello swarm — mai su uno stato parzialmente aggiornato — e che il thread di rete possa aggiornare i dati senza mai bloccare il ciclo di rendering per periodi superiori alla durata dello scambio dei puntatori.

La rappresentazione visiva dei bot nella scena Unity è progettata per comunicare simultaneamente informazioni multiple attraverso canali percettivi diversi, minimizzando il carico cognitivo dell'utente durante il controllo dello swarm. La posizione del GameObject che rappresenta ogni bot nella scena 3D riflette la posizione stimata del bot fisico nel piano di simulazione, convertita da coordinate 2D del piano di simulazione in coordinate 3D della scena Unity tramite una trasformazione affine configurabile che permette di scalare e traslare il sistema di riferimento della simulazione nel sistema di riferimento della scena virtuale. Il colore del materiale del bot riflette il suo stato operativo corrente, con lo stesso schema cromatico dei LED fisici — blu per idle, ciano per movimento, verde per pattern stabile, giallo per batteria scarsa, rosso per emergenza — implementato tramite la proprietà Color del materiale Unity aggiornata ogni frame in funzione dello stato corrente del bot. La dimensione del bot — rappresentato come capsula 3D che riproduce la geometria approssimata del modulo fisico a doppio cono — rimane costante per tutti i bot nella versione base, ma nelle versioni avanzate può essere modulata dal valore del vettore W del framework 4D/5D per fornire un feedback visivo intuitivo dello stato interno del sistema: bot che si gonfiano leggermente quando W è basso — stato fluido — e si contraggono quando W è alto — stato rigido. Le connessioni magnetiche attive tra bot agganciati sono visualizzate come cilindri semi-trasparenti tra i centri dei bot coinvolti, con raggio proporzionale alla forza dell'aggancio e colore che varia dal blu al rosso in funzione del duty cycle corrente delle coil, producendo una visualizzazione strutturale dell'aggregato che rivela immediatamente la topologia degli agganci senza richiedere all'utente di interpretare dati numerici.

Il sistema di tracking della testa in Unity XR Toolkit riceve i dati di orientamento dalla BNO055 del visore tramite una connessione seriale o BLE gestita da un componente C# dedicato — l'HeadTrackingDriver — che interroga il sensore a 100 hertz e applica i quaternioni di orientamento ricevuti alla Main Camera della scena Unity. La frequenza di aggiornamento di 100 hertz — superiore alla frequenza di rendering di 60 hertz — garantisce che il tracking della testa sia sempre il più aggiornato possibile al momento del rendering di ogni frame, riducendo al minimo la Motion-to-Photon latency — il ritardo tra il movimento fisico della testa e l'aggiornamento dell'immagine sullo schermo — che è il fattore principale del motion sickness nei sistemi VR. L'HeadTrackingDriver implementa anche un filtro di smoothing sul segnale di orientamento — tipicamente un filtro complementare con costante di tempo di 5–10 millisecondi — che rimuove il rumore ad alta frequenza del sensore giroscopico senza introdurre ritardi percepibili nel tracking dei movimenti naturali della testa.

Il sistema di interfaccia utente nel visore è implementato come un insieme di pannelli Canvas posizionati nello spazio 3D della scena Unity — la modalità World Space Canvas di

Unity — che rimangono fissi rispetto all'orientamento della testa dell'utente a una distanza di visualizzazione confortevole di circa 60 centimetri. Questo approccio produce un'interfaccia che rimane sempre leggibile indipendentemente dai movimenti della testa — i pannelli seguono la testa come se fossero attaccati agli occhiali dell'utente — senza però occupare in modo invasivo il campo visivo centrale che l'utente utilizza per osservare la scena dello swarm. Il pannello principale espone in forma compatta le informazioni di stato più rilevanti: numero di bot connessi, pattern corrente, livello medio di carica della flotta, valore corrente del vettore W rappresentato come barra di progresso, e un indicatore della qualità della connessione Wi-Fi con il controller. Il pannello di selezione del pattern è visualizzato come una ruota circolare di icone che appaiono nel campo visivo periferico quando l'utente esegue il gesto di attivazione del menu con il wristband, permette la selezione del pattern desiderato tramite il gesto di point-and-dwell — guardare l'icona desiderata per 1.5 secondi senza abbassare lo sguardo — e si nasconde automaticamente dopo la selezione per liberare il campo visivo. Il pannello di parametri è un pannello laterale che mostra i parametri geometrici del pattern corrente come slider 3D manipolabili tramite gesture della mano rilevate dalla camera del visore tramite MediaPipe, permettendo la modifica interattiva dei parametri — raggio del cerchio, ampiezza dell'onda, velocità del vortice — senza mai interrompere la visualizzazione dello swarm.

L'integrazione di MediaPipe Hands nel pipeline di rendering Unity è implementata tramite un componente C# — il GestureRecognitionManager — che esegue MediaPipe in un thread separato per evitare che il carico computazionale del riconoscimento delle gesture impatti la frequenza di rendering. Il thread di MediaPipe riceve i frame video dalla camera del visore a 30 hertz, esegue il rilevamento dei landmark delle mani, calcola la classificazione del gesto corrente tramite l'algoritmo di classificazione delle gesture basato sugli angoli delle articolazioni, e deposita il risultato in una struttura dati condivisa che il GestureRecognitionManager legge nel thread principale di Unity durante ogni frame di rendering. I landmark delle mani rilevati da MediaPipe vengono anche utilizzati per il rendering di overlay visivi che mostrano le mani dell'utente nella scena 3D come mesh semi-trasparenti — una forma di realtà aumentata in cui le mani virtuali coincidono con le mani reali dell'utente nel campo visivo — permettendo all'utente di vedere visivamente come i propri gesti interagiscono con i bot virtuali nella scena.

La sincronizzazione temporale tra il software Unity e il controller fisico è gestita tramite un meccanismo di Network Time Protocol semplificato, in cui il visore invia periodicamente richieste di timestamp al controller e calcola il proprio offset temporale rispetto al clock master del controller tramite la formula classica di calcolo dell'offset NTP: $\text{offset} = ((t_2 - t_1) + (t_3 - t_4)) / 2$, dove t_1 è il timestamp locale al momento dell'invio della richiesta, t_2 è il timestamp del controller al momento della ricezione della richiesta, t_3 è il timestamp del controller al momento dell'invio della risposta, e t_4 è il timestamp locale al momento della ricezione della risposta. Questo offset viene applicato a tutti i timestamp locali del visore prima di includerli nei pacchetti inviati al controller, garantendo che gli eventi generati dal visore — selezione di un pattern, modifica di un parametro, attivazione del comando di emergenza — siano temporalmente coerenti con gli eventi registrati dal controller nel log del sistema.

Punto 29 — Testing, debugging e strumenti di sviluppo

Il testing e il debugging di un sistema come MicroBot presentano sfide qualitativamente diverse rispetto al testing di software convenzionale — sfide che nascono dall'interazione tra componenti che appartengono a domini tecnologici radicalmente diversi e che operano su scale temporali e spaziali incommensurabili. Un bug nel firmware di un MicroBot può manifestarsi non come un messaggio di errore o un crash del programma ma come un aggancio magnetico leggermente meno affidabile del previsto, come una deriva temporale che si accumula lentamente nel clock del bot, come un surriscaldamento che si produce solo dopo venti minuti di operazione continua a una temperatura ambiente di 30 gradi centigradi. Un bug nel motore di simulazione JavaScript può produrre pattern che sembrano corretti nella visualizzazione ma che generano collisioni nel sistema fisico reale perché il modello delle forze magnetiche non include un effetto di attrito che nella realtà è significativo. Un bug nel protocollo di comunicazione può passare inosservato per ore di test in condizioni nominali e manifestarsi solo quando la rete SoftAP è sotto carico con 15 bot connessi simultaneamente. Questa natura distribuita, emergente e spesso intermittente dei bug in un sistema cyber-fisico come MicroBot impone una strategia di testing strutturata a livelli, con strumenti dedicati per ciascun livello e un approccio metodologico che non lascia spazio all'ottimismo ingiustificato — se un componente non è stato testato esplicitamente in condizioni di stress, non si può assumere che funzioni correttamente in quelle condizioni.

La strategia di testing di MicroBot è organizzata in quattro livelli gerarchici che coprono il sistema dalla componente più atomica all'integrazione completa. Il primo livello è il test unitario dei moduli firmware, che verifica il comportamento di ciascun modulo software del firmware — `communication.cpp`, `magnet_driver.cpp`, `power_manager.cpp`, `safety_layer.cpp` — in isolamento rispetto agli altri moduli e rispetto all'hardware fisico. Il test unitario del firmware è implementato tramite framework di test come Unity Test Framework — non il motore di gioco Unity ma l'omonimo framework di test per C/C++ embedded — che permette di eseguire i test su PC in ambiente di simulazione Wokwi o direttamente sull'hardware ESP32 tramite la porta seriale. Per ogni modulo viene definita una suite di test che copre i percorsi di esecuzione principali — il percorso nominale in condizioni normali, i percorsi di gestione degli errori per ogni condizione anomala prevista, e i casi limite che corrispondono ai confini degli intervalli di validità dei parametri. Il test del modulo `safety_layer.cpp`, ad esempio, include casi di test che simulano il superamento di ciascuna delle soglie di protezione — batteria scarica, temperatura critica, timeout di comunicazione, watchdog — e verificano che la risposta del modulo corrisponda esattamente al comportamento atteso: riduzione del duty cycle, disattivazione delle coil, transizione allo stato di emergenza. La copertura del codice dei test unitari — la percentuale di righe di codice eseguite almeno una volta nell'intera suite di test — è monitorata tramite strumenti di profiling e mantenuta al di sopra dell'80% come indicatore della completezza della suite.

Il secondo livello è il test di integrazione tra coppie di moduli, che verifica il comportamento del sistema quando due o più moduli interagiscono attraverso le proprie interfacce. Il caso più critico è l'integrazione tra il modulo di comunicazione e il modulo di controllo delle coil: il test verifica che un pacchetto UDP ricevuto con il comando di attivazione di una coil al 30% di duty cycle produca effettivamente un segnale PWM con duty cycle del 30% sul pin corrispondente del microcontrollore, misurato con un oscilloscopio o con un logger GPIO, entro il ritardo massimo specificato di 5 millisecondi dalla ricezione del pacchetto. Il test di integrazione tra il modulo di gestione della batteria e il safety layer verifica che una simulazione della tensione di batteria che scende progressivamente attraverso le soglie di

avviso, riduzione delle prestazioni ed emergenza produca la sequenza corretta di risposte — cambio del colore del LED, riduzione del duty cycle massimo, disattivazione completa delle coil — con i tempi di risposta corretti per ciascuna soglia. Questi test di integrazione richiedono un ambiente di test più complesso rispetto ai test unitari perché necessitano che tutti i moduli coinvolti siano inizializzati correttamente e che le loro interfacce di comunicazione siano configurate in modo coerente, ma producono una verifica molto più significativa della correttezza del sistema reale.

Il terzo livello è il test di sistema su hardware fisico, che verifica il comportamento del sistema completo — firmware del bot, firmware del controller, simulazione JavaScript — in condizioni operative reali. Il test di sistema è strutturato come una sequenza di scenari di test standardizzati che coprono i casi d'uso principali del sistema: avvio e inizializzazione con verifica della connessione di tutti i bot al controller, transizione verso ciascun tipo di pattern con misura del tempo di convergenza e della precisione di posizionamento, esecuzione di 100 cicli di aggancio e disaggancio magnetico tra due bot con misura del tasso di successo e della forza di tenuta, operazione continuata per 30 minuti con misura del consumo energetico e della temperatura delle coil, simulazione di condizioni di guasto — disconnessione forzata di un bot durante una transizione, inserimento di un pacchetto corrotto nel flusso UDP, esaurimento simulato della batteria — con verifica della risposta del safety layer. Ciascuno scenario di test è documentato in una scheda di test che specifica le condizioni iniziali, la sequenza di operazioni da eseguire, i valori attesi delle metriche di performance e la procedura di verifica. I risultati di ogni esecuzione dei test sono registrati in un log con timestamp e confrontati con i risultati delle esecuzioni precedenti per rilevare regressioni introdotte dalle modifiche al firmware.

Il quarto livello è il test di accettazione con utenti reali, che verifica che il sistema produca un'esperienza utente soddisfacente in condizioni di utilizzo normale non controllate in laboratorio. Il test di accettazione prevede sessioni di utilizzo guidato con utenti che non hanno partecipato allo sviluppo del sistema — preferibilmente con background tecnici diversi, per coprire un ampio spettro di aspettative e di approcci al controllo — durante le quali vengono misurate le metriche di performance dell'interfaccia utente descritte nel capitolo sulla validazione sperimentale e raccolti feedback qualitativi attraverso questionari strutturati. I feedback qualitativi sono particolarmente preziosi per identificare aspetti del sistema che funzionano tecnicamente in modo corretto ma che producono un'esperienza utente confusa o frustrante — gesture che il sistema riconosce correttamente ma che gli utenti trovano difficili da eseguire in modo ripetibile, feedback visivi che trasmettono informazioni tecnicamente accurate ma che gli utenti interpretano erroneamente, transizioni di pattern che il sistema esegue correttamente ma che appaiono caotiche e incontrollabili agli osservatori esterni.

Gli strumenti di debugging del firmware sono un aspetto critico dell'infrastruttura di sviluppo di MicroBot. Il tool principale è il monitor seriale — accessibile tramite Arduino IDE, PlatformIO o qualsiasi terminale seriale — che permette di visualizzare in tempo reale i messaggi di log inviati dal firmware tramite la porta UART dell'ESP32. Il firmware di MicroBot implementa un sistema di logging a livelli — DEBUG, INFO, WARNING, ERROR, CRITICAL — con filtro configurabile a compile-time che permette di abilitare il logging dettagliato nelle build di sviluppo e di disabilitarlo completamente nelle build di produzione per ridurre il carico computazionale e l'uso della memoria. Il sistema di logging è implementato tramite macro C

— LOG_DEBUG(), LOG_INFO(), LOG_WARNING() — che includono automaticamente il nome del modulo sorgente, il numero di riga e il timestamp basato su millis() in ogni messaggio, facilitando la correlazione temporale tra eventi in moduli diversi durante il debugging di problemi di sincronizzazione. Per il debugging di problemi che richiedono la visualizzazione di forme d'onda nel tempo — come la verifica del profilo PWM delle coil o l'analisi della deriva del clock — è disponibile un protocollo di output binario su seriale compatibile con il tool SerialPlot che visualizza in tempo reale i valori numerici inviati dal firmware come grafici temporali, eliminando la necessità di un oscilloscopio hardware per la maggior parte delle analisi di segnale a bassa frequenza.

Il debugging del protocollo di comunicazione è facilitato da un tool di monitoring UDP personalizzato implementato come script Python che si connette alla rete SoftAP del controller come stazione client aggiuntiva e intercetta passivamente i pacchetti UDP scambiati tra il controller e i bot. Il tool decodifica la struttura di ogni pacchetto intercettato — numero di sequenza, timestamp, identificativo del destinatario, tipo di comando, parametri, checksum — e lo visualizza in una tabella interattiva nel terminale con evidenziazione dei pacchetti corrotti, dei pacchetti persi rilevati dalla discontinuità dei numeri di sequenza e dei pacchetti ricevuti fuori ordine. Questo tool è indispensabile per il debugging dei problemi di sincronizzazione temporale, di perdita di pacchetti in condizioni di carico elevato e di comportamenti anomali del protocollo di aggregazione che non sono visibili dal log del singolo bot o del controller ma emergono solo dall'analisi del traffico di rete complessivo. Lo script Python implementa anche un modulo di analisi statistica che calcola automaticamente le distribuzioni di latenza, i tassi di perdita per bot e per tipo di comando, e la correlazione temporale tra i comandi inviati e gli heartbeat ricevuti, producendo un report di qualità della comunicazione che può essere confrontato tra sessioni diverse per rilevare degradamenti graduali delle prestazioni della rete wireless.

Il debugging del motore di simulazione JavaScript è supportato dall'integrazione con gli strumenti di sviluppo del browser — Chrome DevTools o Firefox Developer Tools — che forniscono un debugger JavaScript completo con breakpoint, ispezione delle variabili in tempo reale, profiling delle prestazioni e analisi della memoria. Il profiler delle prestazioni è particolarmente utile per identificare i colli di bottiglia computazionali nel game loop della simulazione: con 50 bot attivi simultaneamente il calcolo delle forze di interazione richiede n^2 operazioni di calcolo della distanza per iterazione, e il profiler permette di misurare il tempo effettivamente impiegato in questa fase rispetto alle altre fasi del game loop e di verificare che l'ottimizzazione con griglia spaziale stia producendo la riduzione del carico computazionale attesa. Il debugger JavaScript permette di inserire breakpoint condizionali che si attivano solo quando si verificano condizioni specifiche — ad esempio, quando la distanza tra due bot scende al di sotto della soglia di aggancio per la prima volta — e di ispezionare lo stato completo del sistema in quel preciso istante, facilitando il debugging di comportamenti emergenti che non si manifestano nelle prime fasi della simulazione ma solo dopo un certo numero di iterazioni del game loop.

Il sistema di continuous integration è l'infrastruttura che garantisce che le modifiche al codice non introducano regressioni non rilevate nel comportamento del sistema. Ogni commit al repository del progetto attiva automaticamente una pipeline di CI che esegue la suite completa di test unitari e di integrazione in un ambiente di simulazione virtuale, compila il firmware per tutte le configurazioni target supportate — ESP32, ESP32-C3, ESP32-S3 — e

produce un report di build che include la dimensione dell'immagine firmware compilata, la copertura dei test e l'esito di ogni test unitario. Le build che non superano la suite di test vengono automaticamente rifiutate e il commit viene etichettato come broken nel sistema di controllo versione, garantendo che il ramo principale del repository contenga sempre una versione del firmware che supera tutti i test automatizzati. Per le modifiche agli algoritmi di coordinamento collettivo, la pipeline di CI include anche una fase di test di regressione sulla simulazione JavaScript che esegue automaticamente gli scenari di test standardizzati e confronta le metriche di convergenza e di precisione di posizionamento con i valori di riferimento stabiliti dalla versione precedente del codice, rilevando automaticamente le regressioni nelle prestazioni degli algoritmi che potrebbero non manifestarsi come errori espliciti nei test unitari ma che degraderebbero le prestazioni del sistema nelle condizioni operative reali.

Punto 30 — Aggiornamento firmware OTA, versionamento e gestione della configurazione

Un sistema di swarm robotics fisico composto da molti moduli identici pone un problema pratico che non esiste nella stessa forma per i sistemi software convenzionali: come si aggiorna il firmware di 15 bot identici quando viene rilasciata una nuova versione che corregge un bug critico nel safety layer o introduce un nuovo algoritmo di coordinamento? Nella versione prototipale con pochi bot, la risposta più semplice è connettere fisicamente ciascun bot al computer di sviluppo tramite cavo USB-C e caricare il nuovo firmware tramite il tool di flashing dell'IDE. Questa procedura è accettabile quando i bot sono due o tre e il processo richiede qualche minuto per l'intera flotta, ma diventa rapidamente impraticabile quando la dimensione dello swarm cresce verso le decine di unità: scollegare ogni bot dalla configurazione corrente, aprire l'involucro se il connettore non è accessibile dall'esterno, collegare il cavo, attendere il flashing, richiudere l'involucro e rimettere il bot nella configurazione — una procedura che richiede 3–5 minuti per bot — produce un downtime della flotta di 45–75 minuti per 15 bot, incompatibile con qualsiasi ritmo di sviluppo iterativo che richieda aggiornamenti frequenti. Il sistema di aggiornamento firmware Over-The-Air è la soluzione a questo problema: permette di distribuire nuove versioni del firmware a tutti i bot della flotta attraverso la rete wireless del sistema, senza richiedere accesso fisico a ciascun modulo, riducendo il tempo di aggiornamento dell'intera flotta da ore a pochi minuti indipendentemente dal numero di bot.

Il meccanismo OTA dell'ESP32 è implementato tramite la libreria ArduinoOTA inclusa nel framework Arduino per ESP32, che espone un'interfaccia di aggiornamento firmware via rete compatibile con il protocollo mDNS per il discovery automatico dei dispositivi aggiornabili. Nella configurazione di MicroBot il sistema OTA è integrato nel firmware di ogni bot come modulo aggiuntivo — `ota_manager.cpp` — che viene inizializzato durante la fase di setup del firmware e gestisce il processo di ricezione e validazione del nuovo firmware durante l'operazione normale del bot. Il modulo OTA è progettato per interferire il meno possibile con il funzionamento normale del bot durante le fasi in cui non è in corso un aggiornamento: richiede risorse computazionali trascurabili quando inattivo e diventa attivo solo quando riceve una richiesta di aggiornamento firmata dal controller. La gestione dell'aggiornamento in corso richiede invece risorse significative — il processore è impegnato nella ricezione e nella scrittura del nuovo firmware sulla flash, il modulo Wi-Fi è sotto carico per la trasmissione del binario — e per questa ragione il modulo OTA sospende tutte le attività non

essenziali durante il processo di aggiornamento, portando il bot in uno stato di manutenzione in cui le coil sono disattivate, il LED lampeggia in bianco per indicare l'aggiornamento in corso e il bot non risponde ai comandi di pattern del controller.

L'ESP32 supporta il meccanismo di aggiornamento OTA attraverso un sistema di partizioni della memoria flash che divide lo spazio disponibile in due slot di firmware — slot A e slot B — più una partizione di configurazione e una partizione di dati persistenti. In condizioni normali il firmware attivo è quello presente nel slot A e il boot loader carica automaticamente il firmware da questo slot. Quando viene ricevuto un aggiornamento OTA il nuovo firmware viene scritto nel slot B, senza sovrascrivere il firmware attivo nel slot A — una garanzia fondamentale di sicurezza che assicura che il bot possa sempre tornare al firmware precedente in caso di problemi. Solo dopo la verifica completa dell'integrità del nuovo firmware — tramite checksum SHA-256 calcolato sull'intero binario e confrontato con il valore atteso trasmesso dal controller — il boot loader viene istruito a caricare il firmware dal slot B al prossimo riavvio. Il bot si riavvia automaticamente, carica il nuovo firmware dal slot B, e dopo un periodo di verifica di 60 secondi durante il quale il nuovo firmware deve completare con successo l'inizializzazione e stabilire la connessione con il controller — il boot loader marca il nuovo firmware come valido e lo promuove a firmware attivo permanente. Se il nuovo firmware non completa con successo la fase di verifica entro il timeout — indicazione di un firmware corrotto o incompatibile con l'hardware — il boot loader esegue automaticamente un rollback al firmware precedente nel slot A, garantendo che il bot rimanga sempre operativo anche in presenza di aggiornamenti falliti.

La gestione del processo di aggiornamento OTA dell'intera flotta è responsabilità del controller, che implementa un algoritmo di distribuzione a onde che aggiorna i bot in gruppi sequenziali invece di aggiornare tutti i bot simultaneamente. L'aggiornamento simultaneo di tutti i bot produrrebbe un picco di carico sulla rete SoftAP e sul storage del controller incompatibile con la qualità della connessione richiesta per il trasferimento affidabile dei binari firmware, e lascerebbe l'intera flotta in stato di manutenzione simultaneamente — una condizione accettabile in un contesto di sviluppo ma indesiderata in un contesto operativo dove è preferibile mantenere almeno una parte della flotta operativa durante l'aggiornamento. L'algoritmo a onde divide i bot in gruppi di 3–5 unità, aggiorna il primo gruppo e attende la conferma del completamento con successo dell'aggiornamento — verificata dalla ricezione degli heartbeat post-riavvio con la nuova versione del firmware — prima di procedere con il gruppo successivo. Questo approccio garantisce che in ogni momento almeno il 70% della flotta sia operativa e che i problemi con il nuovo firmware vengano rilevati sul primo gruppo prima di propagare l'aggiornamento all'intera flotta.

Il versionamento del firmware segue lo schema semantico standard a tre livelli — MAJOR.MINOR.PATCH — con regole precise che governano quando ciascun numero di versione viene incrementato. Il numero MAJOR viene incrementato quando viene introdotta una modifica incompatibile con le versioni precedenti del protocollo di comunicazione — una modifica alla struttura dei pacchetti UDP, all'interfaccia dei moduli firmware o al formato dei file di configurazione — che richiede l'aggiornamento coordinato del firmware di tutti i componenti del sistema prima che la nuova versione possa operare correttamente. Il numero MINOR viene incrementato quando vengono aggiunte nuove funzionalità che sono retrocompatibili con le versioni precedenti — un nuovo tipo di pattern, un nuovo parametro del safety layer, una nuova gesture del wristband — che possono essere introdotte

gradualmente nella flotta senza richiedere l'aggiornamento simultaneo di tutti i componenti. Il numero PATCH viene incrementato per le correzioni di bug che non modificano le interfacce esterne del firmware e che sono sempre retrocompatibili. Il controller mantiene un registro della versione firmware di ogni bot connesso, rilevata dall'heartbeat, e visualizza nel pannello di diagnostica dell'interfaccia utente i bot che stanno eseguendo una versione firmware diversa dalla versione corrente, facilitando l'identificazione dei bot che necessitano di aggiornamento.

La gestione della configurazione è complementare alla gestione del firmware e affronta il problema di come mantenere coerenti i parametri operativi dei bot attraverso aggiornamenti del firmware e sessioni operative diverse. I parametri configurabili del sistema — soglie del safety layer, parametri delle forze del modello matematico, duty cycle nominale delle coil, offset di calibrazione dell'IMU del wristband, matrice di mappatura del framework 4D/5D — sono separati dal codice del firmware e memorizzati in una partizione di configurazione dedicata della flash, in formato JSON con schema definito e validato. Questa separazione garantisce che un aggiornamento del firmware non sovrascriva la configurazione personalizzata del bot, che era stata calibrata empiricamente durante le sessioni di test, e permette di ripristinare la configurazione di default in modo esplicito e controllato senza dover reinstallare il firmware. Il controller mantiene un backup centralizzato della configurazione di ogni bot, sincronizzato al momento della connessione e aggiornato ogni volta che un parametro viene modificato tramite l'interfaccia utente. Questo backup centralizzato permette di ripristinare rapidamente la configurazione di un bot dopo una sostituzione dell'hardware o un reset della flash, senza dover ripetere manualmente il processo di calibrazione per ogni parametro.

Il sistema di versionamento del codice sorgente del progetto MicroBot è gestito tramite Git con una struttura di branch che riflette il ciclo di sviluppo iterativo del progetto. Il branch main contiene sempre la versione stabile e testata del firmware, che corrisponde alla versione attualmente in esecuzione sulla flotta fisica. Il branch develop è il branch di integrazione in cui vengono unificati i contributi delle feature branch prima del rilascio, e viene promosso a main solo dopo il completamento della pipeline di CI e dei test di sistema. Le feature branch — una per ogni nuova funzionalità o correzione di bug significativa — partono sempre da develop e vengono unite tramite pull request che richiedono la revisione del codice e il superamento di tutti i test automatizzati prima dell'approvazione. Ogni merge su main genera automaticamente un tag di versione e un changelog che elenca le modifiche introdotte rispetto alla versione precedente, producendo una storia documentata dell'evoluzione del sistema che permette di risalire in qualsiasi momento alla versione del firmware che era in esecuzione durante una specifica sessione operativa e di riprodurre esattamente le condizioni di quella sessione per il debugging di problemi riscontrati post-hoc.

Punto 31 — Interfaccia web di controllo e monitoraggio remoto

L'interfaccia web di controllo è il componente di MicroBot che democratizza l'accesso al sistema — il canale attraverso cui utenti che non dispongono del visore VR o del wristband possono interagire con lo swarm, monitorare il suo stato in tempo reale e inviare comandi di configurazione da qualsiasi dispositivo connesso alla rete locale del controller. La sua presenza nell'architettura non è una duplicazione delle funzionalità del visore ma un livello di

accesso complementare con caratteristiche proprie: mentre il visore offre un'esperienza immersiva ottimizzata per il controllo in tempo reale durante le sessioni operative, l'interfaccia web offre una visualizzazione più analitica e strutturata dello stato del sistema, adatta alla configurazione pre-sessione, al monitoraggio durante sessioni automatizzate e all'analisi post-sessione dei log operativi. L'interfaccia web è accessibile da qualsiasi browser moderno — desktop, tablet o smartphone — connesso alla rete SoftAP del controller, senza installazione di software aggiuntivo e senza dipendenze da runtime specifici, il che la rende il punto di ingresso predefinito per gli utenti che interagiscono con MicroBot per la prima volta e lo strumento di diagnostica primario quando si verificano problemi che impediscono l'utilizzo del visore.

Il server web è implementato direttamente nel firmware del controller tramite la libreria ESPAsyncWebServer, che permette di servire contenuti HTTP e WebSocket in modo asincrono dall'ESP32-S3 senza bloccare il loop principale del firmware. La scelta di ESPAsyncWebServer rispetto alla libreria WebServer standard di Arduino è motivata dalla sua architettura non bloccante: nella libreria standard ogni richiesta HTTP viene gestita in modo sincrono nel thread principale, il che significa che durante l'elaborazione di una richiesta complessa — come la generazione di un log di sessione esteso — il loop del firmware è bloccato e non può eseguire le funzioni critiche di supervisione dello swarm e di risposta ai pacchetti di heartbeat. ESPAsyncWebServer gestisce le richieste HTTP in callback asincrone che vengono eseguite tra un'iterazione e l'altra del loop principale, garantendo che il server web non interferisca mai con le responsabilità di controllo real-time del controller.

L'interfaccia web è implementata come una Single Page Application — un'applicazione web che carica una sola pagina HTML al momento dell'accesso e aggiorna dinamicamente il contenuto tramite JavaScript senza ricaricare la pagina — che comunica con il controller tramite WebSocket per gli aggiornamenti in tempo reale dello stato dello swarm e tramite richieste HTTP REST per le operazioni di configurazione e di accesso ai log. I file statici dell'applicazione — HTML, CSS, JavaScript minificato — sono memorizzati nella flash del controller in una partizione dedicata del filesystem SPIFFS, il filesystem flash di ESP32 che permette di memorizzare file arbitrari nella memoria flash non utilizzata dal firmware. Le dimensioni totali dei file statici dell'interfaccia web sono mantenute al di sotto di 500 kilobyte tramite minificazione del JavaScript, compressione gzip dei file statici e l'uso di librerie leggere preferite a framework pesanti come React o Angular — una scelta imposta dai limiti dello spazio di archiviazione della flash dell'ESP32 che non consente di ospitare bundle JavaScript dell'ordine di megabyte tipici delle applicazioni web moderne.

La pagina principale dell'interfaccia web è divisa in quattro sezioni principali che coprono le diverse necessità di interazione con il sistema. La sezione di panoramica dello swarm occupa la parte superiore della pagina e visualizza una mappa bidimensionale della configurazione corrente dello swarm — aggiornata in tempo reale tramite WebSocket a 5 hertz — con ogni bot rappresentato come cerchio colorato nella propria posizione stimata, il pattern corrente visualizzato come linee tratteggiate che collegano le posizioni target, e un pannello laterale che mostra per ogni bot connesso l'identificativo, il livello di carica della batteria, la temperatura stimata delle coil, lo stato degli agganci magnetici e la versione del firmware in esecuzione. La mappa è interattiva — cliccando su un bot si espande una scheda di dettaglio che mostra le metriche complete del bot — e permette di selezionare

gruppi di bot tramite rettangolo di selezione per l'invio di comandi a sottoinsiemi specifici della flotta.

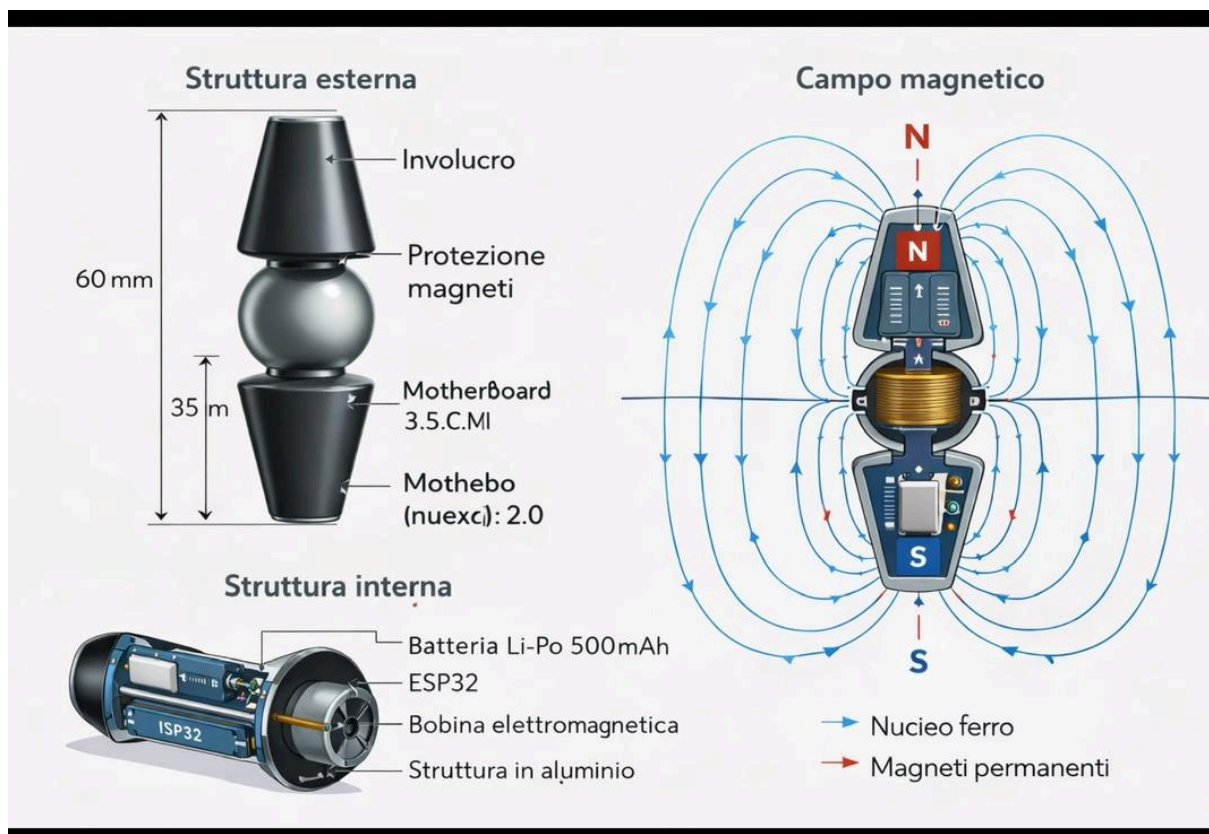
La sezione di controllo del pattern è posizionata sotto la mappa e permette la selezione e la configurazione di tutti i pattern implementati tramite una griglia di pulsanti con anteprime visive e slider per i parametri geometrici. Questa sezione replica le funzionalità del pannello di selezione del pattern del visore VR in una forma più accessibile e dettagliata: mentre il visore espone un sottoinsieme ridotto dei parametri per non sovraccaricare l'interfaccia durante l'uso immersivo, l'interfaccia web espone tutti i parametri configurabili di ogni pattern — raggio, ampiezza, lunghezza d'onda, velocità di rotazione, numero di anelli per il vortice, parametro di passo per la spirale — con campi numerici che permettono l'inserimento di valori precisi oltre ai slider per la regolazione approssimativa. Un pulsante di anteprima permette di visualizzare nella mappa la configurazione target del pattern selezionato con i parametri correnti prima di inviare il comando al controller, permettendo all'utente di verificare visivamente che la configurazione desiderata corrisponda ai parametri inseriti senza dover osservare il comportamento fisico dello swarm durante la transizione.

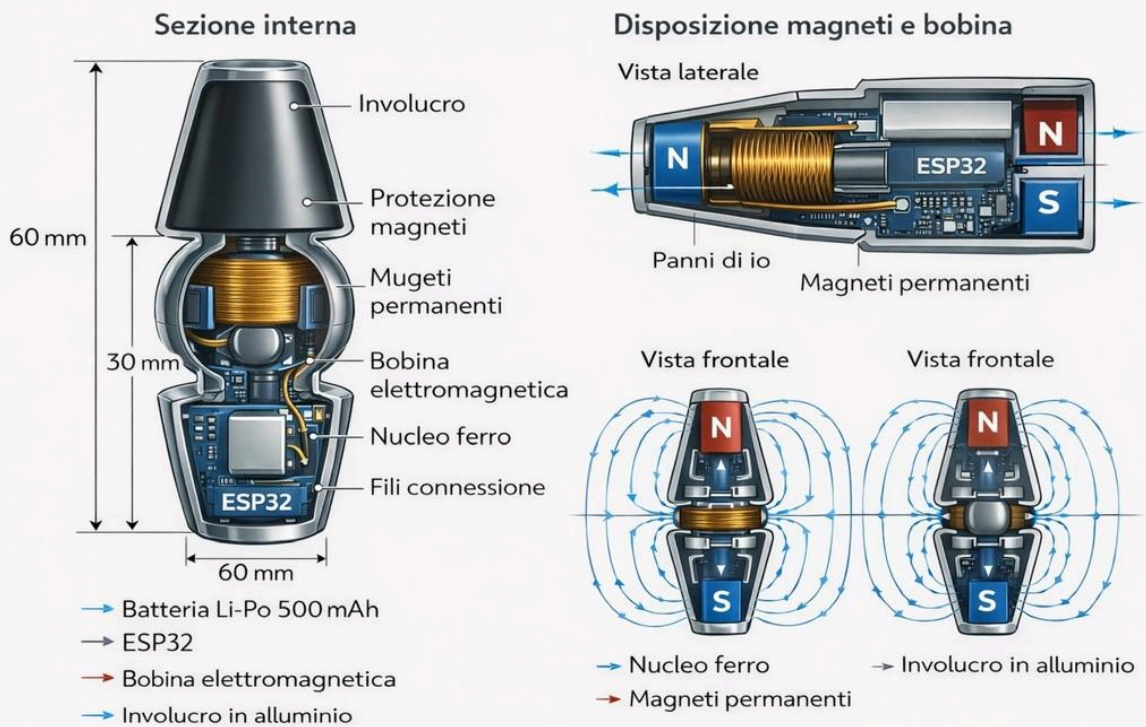
La sezione di configurazione del sistema permette la modifica dei parametri operativi del controller e dei bot attraverso un'interfaccia a form organizzata in tab tematiche. La tab Safety Parameters espone tutte le soglie del safety layer — tensioni di batteria, temperatura critica, timeout di comunicazione, duty cycle massimo — con possibilità di modifica in tempo reale e di ripristino dei valori di default con un singolo clic. La tab Communication Parameters permette di configurare la frequenza degli heartbeat, l'intervallo di sincronizzazione del clock e i parametri di qualità del servizio della rete SoftAP. La tab Framework 4D/5D espone la matrice di mappatura tra parametri acustici e componenti del vettore W, con la possibilità di modificare ogni coefficiente della matrice e di salvare configurazioni di mappatura predefinite per diversi contesti operativi — una configurazione per le sessioni musicali, una per le sessioni di ricerca, una per le dimostrazioni pubbliche. La tab OTA Management espone l'interfaccia per il caricamento di nuovi firmware — tramite un campo di upload file che accetta binari compilati con la toolchain Arduino per ESP32 — e il pannello di gestione dell'aggiornamento OTA che mostra il progresso dell'aggiornamento per ogni bot della flotta in tempo reale.

La sezione di analisi e log è la parte dell'interfaccia web più utile durante le fasi di sviluppo e debugging. Espone il log di sistema del controller in forma di tabella filtrabile e ordinabile — con colonne per timestamp, sorgente dell'evento, livello di severità e descrizione — e permette di scaricare il log completo in formato CSV per l'analisi con strumenti esterni. Un componente di visualizzazione delle metriche nel tempo — implementato tramite la libreria Chart.js per la generazione di grafici nel browser — mostra l'evoluzione temporale delle metriche più rilevanti nelle ultime 60 secondi di operazione: tensioni di batteria dei bot, temperature stimate delle coil, qualità della connessione wireless per bot, latenze dei comandi. Questi grafici temporali sono aggiornati in tempo reale tramite WebSocket e permettono di identificare visivamente tendenze preoccupanti — una batteria che si scarica più rapidamente delle altre, una qualità di connessione che degrada progressivamente in funzione della posizione del bot nella rete — che non sarebbero immediatamente visibili analizzando i valori istantanei senza il contesto temporale.

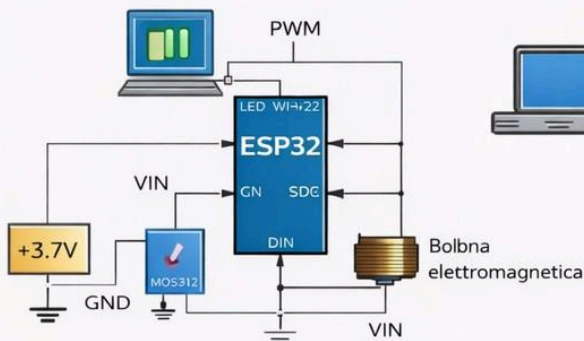
Il sistema di autenticazione dell'interfaccia web è integrato con il sistema di autenticazione biometrica facciale del sistema principale attraverso un meccanismo di token condiviso. Quando l'utente si autentica con successo tramite il sistema biometrico, il controller genera un token di sessione con scadenza configurabile — tipicamente 60 minuti — che viene trasmesso all'interfaccia utente principale e memorizzato localmente nel browser tramite il meccanismo dei cookie HTTP sicuri. Questo token viene incluso in tutte le richieste successive all'interfaccia web come header di autorizzazione HTTP Bearer, permettendo al server web del controller di verificare l'identità dell'utente senza richiedere una nuova autenticazione per ogni richiesta. Le richieste che non includono un token valido vengono reindirizzate a una pagina di accesso che istruisce l'utente a completare l'autenticazione biometrica tramite il sistema principale prima di poter accedere all'interfaccia web, garantendo che l'interfaccia web non rappresenti un canale di bypass del sistema di autenticazione.

La responsività dell'interfaccia web — la sua capacità di adattare il layout alle diverse dimensioni degli schermi dei dispositivi di accesso — è implementata tramite CSS Grid e Flexbox con breakpoint che producono tre layout distinti: il layout desktop con tutte le sezioni visibili simultaneamente su schermi con larghezza superiore a 1024 pixel, il layout tablet con sezioni impilate verticalmente su schermi con larghezza tra 768 e 1024 pixel, e il layout mobile con una singola sezione visibile alla volta navigabile tramite tab su schermi con larghezza inferiore a 768 pixel. Il layout mobile è ottimizzato per il caso d'uso di monitoraggio rapido dello stato dello swarm da smartphone durante sessioni operative in cui le mani dell'utente sono occupate con altri compiti — una verifica del livello di carica dei bot, una visualizzazione dello stato del pattern corrente — e non per il controllo completo del sistema che rimane appannaggio del visore VR e dell'interfaccia desktop.

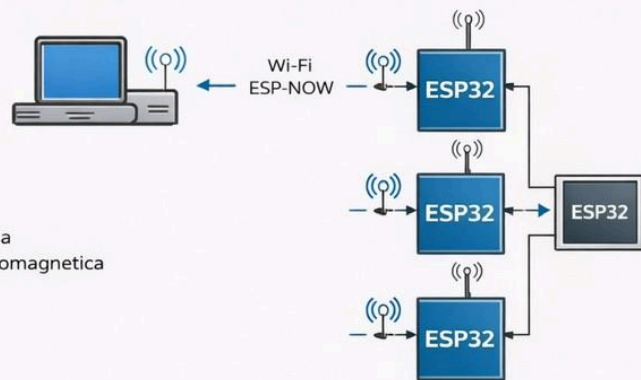




Schema MicroBot



Schema comunicazione controller — MicroBot



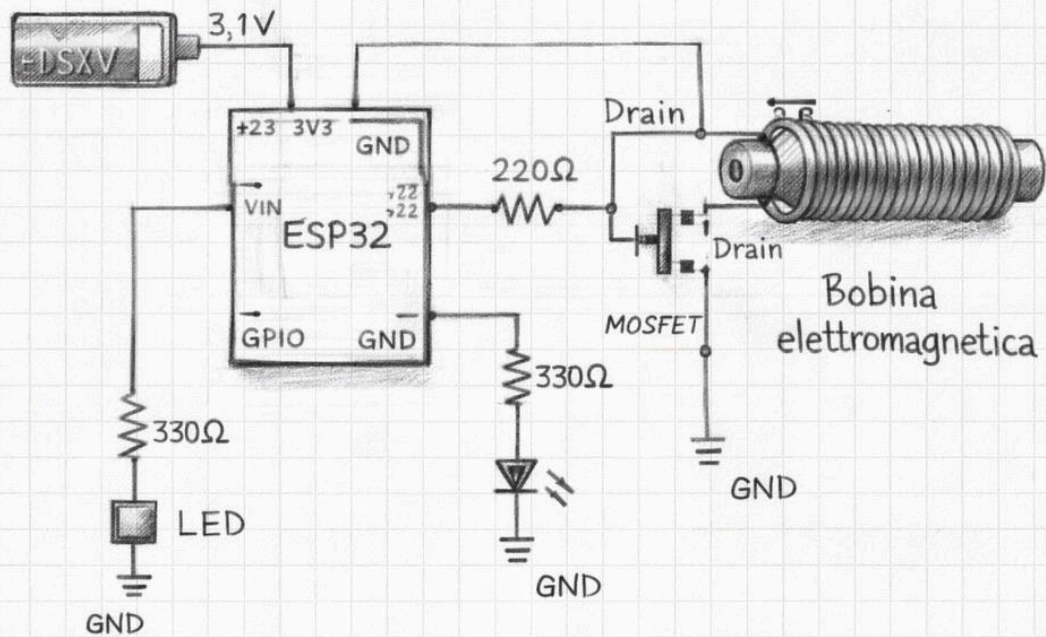
Schema comunicazione controller



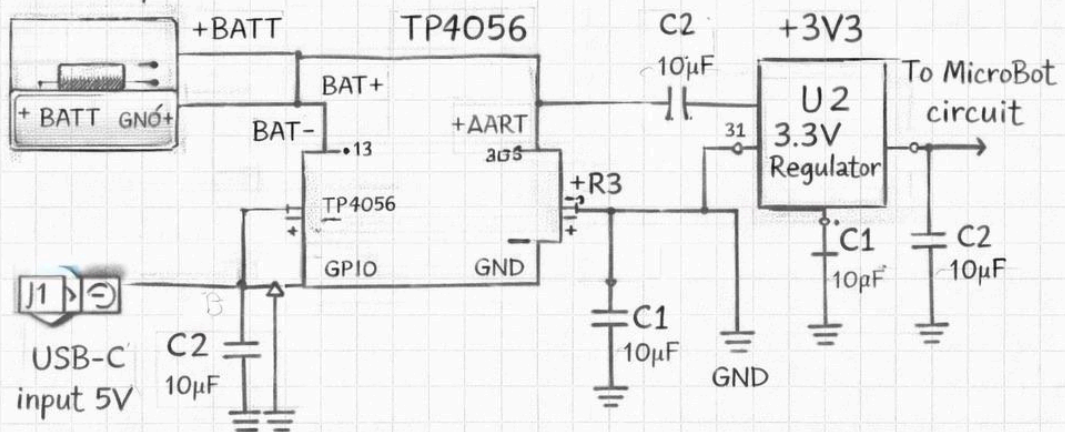
Schema comunicazione MicroBot ↔ MicroBot



Batteria Li-Po 3,7V



BATTERY
3,7V Li-Poly

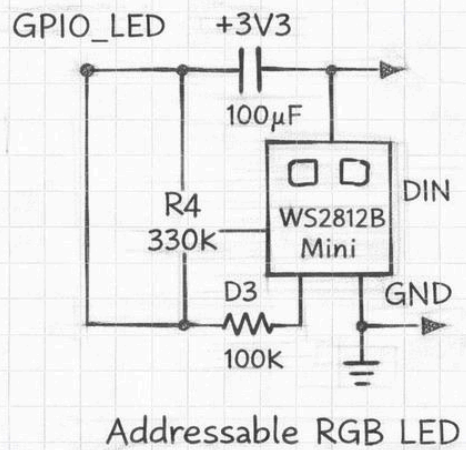
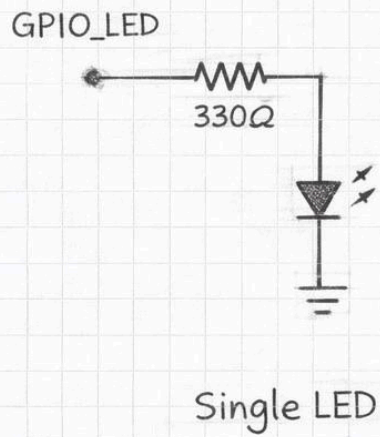


[illegible]

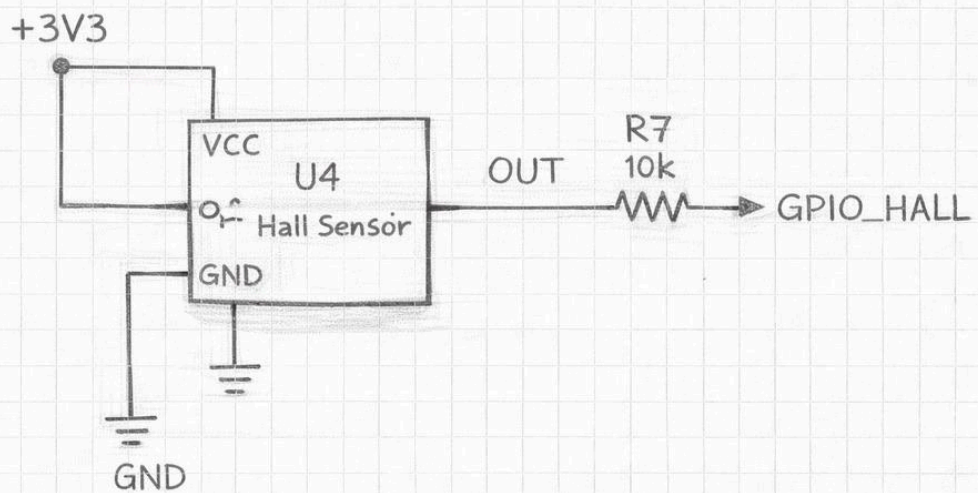
Electromagnetic Coil Driver

The diagram illustrates an electromagnetic coil driver circuit. It features a +BATT 3,7V power supply connected to a GPIO_COIL input. The input signal passes through a 100K resistor (R5) and another 100K resistor before reaching the gate of an N-Channel MOSFET. The MOSFET's source is connected to GND. The drain of the MOSFET is connected to one terminal of the Electromagnetic Coil. A diode D3 is connected in parallel with the coil, with its cathode towards the MOSFET drain and its anode towards the other terminal of the coil. The other terminal of the coil is connected to GND.

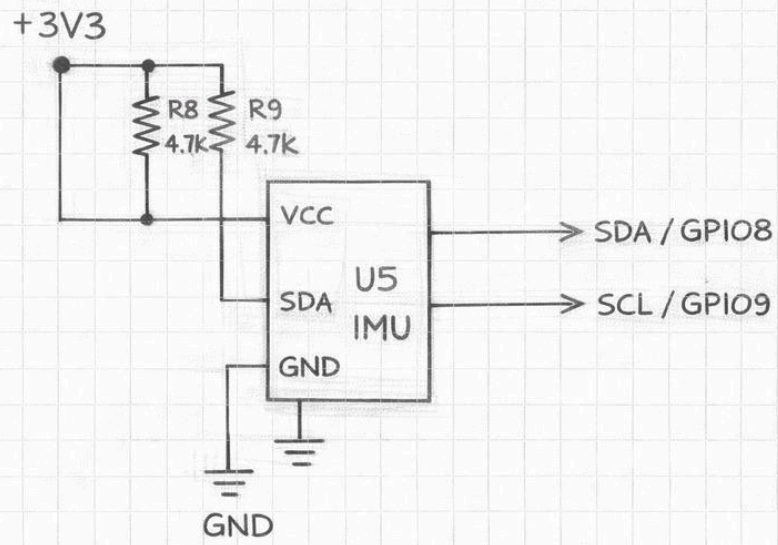
LED Status Indicator



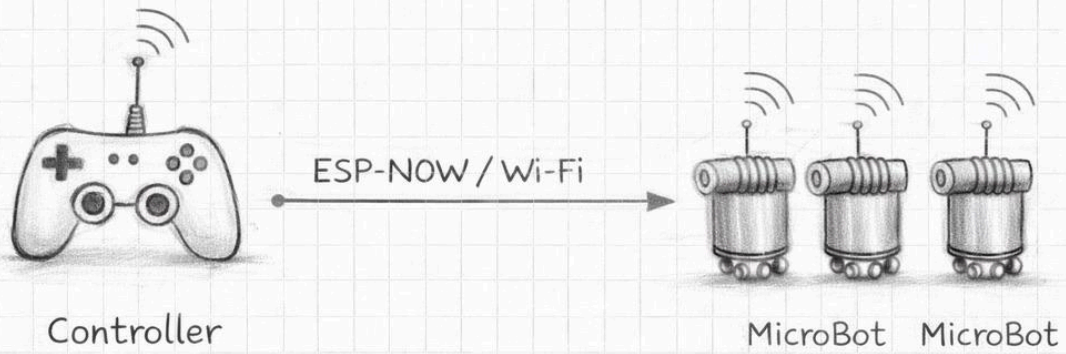
Hall Sensor



IMU



Controller to MicroBot



MicroBot to MicroBot



Punto 32 — Modello di deployment e architettura di rete del sistema

Il deployment di MicroBot — l'insieme delle operazioni necessarie per portare il sistema da uno stato di componenti separati e spenti a uno stato in cui tutti i componenti sono operativi, connessi e pronti a ricevere comandi — è un processo che coinvolge simultaneamente hardware fisico, configurazione di rete, inizializzazione del firmware e verifica dell'integrità del sistema. Progettare il modello di deployment significa progettare l'esperienza di avvio del sistema: quanto tempo richiede passare dallo stato spento allo stato operativo, quante operazioni manuali sono necessarie, quali verifiche vengono eseguite automaticamente e quali richiedono intervento dell'utente, come il sistema gestisce le situazioni in cui non tutti i componenti sono disponibili o funzionanti. Un modello di deployment ben progettato trasforma l'avvio del sistema da una procedura tecnica che richiede competenza specifica a una sequenza di operazioni naturali e guidate che qualsiasi utente può eseguire correttamente, riducendo la barriera all'accesso al sistema e aumentando l'affidabilità delle sessioni operative.

L'architettura di rete di MicroBot è strutturata su tre strati sovrapposti che coprono esigenze di comunicazione diverse. Il primo strato è la rete SoftAP del controller — una rete Wi-Fi locale isolata con SSID dedicato e indirizzamento IP privato nella subnet 192.168.4.0/24 — che costituisce il backbone di comunicazione primario del sistema. Tutti i componenti hardware del sistema — i MicroBot, il wristband in modalità Wi-Fi opzionale, il visore VR, il dispositivo che ospita l'interfaccia web — si connettono a questa rete come stazioni client, con il controller nel ruolo di access point. La separazione della rete SoftAP dalla rete domestica o aziendale preesistente è una scelta progettuale deliberata che produce due vantaggi fondamentali: l'isolamento del traffico di controllo del sistema dal traffico generale della rete, che riduce la latenza e il jitter dei pacchetti UDP di controllo eliminando la competizione con il traffico internet; e la portabilità del sistema, che può essere operato in qualsiasi ambiente senza dipendere dalla disponibilità di infrastruttura di rete preesistente. Il secondo strato è il canale Bluetooth Low Energy tra il wristband e il controller, che opera indipendentemente dalla rete Wi-Fi e fornisce un canale di comunicazione ridondante per i comandi di controllo più critici — il comando di emergenza, la selezione del pattern, la modifica dei parametri principali — che rimangono disponibili anche in caso di degrado della qualità della rete Wi-Fi. Il terzo strato è il canale ESP-NOW tra il controller e i bot nelle versioni più avanzate del sistema — versione 0.4 e successive — che utilizza il protocollo proprietario di Espressif per la comunicazione peer-to-peer a bassa latenza tra dispositivi ESP32 nella stessa rete, complementando il canale UDP per i comandi che richiedono la latenza più bassa possibile.

La sequenza di avvio del sistema segue un ordine prestabilito che garantisce che ogni componente trovi la propria dipendenza già operativa quando tenta di connettersi. Il primo componente da accendere è il controller centrale, che necessita di 3–5 secondi per completare il boot dell'ESP32-S3, inizializzare il filesystem SPIFFS con i file dell'interfaccia web e della configurazione, creare la rete SoftAP con i parametri configurati e avviare i servizi di rete — server UDP, server WebSocket, server HTTP. Il LED del controller completa la sequenza di avvio lampeggiando tre volte in verde per indicare che la rete SoftAP è attiva e il sistema è pronto ad accettare connessioni. Il secondo passo è l'accensione dei MicroBot, che si connettono automaticamente alla rete SoftAP del controller — il cui SSID e la cui password sono memorizzati nella configurazione persistente di ogni bot — completano la

sincronizzazione del clock con il controller e iniziano a trasmettere heartbeat. Il controller visualizza ogni nuova connessione nel log di sistema e aggiorna il contatore di bot connessi nel pannello di stato dell'interfaccia web. Il terzo passo è l'accensione del wristband, che completa la calibrazione dell'IMU attraverso la sequenza guidata di movimenti, stabilisce la connessione BLE con il controller e inizia la trasmissione dei dati di orientamento. Il quarto passo è l'accensione del visore VR — o l'apertura dell'interfaccia web su un dispositivo connesso alla rete SoftAP — che si connette al controller tramite WebSocket, riceve lo snapshot iniziale dello stato completo dello swarm e inizializza la scena Unity con la configurazione corrente. Il quinto e ultimo passo è l'autenticazione biometrica dell'utente, che sblocca l'invio dei comandi al controller e avvia la sessione operativa. L'intera sequenza di avvio richiede tipicamente 60–90 secondi dalla prima accensione allo stato completamente operativo, un tempo accettabile per un sistema di ricerca che non richiede avvii frequenti.

La configurazione della rete SoftAP è un aspetto che richiede attenzione per garantire le prestazioni ottimali della comunicazione wireless nelle condizioni operative tipiche di MicroBot. Il canale Wi-Fi utilizzato dalla rete SoftAP deve essere scelto in modo da minimizzare le interferenze con le reti Wi-Fi preesistenti nell'ambiente operativo: nel caso della banda 2.4 gigahertz — la banda primaria supportata dagli ESP32 — i canali non sovrapposti sono il canale 1, il canale 6 e il canale 11, e il controller seleziona automaticamente quello meno congestionato tramite una scansione dei canali disponibili durante la fase di boot. Il potere trasmissivo della rete SoftAP è impostato al valore minimo che garantisce la copertura dell'area operativa prevista — tipicamente un tavolo o una superficie di lavoro con dimensioni di 1×1 metro — riducendo le interferenze con le reti vicine e il consumo energetico del controller. La potenza trasmissiva di ogni bot è anch'essa configurabile e viene ridotta nelle sessioni in cui tutti i bot si trovano entro 1 metro dal controller, eliminando il consumo associato alla trasmissione a potenza massima in condizioni di prossimità.

Il modello di deployment include anche la gestione della persistenza della configurazione attraverso i riavvii del sistema. I parametri di configurazione del controller — SSID e password della rete SoftAP, lista dei bot autorizzati con i loro identificativi MAC e nomi assegnati, parametri operativi del sistema, configurazione del framework 4D/5D — sono memorizzati nella partizione SPIFFS del controller in formato JSON con schema versionato. Al boot il controller carica questi parametri dalla flash e li utilizza per la configurazione iniziale, garantendo che il sistema riprenda lo stato operativo precedente senza richiedere una nuova configurazione manuale dopo ogni riavvio. I parametri che variano dinamicamente durante l'operazione — come l'offset di sincronizzazione del clock di ogni bot, le statistiche di qualità della connessione e i log delle sessioni operative — sono memorizzati separatamente in una struttura di database leggera basata su file JSON con append incrementale, che permette la consultazione e l'analisi dei dati storici senza richiedere un database server esterno.

La gestione dei conflitti di configurazione — situazioni in cui la configurazione memorizzata nella flash del controller non è compatibile con la configurazione del firmware aggiornato — è gestita attraverso un meccanismo di migrazione della configurazione basato sul numero di versione dello schema. Ogni file di configurazione include un campo version che specifica la versione dello schema con cui è stato generato. Al boot il controller confronta la versione

dello schema del file di configurazione esistente con la versione dello schema supportata dal firmware corrente: se le versioni corrispondono il file viene caricato direttamente, se la versione del file è inferiore a quella supportata dal firmware viene eseguita automaticamente una migrazione che aggiunge i nuovi campi con i valori di default e rimuove i campi obsoleti producendo un file di configurazione compatibile con il nuovo schema. Se la versione del file è superiore a quella supportata dal firmware — situazione che si verifica in caso di rollback a una versione precedente del firmware — il controller genera un avviso nell'interfaccia web e offre all'utente la scelta tra il ripristino della configurazione di default o il caricamento di un backup della configurazione compatibile.

Il sistema di backup e ripristino della configurazione è accessibile dall'interfaccia web attraverso la tab di gestione del sistema. L'utente può creare manualmente un backup dell'intera configurazione del sistema — controller e tutti i bot connessi — in qualsiasi momento, producendo un file JSON compresso che include la configurazione di ogni componente con le proprie versioni di schema. Il backup viene scaricato sul dispositivo dell'utente e può essere ripristinato in qualsiasi momento futuro, permettendo di tornare rapidamente a una configurazione nota-buona dopo esperimenti con parametri non standard o dopo aggiornamenti del firmware che hanno introdotto comportamenti inattesi. Il sistema mantiene automaticamente un backup della configurazione prima di ogni aggiornamento OTA e prima di ogni modifica significativa ai parametri tramite l'interfaccia di configurazione, garantendo che esista sempre un punto di ripristino recente indipendentemente dalla disciplina dell'utente nel creare backup manuali. La funzionalità di confronto tra configurazioni — che evidenzia le differenze tra due file di configurazione selezionati — è uno strumento particolarmente utile per comprendere le cause di variazioni nel comportamento del sistema tra sessioni diverse, permettendo di identificare rapidamente se un comportamento osservato è dovuto a una modifica intenzionale della configurazione o a un aggiornamento del firmware che ha cambiato i valori di default dei parametri.

Il modello di deployment prevede infine un meccanismo di discovery automatico dei bot nella rete SoftAP che elimina la necessità di configurare manualmente l'indirizzo IP o l'identificativo di ogni bot nel controller. Quando un nuovo bot si connette alla rete SoftAP per la prima volta — un bot appena assemblato che non è ancora presente nel database del controller — invia un pacchetto di annuncio con il proprio indirizzo MAC, la versione del firmware e le proprie capacità hardware. Il controller riceve questo annuncio, verifica che il MAC non corrisponda a nessun bot già registrato, genera automaticamente un identificativo univoco per il nuovo bot — un nome generato combinando un aggettivo e un nome di animale da una lista predefinita per produrre identificativi mnemonici come BlueFox-7 o GreenOwl-3 — e registra il nuovo bot nel proprio database con i parametri di configurazione di default. Questo meccanismo di plug-and-play permette di aggiungere un nuovo bot alla flotta semplicemente accendendolo in prossimità del controller, senza operazioni di configurazione manuali, riducendo il tempo necessario per l'integrazione di nuovi moduli nella flotta operativa da decine di minuti a pochi secondi.

Punto 33 — Gestione degli errori, recovery automatico e resilienza del sistema

La resilienza di un sistema cyber-fisico distribuito non si misura dalla frequenza con cui le cose vanno bene ma dalla qualità con cui il sistema gestisce le situazioni in cui le cose vanno male. In un sistema come MicroBot, in cui componenti hardware fisici interagiscono

attraverso canali wireless con componenti software distribuiti su processori embedded con risorse limitate, le situazioni anomale non sono eccezioni rare da gestire con superficialità ma eventi ricorrenti che fanno parte del normale ciclo operativo del sistema: un bot che esaurisce la batteria durante una sessione, una connessione Wi-Fi che si interrompe per un'interferenza momentanea, un MOSFET che si scalda più del previsto per un'attivazione prolungata, un pacchetto UDP corrotto che porta il firmware a uno stato inconsistente. La differenza tra un sistema robusto e un sistema fragile non è nell'assenza di questi eventi — impossibile da garantire nella realtà fisica — ma nella capacità del sistema di rilevarli tempestivamente, rispondere in modo appropriato alle conseguenze immediate e ripristinare il funzionamento normale con il minimo intervento manuale possibile. La gestione degli errori in MicroBot è quindi un sistema progettuale a pieno titolo, con la stessa dignità e la stessa profondità dei componenti funzionali del sistema.

La tassonomia degli errori di MicroBot è organizzata in quattro classi che riflettono la natura e la gravità delle anomalie che il sistema può incontrare. La classe degli errori transitori comprende le anomalie che si risolvono spontaneamente senza intervento del sistema — un pacchetto UDP perso che viene compensato dal pacchetto successivo, un picco di corrente momentaneo che non supera le soglie di protezione, una breve interruzione della connessione Wi-Fi che si risolve con la riconnessione automatica. Per questi errori il sistema non esegue alcuna azione correttiva esplicita ma si limita a registrarli nel log con il livello di severità DEBUG, monitorando la loro frequenza per rilevare situazioni in cui errori transitori singolarmente innocui si manifestano con una frequenza sistematicamente elevata che può indicare un problema strutturale sottostante. La classe degli errori recuperabili comprende le anomalie che richiedono un'azione correttiva esplicita del sistema ma che non compromettono la sessione operativa corrente — un bot che perde la connessione Wi-Fi e si riconnette dopo 2–3 secondi, una coil che raggiunge la soglia di de-rating termico e viene ridotta automaticamente al duty cycle sicuro, un bot che non raggiunge la propria posizione target entro il timeout di convergenza per eccesso di attrito e deve essere riassegnato a una posizione target diversa. Per questi errori il sistema esegue la procedura di recovery appropriata, notifica l'utente attraverso l'interfaccia web e il visore, e registra l'evento nel log con il livello di severità WARNING. La classe degli errori degradanti comprende le anomalie che compromettono parzialmente le prestazioni del sistema ma non ne impediscono il funzionamento — la disconnessione permanente di uno o più bot dalla flotta, la perdita di funzionalità di una coil in un modulo, la riduzione delle prestazioni del canale Wi-Fi per interferenze persistenti. Per questi errori il sistema adatta il proprio comportamento alla capacità ridotta — ridimensionando i pattern alla flotta disponibile, compensando la coil difettosa attraverso una ridistribuzione del carico sulle coil rimanenti — e notifica l'utente con il livello di severità ERROR. La classe degli errori critici comprende le anomalie che richiedono l'interruzione immediata della sessione operativa per garantire la sicurezza del sistema — surriscaldamento critico di un componente, tensione di batteria al di sotto della soglia di emergenza, comportamento del firmware incompatibile con il modello atteso rilevato dal watchdog. Per questi errori il sistema esegue la sequenza di shutdown sicuro e notifica l'utente con il livello di severità CRITICAL, richiedendo un intervento manuale esplicito prima che la sessione possa essere ripresa.

Il meccanismo di recovery dalla perdita di connessione Wi-Fi è il caso di recovery più frequente in MicroBot e il più importante da gestire correttamente perché una perdita di connessione non gestita può lasciare un bot con le coil attive in modalità di mantenimento

senza supervisione del controller, producendo un consumo energetico inutile e potenzialmente un surriscaldamento progressivo se la perdita di connessione persiste. Il firmware del bot implementa un meccanismo di riconnessione automatica con backoff esponenziale: al primo tentativo di riconnessione fallito il bot attende 500 millisecondi prima di ritentare, al secondo fallimento attende 1 secondo, al terzo 2 secondi, al quarto 4 secondi, fino a un intervallo massimo di 30 secondi tra tentativi. Il backoff esponenziale è fondamentale in scenari con molti bot che perdono simultaneamente la connessione — ad esempio a causa di un riavvio del controller — perché senza di esso tutti i bot tenterebbero di riconnettersi contemporaneamente producendo una tempesta di traffico che congestionerebbe il canale Wi-Fi e impedirebbe la riconnessione di tutti. Durante i tentativi di riconnessione il bot mantiene le coil al duty cycle di mantenimento ridotto per un massimo di 10 secondi — sufficiente a mantenere l'aggancio fisico durante una breve interruzione della connessione — trascorsi i quali disattiva completamente le coil e entra in modalità safe-disconnected in attesa della riconnessione. Il LED durante la fase di riconnessione lampeggia in arancione a frequenza crescente per segnalare visivamente il numero di tentativi di riconnessione falliti.

Il recovery dal guasto di un bot durante una transizione di pattern è uno degli scenari più complessi da gestire correttamente perché richiede che il sistema riconosca immediatamente la perdita del bot, rivaluti l'assegnazione delle posizioni target per i bot rimanenti e generi un nuovo piano di transizione che porti la flotta ridotta verso la migliore configurazione possibile del pattern desiderato. Il controller rileva il guasto del bot tramite il timeout di heartbeat — l'assenza di heartbeat per un intervallo superiore a 1500 millisecondi — e avvia immediatamente la procedura di recovery. La procedura valuta la fase corrente della transizione: se la transizione è già al 70% o oltre il completamento, il controller la porta a termine con i bot rimanenti, assegnando le posizioni target del bot mancante ai bot adiacenti tramite l'algoritmo di assegnazione incrementale che minimizza il riorientamento necessario rispetto alla configurazione parzialmente raggiunta. Se la transizione è ancora nelle fasi iniziali, il controller genera un piano di transizione completamente nuovo che esclude il bot mancante, ricalcola le posizioni target del pattern per il numero ridotto di bot e avvia la nuova transizione da zero. In entrambi i casi il controller notifica l'utente attraverso l'interfaccia web e il visore della disconnessione del bot, visualizza la posizione stimata dell'ultimo heartbeat ricevuto nella mappa dello swarm con un'icona di bot disconnesso, e suggerisce le azioni possibili — attendere la riconnessione automatica del bot, rimuovere manualmente il bot dalla configurazione fisica, o procedere con la flotta ridotta.

Il recovery dal surriscaldamento di un singolo bot è gestito attraverso un protocollo a tre stadi che bilancia la necessità di proteggere il componente con la necessità di mantenere la coerenza della struttura aggregata. Il primo stadio è il de-rating automatico delle coil, già descritto nel capitolo sul safety layer, che riduce il duty cycle in funzione della temperatura stimata senza interrompere il pattern corrente — una misura che nella maggior parte dei casi è sufficiente a fermare il surriscaldamento progressivo e a riportare la temperatura nella zona sicura entro 30–60 secondi. Il secondo stadio viene attivato se la temperatura continua a crescere nonostante il de-rating: il bot notifica il controller tramite un flag nell'heartbeat, il controller lo esclude temporaneamente dal piano di aggancio magnetico — il bot mantiene la propria posizione ma non esegue nuovi agganci — e aumenta l'intervallo di aggiornamento degli heartbeat del bot a 1 secondo per ridurre il carico computazionale e l'emissione radio che contribuiscono marginalmente al calore generato. Il terzo stadio viene attivato se la

temperatura supera la soglia critica nonostante le misure dei primi due stadi: il bot disattiva completamente le coil, si disaggancia dai bot vicini e rimane nella propria posizione senza interazioni magnetiche fino al termine del cooldown di 60 secondi, trascorso il quale riprende la normale partecipazione al pattern con un duty cycle massimo ridotto del 30% rispetto al valore nominale fino al termine della sessione operativa corrente.

Il meccanismo di recovery dal freeze del firmware — rilevato dal watchdog hardware che non viene resettato entro il timeout di 5 secondi — produce un riavvio completo dell'ESP32 che porta il bot dallo stato di freeze a uno stato inizializzato pulito in circa 3 secondi. Dopo il riavvio il bot esegue automaticamente la procedura di startup e si riconnette al controller tramite la procedura di riconnessione automatica. Il controller, che ha rilevato il timeout di heartbeat durante il periodo di freeze e il riavvio, accoglie la riconnessione del bot aggiornando la sua voce nella tabella di stato globale e includendolo nel piano di pattern corrente se il pattern è ancora attivo. Crucialmente, il riavvio per watchdog viene registrato nel log con un flag specifico che distingue il riavvio normale — per aggiornamento OTA o comando esplicito — dal riavvio di emergenza per watchdog, permettendo di identificare bot che subiscono freeze frequenti come candidati per l'analisi del firmware alla ricerca di loop bloccanti o memory corruption.

Il sistema di self-healing dello swarm è una capacità emergente che nasce dalla combinazione del recovery automatico dei singoli bot con la gestione adattiva del piano di pattern del controller. Quando più bot subiscono errori recuperabili contemporaneamente — un evento plausibile durante una tempesta di interferenze Wi-Fi o un calo di tensione dell'alimentazione nella stanza — il controller coordina le procedure di recovery individuali in modo da minimizzare la frammentazione temporanea della struttura collettiva, prioritizzando il recovery dei bot che fanno parte di sottostrutture aggregate fisicamente critiche — i bot centrali di un pattern circolare, i bot di giunzione tra due segmenti di una linea — rispetto ai bot periferici la cui assenza temporanea degrada il pattern in modo visivamente meno impattante. Questa prioritizzazione del recovery non richiede logica aggiuntiva esplicita nel controller ma emerge naturalmente dall'algoritmo di riassegnazione adattiva che, nel ricalcolare il piano di pattern per la flotta disponibile, produce configurazioni che mantengono la coerenza geometrica globale anche con sottoinsiemi significativi dei bot temporaneamente non disponibili. Il risultato è un sistema che non crolla in presenza di guasti parziali ma degrada gradualmente e in modo controllato verso configurazioni semplificate che mantengono la leggibilità del pattern inteso anche con una frazione significativa dei bot non disponibili — una proprietà che in letteratura viene chiamata graceful degradation e che distingue i sistemi di swarm robotics maturi dai prototipi che funzionano solo in condizioni ideali.

Punto 34 — Layer di astrazione software e architettura API del sistema

Il layer di astrazione software è il componente architetturale che separa la logica applicativa di alto livello — gli algoritmi di coordinamento dello swarm, la logica del framework 4D/5D, il motore di simulazione — dall'implementazione specifica dei componenti hardware sottostanti — il protocollo UDP con cui il controller comunica con i bot, il formato binario dei pacchetti di heartbeat, la struttura della rete SoftAP. Questa separazione non è una raffinatezza architettonica superflua ma una necessità pratica che diventa evidente nel momento in cui si vuole modificare un aspetto del sistema senza dover riscrivere tutto il

codice che dipende da quel componente: se il protocollo di comunicazione tra controller e bot cambia dalla versione 0.3 alla versione 0.4 — ad esempio introducendo la topologia gerarchica con subcontroller — il codice del motore di simulazione e del framework 4D/5D non dovrebbe richiedere alcuna modifica, perché lavora a un livello di astrazione superiore che non conosce i dettagli del protocollo di comunicazione. Senza un layer di astrazione ben definito questa separazione è impossibile da garantire e qualsiasi modifica strutturale al sistema si propaga in modo imprevedibile attraverso tutti i componenti del codice, producendo il fenomeno noto come software rot — la degradazione progressiva della qualità architetturale del codice dovuta all'accumulo di dipendenze non intenzionali tra componenti.

Il layer di astrazione di MicroBot è implementato come un insieme di interfacce — nel senso delle interfacce dei linguaggi di programmazione orientati agli oggetti, ovvero contratti che specificano quali operazioni un componente deve esporre senza specificare come le implementa — che definiscono i confini tra i diversi strati del sistema. L'interfaccia SwarmController è la più importante di queste astrazioni: definisce l'insieme di operazioni che il motore di simulazione e il framework 4D/5D possono richiedere allo strato di controllo hardware — `invia_comando_pattern`, `ottieni_stato_swarm`, `registra_gestore_eventi`, `attiva_emergenza` — senza specificare se queste operazioni vengono eseguite tramite pacchetti UDP verso un controller ESP32 fisico, tramite messaggi in memoria verso un simulatore JavaScript, o tramite chiamate a un'API REST verso un controller remoto. Il codice del motore di simulazione e del framework 4D/5D dipende esclusivamente dall'interfaccia SwarmController e può quindi essere eseguito senza modifiche contro qualsiasi implementazione concreta di questa interfaccia — una proprietà che è il fondamento della testabilità del sistema.

L'implementazione concreta dell'interfaccia SwarmController per il sistema fisico è la classe `ESP32SwarmController`, che traduce le operazioni dell'interfaccia in operazioni di rete concrete: la chiamata a `invia_comando_pattern` produce la serializzazione del comando nel formato binario del protocollo UDP, l'invio del pacchetto UDP al controller tramite socket di rete e la registrazione del comando nel log locale con il timestamp corrente.

L'implementazione per la simulazione è la classe `SimulatedSwarmController`, che traduce le operazioni dell'interfaccia in aggiornamenti delle strutture dati del motore di simulazione JavaScript senza alcuna comunicazione di rete. Queste due implementazioni sono intercambiabili dal punto di vista del motore di simulazione e del framework 4D/5D, che non possono distinguere se stanno controllando un sistema fisico reale o una simulazione puramente virtuale — una proprietà fondamentale per il testing e per lo sviluppo di nuove funzionalità senza accesso all'hardware fisico.

L'API pubblica del sistema MicroBot è l'insieme di endpoint e interfacce attraverso cui il sistema espone le proprie funzionalità a componenti esterni — applicazioni di terze parti, script di automazione, sistemi di analisi dati — che vogliono interagire con lo swarm senza conoscere i dettagli interni dell'architettura. L'API è implementata come una API REST esposta dal server web del controller tramite `ESPAsyncWebServer`, con endpoint organizzati in risorse logiche che rispecchiano i concetti principali del dominio di MicroBot. La risorsa `/api/swarm` espone le operazioni globali sullo swarm — `GET /api/swarm` per ottenere lo stato completo della flotta, `POST /api/swarm/pattern` per inviare un comando di pattern con i parametri specificati nel corpo della richiesta in formato JSON, `POST /api/swarm/emergency` per attivare il comando di emergenza globale. La risorsa `/api/bots` espone le operazioni sui

singoli bot — GET `/api/bots` per ottenere la lista di tutti i bot connessi con il loro stato, GET `/api/bots/{id}` per ottenere lo stato dettagliato di un bot specifico, POST `/api/bots/{id}/command` per inviare un comando direttamente a un singolo bot. La risorsa `/api/config` espone le operazioni di configurazione — GET `/api/config` per ottenere la configurazione corrente del sistema, PUT `/api/config` per aggiornare la configurazione con i valori specificati nel corpo della richiesta. La risorsa `/api/logs` espone l'accesso ai log del sistema — GET `/api/logs` per ottenere i log recenti con parametri di filtro per livello di severità, sorgente e intervallo temporale, GET `/api/logs/download` per scaricare il log completo della sessione corrente in formato CSV.

Il formato di scambio dati dell'API è JSON per tutte le richieste e le risposte, con uno schema documentato e versionato che include un campo `api_version` in ogni risposta per permettere ai client di verificare la compatibilità con la versione dell'API installata sul controller. La risposta standard di una richiesta GET `/api/swarm` produce un documento JSON con la seguente struttura: un campo `status` con il valore corrente dello stato globale del sistema, un campo `pattern` con il tipo e i parametri geometrici del pattern corrente, un campo `w_vector` con il valore corrente del vettore W del framework 4D/5D, e un campo `bots` che è un array di oggetti ciascuno dei quali descrive lo stato di un singolo bot con i campi `id`, `position`, `battery_voltage`, `temperature_estimated`, `coil_duty_cycle`, `magnet_state`, `firmware_version` e `last_heartbeat_timestamp`. Questo documento JSON viene prodotto dal controller serializzando la tabella di stato globale dello swarm nel formato richiesto, operazione che richiede tipicamente 5–10 millisecondi sull'ESP32-S3 per una flotta di 20 bot — un tempo accettabile per richieste HTTP che non richiedono la stessa latenza delle comunicazioni UDP di controllo.

Il sistema di notifiche push è il meccanismo attraverso cui il controller notifica i client dell'API degli eventi che si verificano nel sistema senza richiedere il polling continuo — una tecnica che con una frequenza di aggiornamento di 10 hertz e 10 client connessi produrrebbe 100 richieste HTTP al secondo sul server del controller, un carico incompatibile con le risorse dell'ESP32-S3. Le notifiche push sono implementate tramite WebSocket — la stessa connessione utilizzata dall'interfaccia web — attraverso cui il controller invia messaggi JSON ai client connessi ogni volta che si verificano eventi significativi nel sistema: connessione o disconnessione di un bot, transizione di pattern completata, attivazione del safety layer, modifica del valore di W , ricezione di un aggiornamento di posizione da tutti i bot. I client dell'API che necessitano di aggiornamenti in tempo reale si connettono al WebSocket del controller e registrano i tipi di eventi che vogliono ricevere tramite un messaggio di sottoscrizione, ricevendo notifiche solo per gli eventi di interesse senza generare traffico di polling inutile.

Il sistema di plugin è il meccanismo che permette di estendere le funzionalità del sistema MicroBot senza modificare il codice core — una capacità essenziale per un progetto di ricerca in cui nuovi esperimenti richiedono frequentemente l'aggiunta di nuovi comportamenti che non erano stati previsti nella progettazione originale. Il sistema di plugin è implementato come un insieme di hook — punti di estensione predefiniti nel ciclo di esecuzione del sistema dove codice esterno può essere iniettato per modificare o aumentare il comportamento standard. Gli hook principali disponibili sono: `pre_pattern_transition`, chiamato prima dell'inizio di ogni transizione di pattern con la configurazione corrente e quella target come parametri, che permette a un plugin di modificare il piano di transizione

prima della sua esecuzione; `post_heartbeat_received`, chiamato dopo la ricezione e la decodifica di ogni pacchetto di heartbeat con i dati del bot come parametro, che permette a un plugin di eseguire analisi personalizzate sui dati di telemetria; `w_vector_update`, chiamato ogni volta che il valore del vettore W del framework 4D/5D viene aggiornato, che permette a un plugin di implementare logiche di mappatura personalizzate tra segnali esterni e stato interno del sistema; `safety_layer_check`, chiamato a ogni iterazione del loop del safety layer, che permette a un plugin di aggiungere condizioni di sicurezza personalizzate alle verifiche standard del sistema. I plugin sono implementati come moduli JavaScript per l'ambiente di simulazione e come librerie C++ per il firmware del controller, con interfacce standardizzate che specificano i tipi dei parametri e i valori di ritorno attesi per ogni hook.

Il meccanismo di dependency injection è lo strumento che rende il sistema di plugin e il layer di astrazione concretamente utilizzabili nel codice del motore di simulazione e del controller. Invece di creare direttamente le istanze delle classi concrete — `ESP32SwarmController`, `UDPCommunicationModule`, `LocalStorageLogger` — il codice del motore di simulazione e del framework 4D/5D riceve le istanze delle proprie dipendenze attraverso il costruttore o tramite un container di dependency injection configurato all'avvio del sistema. Questa tecnica permette di configurare il sistema con implementazioni diverse delle stesse interfacce senza modificare il codice che le utilizza: in un ambiente di test si iniettano implementazioni mock che producono comportamenti controllati e verificabili, in un ambiente di produzione si iniettano le implementazioni reali che comunicano con l'hardware fisico, in un ambiente di sviluppo si iniettano implementazioni ibride che combinano componenti reali e simulati in funzione della disponibilità dell'hardware. Il container di dependency injection di MicroBot è implementato come un registro di factory functions — funzioni che producono istanze di componenti con le dipendenze appropriate — configurabile tramite file JSON che specifica quale implementazione concreta deve essere fornita per ogni interfaccia in ciascun ambiente di esecuzione.

La documentazione dell'API pubblica è generata automaticamente dal codice sorgente tramite commenti in formato JSDoc per i componenti JavaScript e Doxygen per i componenti C++ del firmware, producendo documentazione HTML interattiva con esempi di richiesta e risposta per ogni endpoint, descrizione dei parametri accettati e dei codici di errore restituiti. Questa documentazione è ospitata direttamente sul server web del controller all'endpoint `/api/docs`, accessibile da qualsiasi browser connesso alla rete SoftAP, eliminando la necessità di documentazione separata che può divergere dal comportamento effettivo del codice — un problema comune nei sistemi in rapida evoluzione in cui la documentazione manuale viene aggiornata meno frequentemente del codice che descrive.

Punto 35 — Pipeline di dati, analisi sperimentale e machine learning

La pipeline di dati di MicroBot è l'infrastruttura che trasforma le osservazioni grezze del sistema — i pacchetti di heartbeat con le metriche di ogni bot, i log del safety layer, i timestamp degli eventi di coordinamento, le traiettorie dei bot durante le transizioni di pattern — in conoscenza strutturata che può essere analizzata, visualizzata e utilizzata per migliorare il comportamento del sistema nelle sessioni successive. Senza questa infrastruttura, ogni sessione operativa produce dati che vengono consumati in tempo reale per il controllo del sistema e poi dispersi senza lasciare traccia permanente, rendendo impossibile l'analisi comparativa tra sessioni diverse, l'identificazione di tendenze nel

comportamento del sistema nel tempo e l'addestramento di modelli di apprendimento automatico che potrebbero migliorare le prestazioni del coordinamento collettivo. Con questa infrastruttura, ogni sessione operativa produce un contributo permanente alla base di conoscenza del sistema — un deposito di esperienza strutturata che cresce con ogni sessione e che abilita capacità di analisi e di apprendimento che non sarebbero possibili con i soli dati della sessione corrente.

La raccolta dei dati avviene in tempo reale durante ogni sessione operativa attraverso tre canali distinti che coprono diversi aspetti del comportamento del sistema. Il canale di telemetria raccoglie i dati di stato di ogni bot a ogni ricezione di heartbeat — tensione di batteria, temperatura stimata, duty cycle delle coil, stato degli agganci magnetici, qualità della connessione wireless, posizione stimata — e li memorizza in una struttura di time series organizzata per bot e per timestamp. Il canale degli eventi raccoglie gli eventi discreti del sistema — connessioni e disconnessioni dei bot, transizioni di pattern, attivazioni del safety layer, comandi dell'utente — con il timestamp preciso al millisecondo di ciascun evento e il contesto necessario per la sua interpretazione — il tipo di pattern richiesto, i parametri geometrici, il numero di bot disponibili al momento del comando. Il canale delle traiettorie raccoglie le posizioni stimate di tutti i bot a frequenza regolare — tipicamente a 5 hertz — durante le fasi di transizione tra pattern, producendo una rappresentazione cinematografica del comportamento collettivo dello swarm che può essere analizzata per calcolare le metriche di convergenza, precisione di posizionamento e qualità della traiettoria descritte nel capitolo sulla validazione sperimentale.

Il formato di memorizzazione dei dati è progettato per essere leggibile sia dall'interfaccia web del controller per la visualizzazione in tempo reale che dagli strumenti di analisi offline. I dati di telemetria sono memorizzati in formato CSV con colonne per timestamp, identificativo del bot e valore di ciascuna metrica — un formato compatibile con pandas di Python, con Excel e con qualsiasi tool di analisi dati generico senza preprocessing. I dati degli eventi sono memorizzati in formato JSON Lines — un file di testo in cui ogni riga è un documento JSON valido che descrive un singolo evento — che combina la struttura flessibile del JSON con la facilità di elaborazione riga per riga tipica dei file di testo. Le traiettorie sono memorizzate in formato NumPy .npz compresso per i file di grandi dimensioni prodotti da sessioni lunghe con molti bot, che riduce significativamente lo spazio di archiviazione rispetto al CSV a scapito della leggibilità diretta senza librerie Python. Tutti i file di dati di una sessione sono raggruppati in una cartella con nome generato dal timestamp di inizio sessione e da un identificativo descrittivo specificato dall'utente — ad esempio 20260313_143022_cerchio_8bot — e possono essere scaricati dall'interfaccia web come archivio compresso per l'analisi offline.

Il sistema di analisi offline è implementato come un insieme di notebook Jupyter che forniscono template riutilizzabili per i tipi di analisi più comuni. Il notebook di analisi delle prestazioni carica i dati di una o più sessioni, calcola automaticamente le metriche di convergenza, precisione di posizionamento e consumo energetico per ogni transizione di pattern registrata, e produce grafici comparativi che mostrano l'evoluzione delle prestazioni nel tempo — permettendo di verificare se le modifiche al firmware introdotte tra una sessione e l'altra hanno prodotto miglioramenti reali nelle metriche obiettivo. Il notebook di analisi delle traiettorie visualizza le traiettorie dei bot durante le transizioni di pattern come animazioni che possono essere riprodotte a velocità variabile, con sovrapposizione delle

forze calcolate dal modello matematico per ogni bot in ogni istante — uno strumento fondamentale per la comprensione intuitiva del comportamento emergente dello swarm e per l'identificazione delle fasi della transizione in cui il comportamento reale si discosta maggiormente dal comportamento modellato. Il notebook di analisi delle anomalie applica algoritmi di rilevamento delle anomalie — isolation forest, local outlier factor — ai dati di telemetria per identificare automaticamente i bot che si comportano in modo anomalo rispetto agli altri — consumano significativamente più energia, si surriscaldano più rapidamente, hanno una qualità di connessione sistematicamente inferiore — segnalando i candidati per l'ispezione fisica e la manutenzione preventiva prima che i problemi divengano critici durante una sessione operativa.

L'integrazione del machine learning nel sistema MicroBot è pianificata come una progressione graduale che introduce capacità di apprendimento senza comprometterne la prevedibilità e la sicurezza. Il primo livello di integrazione — previsto nella versione 0.4 — è l'apprendimento supervisionato per la classificazione delle gesture del wristband. Il dataset di training è generato automaticamente durante le sessioni operative registrando le sequenze di dati IMU del wristband insieme all'etichetta del gesto corrispondente — confermata dall'esito corretto del riconoscimento da parte dell'algoritmo a soglie correnti. Con un dataset di 1000–2000 esempi per tipo di gesture — raggiungibili in poche sessioni operative — è possibile addestrare un classificatore leggero come un Support Vector Machine o una rete neurale con due strati nascosti di piccole dimensioni, che può essere eseguito in tempo reale sull'ESP32-C3 del wristband grazie alla libreria TensorFlow Lite for Microcontrollers. Il classificatore addestrato sui dati reali dell'utente specifico produce tipicamente un tasso di riconoscimento superiore del 5–10% rispetto all'algoritmo a soglie universale, perché apprende le caratteristiche cinematiche specifiche dei gesti di quel particolare utente — la velocità di esecuzione, l'ampiezza del movimento, il profilo di accelerazione — invece di applicare soglie universali che non tengono conto delle variazioni individuali nello stile di esecuzione dei gesti.

Il secondo livello di integrazione — previsto nella versione 0.5 — è l'apprendimento per rinforzo per l'ottimizzazione dei parametri di coordinamento dello swarm. L'idea è di addestrare un agente di reinforcement learning che osserva lo stato globale dello swarm — posizioni dei bot, stato degli agganci, valore del vettore W , tipo di pattern corrente — e decide i valori ottimali dei parametri di coordinamento — duty cycle delle coil, soglie di aggancio, velocità di propagazione dell'onda — che massimizzano una funzione di ricompensa definita come combinazione della velocità di convergenza al pattern target, della stabilità degli agganci e del consumo energetico. L'addestramento dell'agente avviene inizialmente nella simulazione JavaScript — dove è possibile eseguire migliaia di episodi di training in pochi minuti — e il modello addestrato viene poi trasferito al firmware del controller per l'esecuzione in tempo reale sul sistema fisico. Il processo di transfer learning dalla simulazione alla realtà — noto in letteratura come sim-to-real transfer — introduce tipicamente una degradazione delle prestazioni dovuta alle differenze tra il modello simulato e la fisica reale, che viene compensata attraverso una fase di fine-tuning sul sistema fisico con un numero ridotto di episodi di esplorazione che adattano il modello appreso nella simulazione alle specificità del sistema reale.

Il terzo livello di integrazione — oltre la versione 0.5, nella prospettiva a lungo termine — è l'apprendimento della struttura ottimale dei pattern in funzione delle preferenze dell'utente.

Un sistema di raccomandazione che osserva le sequenze di pattern selezionate dall'utente nelle sessioni operative — quali pattern vengono scelti in quale ordine, quali parametri vengono modificati e in quale direzione, quali pattern vengono abbandonati rapidamente e quali vengono mantenuti a lungo — può imparare un modello delle preferenze estetiche dell'utente e proporre proattivamente pattern e transizioni che l'utente troverebbe interessanti, trasformando l'interfaccia di controllo da un sistema passivo che esegue i comandi ricevuti a un sistema attivo che anticipa e suggerisce possibilità creative che l'utente non avrebbe esplorato autonomamente. Questo livello di integrazione richiede la raccolta di dati comportamentali dell'utente nel tempo — una base di dati che per sua natura cresce con l'uso prolungato del sistema — e algoritmi di apprendimento collaborativo filtrato che possono generalizzare le preferenze di un utente specifico anche con un numero limitato di osservazioni, estraendo pattern comuni dalle preferenze di utenti diversi per compensare la scarsità dei dati individuali nelle prime sessioni di utilizzo.

Il sistema di privacy dei dati raccolti è un aspetto che richiede attenzione esplicita nella progettazione della pipeline perché i dati comportamentali dell'utente — le sequenze di gesture, le preferenze di pattern, i ritmi di utilizzo del sistema — sono dati personali che meritano la stessa protezione dei dati biometrici utilizzati nell'autenticazione facciale. La pipeline di MicroBot implementa il principio di privacy by design adottando tre misure complementari. La prima è la minimizzazione dei dati: vengono raccolti solo i dati strettamente necessari per le analisi previste, escludendo esplicitamente i dati che potrebbero consentire l'identificazione o il tracciamento dell'utente al di fuori del contesto operativo di MicroBot. La seconda è la localizzazione dei dati: tutti i dati vengono memorizzati localmente sul controller e sui dispositivi dell'utente, senza trasmissione a server esterni o servizi cloud, garantendo che l'utente mantenga il controllo completo sui propri dati operativi. La terza è la trasparenza: l'interfaccia web espone un pannello dedicato alla gestione dei dati che mostra all'utente esattamente quali dati sono stati raccolti, per quale periodo e per quale scopo, con la possibilità di cancellare selettivamente o integralmente i dati raccolti in qualsiasi momento senza conseguenze sulle funzionalità operative del sistema.

Punto 36 — Protocollo di comunicazione avanzato: affidabilità, sicurezza e ottimizzazione

Il protocollo di comunicazione di MicroBot è stato introdotto nel punto 8 nella sua struttura fondamentale — topologia a stella SoftAP, trasporto UDP, struttura del pacchetto, meccanismo di heartbeat e sincronizzazione temporale. Quel capitolo ha stabilito le fondamenta architetturali del protocollo, sufficienti per comprendere il funzionamento del sistema nella versione 0.3. Questo capitolo approfondisce gli aspetti avanzati del protocollo che determinano la sua affidabilità in condizioni operative reali, la sua resistenza agli attacchi di sicurezza e la sua scalabilità verso configurazioni con un numero elevato di bot e requisiti di latenza più stringenti. La differenza tra un protocollo che funziona in laboratorio in condizioni ideali e un protocollo che funziona in modo affidabile nelle condizioni variabili di un utilizzo reale — interferenze Wi-Fi, variazioni della qualità del canale, carico variabile della rete — è determinata da questi aspetti avanzati che raramente emergono durante i test preliminari ma che diventano la causa principale dei problemi nelle sessioni operative prolungate.

La scelta di UDP come protocollo di trasporto è stata motivata nel capitolo 8 principalmente dalla sua latenza ridotta rispetto a TCP. È opportuno approfondire questa motivazione con maggiore rigore tecnico per chiarire i trade-off che questa scelta comporta. UDP è un protocollo connectionless e unreliable che non garantisce la consegna dei datagrammi, non garantisce l'ordine di ricezione e non implementa alcun meccanismo di controllo della congestione. Queste caratteristiche, che nel contesto delle applicazioni internet convenzionali sono limitazioni inaccettabili, nel contesto del controllo in tempo reale di uno swarm robotico sono proprietà desiderabili: la mancanza di garanzia di consegna significa che un pacchetto perso non blocca la trasmissione dei pacchetti successivi in attesa di ritrasmissione — come invece avviene con TCP — garantendo che il sistema riceva sempre i comandi più recenti anche in presenza di perdite occasionali. La mancanza di controllo della congestione significa che il controller può inviare pacchetti di comando alla frequenza necessaria senza che il protocollo di trasporto riduca autonomamente il rate di trasmissione in risposta alla congestione della rete, una riduzione che in un sistema di controllo real-time produrrebbe degradamenti imprevedibili delle prestazioni. Il prezzo di questi vantaggi è che il protocollo applicativo — il codice del firmware del controller e dei bot — deve implementare esplicitamente i meccanismi di affidabilità necessari per il suo specifico caso d'uso, invece di affidarsi alle garanzie fornite dal protocollo di trasporto.

I meccanismi di affidabilità implementati a livello applicativo nel protocollo di MicroBot sono progettati per garantire le proprietà di correttezza necessarie per il sistema senza introdurre la latenza aggiuntiva che sarebbe inevitabile con TCP. Il meccanismo di acknowledgement selettivo è il primo di questi strumenti: per i comandi che richiedono conferma di ricezione — tipicamente i comandi di transizione di pattern e i comandi di emergenza, ma non i comandi di aggiornamento continuo del duty cycle che sono tolleranti alla perdita — il controller attende un pacchetto di acknowledgement dal bot entro un timeout di 50 millisecondi, e in assenza di acknowledgement ritrasmette il comando fino a un massimo di tre tentativi con intervalli esponenzialmente crescenti di 50, 100 e 200 millisecondi. Questo meccanismo garantisce la consegna dei comandi critici con una probabilità di successo del $1 - p^3$ dove p è la probabilità di perdita di un singolo pacchetto: con un tasso di perdita del 2% — il valore target specificato nel capitolo sulla validazione — la probabilità di fallimento della consegna dopo tre tentativi è dell'ordine di 8×10^{-6} , un valore trascurabile per le applicazioni di MicroBot. Il meccanismo di numeri di sequenza è il secondo strumento di affidabilità: ogni pacchetto UDP include un numero di sequenza a 16 bit che il ricevitore usa per rilevare i duplicati — pacchetti ritrasmessi che arrivano dopo che il ricevitore ha già processato il pacchetto originale — e per rilevare i pacchetti fuori ordine — situazione in cui il pacchetto $n+1$ arriva prima del pacchetto n a causa di percorsi di routing diversi nella rete. Il ricevitore ignora i duplicati e gestisce i pacchetti fuori ordine applicando la politica del pacchetto più recente — processa sempre il pacchetto con il numero di sequenza più alto ricevuto fino a quel momento, ignorando i pacchetti con numero di sequenza inferiore. Questa politica è appropriata per i comandi di aggiornamento continuo del duty cycle e della posizione target, dove il valore più recente è sempre il più rilevante, ma non per i comandi che devono essere eseguiti nell'ordine in cui sono stati emessi — per questi viene utilizzato il meccanismo di acknowledgement selettivo che garantisce la consegna ordinata.

La sicurezza del protocollo di comunicazione è un aspetto che nelle versioni prototipali di MicroBot è implementato con soluzioni leggere compatibili con le risorse limitate dell'ESP32, ma che assume maggiore importanza nelle versioni più mature in cui il sistema potrebbe

essere esposto a reti meno controllate. Il meccanismo di autenticazione dei bot alla rete SoftAP — già descritto nel capitolo sul controller — garantisce che solo i moduli fisici autorizzati possano connettersi alla rete del sistema, impedendo la connessione di dispositivi non autorizzati che potrebbero intercettare il traffico di controllo o inviare comandi non autorizzati allo swarm. La protezione del traffico di controllo dalla modifica non autorizzata è implementata tramite il checksum a 16 bit incluso in ogni pacchetto, che permette di rilevare la corruzione accidentale dei dati dovuta a interferenze elettromagnetiche o errori di trasmissione, ma non fornisce protezione crittografica contro modifiche intenzionali da parte di un attaccante che può intercettare e riformulare i pacchetti. Per le versioni future del sistema in cui questa protezione è necessaria, il protocollo prevede l'aggiunta di un Message Authentication Code — calcolato come HMAC-SHA256 del contenuto del pacchetto con una chiave condivisa tra controller e bot — che garantisce sia l'integrità sia l'autenticità dei pacchetti senza introdurre la latenza di una cifratura simmetrica completa.

La protezione contro i replay attack — attacchi in cui un aggressore cattura un pacchetto di comando legittimo e lo ritrasmette successivamente per replicare l'effetto del comando originale — è implementata tramite il timestamp incluso in ogni pacchetto combinato con una finestra di accettazione temporale configurabile. Il bot riceve un pacchetto e verifica che il timestamp incluso nel pacchetto sia compreso nella finestra di accettazione — tipicamente ± 500 millisecondi rispetto al proprio clock sincronizzato con il controller — prima di processare il comando. Un pacchetto catturato e ritrasmesso dopo la scadenza della finestra di accettazione viene silenziosamente scartato dal bot ricevente, rendendo il replay attack inefficace anche in assenza di cifratura del traffico. La finestra di accettazione di ± 500 millisecondi è scelta come compromesso tra la protezione dal replay — che migliora riducendo la finestra — e la tolleranza alla deriva del clock dei bot — che richiede una finestra sufficientemente ampia da coprire la deriva residua tra due sincronizzazioni successive a 10 hertz.

L'ottimizzazione del throughput del canale Wi-Fi è un aspetto operativo che diventa rilevante quando il numero di bot connessi aumenta verso la decina e il traffico aggregato di heartbeat e comandi inizia a competere per la banda disponibile. Il canale Wi-Fi 802.11g a 2.4 gigahertz ha una banda teorica massima di 54 megabit al secondo, ma la banda effettiva disponibile per il traffico applicativo è tipicamente il 40–50% del valore teorico a causa degli overhead del protocollo 802.11 — header, ACK di livello MAC, DIFS e backoff — che producono un throughput effettivo di 20–25 megabit al secondo in condizioni ottimali. Con pacchetti di heartbeat di 32 byte, un intervallo di 500 millisecondi tra heartbeat e 20 bot connessi, il traffico aggregato di heartbeat è dell'ordine di $20 \times 32 \times 8 / 0.5 = 10.240$ bit al secondo — un valore trascurabile rispetto alla banda disponibile anche in condizioni degradate. Il traffico di comandi è più variabile e dipende dalla frequenza con cui vengono inviati aggiornamenti dei pattern: in modalità Pattern Hold il controller invia aggiornamenti a 2 hertz con pacchetti di 16 byte per bot — $20 \times 16 \times 8 \times 2 = 5.120$ bit al secondo — mentre in modalità Vortex Attivo con aggiornamenti delle posizioni target a 20 hertz il traffico aumenta a $20 \times 16 \times 8 \times 20 = 51.200$ bit al secondo — ancora ben al di sotto della banda disponibile ma in un intervallo in cui l'ottimizzazione del formato dei pacchetti inizia a produrre benefici misurabili.

La compressione delta dei pacchetti di aggiornamento delle posizioni target è la tecnica di ottimizzazione più efficace per ridurre il traffico di rete nei pattern dinamici ad alta frequenza di aggiornamento. Invece di inviare le coordinate assolute della posizione target di ogni bot a ogni aggiornamento — che richiederebbe $4 \text{ byte per coordinata} \times 2 \text{ coordinate} \times 20 \text{ bot} = 160 \text{ byte per pacchetto broadcast}$ — il controller invia solo la variazione rispetto alla posizione target dell'aggiornamento precedente per i bot la cui posizione target è cambiata significativamente. Per il pattern Vortex in cui tutte le posizioni target ruotano di un angolo costante ad ogni aggiornamento, la compressione delta può essere ulteriormente semplificata inviando un singolo parametro — l'angolo di rotazione incrementale — invece delle variazioni di posizione per ogni bot, riducendo il pacchetto di aggiornamento da 160 byte a 4 byte — una riduzione del 97.5% che lascia abbondante margine di banda per l'eventuale aumento del numero di bot. La decompressione delta è implementata nel firmware di ogni bot come un'operazione di aggiornamento locale che aggiunge la variazione ricevuta alla propria posizione target corrente, producendo la nuova posizione target senza richiedere che il bot riceva il valore assoluto completo ad ogni aggiornamento.

Il protocollo di multicast è la soluzione adottata per l'invio efficiente di comandi a sottogruppi di bot senza ricorrere all'invio individuale o al broadcast globale. UDP supporta il multicast a livello IP attraverso gli indirizzi della classe D — 224.0.0.0/4 — che permettono di inviare un singolo pacchetto UDP che viene consegnato automaticamente dalla rete a tutti i dispositivi iscritti al gruppo multicast corrispondente. In MicroBot ogni pattern prevede un gruppo multicast dedicato: tutti i bot che partecipano al pattern circolare si iscrivono al gruppo multicast del pattern circolare, tutti i bot che partecipano al pattern a onda al gruppo dell'onda, e così via. Il controller invia gli aggiornamenti delle posizioni target come pacchetti multicast al gruppo del pattern corrente, e solo i bot iscritti a quel gruppo ricevono e processano il pacchetto, eliminando la necessità di demultiplexing a livello applicativo e riducendo il carico computazionale dei bot che non partecipano al pattern corrente — particolarmente rilevante nelle sessioni in cui solo un sottoinsieme della flotta è attivo in un dato momento.

Punto 37 — Interazione uomo-macchina: ergonomia, feedback e design dell'esperienza

L'interazione uomo-macchina è la dimensione di MicroBot che determina in ultima analisi se il sistema è uno strumento utile o un oggetto da laboratorio che funziona tecnicamente ma che nessuno vuole usare. Un sistema di swarm robotics può avere algoritmi di coordinamento impeccabili, un protocollo di comunicazione ottimizzato e un safety layer robusto, e tuttavia fallire nel suo scopo fondamentale se l'interfaccia attraverso cui l'utente interagisce con lo swarm è confusa, faticosa o non responsiva. La qualità dell'interazione uomo-macchina non è una proprietà decorativa che si aggiunge al sistema dopo aver risolto i problemi tecnici — è una proprietà strutturale che condiziona tutte le scelte progettuali dall'inizio, perché le decisioni sull'architettura del sistema hardware e software hanno conseguenze dirette e spesso irreversibili sull'esperienza che l'utente vive quando usa il sistema. Progettare l'interazione uomo-macchina di MicroBot significa adottare la prospettiva dell'utente come criterio primario di valutazione delle scelte progettuali — non come criterio secondario da applicare dopo aver ottimizzato le prestazioni tecniche del sistema.

Il modello mentale dell'utente — la rappresentazione interna che l'utente forma del sistema attraverso l'esperienza di utilizzo — è il concetto centrale attorno a cui ruota la progettazione dell'interazione di MicroBot. Un modello mentale accurato permette all'utente di prevedere come il sistema risponderà alle proprie azioni, di interpretare correttamente i feedback ricevuti e di recuperare dagli errori in modo autonomo senza dover consultare la documentazione. Un modello mentale inaccurato — costruito da un'interfaccia che nasconde il funzionamento del sistema dietro astrazioni fuorvianti o che fornisce feedback inconsistenti — produce frustrazione, errori ripetuti e una sensazione di mancanza di controllo che degrada irrimediabilmente l'esperienza di utilizzo. Il sistema MicroBot è intrinsecamente complesso — è un sistema distribuito con molti componenti che interagiscono in modo non lineare — e il compito dell'interfaccia è di esporre questa complessità in modo progressivo e stratificato, mostrando all'utente principiante una visione semplificata del sistema che è comunque sufficiente per il controllo base, e rivelando gradualmente i livelli di complessità aggiuntiva man mano che l'utente acquisisce familiarità con il sistema e desidera esplorare le sue capacità più avanzate.

Il principio di immediatezza del feedback è la proprietà dell'interfaccia che più direttamente determina la sensazione di controllo dell'utente durante l'interazione con lo swarm. L'immediatezza non significa solo bassa latenza — anche se la latenza è un componente fondamentale — ma significa che ogni azione dell'utente produce immediatamente una risposta percepibile che conferma la ricezione dell'azione da parte del sistema, indipendentemente dalla latenza della risposta fisica dei bot. Questa distinzione è cruciale: se l'utente esegue un gesto di selezione del pattern circolare e deve attendere 2–3 secondi prima di vedere la risposta fisica dei bot — un tempo accettabile per la fisica dell'aggancio magnetico — il periodo di attesa senza feedback è psicologicamente disorientante e produce la sensazione che il gesto non sia stato recepito, spingendo l'utente a ripetere il gesto. La soluzione è il feedback immediato a due stadi: un feedback istantaneo — entro 50 millisecondi dal gesto — che conferma la ricezione del comando attraverso un segnale percepibile all'utente senza aspettare la risposta fisica dei bot, seguito dal feedback fisico dei bot — il cambio di colore dei LED, l'inizio del movimento — che arriva con la latenza fisiologica del sistema. Il feedback immediato è implementato attraverso tre canali simultanei: il feedback audio — un breve suono di conferma prodotto dal sistema al momento della ricezione del gesto — il feedback visivo nel visore — un'animazione di flash sul pattern selezionato nell'interfaccia 3D — e il feedback aptico nel wristband — una breve vibrazione prodotta da un piccolo motore eccentrico opzionale integrato nell'involucro.

Il design del sistema di feedback luminoso dei LED dei bot è uno degli aspetti dell'interazione più visibili e più importanti per la leggibilità dello stato dello swarm durante le sessioni operative. I LED dei bot sono l'unico canale di feedback diretto dal sistema fisico all'utente — indipendente dal visore e dall'interfaccia web — e devono comunicare informazioni sufficienti per permettere all'utente di comprendere lo stato del sistema semplicemente osservando i bot con i propri occhi. La scelta dei colori e dei pattern di lampeggio segue principi consolidati di design dell'informazione: i colori freddi — blu e ciano — indicano stati di funzionamento normale e attività, i colori caldi — giallo e rosso — indicano stati di attenzione ed emergenza, il verde indica il raggiungimento di uno stato obiettivo. La distinzione tra stati diversi all'interno della stessa categoria cromatica è implementata tramite pattern di lampeggio: il blu fisso indica idle in attesa di comandi, il ciano lampeggiante lento indica movimento in corso, il ciano lampeggiante veloce indica

aggancio attivo in fase critica. La ridondanza tra i canali cromatico e cinematografico del LED — entrambi il colore e il pattern di lampeggio cambiano in funzione dello stato — garantisce che le informazioni siano leggibili anche in condizioni di illuminazione ambiente che potrebbero rendere difficile la distinzione tra colori simili, e in contesti in cui l'utente ha una forma di daltonismo che impedisce la corretta percezione di alcune differenze cromatiche.

La progettazione delle gesture del wristband segue principi ergonomici che massimizzano la comodità e la naturalezza dei movimenti richiesti. Le gesture più frequenti — la selezione del pattern e la modifica dei parametri principali — sono mappate sui movimenti di polso che il corpo umano esegue con minor fatica e maggiore precisione: la rotazione del polso attorno all'asse longitudinale dell'avambraccio è il movimento più naturale e preciso del polso, e per questa ragione è utilizzata per la modifica del parametro principale del pattern corrente — il raggio per il cerchio, l'ampiezza per l'onda, la velocità per il vortice. Il pitch del polso — l'inclinazione verso l'alto o verso il basso — è il secondo movimento più naturale e viene utilizzato per un parametro secondario del pattern. Le gesture discrete — la scossa singola per il cambio di pattern, la doppia scossa per l'emergenza — sono progettate per essere facilmente distinguibili tra loro e difficilmente attivabili accidentalmente durante i movimenti normali del polso: la doppia scossa richiede due accelerazioni separate da un intervallo di 200–400 millisecondi che non si verifica nei movimenti normali del braccio, eliminando i falsi positivi durante le sessioni operative. La forza e l'ampiezza minima delle gesture richieste sono calibrate sull'utente durante la fase di calibrazione iniziale del wristband — il sistema impara il range di movimento caratteristico dell'utente specifico e adatta le soglie di riconoscimento di conseguenza — garantendo che gesture troppo piccole per un utente robusto non vengano confuse con rumore di fondo e che gesture che sarebbero eccessivamente ampie per un utente con mobilità ridotta del polso siano comunque riconoscibili entro un range di movimento confortevole.

La curva di apprendimento del sistema è progettata con un approccio progressivo che permette all'utente di iniziare a usare il sistema in modo produttivo fin dalla prima sessione, pur lasciando spazio a un approfondimento progressivo delle funzionalità avanzate nelle sessioni successive. Il livello base dell'interazione — controllare lo swarm attraverso tre o quattro gesture fondamentali del wristband per selezionare i pattern più semplici — è apprendibile in 5–10 minuti di utilizzo guidato, un tempo compatibile con una sessione introduttiva. Il livello intermedio — modulare i parametri geometrici dei pattern, utilizzare il framework 4D/5D per modulare lo swarm con il suono, gestire la flotta attraverso l'interfaccia web — richiede 2–3 sessioni di pratica per essere padroneggiato con fluidità. Il livello avanzato — configurare la matrice di mappatura del framework 4D/5D, creare pattern personalizzati, analizzare i log delle sessioni, eseguire aggiornamenti OTA — è accessibile agli utenti che hanno sviluppato una comprensione profonda del sistema attraverso l'uso prolungato e che hanno interesse a esplorarne le capacità più tecniche. Questa struttura a livelli non è implementata come un sistema di blocchi espliciti — l'utente non deve sbloccare livelli come in un videogioco — ma emerge naturalmente dalla struttura dell'interfaccia, che espone le funzionalità più complesse in posizioni meno prominenti dell'interfaccia web e del menu del visore, richiedendo all'utente di esplorare attivamente per scoprirle.

Il feedback sonoro del framework 4D/5D merita una trattazione specifica perché svolge un ruolo unico nell'interazione: è al tempo stesso un canale di input verso il sistema — il suono modula lo stato interno W — e un canale di feedback dal sistema verso l'utente — lo swarm

risponde al suono con cambiamenti visibili nel comportamento fisico. Questa dualità trasforma il suono da un canale di controllo unidirezionale in un canale di interazione bidirezionale che crea un dialogo tra l'utente e lo swarm mediato dall'elemento sonoro: l'utente produce suono, il suono modula il comportamento dello swarm, il comportamento dello swarm modifica la risposta emotiva e creativa dell'utente che a sua volta modifica il suono prodotto. Questa retroazione tra produzione sonora e risposta visiva dello swarm è il nucleo dell'esperienza più ricca che MicroBot può offrire — una forma di improvvisazione guidata in cui lo swarm è sia strumento sia partner creativo dell'utente. Il sistema di analisi audio che alimenta il framework 4D/5D è progettato per rispondere a questa intenzione con una latenza massima di 50 millisecondi tra l'evento sonoro e la modifica del vettore W, sufficientemente bassa da mantenere la sensazione di connessione diretta tra gesto sonoro e risposta fisica dello swarm.

La documentazione dell'utente è progettata come componente integrale dell'esperienza di interazione e non come appendice tecnica separata dal sistema. Il manuale di avvio rapido è una card laminata di 10×15 centimetri con la sequenza di avvio del sistema e le gesture fondamentali del wristband rappresentate come pittogrammi — un formato che l'utente può consultare mentre indossa il visore senza dover toglierlo. Il tutorial interattivo integrato nel visore è una sequenza guidata di esercizi che il sistema propone alla prima sessione di utilizzo: un assistente virtuale nella scena 3D guida l'utente attraverso le gesture fondamentali, le conferma quando eseguite correttamente e le ripete quando non vengono riconosciute, costruendo la competenza di base nel contesto operativo reale invece che attraverso la lettura di documentazione teorica. La documentazione avanzata è disponibile nell'interfaccia web nella sezione /docs come hypertext navigabile con esempi di configurazione, descrizioni dettagliate di tutti i parametri e una sezione di troubleshooting che guida l'utente nella diagnosi e nella risoluzione dei problemi più comuni attraverso una serie di domande a scelta multipla che conducono alla causa radice del problema e alla procedura di risoluzione appropriata.

Punto 38 — Comunicazione inter-bot: ESP-NOW, mesh networking e autonomia locale

I capitoli precedenti sulla comunicazione hanno descritto il protocollo di scambio tra il controller centrale e i singoli bot — una topologia a stella in cui il controller è l'unico nodo che detiene la visione globale dello swarm e che coordina il comportamento di tutti i bot attraverso comandi individuali e broadcast. Questa topologia è appropriata per la versione 0.3 del sistema, in cui il numero di bot è limitato, la semplicità implementativa è una priorità e la dipendenza da un nodo centrale è accettabile. Ma è una topologia che presenta un limite strutturale fondamentale: ogni comunicazione tra due bot — anche la più semplice, come la coordinazione dell'aggancio magnetico tra due moduli adiacenti — deve passare attraverso il controller, percorrendo un cammino di comunicazione che include due hop wireless, due elaborazioni a livello applicativo nel controller e due trasmissioni di ritorno verso i bot coinvolti. Questa indirezione introduce una latenza minima di 20–40 millisecondi per ogni decisione coordinata tra bot adiacenti — un valore che per operazioni semplici è accettabile ma che per coordinazioni ad alta frequenza come la modulazione real-time della rigidità di un aggregato in risposta a perturbazioni esterne diventa un collo di bottiglia che limita la reattività del sistema. La comunicazione inter-bot diretta — la capacità di ogni bot di scambiare informazioni con i bot fisicamente vicini senza mediazione del controller — è la

soluzione a questo limite e la direzione verso cui si evolve l'architettura di comunicazione di MicroBot nelle versioni successive.

ESP-NOW è il protocollo di comunicazione peer-to-peer proprietario di Espressif Systems che permette la comunicazione diretta tra dispositivi ESP32 nella stessa area senza richiedere un access point intermedio. ESP-NOW opera a livello di data link — al di sotto del livello IP — inviando frame di dati direttamente agli indirizzi MAC dei dispositivi destinatari con una latenza tipica di 1–2 millisecondi, dieci volte inferiore alla latenza del percorso UDP attraverso il controller. Il protocollo supporta la modalità unicast — comunicazione da un dispositivo a un singolo destinatario identificato dal MAC — e la modalità broadcast — comunicazione a tutti i dispositivi ESP32 nel raggio di ricezione — con un payload massimo di 250 byte per frame, sufficiente per i tipi di messaggi inter-bot previsti in MicroBot. La limitazione principale di ESP-NOW è la sua natura stateless: non fornisce garanzie di consegna, non gestisce la ritrasmissione dei frame persi e non supporta nativamente il routing tra nodi non direttamente in comunicazione radio. Nella versione 0.4 di MicroBot ESP-NOW viene adottato come canale di comunicazione supplementare al canale UDP principale, utilizzato specificamente per i messaggi inter-bot a bassa latenza — notifiche di prossimità, coordinate dell'aggancio magnetico, aggiornamenti di stato locali — mentre il canale UDP con il controller rimane il canale primario per i comandi di pattern e la supervisione globale.

Il protocollo di comunicazione inter-bot costruito sopra ESP-NOW segue una struttura deliberatamente semplice che riflette la natura locale e ad alta frequenza delle informazioni scambiate tra bot vicini. Ogni frame inter-bot contiene un header di 8 byte — identificativo del mittente a 2 byte, tipo di messaggio a 1 byte, numero di sequenza a 2 byte, timestamp a 3 byte — seguito da un payload variabile di dimensione massima 242 byte. I tipi di messaggio inter-bot definiti nella versione 0.4 sono cinque. Il messaggio PROXIMITY_BEACON è trasmesso da ogni bot in broadcast a intervalli di 100 millisecondi e contiene la posizione stimata del bot, il suo stato operativo e il livello di carica della batteria — un'implementazione del principio di stigmergia in cui ogni bot diffonde continuamente informazioni sulla propria presenza e sul proprio stato nell'ambiente locale, permettendo ai bot vicini di costruire autonomamente una mappa parziale del vicinato senza interrogare il controller. Il messaggio DOCK_REQUEST è trasmesso in unicast verso il bot target quando il sistema di aggancio decide di iniziare la procedura di aggancio, e contiene i parametri di aggancio — lato del cono da agganciare, duty cycle richiesto, timestamp di inizio sincronizzato. Il messaggio DOCK_CONFIRM è la risposta del bot target che conferma la disponibilità all'aggancio e sincronizza i parametri operativi. Il messaggio DOCK_RELEASE è trasmesso in unicast per coordinare il disaggancio, specificando il timestamp sincronizzato dell'inizio dell'attivazione inversa delle coil in entrambi i bot. Il messaggio STATE_SYNC è trasmesso periodicamente tra bot agganciati per sincronizzare il loro stato operativo — duty cycle corrente delle coil, temperatura stimata, livello di carica — permettendo di adattare i parametri dell'aggancio in funzione delle condizioni di entrambi i moduli senza richiedere l'intervento del controller.

Il meccanismo di scoperta dei vicini è il componente del protocollo inter-bot che permette a ogni bot di costruire e mantenere aggiornata la propria mappa locale del vicinato — l'insieme dei bot fisicamente vicini con cui la comunicazione diretta è possibile. La scoperta avviene passivamente attraverso la ricezione dei PROXIMITY_BEACON trasmessi in broadcast dai

bot vicini: ogni bot mantiene una tabella dei vicini che mappa l'identificativo di ogni bot ricevente alla propria posizione stimata, al timestamp dell'ultimo beacon ricevuto e alla qualità del segnale radio misurata dall'RSSI — Received Signal Strength Indicator — del frame ESP-NOW. Un bot che non trasmette beacon per un intervallo superiore a 500 millisecondi viene rimosso dalla tabella dei vicini, garantendo che la mappa locale rifletta sempre la topologia corrente del vicinato fisico e non includa bot che si sono allontanati o che si sono spenti. La qualità del segnale radio è un proxy della distanza fisica tra i bot — con la certa approssimazione introdotta dalle variazioni di orientamento delle antenne e dalla presenza di superfici riflettenti — e viene utilizzata dal sistema di aggancio per stimare quali bot sono candidati all'aggancio fisico nella sessione corrente, riducendo il numero di tentative di aggancio verso bot troppo lontani per essere fisicamente raggiunti.

L'autonomia locale è la proprietà emergente che nasce dalla combinazione della comunicazione inter-bot con una logica decisionale locale sufficientemente ricca da permettere ai bot di coordinare alcune azioni senza l'intervento del controller. Nelle versioni prototipali questa autonomia è limitata alla gestione locale dell'aggancio magnetico — i due bot che si stanno agganciando coordinano direttamente i propri profili di corrente nelle coil tramite messaggi DOCK_REQUEST e DOCK_CONFIRM senza passare attraverso il controller, riducendo la latenza dell'aggancio da 40–60 millisecondi a 3–5 millisecondi. Nelle versioni più avanzate l'autonomia locale si estende alla gestione delle perturbazioni locali: quando un bot agganciato rileva una forza esterna che tende a disagganciare la struttura — rilevata come una riduzione dell'RSSI del beacon del bot vicino o come un incremento anomalo della corrente richiesta per mantenere il duty cycle di mantenimento — comunica direttamente con il bot vicino tramite messaggio STATE_SYNC e i due bot coordinano autonomamente un incremento temporaneo del duty cycle delle proprie coil per aumentare la forza di tenuta dell'aggancio, senza attendere il comando del controller che arriverebbe con una latenza 10–20 volte superiore.

La topologia mesh è l'estensione naturale della comunicazione inter-bot verso una rete in cui ogni bot può instradare messaggi verso bot non direttamente raggiungibili, creando un grafo di connettività che copre l'intero swarm indipendentemente dalla posizione del controller. In una topologia mesh ogni bot è sia un nodo terminale — che genera e consuma messaggi propri — sia un router — che instrada i messaggi dei bot vicini verso destinazioni più lontane. Il routing in una rete mesh di swarm robotics è un problema particolarmente complesso perché la topologia della rete cambia continuamente con il movimento dei bot e con i cambiamenti degli agganci magnetici, rendendo inutilizzabili gli algoritmi di routing convenzionali basati su tabelle di routing statiche. La soluzione adottata in MicroBot per la versione 0.4 è il routing geografico — ogni messaggio viene instradato verso il bot vicino la cui posizione stimata è più vicina alla posizione del destinatario — che non richiede tabelle di routing e si adatta automaticamente ai cambiamenti della topologia senza protocolli di aggiornamento espliciti. Il routing geografico ha la nota limitazione dei buchi nella copertura — situazioni in cui nessun bot vicino è più vicino al destinatario di quanto lo sia il mittente, producendo un loop di routing — che viene gestita tramite una strategia di perimeter routing che aggira l'ostacolo navigando lungo il perimetro della zona di buca di copertura fino a trovare un percorso verso il destinatario.

La coesistenza del canale ESP-NOW inter-bot con il canale UDP verso il controller richiede una gestione attenta delle risorse radio dell'ESP32, che ha un unico modulo Wi-Fi che deve

supportare simultaneamente la connessione all'access point SoftAP del controller e la comunicazione ESP-NOW con i bot vicini. Questa coesistenza è tecnicamente possibile perché l'ESP32 supporta la modalità AP+Station simultanea — in cui opera contemporaneamente come stazione connessa alla rete SoftAP del controller e come interfaccia ESP-NOW verso i bot vicini — ma richiede una gestione del tempo di accesso al canale radio che minimizza le interferenze tra i due canali. La libreria ESP-NOW di Espressif gestisce automaticamente la coesistenza con il modulo Wi-Fi attraverso un meccanismo di time-slicing che alterna l'accesso al canale radio tra le due funzionalità con una frequenza sufficientemente alta da garantire la continuità di entrambe le connessioni, ma la configurazione ottimale dei parametri di time-slicing — dimensione degli slot temporali, priorità relativa dei due canali — richiede una calibrazione empirica per ogni configurazione specifica di MicroBot in funzione del numero di bot e della frequenza dei messaggi inter-bot.

La retrocompatibilità del protocollo inter-bot con i bot che non supportano ESP-NOW — i bot della versione 0.3 che utilizzano solo il canale UDP — è garantita attraverso un meccanismo di degradazione automatica: quando un bot della versione 0.4 tenta di stabilire una comunicazione ESP-NOW con un bot vicino e non riceve risposta entro il timeout di 10 millisecondi, assume automaticamente che il bot vicino non supporti ESP-NOW e delega la coordinazione attraverso il controller tramite il canale UDP standard. Questa degradazione automatica permette di aggiornare gradualmente la flotta alla versione 0.4 senza dover sostituire tutti i bot simultaneamente — i bot già aggiornati beneficiano della comunicazione inter-bot diretta tra loro, mentre continuano a comunicare correttamente con i bot non ancora aggiornati attraverso il controller — garantendo la continuità operativa del sistema durante il processo di aggiornamento progressivo della flotta.

Punto 39 — Piano delle simulazioni: elettrica, firmware e comportamentale

Le simulazioni di MicroBot non sono un'alternativa alla prototipazione fisica ma un prerequisito indispensabile che precede, accompagna e integra il lavoro sull'hardware reale lungo tutto il ciclo di sviluppo del progetto. La distinzione fondamentale tra simulazione e prototipazione non è la fedeltà al sistema reale — entrambe approssimano la realtà in modi diversi — ma il costo dell'iterazione: modificare un parametro nella simulazione richiede secondi, modificare il layout di un PCB richiede giorni. Questa asimmetria del costo di iterazione è la ragione per cui il piano delle simulazioni di MicroBot è strutturato come un percorso di raffinamento progressivo che sposta gradualmente il peso del testing dalla simulazione all'hardware fisico man mano che la comprensione del sistema aumenta e le scelte progettuali si cristallizzano in implementazioni concrete. Le simulazioni non vengono abbandonate quando il prototipo fisico diventa disponibile — continuano a svolgere funzioni specifiche che l'hardware fisico non può replicare con la stessa efficienza, come il testing di condizioni di guasto pericolose, la validazione di nuovi algoritmi prima del deployment sul firmware e la generazione di dataset sintetici per l'addestramento dei modelli di machine learning.

Il piano delle simulazioni di MicroBot è organizzato in tre livelli gerarchici che corrispondono alle tre astrazioni principali del sistema — il livello elettrico, il livello firmware e il livello comportamentale — ciascuno con strumenti dedicati, obiettivi specifici e criteri di completamento definiti che determinano quando il sistema è sufficientemente compreso a quel livello per procedere alla fase successiva di sviluppo. I tre livelli non sono sequenziali

ma paralleli e interdipendenti: i risultati della simulazione elettrica informano i parametri del safety layer che viene testato nella simulazione firmware, e i parametri del safety layer informano i vincoli operativi che vengono rispettati dalla simulazione comportamentale. Questa interdipendenza richiede che il piano delle simulazioni sia gestito come un progetto integrato con punti di sincronizzazione espliciti in cui i risultati dei tre livelli vengono riconciliati e le eventuali inconsistenze vengono risolte prima di procedere.

Il livello di simulazione elettrica ha l'obiettivo di caratterizzare il comportamento del circuito di pilotaggio delle coil in tutte le condizioni operative rilevanti — dal funzionamento nominale alle condizioni di stress termico, dai picchi di corrente durante l'aggancio alla scarica progressiva della batteria — producendo i parametri quantitativi che definiscono i limiti operativi sicuri del sistema. Gli strumenti utilizzati a questo livello sono LTspice per le analisi di precisione e Falstad per la visualizzazione intuitiva durante le fasi esplorative. Il piano di simulazione a questo livello prevede quattro campagne di simulazione distinte che coprono gli aspetti più critici del comportamento elettrico del sistema. La prima campagna è la simulazione del transitorio di accensione di una singola coil: partendo dallo stato iniziale con coil completamente scarica — corrente nulla — il segnale PWM viene applicato al gate del MOSFET e si osserva l'evoluzione della corrente nella coil fino al regime stazionario, misurando il tempo di rise della corrente, il picco di corrente nel primo ciclo PWM e il ripple di corrente a regime. I risultati attesi sono un tempo di rise dell'ordine di L/R — tipicamente 50–150 microsecondi per le coil di MicroBot — e un ripple di corrente inversamente proporzionale all'induttanza e alla frequenza PWM. Qualsiasi deviazione significativa da questi valori indica un errore nel modello — un'induttanza reale diversa da quella calcolata, una resistenza di avvolgimento non corretta — che deve essere investigata e corretta prima di procedere.

La seconda campagna è la simulazione del picco di tensione durante lo spegnimento del MOSFET, il fenomeno più pericoloso per l'integrità del componente. Quando il MOSFET si spegne la corrente nella coil — che non può variare istantaneamente per la legge di Lenz — deve trovare un percorso alternativo attraverso il diodo di ricircolo, producendo un picco di tensione al drain del MOSFET che dipende dalla velocità di spegnimento del MOSFET, dall'induttanza della coil e dalle capacità parassite del circuito. La simulazione misura l'ampiezza e la durata di questo picco in funzione della velocità di spegnimento del MOSFET — controllata dalla resistenza di gate — e verifica che il picco rimanga al di sotto del 70% della tensione di breakdown del componente con un margine di sicurezza adeguato. Se il picco simulato supera questo limite, la simulazione permette di esplorare soluzioni correttive — un diodo di ricircolo più veloce, una resistenza di smorzamento in parallelo alla coil, un condensatore di snubber al drain del MOSFET — e di quantificare la loro efficacia prima di implementarle nell'hardware reale. La terza campagna è la simulazione termica equivalente del circuito con quattro coil attive a diversi duty cycle, che calcola la potenza dissipata in ciascun componente e la stima dell'incremento di temperatura in funzione della resistenza termica equivalente del package. Questa campagna produce la mappa di temperatura del PCB nelle condizioni operative peggiori — quattro coil attive al 100% di duty cycle — e verifica che nessun componente superi la propria temperatura massima di giunzione. La quarta campagna è la simulazione del comportamento della batteria durante una sessione operativa completa, che modella la scarica della batteria come una sorgente di tensione con resistenza interna variabile in funzione del livello di carica e della corrente di scarica, e calcola l'evoluzione della tensione di alimentazione nel tempo in risposta al profilo di

corrente della sessione operativa — alternanza di fasi di mantenimento a basso consumo e fasi di aggancio ad alto consumo. Questa campagna produce le curve di autonomia del sistema in funzione del profilo operativo e identifica le condizioni in cui la caduta di tensione sulla resistenza interna della batteria può portare la tensione di alimentazione al di sotto della soglia di reset dell'ESP32 — un fenomeno che produce riavvii inattesi del firmware e che deve essere prevenuto attraverso la scelta di condensatori di bypass adeguati sul circuito di alimentazione.

Il livello di simulazione firmware ha l'obiettivo di verificare il comportamento della logica software del firmware in condizioni operative diverse, identificando i bug logici, le race condition e i problemi di timing prima che si manifestino sull'hardware fisico dove il debugging è significativamente più difficile. Lo strumento principale è Wokwi, un simulatore di microcontrollori ESP32 accessibile via browser che esegue il firmware compilato in un ambiente virtuale che simula i pin GPIO, le interfacce seriali, il modulo Wi-Fi e le periferiche hardware dell'ESP32. Wokwi non è un simulatore di livello elettrico — non modella il comportamento fisico dei segnali sui pin — ma è un simulatore di livello firmware che esegue le stesse istruzioni macchina che verrebbero eseguite sull'hardware fisico, producendo un ambiente di test che è funzionalmente equivalente all'hardware per la verifica della logica software. Il piano di simulazione firmware prevede cinque scenari di test sistematici. Il primo scenario è la verifica della sequenza di inizializzazione: il firmware viene avviato nella simulazione e si verifica che la sequenza di init — configurazione dei pin GPIO, inizializzazione del driver PWM, connessione Wi-Fi, sincronizzazione del clock, transizione allo stato IDLE — si completi correttamente entro il timeout di avvio configurato. Il secondo scenario è la verifica della macchina a stati finiti: il sistema viene portato attraverso tutte le transizioni di stato previste — IDLE → MOVING → DOCKING → LOCKED → DOCKING → IDLE — tramite comandi UDP simulati, e si verifica che ogni transizione produca esattamente le azioni attese — variazione del duty cycle, cambio del colore del LED, aggiornamento dello stato nell'heartbeat. Il terzo scenario è la verifica del safety layer sotto stress: i parametri di stato del sistema — tensione di batteria, temperatura stimata, qualità della connessione — vengono manipolati tramite iniezione diretta di valori nelle variabili del firmware per simulare il superamento di ciascuna soglia di protezione, e si verifica che il safety layer risponda con le azioni correttive appropriate entro il tempo massimo specificato. Il quarto scenario è la verifica della robustezza del protocollo di comunicazione: il canale UDP simulato viene configurato per introdurre artificialmente perdita di pacchetti, ritardi casuali e corruzione del checksum, e si verifica che il firmware gestisca correttamente tutte queste condizioni senza entrare in stati inconsistenti o produrre comportamenti non previsti. Il quinto scenario è la verifica del watchdog: l'esecuzione del loop principale viene bloccata artificialmente per simulare un freeze del firmware, e si verifica che il watchdog hardware produca il riavvio del sistema entro il timeout configurato.

Il livello di simulazione comportamentale ha l'obiettivo di validare gli algoritmi di coordinamento collettivo e il framework 4D/5D attraverso l'esecuzione della simulazione JavaScript con migliaia di iterazioni del game loop che coprono tutti i pattern implementati, le transizioni tra pattern e le condizioni di stress della flotta. Questo livello di simulazione è il più direttamente collegato alle prestazioni percepibili del sistema — la fluidità delle transizioni, la precisione del posizionamento, la qualità dell'integrazione con il suono — e i suoi risultati sono direttamente confrontabili con le metriche di performance misurate sull'hardware fisico. Il piano di simulazione comportamentale prevede tre fasi progressive.

La prima fase è la validazione dei singoli pattern in isolamento: per ogni tipo di pattern vengono eseguiti 100 test di convergenza partendo da configurazioni iniziali casuali diverse, misurando per ciascuno il tempo di convergenza, la deviazione media dalla configurazione target e il numero di agganci e disagganci prodotti durante la transizione. I risultati vengono aggregati in distribuzioni statistiche — media, deviazione standard, percentili — che costituiscono i valori di riferimento attesi per le corrispondenti misurazioni sull'hardware fisico. La seconda fase è la validazione delle transizioni tra pattern: per ogni coppia di pattern consecutivi vengono eseguiti 50 test di transizione, misurando le metriche di qualità della transizione — fluidità cinematica, numero di collisioni rilevate, tempo totale dalla configurazione iniziale alla configurazione finale stabile. La terza fase è la validazione del framework 4D/5D: il sistema viene operato con il vettore W modulato da segnali audio sintetici di diversa natura — suoni puri, rumore bianco, segnali ritmici a diverse frequenze — e si verifica che la risposta dello swarm corrisponda alle aspettative teoriche derivate dalla matrice di mappatura configurata, misurando la correlazione tra il segnale di ingresso e la risposta osservabile del sistema come metrica di qualità dell'integrazione.

Il criterio di completamento del piano delle simulazioni è definito in termini di accordo quantitativo tra i risultati della simulazione e i risultati delle misurazioni sull'hardware fisico: il piano è considerato completato quando la deviazione media tra i valori predetti dalla simulazione e i valori misurati sull'hardware è inferiore al 20% per tutte le metriche principali di ciascun livello di simulazione. Questo criterio non richiede una corrispondenza perfetta tra simulazione e realtà — fisicamente impossibile dato il numero di effetti non modellati — ma richiede che il modello sia sufficientemente accurato da essere utile per le decisioni progettuali, guidando le scelte di dimensionamento e di configurazione verso soluzioni che funzioneranno correttamente sull'hardware fisico senza richiedere iterazioni eccessive di prova ed errore.

Punto 40 — Analisi dei costi, accessibilità e riproducibilità del progetto

La questione del costo di MicroBot è una questione progettuale prima ancora che economica. Un sistema di swarm robotics che costa decine di migliaia di euro per essere replicato è un sistema che rimane confinato nei laboratori delle istituzioni accademiche meglio finanziate, accessibile a una comunità ristretta di ricercatori con accesso privilegiato a risorse economiche e a filiere di approvvigionamento specializzate. Un sistema che costa qualche centinaio di euro per un prototipo funzionale di piccola scala è un sistema che può essere replicato da un maker curioso, da uno studente di ingegneria con un budget limitato, da un artista digitale che vuole esplorare la robotica come medium espressivo, da un insegnante che vuole introdurre i concetti di comportamento emergente in un corso universitario. La democratizzazione dell'accesso alla tecnologia — il principio che la conoscenza e gli strumenti per produrla dovrebbero essere disponibili al maggior numero possibile di persone indipendentemente dalle loro risorse economiche — è uno dei valori fondanti del progetto MicroBot, e l'analisi dei costi non è un esercizio contabile ma la verifica di quanto questo valore venga effettivamente realizzato nelle scelte concrete di componentistica e di processo produttivo.

Il costo di un singolo MicroBot nella versione 0.3 è stimato sommando i costi di approvvigionamento di ciascun componente ai prezzi di mercato attuali per acquisti di piccole quantità — ordini da 5 a 10 unità dello stesso componente, compatibili con le

necessità di un prototipo di ricerca. Il microcontrollore ESP32-C3 SuperMini ha un costo di approvvigionamento di circa 2–4 euro per unità acquistato su piattaforme di vendita online come AliExpress o Amazon, con variazioni significative in funzione del fornitore e del tempo di consegna. I quattro MOSFET BSS138 in package SOT-23 costano circa 0.10–0.20 euro ciascuno, per un totale di 0.40–0.80 euro. I componenti passivi — resistenze, condensatori, diodi di ricircolo — costano complessivamente meno di 0.50 euro per unità in acquisti singoli e praticamente zero se si dispone già di un assortimento base di componentistica SMD. Il regolatore LDO in package SOT-23 costa circa 0.30–0.50 euro. Il LED RGB SMD 3535 o WS2812B costa circa 0.20–0.50 euro per unità. La batteria Li-Po da 50 milliampereora costa circa 2–4 euro per unità, con variazioni significative in funzione della qualità e del fornitore — le batterie di qualità superiore con circuito di protezione integrato tendono a costare il doppio delle batterie di base ma offrono maggiore affidabilità e durata nel tempo. Il connettore JST-PH a 2 pin costa circa 0.20–0.30 euro. I magneti permanenti neodimio N52 da 5×2 millimetri costano circa 0.10–0.20 euro ciascuno per acquisti di piccole quantità, per un totale di 0.60–1.20 euro per sei magneti per modulo. Il filo di rame AWG 32 per le quattro coil — stimando un consumo di circa 5 metri per modulo — costa circa 0.20–0.40 euro al prezzo di un rotolo da 100 metri. Il PCB custom circolare da 15 millimetri può essere prodotto da servizi di prototipazione PCB come JLCPCB o PCBWay a un costo di circa 1–2 euro per unità per ordini minimi di 5 pezzi — il costo di setup è fisso e il costo per unità diminuisce significativamente all'aumentare della quantità. Il filamento PLA per il guscio stampato in 3D — stimando un consumo di circa 15–20 grammi per modulo — costa circa 0.30–0.50 euro al prezzo di un chilogrammo di filamento.

La somma di questi contributi produce un costo totale per singolo MicroBot nell'intervallo di 8–15 euro per unità in acquisti singoli di piccola quantità, con una riduzione significativa verso il basso con acquisti in quantità maggiori — un costo nell'intervallo di 5–10 euro per unità per ordini di 20 o più unità dello stesso componente. Questo costo unitario colloca MicroBot in una fascia di accessibilità eccezionale per un sistema di swarm robotics fisico: i sistemi comparabili disponibili commercialmente — come il Kilobot di Harvard, che ha un costo di circa 80–100 dollari per unità — costano da 8 a 20 volte di più per unità, rendendo la costituzione di una flotta di 20–30 unità un investimento di 1600–3000 dollari contro i 150–300 euro di MicroBot. Questa differenza di costo non è solo una differenza quantitativa ma una differenza qualitativa che determina chi può permettersi di accedere alla tecnologia: a 100 dollari per unità una flotta di 20 bot richiede l'approvazione di un budget di ricerca istituzionale, mentre a 10 euro per unità la stessa flotta è accessibile con le risorse personali di uno studente motivato.

Il costo del controller centrale è stimato nell'intervallo di 15–25 euro per unità, includendo il modulo ESP32-S3, la batteria Li-Po da 1500 milliampereora, il circuito di alimentazione, il PCB e l'involucro stampato in 3D. Il costo del wristband è stimato nell'intervallo di 20–30 euro per unità, includendo il modulo ESP32-C3 mini, la BNO055, la batteria da 500 milliampereora e l'involucro. Il costo del visore VR prototipale nella versione con smartphone — che utilizza un telefono Android di seconda mano come display — è stimato nell'intervallo di 30–50 euro per il solo telaio stampato e le lenti ottiche, escludendo il costo dello smartphone che si assume già disponibile. La webcam per il sistema di autenticazione biometrica facciale ha un costo di 15–30 euro per una webcam USB standard con risoluzione HD.

Il costo totale di un sistema MicroBot completo con 10 bot, controller, wristband, visore e webcam è quindi stimato nell'intervallo di 200–400 euro, a seconda delle scelte di componentistica e del livello di qualità dei singoli componenti. Questo costo include tutti gli strumenti di interazione — controller, wristband, visore — ma non include il costo degli strumenti di produzione — stampante 3D, saldatore, multimetro — che si assume già disponibili o condivisi. Se si include anche il costo degli strumenti di produzione necessari che non si posseggono già, il costo complessivo può aumentare di 150–300 euro per una stampante 3D FDM di entry-level, 30–50 euro per un saldatore a temperatura controllata di qualità accettabile e 20–30 euro per un multimetro digitale con risoluzione adeguata. Anche includendo questi costi di attrezzatura, il budget totale per la realizzazione di un sistema MicroBot funzionale rimane al di sotto di 700 euro — un investimento accessibile per un progetto di ricerca universitario, per un maker ben equipaggiato o per un collettivo di appassionati che condividono le attrezzature.

La riproducibilità del progetto è la dimensione complementare all'accessibilità economica: non è sufficiente che i componenti siano economicamente accessibili se le competenze e le conoscenze necessarie per assemblarli in un sistema funzionante sono accessibili solo a specialisti. MicroBot è progettato per essere riproducibile da chiunque abbia competenze di base in elettronica — capacità di saldare componenti SMD di taglia 0603 o superiore — programmazione embedded — familiarità con l'ecosistema Arduino per ESP32 — e stampa 3D — capacità di calibrare una stampante FDM e di ottimizzare i parametri di slicing per un guscio di piccole dimensioni con geometria curvilinea. Queste competenze sono accessibili e largamente diffuse nella comunità maker e nel mondo accademico tecnico-scientifico: la combinazione Arduino più stampa 3D è ormai lo standard di facto per la prototipazione rapida di sistemi embedded fisici, e le competenze necessarie sono insegnate in corsi universitari di livello triennale in molte facoltà di ingegneria. Le competenze più specialistiche — l'avvolgimento manuale delle coil, il montaggio e la calibrazione dei magneti permanenti — sono apprendibili in poche ore di pratica seguendo le istruzioni documentate nel presente documento, senza richiedere formazione specifica nel settore della robotica o dell'elettromeccanica.

La documentazione è il terzo pilastro della riproducibilità, insieme alla scelta di componenti standard e all'adozione di processi produttivi accessibili. Una documentazione completa, accurata e progressivamente strutturata — come quella prodotta in questo documento — è la condizione necessaria per permettere a un ricercatore o a un maker che non ha partecipato allo sviluppo originale del sistema di replicarlo con successo partendo da zero. Questa documentazione non deve essere solo descrittiva — limitarsi a descrivere cosa è stato fatto senza spiegare perché — ma deve essere esplicativa delle motivazioni dietro ogni scelta progettuale, in modo che il replicatore possa comprendere il sistema sufficientemente da adattarlo alle proprie specifiche esigenze senza dover partire da zero ogni volta che un componente standard non è disponibile nel proprio contesto o deve essere sostituito con un equivalente funzionale. La pubblicazione del codice sorgente completo del sistema — firmware del bot, firmware del controller, simulazione JavaScript, software Unity — sotto una licenza open source permissiva come MIT o Apache 2.0 è il complemento naturale della documentazione hardware e trasforma MicroBot da un prototipo di ricerca in una piattaforma aperta su cui la comunità può costruire varianti, estensioni e applicazioni che arricchiscono il progetto originale in direzioni che nessun singolo team di sviluppo potrebbe anticipare o perseguire autonomamente.

Il modello di sviluppo open source che MicroBot adotta implica anche una strategia di contribuzione dalla comunità che permette di beneficiare dei miglioramenti apportati da replicatori esterni. La repository del progetto su GitHub include le linee guida per i contributi — standard di codifica, processo di revisione delle pull request, criteri di accettazione delle nuove funzionalità — e un sistema di issue tracking che permette ai replicatori di segnalare problemi, richiedere chiarimenti sulla documentazione e proporre miglioramenti. Questa apertura alla contribuzione esterna non è una concessione al modello open source ma una strategia deliberata per moltiplicare le risorse di sviluppo del progetto: ogni replicatore che incontra un problema e lo risolve, ogni maker che adatta MicroBot a un'applicazione specifica e documenta l'adattamento, ogni ricercatore che sperimenta una variante algoritmica e ne pubblica i risultati, contribuisce alla base di conoscenza collettiva del progetto e riduce il lavoro necessario per i replicatori successivi. In questo senso la comunità che si forma attorno a MicroBot è essa stessa un componente del progetto — non un'appendice esterna ma una parte integrante del sistema che ne determina la vitalità e l'evoluzione nel tempo.

Punto 41 — Etica, impatto sociale e responsabilità nella swarm robotics

La riflessione etica su MicroBot non è un esercizio accademico aggiunto alla fine del documento per soddisfare una convenzione della scrittura scientifica — è una componente del progetto che merita la stessa attenzione metodologica riservata alla progettazione del circuito di pilotaggio delle coil o alla validazione sperimentale degli algoritmi di coordinamento. La ragione è semplice: un sistema in grado di coordinare il comportamento collettivo di oggetti fisici nello spazio reale, di autenticare l'identità dei suoi utenti tramite biometria facciale, di apprendere i pattern comportamentali degli utenti attraverso sessioni operative prolungate e di ricevere futuri aggiornamenti che ne espandono le capacità, è un sistema che ha impatti nel mondo reale che vanno oltre la sua funzione tecnica immediata. Questi impatti — sulla privacy degli utenti, sulla sicurezza delle persone che si trovano nell'ambiente operativo del sistema, sulla distribuzione delle opportunità di accesso alla tecnologia, sull'evoluzione delle norme sociali relative all'autonomia dei sistemi robotici — non possono essere ignorati dai progettisti di MicroBot senza rinunciare alla responsabilità che accompagna inevitabilmente la capacità tecnica di costruire sistemi con queste caratteristiche.

La prima dimensione etica di MicroBot riguarda la raccolta e il trattamento dei dati biometrici degli utenti. Il sistema di autenticazione facciale raccoglie e processa immagini del volto degli utenti autorizzati, estrae caratteristiche biometriche da queste immagini e le memorizza in un database cifrato sul controller. Questi dati sono tra le categorie più sensibili di dati personali previste dalle normative sulla protezione dei dati — in Europa dal Regolamento Generale sulla Protezione dei Dati, il GDPR, che classifica i dati biometrici come categoria speciale di dati che richiedono misure di protezione rafforzate e basi giuridiche specifiche per il loro trattamento. MicroBot adotta il principio di minimizzazione dei dati biometrici — memorizzando solo i vettori di caratteristiche estratti dai volti e non le immagini originali — e il principio di proporzionalità — utilizzando i dati biometrici esclusivamente per la funzione di autenticazione per cui sono stati raccolti e non per nessuna altra finalità. Il meccanismo di cancellazione dei dati biometrici è accessibile all'utente tramite l'interfaccia web e produce la cancellazione irreversibile di tutti i template biometrici memorizzati senza richiedere la disinstallazione o la reinizializzazione del sistema. L'informativa sul trattamento dei dati

biometrici — che descrive quali dati vengono raccolti, per quale finalità, per quanto tempo vengono conservati e quali diritti ha l'utente rispetto a questi dati — è parte integrante della documentazione del sistema e deve essere letta e accettata esplicitamente dall'utente prima che il sistema di autenticazione venga attivato.

La seconda dimensione etica riguarda la sicurezza delle persone nell'ambiente operativo del sistema. Un sistema di swarm robotics con aggancio magnetico controllato è intrinsecamente capace di esercitare forze fisiche su oggetti nell'ambiente — e potenzialmente su persone che si trovano in prossimità dei bot durante le sessioni operative. La forza di aggancio di 0.5–1.5 newton che MicroBot produce non è pericolosa per gli adulti in condizioni di utilizzo normale, ma può essere fonte di disagio per bambini piccoli o per persone con ipersensibilità tattile se vengono a contatto involontario con i bot durante una sessione operativa. Il safety layer di MicroBot include la disattivazione immediata di tutte le coil in risposta a determinati pattern di perturbazione rilevati dai sensori — una misura che riduce il rischio di contatto indesiderato ma non lo elimina completamente. La responsabilità primaria per la sicurezza dell'ambiente operativo è dell'utente che sceglie il contesto di utilizzo del sistema: MicroBot è progettato per l'utilizzo su superfici di lavoro controllate in ambienti in cui l'accesso dei non utenti è supervisionato, e il suo utilizzo in ambienti pubblici non controllati — installazioni artistiche aperte al pubblico, dimostrazioni in spazi frequentati da bambini — richiede misure aggiuntive di separazione fisica tra i bot e il pubblico che devono essere pianificate e implementate dall'organizzatore dell'evento.

La terza dimensione etica riguarda le implicazioni della crescente autonomia del sistema nelle versioni future. Nella versione 0.3 MicroBot è un sistema completamente eteronomo — ogni azione dei bot è il risultato di un comando esplicito del controller, che a sua volta esegue le istruzioni dell'utente autenticato. Nelle versioni future — particolarmente dalla versione 0.4 con la comunicazione inter-bot diretta e dalla versione 0.5 con gli algoritmi di reinforcement learning — il sistema acquisisce gradi crescenti di autonomia locale: i bot coordinano direttamente la procedura di aggancio senza l'intervento del controller, il sistema di reinforcement learning sceglie autonomamente i parametri di coordinamento che massimizzano la funzione di ricompensa, il sistema di raccomandazione propone proattivamente pattern che l'utente non ha esplicitamente richiesto. Ogni incremento di autonomia riduce la granularità del controllo umano sul comportamento del sistema e aumenta la difficoltà di prevedere e spiegare le azioni del sistema in termini comprensibili all'utente. La letteratura sull'etica dell'intelligenza artificiale ha sviluppato il concetto di meaningful human control — controllo umano significativo — come requisito fondamentale per i sistemi autonomi: non è sufficiente che un essere umano sia formalmente in controllo del sistema se il sistema prende decisioni a una velocità o a un livello di complessità che supera la capacità dell'umano di comprenderle e di intervenire in modo informato. MicroBot adotta il principio di autonomia incrementale con supervisione — ogni incremento di autonomia è accompagnato da strumenti di trasparenza che permettono all'utente di comprendere le decisioni autonome del sistema — e il principio di interruzione immediata — l'utente può in qualsiasi momento disattivare completamente l'autonomia locale e riprendere il controllo manuale completo tramite il comando di emergenza globale.

La quarta dimensione etica riguarda l'impatto ambientale del sistema. MicroBot utilizza batterie Li-Po che contengono litio, cobalto e altri materiali critici con impatti significativi sulla catena di approvvigionamento globale — estrazione mineraria in condizioni spesso

problematiche, trasporto su lunghe distanze, smaltimento difficile a fine vita. Il progetto affronta questa dimensione attraverso due approcci complementari. Il primo è l'estensione della vita operativa dei componenti: le batterie di MicroBot sono progettate per essere facilmente sostituibili senza aprire l'involucro — grazie al connettore JST-PH accessibile — e senza danneggiare il guscio stampato, permettendo di sostituire solo la batteria esaurita invece di smaltire l'intero modulo. Il secondo è l'adozione di materiali di stampa biodegradabili: il PLA utilizzato per i gusci è derivato da acido polilattico di origine vegetale e è compostabile in condizioni industriali, producendo un impatto ambientale significativamente inferiore rispetto ai polimeri derivati dal petrolio come ABS o PETG. Per i componenti elettronici, la cui sostenibilità è più difficile da garantire a livello di singolo progetto, MicroBot adotta il principio di modularità riparabile — ogni componente del PCB è sostituibile individualmente senza richiedere la sostituzione dell'intero PCB — che estende la vita utile del sistema riducendo la necessità di smaltimento e riacquisto di componenti ancora funzionanti.

La quinta dimensione etica riguarda la distribuzione dei benefici della tecnologia di swarm robotics. MicroBot è progettato come piattaforma aperta e accessibile con l'obiettivo esplicito di democratizzare l'accesso alla ricerca sulla swarm robotics, ma la democratizzazione dell'accesso tecnologico non avviene automaticamente semplicemente rendendo il codice disponibile su GitHub e i componenti acquistabili su AliExpress. Richiede un investimento attivo nella riduzione delle barriere non economiche all'accesso — la lingua della documentazione, la complessità delle competenze prerequisite, la disponibilità di canali di supporto per chi incontra difficoltà nella replicazione. MicroBot affronta questa sfida attraverso la produzione di documentazione multilingue — con traduzione prioritaria in italiano, inglese, spagnolo e cinese — e attraverso la creazione di versioni semplificate del sistema adatte a contesti educativi con competenze tecniche ridotte — una versione MicroBot-Edu con un numero ridotto di componenti, processo di assemblaggio semplificato e firmware con funzionalità ridotte ma più facili da comprendere e da modificare da studenti alle prime armi con l'elettronica embedded. La collaborazione con istituzioni educative — scuole superiori tecnico-scientifiche, università, istituti di formazione professionale — per l'adozione di MicroBot come strumento didattico nei corsi di robotica, di sistemi embedded e di intelligenza artificiale è una priorità strategica del progetto che moltiplica l'impatto della tecnologia ben oltre il numero di replicatori individuali che costruiscono il sistema per utilizzo personale o di ricerca.

La sesta dimensione etica riguarda la trasparenza del sistema verso il pubblico che osserva le sue installazioni e dimostrazioni pubbliche. Quando MicroBot viene utilizzato in un contesto pubblico — una galleria d'arte, un festival tecnologico, una fiera scientifica — le persone che osservano il sistema in funzione non hanno informazioni sul suo funzionamento, sulle sue capacità e sui suoi limiti se queste informazioni non vengono attivamente comunicate. Il sistema di autenticazione biometrica facciale, in particolare, deve essere chiaramente comunicato ai visitatori di un'installazione pubblica: la presenza di una telecamera che riprende i volti delle persone in uno spazio pubblico, anche se utilizzata esclusivamente per l'autenticazione degli operatori del sistema e non per il riconoscimento dei visitatori, richiede una segnaletica esplicita e chiara che informi i visitatori della presenza e della funzione della telecamera. Più in generale, MicroBot adotta il principio di trasparenza pubblica — la disponibilità a spiegare il funzionamento del sistema in termini comprensibili a un pubblico non tecnico, a rispondere alle domande sui dati raccolti e sul loro utilizzo, e a

rendere visibili i limiti e le incertezze del sistema accanto alle sue capacità — come parte integrante della sua filosofia progettuale e non come obbligo regolatorio da soddisfare con il minimo sforzo possibile.

Punto 42 — Confronto con sistemi esistenti e posizionamento nella letteratura

Collocare MicroBot nel panorama della swarm robotics esistente è un esercizio necessario per due ragioni complementari. La prima è intellettuale: comprendere come MicroBot si relaziona con i sistemi che lo hanno preceduto — cosa eredita, cosa modifica, cosa introduce di genuinamente nuovo — è parte integrante della comprensione del progetto stesso. La seconda è scientifica: la validità di un contributo originale alla ricerca non può essere affermata in astratto ma deve essere dimostrata rispetto allo stato dell'arte, identificando le lacune nella letteratura esistente che il progetto si propone di colmare e i risultati dei sistemi comparabili rispetto ai quali le prestazioni di MicroBot vengono misurate. Un progetto che non conosce i propri predecessori rischia di reinventare soluzioni già note, di trascurare lezioni già apprese e di perdere l'opportunità di costruire su fondamenta solide invece di partire da zero.

Il Kilobot, sviluppato dal laboratorio di robotica della Harvard University e descritto in una serie di pubblicazioni a partire dal 2012, è il punto di riferimento più diretto per MicroBot nel panorama della swarm robotics su superfici piane. Il Kilobot è un robot circolare del diametro di 33 millimetri e del peso di 22 grammi, equipaggiato con un microcontrollore ATmega328 — lo stesso chip dell'Arduino Uno — due motori a vibrazione che producono il movimento per locomozione differenziale strisciando sulla superficie di appoggio, un emettitore e un ricevitore a infrarossi per la comunicazione locale tra bot vicini, e una batteria ricaricabile tramite un sistema di ricarica simultanea a induzione che permette di ricaricare centinaia di bot contemporaneamente senza connessioni fisiche individuali. Il Kilobot è stato progettato specificamente per la ricerca su swarm di grandi dimensioni — il paper originale di Rubenstein, Cornejo e Nagpal del 2014 dimostra la formazione di pattern con 1024 Kilobot — e la sua architettura riflette questa priorità di scalabilità: la comunicazione a infrarossi locale permette a ogni bot di rilevare la distanza dai vicini senza un sistema di localizzazione globale, e gli algoritmi di coordinamento implementati sui Kilobot sono interamente decentralizzati. Il costo originale di un Kilobot era di circa 14 dollari per unità in grandi quantità, ma i kit attuali disponibili commercialmente si posizionano nell'intervallo di 80–100 dollari per unità, significativamente superiore al costo target di MicroBot.

Il confronto tra MicroBot e Kilobot rivela complementarità più che competizione diretta. Il Kilobot è ottimizzato per la scalabilità verso grandi swarm con coordinamento completamente decentralizzato — una piattaforma di ricerca per l'emergenza collettiva pura. MicroBot è ottimizzato per l'interazione ricca con l'utente, l'aggregazione fisica tramite aggancio magnetico e l'integrazione con interfacce multimodali avanzate — una piattaforma per l'esplorazione del confine tra controllo umano e autonomia collettiva. Il Kilobot non ha aggancio fisico tra moduli — ogni bot rimane un'entità indipendente che si muove per vibrazione — mentre l'aggancio magnetico di MicroBot permette la formazione di strutture rigide composite che il Kilobot non può realizzare. MicroBot ha una connessione wireless con il controller centrale che il Kilobot non ha — il Kilobot è completamente autonomo e non riceve comandi dall'esterno durante l'operazione — ma questa differenza architettonica

riflette obiettivi diversi: la purezza del coordinamento decentralizzato nel Kilobot, la ricchezza dell'interazione uomo-macchina in MicroBot.

Il sistema M-Blocks del Massachusetts Institute of Technology, sviluppato dal gruppo di ricerca di Daniela Rus e descritto in pubblicazioni a partire dal 2013, esplora una direzione complementare alla swarm robotics su superfici piane: la modular robotics tridimensionale con aggregazione fisica tra moduli. Gli M-Blocks sono cubi di 50 millimetri di lato equipaggiati con magneti permanenti sulle facce e con un meccanismo di accelerazione interno — un volano ad alta velocità — che permette ai moduli di saltare, ruotare e agganciarsi ad altri moduli in configurazioni tridimensionali. Il sistema di aggancio degli M-Blocks è basato esclusivamente su magneti permanenti senza coil elettromagnetiche attive — una scelta che semplifica l'hardware a scapito della controllabilità del disaggancio, che negli M-Blocks richiede l'applicazione di forze meccaniche esterne piuttosto che l'inversione di corrente nelle coil come in MicroBot. Il confronto con M-Blocks è particolarmente rilevante per la scelta architetturale del sistema di aggancio magnetico di MicroBot: la combinazione di magneti permanenti per il pre-allineamento passivo e coil elettromagnetiche per l'aggancio e il disaggancio controllato è una soluzione originale che non trova precedenti diretti né nel Kilobot né negli M-Blocks, e che produce un sistema di aggancio con caratteristiche di controllabilità superiori agli M-Blocks mantenendo la semplicità costruttiva dei magneti permanenti come elemento primario di tenuta.

Il sistema Zooids, sviluppato dall'Inria e dalla Stanford University e pubblicato nel 2016, è il sistema di swarm robotics più simile a MicroBot per l'enfasi sulla qualità dell'interazione uomo-macchina. I Zooids sono piccoli robot circolari del diametro di 26 millimetri che si muovono su una superficie di lavoro retroilluminata di 1.4 metri di diagonale, localizzati tramite un proiettore ottico che illumina ciascun bot con un pattern luminoso codificato che permette al sistema di calcolare la posizione di tutti i bot con precisione millimetrica a 60 hertz. I Zooids sono controllati da un computer centrale che invia comandi di velocità a ciascun bot tramite radio a 2.4 gigahertz, e l'interfaccia utente permette all'utente di interagire direttamente con i bot attraverso touch, gesture e manipolazione diretta sulla superficie di lavoro. Il confronto con Zooids è illuminante per la comprensione del contributo originale di MicroBot nell'area dell'interazione uomo-macchina: Zooids ha un sistema di localizzazione di alta precisione basato su proiettore ottico che MicroBot non ha nella versione prototipale, ma non ha aggancio fisico tra moduli, non ha integrazione con visore VR, non ha framework 4D/5D con modulazione sonora e non ha sistema di autenticazione biometrica. Il costo del sistema Zooids — dominato dal proiettore di posizionamento ad alta risoluzione e dalla superficie di lavoro retroilluminata — è significativamente superiore al costo di MicroBot, posizionandolo in una fascia di accessibilità diversa.

Il sistema Topobo, sviluppato da Hayes Soffer e Mitchel Resnick al MIT Media Lab, esplora la dimensione della materia programmabile con aggancio fisico da una prospettiva educativa e creativa: moduli che possono essere connessi fisicamente e che ricordano e ripetono i movimenti che vengono loro impressi manualmente, producendo strutture animate che trasformano la memoria del gesto umano in comportamento autonomo del sistema. Topobo non è strettamente un sistema di swarm robotics — i moduli non comunicano tra loro né prendono decisioni autonome — ma è rilevante per MicroBot come precedente nella visione della materia fisicamente programmabile attraverso l'interazione diretta dell'utente. La visione di MicroHand come evoluzione futura di MicroBot condivide con Topobo l'ambizione

di creare strutture fisiche che trasformano l'intenzione umana in configurazioni materiali, portando questa visione in un contesto di swarm autonomo e interattivo che Topobo non realizza.

Il sistema Programmable Matter di Seth Goldstein e Todd Mowry della Carnegie Mellon University — descritto in pubblicazioni a partire dal 2005 — è il precedente teorico più rilevante per la visione di lungo termine di MicroBot. Il progetto Claytronics di Goldstein e Mowry immagina moduli robotici della dimensione di un millimetro — i catoms, da claytronic atoms — capaci di aggregarsi in strutture tridimensionali arbitrarie che possono replicare la forma di qualsiasi oggetto fisico. Questa visione di materia programmabile a grana fine è il punto di arrivo teorico verso cui MicroBot si orienta, consapevole che la distanza tra i moduli da 50–70 millimetri della versione prototipale e i catoms da 1 millimetro di Claytronics è una distanza che richiede decenni di sviluppo tecnologico in miniaturizzazione, efficienza energetica e algoritmi distribuiti. MicroBot contribuisce a questo percorso sviluppando e validando architetture hardware e software che possono essere progressivamente miniaturizzate man mano che le tecnologie abilitanti — microcontrollori sempre più compatti, batterie sempre più dense, fabbricazione sempre più precisa — raggiungono le prestazioni necessarie.

Il posizionamento di MicroBot nella letteratura della swarm robotics può essere sintetizzato identificando tre contributi originali che distinguono il progetto dai sistemi precedenti. Il primo contributo è il framework 4D/5D per la modulazione del comportamento dello swarm attraverso stati interni dipendenti dal suono — un approccio che non ha precedenti diretti nella letteratura e che apre una nuova direzione di ricerca nell'interazione multimodale tra utente e swarm. Il secondo contributo è l'architettura di aggancio magnetico ibrido — magneti permanenti per il pre-allineamento passivo e coil elettromagnetiche per l'aggancio e il disaggancio controllato — che produce caratteristiche di controllabilità superiori ai sistemi basati su soli magneti permanenti come M-Blocks e superiori ai sistemi basati su soli elettromagneti come alcuni sistemi di modular robotics che richiedono alimentazione continua per mantenere l'aggancio. Il terzo contributo è l'architettura integrata di interazione uomo-macchina — visore VR, wristband IMU, riconoscimento gesture, modulazione sonora, autenticazione biometrica — che supera per ricchezza e naturalezza dell'interazione qualsiasi sistema di swarm robotics esistente, aprendo la possibilità di esplorare modalità di controllo dello swarm che non sono state indagate da nessun sistema precedente.

Punto 43 — Roadmap dettagliata: dalla versione 0.1 alla versione 0.5 e oltre

La roadmap di MicroBot non è un documento di marketing che promette funzionalità future senza una base tecnica solida — è un piano di sviluppo iterativo radicato nella comprensione concreta dei vincoli tecnologici, delle dipendenze tra componenti e delle risorse disponibili per ciascuna fase. Ogni versione della roadmap è definita da un insieme coerente di obiettivi raggiungibili con le tecnologie e le competenze disponibili in quella fase, da criteri di completamento verificabili che determinano quando una versione è sufficientemente matura per procedere alla successiva, e da un insieme di lezioni apprese che informano la progettazione della versione successiva. Questo approccio incrementale e verificabile è la risposta pratica alla complessità del sistema: invece di pianificare un sistema completo che viene realizzato tutto in una volta — con il rischio concreto che errori architetturali fondamentali vengano scoperti solo quando tutto il sistema è già stato costruito

— MicroBot evolve per stadi successivi in cui ogni stadio aggiunge funzionalità verificabili al sistema precedente, permettendo di rilevare e correggere i problemi prima che si propaghino all'intera architettura.

La versione 0.1 è la versione fondativa del progetto — la fase in cui l'architettura concettuale del sistema viene definita, documentata e validata attraverso la simulazione astratta senza alcun componente hardware fisico. Gli obiettivi di questa versione sono esclusivamente intellettuali e documentali: definire l'architettura a cinque livelli del sistema, specificare i protocolli di comunicazione, formalizzare il modello matematico dello swarm, documentare le scelte progettuali con le loro motivazioni. Il prodotto principale della versione 0.1 è il documento che state leggendo — questa documentazione master a 45 punti che costituisce la base di riferimento per tutte le versioni successive. Il secondo prodotto è la simulazione JavaScript del motore di simulazione comportamentale, che implementa il modello matematico descritto nel documento e permette di visualizzare e validare i pattern geometrici e gli algoritmi di coordinamento in un ambiente puramente virtuale. Il terzo prodotto è lo schema elettrico del circuito di pilotaggio delle coil con i modelli SPICE dei componenti critici, che permette di verificare la correttezza del dimensionamento prima di acquistare e assemblare i componenti fisici. Il criterio di completamento della versione 0.1 è la produzione di documentazione sufficiente a permettere la replicazione del sistema da parte di un team esterno con competenze tecniche appropriate, e la validazione della simulazione JavaScript contro le metriche di convergenza teoricamente attese dal modello matematico.

La versione 0.2 è la versione di simulazione hardware — la fase in cui il comportamento del firmware viene verificato in ambiente virtuale prima di essere caricato sull'hardware fisico reale. Il componente principale di questa versione è la simulazione del firmware su Wokwi, che permette di sviluppare e testare la logica del codice embedded — la macchina a stati finiti, il safety layer, il protocollo di comunicazione, il sistema di logging — senza disporre ancora di ESP32 fisici. Un aspetto caratteristico della versione 0.2 è la simulazione del comportamento di una flotta di bot attraverso una serie di LED che replicano la segnalazione luminosa dei MicroBot fisici: in assenza di robot veri, LED di diversi colori collegati ai pin GPIO di un ESP32 di sviluppo rappresentano i singoli bot della flotta, lampeggiando in schemi che riproducono i pattern luminosi che i LED fisici dei MicroBot produrranno nella versione 0.3. Questo approccio permette di verificare visivamente la correttezza della logica di coordinamento — la sequenza di attivazione dei bot durante una transizione di pattern, la risposta del sistema alle condizioni di emergenza simulate, la coerenza del timing tra i diversi bot — senza richiedere hardware robotico fisico. La versione 0.2 include anche lo sviluppo del firmware del controller nella sua versione base — gestione della rete SoftAP, ricezione degli heartbeat, invio dei comandi di pattern — testato nella simulazione Wokwi con un controller virtuale che comunica con un insieme di client virtuali che simulano i bot. Il criterio di completamento della versione 0.2 è il superamento di tutti i test unitari e di integrazione del firmware su piattaforma Wokwi, e la dimostrazione visiva della sequenza corretta di lampeggio dei LED per ciascuno dei pattern implementati.

La versione 0.3 è la versione prototipale fisica — la fase in cui tutti i componenti hardware vengono costruiti e integrati per la prima volta producendo un sistema fisicamente funzionale. Questa è la versione più critica dell'intera roadmap perché è il primo confronto diretto tra il sistema progettato sulla carta e la realtà fisica, e quasi certamente produrrà

sorprese — componenti che non si comportano come previsto, tolleranze dimensionali che non sono sufficienti, effetti fisici non modellati che degradano le prestazioni rispetto alle attese. Il piano di assemblaggio della versione 0.3 segue la sequenza descritta nel capitolo sulla prototipazione fisica, procedendo da un singolo bot verso la flotta completa attraverso stadi di verifica progressiva. Il primo stadio è l'assemblaggio e il test di un singolo bot con firmware base — connessione Wi-Fi, heartbeat, pilotaggio delle coil, segnalazione LED — che verifica la correttezza del design elettrico e del firmware prima di replicare il bot in più esemplari. Il secondo stadio è la replica del bot verificato in una flotta minima di quattro unità e il test della comunicazione multi-bot con il controller — connessioni simultanee, sincronizzazione del clock, risposta coordinata ai comandi di pattern. Il terzo stadio è il test dell'aggancio magnetico tra due bot — verifica delle forze di aggancio, del tasso di successo e della procedura di disaggancio controllato — che valida il design del sistema magnetico. Il quarto stadio è l'integrazione del wristband e del sistema di autenticazione biometrica — verifica del riconoscimento delle gesture, dell'autenticazione facciale e del controllo del sistema tramite wristband. Il quinto stadio è l'integrazione del visore VR — verifica del tracking della testa, del rendering della scena 3D e della comunicazione bidirezionale tra il visore e il controller. Il criterio di completamento della versione 0.3 è il superamento dei test di accettazione con una flotta di almeno 8 bot, con metriche di performance entro il 30% dei valori target specificati nel capitolo sulla validazione sperimentale.

La versione 0.4 introduce tre famiglie di funzionalità che espandono significativamente le capacità del sistema rispetto alla versione 0.3. La prima famiglia è la comunicazione inter-bot tramite ESP-NOW, con implementazione del protocollo di scoperta dei vicini, dei messaggi di coordinazione locale dell'aggancio e del meccanismo di degradazione automatica verso il canale UDP per i bot non aggiornati. La seconda famiglia è l'autonomia locale del sistema di aggancio, con implementazione della procedura di aggancio coordinato direttamente tra bot vicini senza mediazione del controller e del meccanismo di rinforzo locale dell'aggancio in risposta alle perturbazioni esterne rilevate localmente. La terza famiglia è il framework 4D/5D nella sua versione completa, con implementazione dell'analisi audio in tempo reale, della mappatura parametrica tra caratteristiche acustiche e componenti del vettore W , e dell'integrazione della modulazione W nei parametri di coordinamento del motore di simulazione. Il criterio di completamento della versione 0.4 è la dimostrazione della riduzione della latenza di aggancio da 40–60 millisecondi nella versione 0.3 a 5–10 millisecondi con la comunicazione inter-bot ESP-NOW, e la dimostrazione della correlazione misurabile tra il segnale audio di input e il comportamento osservabile dello swarm attraverso la modulazione del vettore W .

La versione 0.5 rappresenta il punto di maturità del progetto nella sua configurazione prototipale e introduce le funzionalità di intelligenza distribuita che avvicinano MicroBot alla visione di lungo termine della swarm robotics autonoma e adattiva. Il componente principale è il sistema di reinforcement learning per l'ottimizzazione dei parametri di coordinamento, con il ciclo completo di addestramento in simulazione, transfer learning sul sistema fisico e aggiornamento del modello tramite OTA. Il secondo componente è il sistema di anticipazione dei pattern — un modulo predittivo che osserva le sequenze di comandi dell'utente e inizia a prepararsi per la transizione successiva prima che il comando esplicito venga emesso, riducendo la latenza percepita del sistema. Il terzo componente è il sistema di adattamento autonomo al contesto operativo — il sistema rileva automaticamente le caratteristiche della superficie di lavoro attraverso l'analisi delle forze di aggancio richieste, adatta i parametri di

coordinamento all'attrito e alle irregolarità della superficie specifica, e memorizza le configurazioni ottimali per superfici ricorrenti. Il criterio di completamento della versione 0.5 è la dimostrazione di un miglioramento misurabile delle metriche di performance — tempo di convergenza, precisione di posizionamento, tasso di successo dell'aggancio — rispetto alla versione 0.4 dopo 10 sessioni di training del sistema di reinforcement learning, e la dimostrazione di un tasso di riconoscimento gesture superiore al 92% con il classificatore addestrato sui dati reali dell'utente rispetto al 90% dell'algoritmo a soglie della versione 0.3.

Oltre la versione 0.5 il progetto entra in una fase di sviluppo che va oltre la roadmap pianificata con certezza e si apre a direzioni di ricerca che dipendono dai risultati ottenuti nelle versioni precedenti e dalle opportunità di collaborazione che si presentano lungo il percorso. Le direzioni principali identificate sono tre. La prima è la scalabilità verso flotte di 50–100 bot con topologia di comunicazione gerarchica a due livelli — controller, subcontroller, bot — che richiede la riduzione del costo unitario dei bot verso i 5 euro attraverso l'ottimizzazione del design del PCB per la produzione in volume e la selezione di microcontrollori ancora più economici. La seconda è la miniaturizzazione verso bot di dimensioni inferiori a 30 millimetri — una riduzione del volume di un fattore 8 rispetto alla versione 0.3 — che richiede avanzamenti significativi nella densità energetica delle batterie, nella potenza di elaborazione dei microcontrollori ultra-compatti e nelle tecniche di assemblaggio di componenti su PCB di dimensioni inferiori al centimetro. La terza è lo sviluppo della MicroHand — la biforcazione evolutiva più ambiziosa di MicroBot — che richiede l'integrazione di moduli specializzati con attuatori meccanici nelle strutture aggregate dello swarm per produrre appendici robotiche riconfigurabile capaci di manipolazione di oggetti.

La gestione della roadmap oltre la versione 0.5 adotta un modello di sviluppo guidato dalla comunità — research-driven open development — in cui le priorità di sviluppo vengono determinate non solo dal team originale del progetto ma dalla comunità di ricercatori e maker che hanno adottato MicroBot come piattaforma di ricerca e che portano al progetto prospettive, competenze e applicazioni che il team originale non avrebbe potuto anticipare. Questo modello riconosce che il valore di una piattaforma di ricerca aperta non è solo nei risultati che il team originale produce con essa ma nella moltiplicazione di direzioni di ricerca che diventa possibile quando la piattaforma è accessibile a una comunità allargata — una moltiplicazione che non può essere pianificata ma solo abilitata attraverso la qualità della documentazione, la robustezza dell'architettura e la disponibilità del team originale a collaborare con i membri della comunità che vogliono estendere il sistema in direzioni nuove e inattese.

Punto 44 — Contributi originali, pubblicazioni e disseminazione della ricerca

Un progetto di ricerca non completa il proprio ciclo vitale nel momento in cui il prototipo funziona correttamente nel laboratorio dove è stato sviluppato — completa il proprio ciclo solo quando i risultati, i metodi e le lezioni apprese vengono comunicati alla comunità scientifica e tecnica in modo sufficientemente dettagliato da permettere ad altri ricercatori di comprenderli, criticarli, replicarli e costruirci sopra. La disseminazione della ricerca non è un'attività accessoria che si aggiunge alla fine del lavoro tecnico ma è parte integrante del lavoro stesso: senza disseminazione il contributo di MicroBot alla conoscenza collettiva della swarm robotics rimane confinato al team che lo ha sviluppato, e i risultati ottenuti — anche

se significativi — non producono l'impatto moltiplicativo che è la ragione fondamentale per cui la ricerca viene condotta e condivisa. Questo capitolo identifica i contributi originali di MicroBot alla letteratura scientifica del settore, delinea la strategia di disseminazione dei risultati attraverso i canali appropriati e riflette sulle condizioni necessarie affinché MicroBot diventi non solo un prototipo funzionante ma un punto di riferimento per la ricerca futura nella swarm robotics interattiva.

L'identificazione rigorosa dei contributi originali richiede di distinguere con precisione tra ciò che MicroBot ha ereditato dalla letteratura esistente — e che viene correttamente attribuito ai suoi predecessori — e ciò che introduce genuinamente di nuovo. Questa distinzione non è sempre netta: molti contributi originali sono ricombinazioni creative di elementi esistenti in configurazioni nuove che producono proprietà emergenti non presenti in nessuno degli elementi originali. La novità di MicroBot non risiede nell'uso dell'ESP32 — ampiamente diffuso nell'ecosistema maker — né nell'uso dei magneti permanenti per l'aggancio fisico — già presente in M-Blocks e in altri sistemi di modular robotics — né nella simulazione JavaScript del comportamento dello swarm — una tecnica standard nello sviluppo di sistemi multi-agente. La novità risiede nell'integrazione di questi elementi in un'architettura coerente che produce proprietà sistemiche assenti in qualsiasi sistema precedente, e nell'introduzione del framework 4D/5D che è genuinamente senza precedenti nella letteratura della swarm robotics.

Il primo contributo originale identificato è il framework 4D/5D per la modulazione del comportamento dello swarm attraverso stati interni dipendenti da segnali contestuali — il suono come caso primario implementato ma il framework è generalizzabile a qualsiasi segnale ambientale misurabile. Questo contributo è originale in due dimensioni distinte. La prima è la dimensione concettuale: l'idea di introdurre uno stato interno del sistema swarm che non corrisponde a nessuna variabile fisica direttamente osservabile — la posizione dei bot, la loro velocità, lo stato degli agganci — ma che parametrizza le relazioni tra questi osservabili è una novità concettuale che non ha precedenti nella letteratura. La seconda è la dimensione implementativa: la realizzazione di un framework che mappa in tempo reale le caratteristiche acustiche del segnale audio su questo stato interno, e che produce comportamenti fisicamente osservabili dello swarm che variano in modo continuo e naturale in risposta al suono, è una realizzazione tecnica che non era stata tentata in nessun sistema precedente. La pubblicazione di questo contributo richiede la formalizzazione matematica del framework — già sviluppata nel capitolo sul modello matematico — e la presentazione di risultati sperimentali quantitativi che dimostrano la correlazione tra segnale audio e risposta comportamentale dello swarm con le metriche definite nel capitolo sulla validazione sperimentale.

Il secondo contributo originale è l'architettura ibrida di aggancio magnetico che combina magneti permanenti per il pre-allineamento passivo e coil elettromagnetiche per l'aggancio e il disaggancio controllato con modulazione continua della rigidità dell'aggregato. Questo contributo è caratterizzato da un'analisi comparativa rispetto alle architetture di aggancio esistenti — soli magneti permanenti come in M-Blocks, soli elettromagneti come in alcuni sistemi di modular robotics — che dimostra vantaggi quantitativi in termini di controllabilità del disaggancio, efficienza energetica del mantenimento dell'aggancio e qualità del pre-allineamento passivo. La pubblicazione di questo contributo richiede una caratterizzazione sperimentale completa del sistema di aggancio — le curve forza-distanza

in funzione del duty cycle delle coil, le statistiche del tasso di successo in funzione dell'allineamento iniziale, le curve di consumo energetico nelle diverse fasi dell'aggancio — che costituiscono i dati quantitativi necessari per la comparazione con i sistemi alternativi.

Il terzo contributo originale è l'architettura integrata di interazione uomo-macchina per il controllo dello swarm che combina visore VR, wristband IMU, riconoscimento gesture multimodale, autenticazione biometrica facciale e modulazione sonora in un sistema coerente. Questo contributo è rilevante non solo per la comunità della swarm robotics ma anche per la comunità dell'interazione uomo-macchina e della realtà mista, perché esplora modalità di controllo di sistemi fisici distribuiti che non erano state investigate in precedenza. La pubblicazione di questo contributo richiede uno studio utente formale con un numero statisticamente significativo di partecipanti che valuti le metriche di usabilità, la curva di apprendimento e la qualità dell'esperienza soggettiva del controllo dello swarm attraverso l'interfaccia integrata di MicroBot, confrontandola con interfacce di controllo alternative — tastiera e mouse, joystick, touchscreen — per quantificare i vantaggi delle modalità di interazione multimodale implementate.

Il quarto contributo originale è il modello di gestione energetica adattiva per swarm di bot miniaturizzati con batterie di piccola capacità, che combina la modalità operativa a tre livelli — Full Active, Pattern Hold, Standby — con l'algoritmo di energy-aware scheduling che minimizza il consumo energetico complessivo dello swarm durante le transizioni di pattern. Questo contributo è rilevante per tutta la comunità della swarm robotics con moduli alimentati a batteria e risponde a una sfida pratica — come massimizzare l'autonomia di una flotta di robot con batterie di piccola capacità senza compromettere le prestazioni di coordinamento — che è condivisa da molti sistemi della letteratura ma che raramente viene affrontata con il rigore quantitativo che MicroBot adotta. La pubblicazione di questo contributo richiede dati sperimentali sull'autonomia del sistema nelle diverse modalità operative e la dimostrazione quantitativa del vantaggio dell'algoritmo energy-aware rispetto a strategie di scheduling alternative.

La strategia di pubblicazione dei contributi di MicroBot è articolata su tre livelli di venue che corrispondono a diversi gradi di formalizzazione dei risultati e a diverse comunità di destinatari. Il primo livello è la pubblicazione tecnica open access su piattaforme come arXiv o HAL, che permette la condivisione immediata dei risultati in formato preprint senza i tempi di revisione tipici delle pubblicazioni scientifiche formali. I preprint sono particolarmente adatti per i contributi tecnici dettagliati — la specifica del protocollo di comunicazione, la documentazione del framework 4D/5D, la descrizione dell'architettura hardware — che beneficiano di una disseminazione rapida verso la comunità di sviluppatori e maker che possono iniziare a sperimentare con il sistema mentre il processo di revisione formale è ancora in corso. Il secondo livello è la pubblicazione su conferenze scientifiche del settore — IEEE International Conference on Robotics and Automation, ACM CHI Conference on Human Factors in Computing Systems, IEEE RoboSoft — che offrono revisione tra pari, feedback della comunità e una vetrina per la presentazione live del sistema a una platea di esperti. La presentazione live è particolarmente importante per un sistema come MicroBot le cui proprietà più significative — la fluidità del comportamento collettivo, la naturalezza dell'interazione uomo-macchina, la risposta del sistema al suono — sono difficili da comunicare attraverso testo e immagini statiche e richiedono una dimostrazione dal vivo per essere pienamente apprezzate. Il terzo livello è la pubblicazione su riviste scientifiche con

peer review esteso — come IEEE Transactions on Robotics, ACM Transactions on Human-Robot Interaction o Soft Robotics — che richiedono tempi più lunghi ma offrono la maggiore visibilità e permanenza nella letteratura scientifica.

La disseminazione verso la comunità non accademica — maker, hobbisti, educatori, artisti digitali — avviene attraverso canali distinti e appropriati a questo pubblico diverso. Il repository GitHub del progetto con documentazione completa in più lingue è il canale primario di disseminazione tecnica verso la comunità maker. I tutorial video — serie di video pubblicati su piattaforme come YouTube che guidano passo per passo attraverso l'assemblaggio di un MicroBot, il caricamento del firmware, la configurazione del sistema e il primo test di aggancio magnetico — sono il canale più efficace per raggiungere un pubblico con competenze tecniche variabili che apprende meglio attraverso la dimostrazione visiva che attraverso la lettura di documentazione scritta. Le partecipazioni a maker faire, hackathon e festival della tecnologia sono le occasioni per portare il sistema fisicamente a contatto con comunità di potenziali replicatori e collaboratori, ricevere feedback diretti sull'esperienza di utilizzo e identificare applicazioni e varianti del sistema che il team originale non aveva considerato. La collaborazione con università e istituti tecnici per l'adozione di MicroBot come strumento didattico nei corsi di robotica e sistemi embedded è il canale di disseminazione con il maggiore impatto moltiplicativo nel lungo periodo: ogni corso che adotta MicroBot come piattaforma di laboratorio forma una nuova generazione di studenti che portano con sé la familiarità con il sistema nelle loro future attività di ricerca e sviluppo, espandendo organicamente la comunità del progetto senza richiedere investimenti aggiuntivi di marketing o di comunicazione.

Il meccanismo di feedback dalla comunità verso il progetto è altrettanto importante della disseminazione unidirezionale dei risultati. Il sistema di issue tracking su GitHub permette a chiunque abbia replicato il sistema di segnalare problemi, proporre miglioramenti e condividere varianti del design che hanno funzionato meglio nella propria specifica applicazione. Un forum dedicato — ospitato su piattaforme come Discourse o Reddit — offre uno spazio di discussione meno formale dove i membri della comunità possono scambiare esperienze, aiutarsi reciprocamente nella risoluzione dei problemi e coordinare collaborazioni su varianti e estensioni del sistema. La raccolta sistematica di questi feedback e la loro integrazione nelle versioni successive del sistema e della documentazione è la forma più efficace di sviluppo collaborativo — un processo in cui la comunità degli utenti diventa co-sviluppatrice del sistema, contribuendo con una diversità di contesti operativi, competenze e prospettive che nessun team ristretto di sviluppo potrebbe replicare autonomamente.

La documentazione della storia dello sviluppo del progetto — incluse le decisioni progettuali che si sono rivelate sbagliate, le soluzioni che non hanno funzionato, i problemi che hanno richiesto multiple iterazioni per essere risolti — è una forma di contributo alla conoscenza collettiva particolarmente preziosa e sistematicamente sottorappresentata nella letteratura scientifica, che tende a pubblicare solo i risultati positivi presentati nella loro forma migliore. MicroBot si impegna a documentare pubblicamente anche i fallimenti e le false partenze del processo di sviluppo — i design di PCB che non hanno funzionato, gli algoritmi di coordinamento che hanno prodotto comportamenti inattesi, le scelte architetturelle che si sono rivelate meno appropriate del previsto — perché questa conoscenza negativa è altrettanto preziosa per i ricercatori e i maker che affrontano sfide simili, e la sua assenza

dalla letteratura è una delle cause principali del fenomeno noto come rediscovery tax — il costo pagato da ogni nuovo progetto nel ripercorrere cammini già battuti da altri che non hanno documentato i propri errori.

Punto 45 — MicroHand: visione, architettura e traiettoria evolutiva

MicroHand è il punto di arrivo della visione di MicroBot — non una versione successiva del sistema ma una biforcazione evolutiva che porta i principi fondanti del progetto in un dominio applicativo radicalmente diverso e significativamente più ambizioso. Se MicroBot esplora il confine tra controllo umano e comportamento collettivo emergente su una superficie piana, MicroHand sposta questo confine nello spazio tridimensionale della manipolazione fisica di oggetti reali, trasformando lo swarm da un sistema di pattern estetici e scientificamente interessanti in uno strumento di interazione fisica con il mondo materiale. Questa transizione non è banale — la manipolazione di oggetti impone vincoli fisici e requisiti di precisione che non hanno equivalenti nel coordinamento su superficie piana — ma è naturale nella logica evolutiva del progetto: ogni componente architetturale sviluppato per MicroBot — il sistema di aggancio magnetico ibrido, il framework 4D/5D, l'interfaccia di controllo multimodale, il protocollo di comunicazione inter-bot — trova in MicroHand un campo di applicazione che ne valorizza le caratteristiche in modi che la superficie piana non permette.

La denominazione MicroHand non indica semplicemente un robot con dita articolate — sarebbe una descrizione riduttiva che non cattura la natura fondamentale diversa del sistema rispetto alle mani robotiche convenzionali. Una mano robotica convenzionale è una struttura meccanica a geometria fissa con un numero determinato di gradi di libertà — tipicamente cinque dita con tre o quattro articolazioni ciascuna — progettata per eseguire un repertorio predefinito di prese che coprono le configurazioni di manipolazione più comuni. MicroHand è invece una struttura robotica a geometria variabile che emerge dall'aggregazione dinamica di moduli identici — i MicroBot nella loro versione evoluta — capace di assumere configurazioni morfologicamente diverse in risposta alla forma dell'oggetto da manipolare, alle forze fisiche che l'oggetto richiede e all'intenzione dell'utente che guida la manipolazione. Questa adattabilità morfologica è la proprietà che distingue fondamentalemente MicroHand dalle mani robotiche convenzionali e che apre applicazioni impossibili per sistemi a geometria fissa: la presa di oggetti di forma irregolare e variabile — frutti di forma non standard, tessuti deformabili, oggetti fragili di geometria complessa — per i quali non esiste una configurazione di presa ottimale universale e la capacità di adattamento morfologico è la conditio sine qua non di una manipolazione sicura ed efficace.

L'architettura hardware di MicroHand si distingue da quella di MicroBot standard per l'introduzione di moduli specializzati con funzionalità diverse che si aggregano con i moduli base per formare strutture funzionalmente eterogenee. Il modulo base rimane il MicroBot nella sua versione 0.5 — con ESP32, sistema di aggancio magnetico ibrido, LED e batteria — che costituisce l'elemento strutturale e di comunicazione della struttura aggregata. Il modulo di attuazione è la novità architetturale principale di MicroHand: integra nel volume del MicroBot standard un attuatore fisico che produce un grado di libertà attivo aggiuntivo rispetto al semplice aggancio magnetico. Nelle versioni prototipali i candidati più promettenti per questo attuatore sono due tecnologie complementari con caratteristiche diverse. Il servomotore digitale subminiatura — disponibile in package di dimensioni compatibili con il volume interno di un MicroBot modificato, con coppia dell'ordine di 0.1–0.3 newton-metro e

risoluzione angolare di 1 grado — permette la realizzazione di articolazioni con angolo configurabile tra moduli adiacenti, trasformando la struttura aggregata da un corpo rigido in una struttura articolata con geometria controllabile. L'attuatore a memoria di forma — in lega NiTiNOL riscaldata per effetto Joule attraverso la corrente delle coil esistenti — permette la realizzazione di deformazioni continue e silenziose tra configurazioni predeterminate, con una forza disponibile dell'ordine di alcuni newton in un volume comparabile a quello delle coil elettromagnetiche già presenti nel MicroBot. La scelta tra queste tecnologie di attuazione dipende dal compromesso accettabile tra precisione angolare — superiore nel servomotore — e silenziosità e semplicità meccanica — superiori nell'attuatore a memoria di forma.

Il modulo sensoriale è il secondo tipo di modulo specializzato previsto nell'architettura di MicroHand. Integra sensori per la percezione dello stato di contatto con l'oggetto manipolato — sensori di forza e pressione tattile che misurano l'intensità e la distribuzione del contatto tra la struttura aggregata e la superficie dell'oggetto — e sensori di prossimità a infrarossi o a ultrasuoni che rilevano la posizione e la forma dell'oggetto prima del contatto fisico, permettendo la pianificazione della configurazione di presa ottimale prima dell'avvicinamento. Il modulo sensoriale comunica le proprie misurazioni al controller tramite il protocollo di heartbeat esteso con i nuovi campi sensoriali, e i dati sensoriali vengono utilizzati dall'algoritmo di controllo della presa per adattare in tempo reale la configurazione della struttura aggregata alla forma e alla deformabilità dell'oggetto manipolato. La distribuzione dei moduli sensoriali nella struttura aggregata — tipicamente nei moduli posizionati nelle zone di contatto primario con l'oggetto — produce una mappa di pressione tattile distribuita che permette di rilevare la distribuzione delle forze di contatto e di aggiustare la configurazione di presa per minimizzare le pressioni di picco sulle zone più fragili dell'oggetto.

L'algoritmo di controllo della presa di MicroHand è il componente computazionale più complesso del sistema e richiede lo sviluppo di capacità algoritmiche che vanno ben oltre quelle del sistema MicroBot base. Il problema fondamentale è la pianificazione della configurazione di presa ottimale — determinare quali moduli devono agganciarsi in quale configurazione per produrre una struttura che sia in grado di afferrare e manipolare l'oggetto target senza danneggiarlo. Questo problema appartiene alla categoria della grasp planning, ampiamente studiata nella letteratura della robotica ma tipicamente affrontata per mani robotiche a geometria fissa — il problema diventa significativamente più complesso quando la geometria della mano è essa stessa una variabile ottimizzabile. L'approccio adottato in MicroHand combina un modello geometrico dell'oggetto — ottenuto attraverso la ricostruzione tridimensionale dalla camera del visore VR tramite algoritmi di structure from motion o depth estimation — con un algoritmo di ottimizzazione che cerca la configurazione di aggregazione che massimizza la qualità della presa definita come combinazione della stabilità della presa — resistenza alle forze esterne di perturbazione — e della minimalità della pressione di contatto — prevenzione del danneggiamento di oggetti fragili. Nella versione prototipale di MicroHand questo algoritmo di ottimizzazione è eseguito sul computer che ospita il visore VR piuttosto che sull'ESP32-S3 del controller, la cui potenza computazionale è insufficiente per i calcoli di geometria tridimensionale richiesti dalla pianificazione della presa.

L'interfaccia di controllo di MicroHand estende l'interfaccia di MicroBot con modalità di interazione specifiche per la manipolazione di oggetti. Il visore VR mostra una

rappresentazione tridimensionale dell'oggetto target con una sovrapposizione della configurazione di presa pianificata — le zone di contatto previste colorate in funzione della pressione stimata, i moduli che formeranno la struttura di presa evidenziati nella mappa dello swarm — permettendo all'utente di validare visivamente la configurazione prima di eseguirla fisicamente. Il wristband rileva i movimenti della mano dell'utente nello spazio tridimensionale e li mappa sui movimenti della struttura MicroHand nello spazio operativo, producendo un sistema di teleoperation intuitivo in cui i gesti naturali della mano dell'utente si traducono in movimenti corrispondenti della struttura robotica — aprire la mano produce il disaggancio dei moduli e l'ampliamento della struttura, chiuderla produce l'aggregazione e la chiusura della presa. Il feedback aptico del wristband — prodotto da un array di attuatori vibratori posizionati sulle dita — restituisce all'utente una rappresentazione tattile delle forze di contatto misurate dai sensori di pressione della struttura MicroHand, chiudendo il ciclo sensorimotore tra il corpo dell'utente e il sistema robotico in modo che la manipolazione di oggetti tramite MicroHand produca una sensazione di contatto diretto che supera il semplice controllo visivo remoto.

Le applicazioni di MicroHand che motivano l'investimento nello sviluppo di questo sistema più complesso rispetto al MicroBot base sono distribuite su tre domini principali. Nel dominio della robotica di servizio MicroHand affronta il problema della manipolazione di oggetti di forma variabile in ambienti non strutturati — la cucina domestica, il magazzino di logistica, il laboratorio di ricerca — dove la varietà degli oggetti da manipolare rende impraticabile la progettazione di un gripper specializzato per ogni tipo di oggetto. La capacità di MicroHand di adattare morfologicamente la propria struttura alla forma dell'oggetto specifico da manipolare — un pomodoro, una bottiglia, un circuito stampato, un foglio di carta — senza richiedere la sostituzione del tool o la riprogrammazione del sistema è il vantaggio competitivo fondamentale rispetto ai gripper convenzionali a geometria fissa. Nel dominio della riabilitazione motoria MicroHand trova applicazione come dispositivo di assistenza per persone con disabilità motorie degli arti superiori — persone con lesioni midollari, amputazioni parziali o condizioni neurologiche che limitano la capacità di presa manuale — che possono controllare la struttura MicroHand tramite segnali EMG residui o tramite interfacce di controllo adattate alle loro capacità motorie specifiche. La morfologia adattiva di MicroHand è particolarmente preziosa in questo contesto perché permette di compensare le variazioni nella capacità di controllo dell'utente — un utente con forza muscolare ridotta può controllare una struttura MicroHand di morfologia semplificata che richiede meno precisione di controllo, e progredire verso strutture più complesse man mano che le capacità migliorano attraverso la riabilitazione. Nel dominio dell'arte e della performance MicroHand diventa uno strumento espressivo che estende le capacità creative dell'artista oltre i limiti biologici della mano umana — producendo configurazioni di presa e di manipolazione impossibili per una mano biologica, eseguendo gesti di precisione e di forza che superano le capacità fisiche umane, operando in ambienti — sott'acqua, in alta temperatura, in spazi confinati — inaccessibili alla mano umana.

Il percorso evolutivo da MicroBot a MicroHand non è una discontinuità ma una progressione naturale che richiede tre condizioni di maturità del sistema base prima di poter essere intrapresa con successo. La prima condizione è la solidità del sistema di aggancio magnetico ibrido — verificata attraverso le campagne di test della versione 0.3 e 0.4 — che deve dimostrare un tasso di successo dell'aggancio superiore al 95% e una forza di tenuta sufficientemente elevata da supportare i carichi meccanici della manipolazione di oggetti. La

seconda condizione è la maturità del protocollo di comunicazione inter-bot — verificata nella versione 0.4 — che deve garantire latenze di coordinamento inferiori a 10 millisecondi tra moduli adiacenti per permettere il controllo in tempo reale della configurazione articolata della struttura durante la manipolazione. La terza condizione è la disponibilità di moduli attuatori sufficientemente miniaturizzati e affidabili — una condizione che dipende dall'evoluzione delle tecnologie di attuazione subminiatura e che costituisce il principale rischio tecnico del percorso evolutivo verso MicroHand. Se queste tre condizioni sono soddisfatte dalla versione 0.5 di MicroBot — e l'architettura del sistema è stata progettata con l'obiettivo esplicito di soddisfarle — il passaggio a MicroHand diventa un'estensione naturale che richiede l'aggiunta di moduli specializzati e lo sviluppo di nuovi algoritmi di controllo della presa su una piattaforma hardware e software già matura e verificata.

MicroHand è dunque il punto di chiusura della visione di MicroBot — il momento in cui lo swarm di moduli magneticamente aggregabili smette di essere un sistema di ricerca sulla swarm robotics e diventa uno strumento fisico capace di interagire con il mondo materiale con la stessa ricchezza e adattabilità con cui la mano biologica interagisce con l'ambiente che la circonda. Questo punto di arrivo non è una certezza — dipende da progressi tecnologici che non possono essere garantiti a priori e da scelte progettuali che verranno prese man mano che il sistema evolve attraverso le versioni pianificate — ma è un obiettivo che orienta le scelte presenti in modo concreto e verificabile, garantendo che ogni passo del percorso di sviluppo di MicroBot contribuisca alla costruzione delle fondamenta su cui MicroHand potrà essere realizzato. In questo senso MicroHand non è solo la versione futura di MicroBot — è la ragione per cui MicroBot è stato progettato nel modo in cui è stato progettato, il fine che giustifica i mezzi e che conferisce coerenza all'intera traiettoria evolutiva del progetto.

